面试准备材料 · 对外分享版（量化细节已脱敏）

华为「AI 模型工程师 多模态大模型」面试手册

候选人背景：多模态感知 / 3D 占据 / 端侧部署
目标岗位方向：多模态大模型算法（2012 · 诺亚 · 车 BU · 终端）

本册包含

1. **面试速成指南** — 岗位定位、简历关键词映射（8 条）、四个 STAR 故事、12 问口头速记、三天冲刺表、反问
2. **八股知识问答 74 问** — 视觉 token / 图文对齐 / 架构训练 / 幻觉评测 / 视频生成 / 推理部署 / 项目接口话术
3. **手撕 AI 题（C++ 版）** — 力扣 C++ 模板与 20 题清单、AI 题的 C++ 展开实现（含 26 项断言）
4. **手撕 AI 题（Python 版）** — Python/numpy 语法地基 + 六题逐行讲解 + torch 版（含 39 项断言）

配套可运行代码：`ai_interview_cpp_手撕题库已验证.cpp`（26 项断言） · `ai_interview_py_手撕题库已验证.py`（39 项断言，含 torch 与官方算子对拍）
生成日期 2026-10-29

华为「AI 模型工程师 — 多模态大模型」面试速成指南

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 用法：第 1 章改简历（半天），第 3 章背代码（两个半天），第 2、5 章通勤/睡前过。 全文约 3 天可闭环。红线在后面，先看不吃亏。

0. 先定调：你的牌面和这个岗位的接口在哪

你不是"零基础转多模态"，你的真实资产是**多模态融合 + 几何对齐 + 大规模训练调优 + 端侧部署**四件事的实操。华为这个岗（2012/诺亚/车 BU/终端都有类似坑位）真正要的是三类人之一：

岗位方向	关键词	你的匹配度
VLM 算法（图文/视频理解、对齐、SFT）	CLIP、Q-Former/MLP projector、LLaVA、数据配比、RLHF/DPO	中：训练调优、数据闭环迁移得过去，但**没训过 LLM**
多模态感知/空间智能（车 BU、具身）	BEV、Occ、深度、点云-图像融合、3D grounding	**高**：这是你的主场
端侧大模型/推理加速	量化、蒸馏、算子、昇腾 CANN/ATC、显存	**高**：ATC→MDC610 落地过

定位话术（开场 30 秒自述就用它）：

> "我做过的是把视觉、激光两种异构模态在统一特征网格上做对齐融合，并负责过从数据生产、指标归因到大模型训练稳定性、再到 ONNX→ATC→MDC 端侧部署的整条链路。多模态大模型里我最有底气的是**跨模态对齐、长序列建模、大规模训练调优和推理落地**这四块，模型侧的 VLM 结构和 SFT/偏好对齐我在系统补齐。"

诚实但不掉价，且直接命中华为最看重的"能落地 + 能调优"。

1. 简历往多模态靠（不改事实，只换词汇）

1.1 关键词映射表（左边是你现在的写法，右边是 HR/技术面想看到的写法）

你现有表述	改成（同义、但要能扛住追问）
视觉支路 + 激光支路 BEV 融合（BevFusionV2、SE、CBAM）	**异构模态特征对齐与门控融合**：视觉 128ch@256×256 与点云 128ch@256×256 在统一 BEV token 网格上对齐，SE 通道门控解决两模态密度/尺度不匹配
ViewTransformer + LSS 深度分布（D=99, sid 模式）	**几何感知的视觉 token 重组**：7 视角 3×384×704 经深度分布加权投影为 65,536 个 BEV query，实现图像 token 到 3D 空间的显式定位（空间智能/VLA 的地基能力）
temporal_history + scene-aligned 8 段时序切分	**长序列时序建模的显存-长度权衡**：按 scene 对齐切段避免跨场景污染，时序历史缓存控制在 X 帧，平衡 mIoU 与显存
多尺度深监督 + ASPP 粗头 w=0.2（loss 降一个量级）	**多尺度深监督抑制收敛不稳**：粗头辅助损失加权 0.2，收敛期 loss 降一个数量级，解决深层任务头梯度稀疏
32×32 混合矩阵替 3-tap（显存降约 8%）	**显存墙下的结构重构**：用 32×32 混合矩阵替代 3-tap 大核，显存 −7.1GB（−8%），保住 batch 与分辨率
OCC GT 数据闭环（数万个场景/百万级关键帧/ray-casting 三态）	**大规模多模态数据生产线与评测基准构建**：ray-casting 区分 free/occupied/unknown 三态，scene 级时序采样，零激光评测协议（万级帧）+ 双 seed 噪声地板
ONNX→ATC/AOE→MDC610 .om；Decode/TopK/NMS 解耦	**端侧推理优化与算子改造**：不支持的算子解耦 + OpenCV 向量化，ROI 超分误检 约 5%→约 1.4%，完成 opset11/16 到昇腾 .om 全链路
标定三层法（4-DOF 同位变换/PCA 切线拟合/递进降级）	**多模态时空对齐与鲁棒性设计**：scale-invariant 残差 + 状态机退化恢复，角点补全 约 9px→约 3px、失败恢复率 约 87%

1.2 三条 bullet 的改写示例（直接可贴）

改前：负责视觉与激光 BEV 融合模块开发与训练调试。

改后：

> 主导 7 相机 + 4 激光的异构模态融合主干（BevFusionV2 + SE 门控）：两条支路在 0.6m/格、256×256 的 BEV token 网格上对齐，SE 通道重加权缓解模态密度差；8 卡训练集群 训练 12/24 epoch EMA，显存峰值 约 89GB（十万级行 batch点云），通过混合矩阵重构降到 约 82GB 后保住分辨率。

改前：负责 Occupancy 真值生产与评测。

改后：

> 负责 Occ 监督生产线与评测基准（数万个场景 / 百万级关键帧）：ray-casting 区分 free/occupied/unknown 三态，标注口径与转换器版本双向核查（发现 filter 版本反转导致 mask 体积 约 0.64 倍、occ mIoU 降约 3 个点 的隐性 bug）；设计 万级帧评测协议与双 seed 噪声地板，把"模型变好"与"评测抖动"分开。

改前：负责模型在 MDC 平台的部署。

改后：

> 完成感知模型 ONNX(opset11/16)→ATC/AOE→MDC610 .om 的端侧落地：对 NPU 不支持的 Decode/TopK/NMS 做算子解耦 + OpenCV 向量化重写，端侧 ROI 超分误检 约 5%→约 1.4%。

1.3 必须补进简历的"多模态新词"（真实可辩护的最小增量）

只在**你真的动过手/读透过**的前提下加，且要能接住追问：

- **CLIP / InfoNCE 对比学习**：把"视觉-文本对齐"接到你现成的"视觉-激光对齐"经验上（对齐就是对齐，差别在监督信号与 loss）。
- **ViT / patch tokenization**：你在 backbone 里做的就是多尺度特征 token 化，能讲清 patch 划分、位置编码、CLS token、计算复杂度 $O(N^2)$ 与窗口注意力。
- **projector（MLP / Q-Former）**：讲清"冻结视觉塔 + 只训二层 MLP"为什么在 LLaVA 里够用，Q-Former 的 query 压缩为什么在 BLIP-2 里用来降 token 数——**这个你可以用"BEV query 压缩/重排"经验直接类比**，是你最容易讲出深度的点。
- **LoRA / 混合精度 / ZeRO**：你有显存墙实战，讲 LoRA 的 rank/alpha、ZeRO-2/3 的切分对象即可。
- **评测集**：MMBench / MMMU / SEED / POPE（幻觉）、RefCOCO（grounding）各评什么、指标是什么。

1.4 红线（比写得好更重要）

1. **不说自己训过 LLM/VLM**。说"补课中/做过同构的跨模态对齐与大规模训练调优"。
2. **不报参数量**（百万级参数量 这种会被"这么点也叫大模型"打穿）。改说结构设计意图与单变量控制（恒等初始化保证可比）。
3. **不说论文已发表**。
4. **不说做过 SLAM**（只说视觉几何、位姿对齐、ICP、4-DOF 同位变换）。
5. **不跨协议比指标**。被问延迟/精度，先报口径（帧数、平台、是否含预处理）再报数。

---

2. 四个可复用的答题故事（STAR，面试官会反复挖）

华为技术面最爱链路化追问："为什么这么设计→试过哪些替代→怎么证明是它起作用→失败怎么定位"。这四段准备好，能覆盖 80% 的项目提问。

1. **跨模态对齐**：两模态分辨率/密度不一致 → 统一到 0.6m BEV 网格；对比 concat / SE 门控 / CBAM 的差异；用逐类 fn/fp 成分解剖证明增益不是超参噪声。

2. **训练稳定性三连**：CE target 越界（cls_num 参数名坑）→ CUBLAS_STATUS_ALLOC_FAILED（3→8 GEMM workspace，改 depthwise+pool）→ GT-Paste spconv 越界。**讲"怎么定位"比讲"修了什么"值分**：看日志→二分变量→最小复现。
3. **数据/评测的口径 bug**：转换器两个方向相反的 filter 版本 → mask 体积对不上（约 0.64 倍）→ 用逐元素对比 + IoU=1.0 锁定版本 → 结论是"掉分是转换器 bug 不是类别树变差"。这是**极强的工程严谨性证明**。
4. **端侧部署**：NPU 不支持算子 → 解耦 + 向量化 → 误检下降。升华句："端侧不是把模型塞进去，是把不可部署的语义换成可部署的等价语义。"

---

3. 手撕题（核心）——机器学习部分

> 面试手撕的机器学习和力扣的区别：**力扣考正确性，机器学习手撕考"你知不知道工业实现的坑"**。 写完代码必须主动补 2 句追问点，这是加分位。

3.1 Softmax（+ 数值稳定 + 交叉熵）

```python
import numpy as np

def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)      # 防 exp 溢出，数学上恒等
    e = np.exp(x)
    return e / e.sum(axis=axis, keepdims=True)

def log_softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)
    return x - np.log(np.exp(x).sum(axis=axis, keepdims=True))
```

PyTorch 版交叉熵（**高频手撕**）：

```python
import torch
def cross_entropy(logits, target, weight=None):      # logits:[N,C] target:[N] int
    m = logits.max(dim=1, keepdim=True).values
    lse = m.squeeze(1) + torch.log(torch.exp(logits - m).sum(dim=1))
    nll = lse - logits.gather(1, target[:, None]).squeeze(1)
    if weight is not None:
        w = weight[target]
        return (nll * w).sum() / w.sum()
    return nll.mean()
```

追问必答： - 为什么要减 max？exp 上溢成 inf，再除 inf 得 nan；减常数不改变结果。 - log_softmax 比 softmax 稳在哪？交叉熵直接在对数域算，避免 "先算小概率→取 log" 的下溢；用 logsumexp 恒等式。 - softmax 温度 T 的作用：T↑ 分布更平（蒸馏/采样多样性），T↓ 更尖（argmax 近似）。 - 带 mask 的 softmax：被屏蔽位置置 **-1e9 或 -inf**，注意**整行全屏蔽会出 nan**，工业实现会 `scores.masked_fill(mask, -1e4)` 或对全屏蔽行补 0。 - Softmax vs Sigmoid：多分类互斥 vs 多标签独立（BCEWithLogits 更稳，因为它自带 logsumexp 形式）。 - Label Smoothing 怎么改？target 从 one-hot 变 (1-ε) 的 one-hot + ε/C 均匀。

3.2 K-Means

```python
import numpy as np

def kmeans(X, k, iters=100, tol=1e-4, seed=0, n_init=1):
    X = np.asarray(X, dtype=np.float64)
    rng = np.random.default_rng(seed)
    best_labels, best_centers, best_inertia = None, None, np.inf
    for _ in range(n_init):
        idx = rng.choice(len(X), k, replace=False)        # 简单随机初始化
        centers = X[idx].copy()
        labels = np.zeros(len(X), dtype=int)
        for _ in range(iters):
            d = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(-1)   # n*k
            new_labels = d.argmin(1)
            new_centers = centers.copy()
            for j in range(k):
                m = new_labels == j
                if m.any():                               # 空簇：保留旧中心
```

```
                    new_centers[j] = X[m].mean(0)
            shift = np.linalg.norm(new_centers - centers)
            centers, labels = new_centers, new_labels
            if shift < tol:
                break
        inertia = ((X - centers[labels]) ** 2).sum()
        if inertia < best_inertia:
            best_labels, best_centers, best_inertia = labels, centers, inertia
    return best_labels, best_centers
```

追问必答： - 复杂度 O(n · k · d · iters)，k-means++ 初始化把随机初始化的最坏情况压掉。 - k 怎么定：肘部法（inertia 拐点）、轮廓系数 silhouette、Gap statistic、业务约束。 - 假设：簇是**凸的、各向同性、方差相近**；对异常值敏感（可用 k-medoids / trimmed）。 - 与 EM/GMM：k-means 是 GMM 的硬分配特例（等协方差），EM 的 E 步=软分配。 - 空簇处理：保留旧中心 / 取离其他中心最远点。 - 加速：MiniBatch、kd-tree/球树（低维）、三角不等式剪枝（Elkan）。 - 面试官可能顺手问 DBSCAN：密度可达、不需 k、能找任意形状、对 eps/minPts 敏感、高维失效。

3.3 Self-Attention / Multi-Head Attention（最可能被要求写的一个）

先写 Self-Attention（单头），再扩多头，最后加 mask。用 PyTorch：

```
import math, torch, torch.nn as nn, torch.nn.functional as F

def self_attention(x, Wq, Wk, Wv, mask=None):
    # x: [B, L, D]; Wq/Wk/Wv: [D, D]
    Q, K, V = x @ Wq, x @ Wk, x @ Wv                  # [B, L, D]
    scores = Q @ K.transpose(-2, -1) / math.sqrt(Q.size(-1))   # [B, L, L]
    if mask is not None:
        scores = scores.masked_fill(mask, float('-inf'))       # mask=True 表示屏蔽
    attn = torch.softmax(scores, dim=-1)
    return attn @ V, attn
```

多头（完整、含 mask、可跑）：

```
class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_head, dropout=0.0):
        super().__init__()
        assert d_model % n_head == 0
        self.h = n_head
        self.d_k = d_model // n_head
        self.w_q = nn.Linear(d_model, d_model)
        self.w_k = nn.Linear(d_model, d_model)
        self.w_v = nn.Linear(d_model, d_model)
        self.w_o = nn.Linear(d_model, d_model)
        self.drop = nn.Dropout(dropout)

    def forward(self, q, k, v, attn_mask=None, key_padding_mask=None):
        B, Lq, _ = q.shape
        Lk = k.size(1)
        # 1) 线性投影 + 拆头: [B,L,D] -> [B,h,L,dk]
        Q = self.w_q(q).view(B, Lq, self.h, self.d_k).transpose(1, 2)
        K = self.w_k(k).view(B, Lk, self.h, self.d_k).transpose(1, 2)
        V = self.w_v(v).view(B, Lk, self.h, self.d_k).transpose(1, 2)
        # 2) 缩放点积
        scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k)    # [B,h,Lq,Lk]
        # 3) 两种 mask: 因果/结构 mask [Lq,Lk]; padding mask [B,Lk]
        if attn_mask is not None:
            scores = scores.masked_fill(attn_mask.bool(), float('-inf'))
        if key_padding_mask is not None:
            pad = key_padding_mask[:, None, None, :].bool()       # [B,1,1,Lk]
            scores = scores.masked_fill(pad, float('-inf'))
        # 4) softmax 后合并
        attn = self.drop(torch.softmax(scores, dim=-1))
        out = attn @ V                                            # [B,h,Lq,dk]
        out = out.transpose(1, 2).contiguous().view(B, Lq, self.h * self.d_k)
        return self.w_o(out), attn
```

Numpy 版（若面试官说"别用框架"）：

```
def mha_numpy(x, Wq, Wk, Wv, Wo, h, mask=None):
    B, L, D = x.shape
    dk = D // h
    def split(t):                                  # [B,L,D] -> [B,h,L,dk]
```

```
        return t.reshape(B, L, h, dk).transpose(0, 2, 1, 3)
    Q, K, V = split(x @ Wq), split(x @ Wk), split(x @ Wv)
    scores = Q @ K.transpose(0, 1, 3, 2) / np.sqrt(dk)           # [B,h,L,L]
    if mask is not None:
        scores = np.where(mask, -1e9, scores)
    scores = scores - scores.max(-1, keepdims=True)
    e = np.exp(scores)
    attn = e / e.sum(-1, keepdims=True)
    out = (attn @ V).transpose(0, 2, 1, 3).reshape(B, L, D)
    return out @ Wo, attn
```

追问必答（**这部分比代码更值分**）：- **为什么除 √d_k**：q·k 的方差随 d_k 线性增长，不缩放会让 softmax 落到饱和区、梯度消失；缩放使方差归一。- **复杂度**：时间/显存 O($L^2 \cdot d$)。这就是长上下文贵的根因 → 引出去向：FlashAttention（分块 + 在线 softmax，**省显存不省 FLOPs**）、MQA/GQA（共享 K/V 头，省 KV cache）、稀疏/滑窗注意力、线性注意力、Mamba/SSM。- **O(L^2) 显存怎么省**：FlashAttention 的 tiling + 重计算（不落 L×L 矩阵）、KV cache 量化、GQA。- **view 后为什么必须 contiguous**：transpose 后内存不连续，view 要求连续，否则报错——实际实现写 `reshape` 或加 `.contiguous()`。- **mask 语义一定要说清**：True=屏蔽（-inf）、False=保留；padding mask 是 batch 维、causal mask 是 Lq×Lk 下三角。- **为什么要多头**：不同头学不同子空间/不同依赖模式（位置、句法、指代），可类比 CNN 多通道。- **LayerNorm 放哪**：Post-LN（原始 Transformer，需 warmup）vs Pre-LN（现代主流，更稳，可去 warmup）；RMSNorm 少了均值中心化，更快。- **Q/K/V 不同源**：self-attention（同源）vs cross-attention（Q 来自 decoder/query，K/V 来自 encoder，就是 Flamingo/BLIP-2/多模态融合里的融合入口——**这里一定要接上你的经验**）。- 位置编码：绝对正弦、可学习、RoPE（相对位置、可外推、LLaMA 系）、ALiBi。**RoPE 为什么能外推/怎么扩 context（NTK 插值/位置内插）** 是当前高频追问。

3.4 分类树（CART，ID3/C4.5 关系要会讲）

```
import numpy as np

class Node:
    __slots__ = ("feat", "thr", "left", "right", "value")
    def __init__(self): self.feat = self.thr = self.left = self.right = self.value = None

def _gini(y, n_cls):
    if len(y) == 0: return 0.0
    p = np.bincount(y, minlength=n_cls) / len(y)
    return 1.0 - (p ** 2).sum()

def _build(X, y, depth, max_depth, min_samples_leaf, min_gain, n_cls):
    node = Node()
    node.value = np.bincount(y, minlength=n_cls).argmax()
    # 停止条件：深度、纯节点、样本太少
    if depth >= max_depth or len(np.unique(y)) == 1 or len(y) < 2 * min_samples_leaf:
        return node
    best_gain, best = min_gain, None
    parent = _gini(y, n_cls)
    n = len(y)
    for f in range(X.shape[1]):
        vals = np.unique(X[:, f])
        thrs = (vals[:-1] + vals[1:]) / 2.0
        for t in thrs:
            m = X[:, f] <= t
            nl, nr = m.sum(), n - m.sum()
            if nl < min_samples_leaf or nr < min_samples_leaf:
                continue
            g = parent - (nl * _gini(y[m], n_cls) + nr * _gini(y[~m], n_cls)) / n
            if g > best_gain:
                best_gain, best = g, (f, t, m)
    if best is None:
        return node
    f, t, m = best
    node.feat, node.thr = f, t
    node.left  = _build(X[m],  y[m],  depth + 1, max_depth, min_samples_leaf, min_gain, n_cls)
    node.right = _build(X[~m], y[~m], depth + 1, max_depth, min_samples_leaf, min_gain, n_cls)
    return node

def _predict_one(node, x):
    while node.feat is not None:
        node = node.left if x[node.feat] <= node.thr else node.right
    return node.value

class DecisionTreeClassifier:
```

```
    def __init__(self, max_depth=5, min_samples_leaf=1, min_gain=1e-7):
        self.max_depth, self.min_samples_leaf, self.min_gain = max_depth, min_samples_leaf, min_gain
    def fit(self, X, y):
        self.n_cls = int(y.max()) + 1
        self.root = _build(np.asarray(X, float), np.asarray(y, int), 0,
                           self.max_depth, self.min_samples_leaf, self.min_gain, self.n_cls)
        return self
    def predict(self, X):
        return np.array([_predict_one(self.root, x) for x in np.asarray(X, float)])
```

追问必答：- **划分准则**：ID3 信息增益（偏多值特征，倾向多叉）、C4.5 信息增益率（除法惩罚）、CART 基尼指数（**二叉树**，分类用基尼、回归用 MSE，叶子取均值）。- **为什么 CART 用基尼不用熵**：二次形式没有 log，快；两者结论高度一致。- **剪枝**：预剪枝（max_depth/min_samples_leaf/min_gain，快但可能欠拟合，贪心视野短）vs 后剪枝（代价复杂度 CART 的 α，从全树自底向上合并，更准但贵）。- **过拟合原因**：划分太细，叶子只剩 1 个样本 → 方差爆炸。控制树深 / 叶最小样本 / 后剪枝。- **连续特征处理**：排序后取中点枚举 O(n · log n · d) 的排序 + O(n · d) 扫描；大样本用分位分桶近似（LightGBM 直方图）。- **特征重要性**：按该特征分裂带来的加权不纯度下降累积（sklearn 的 gini_importance），偏多值特征；更可靠是置换重要性。- **推导到 GBDT**：残差/梯度拟合、XGBoost 的二阶 Taylor + 正则（$\gamma T + \lambda w^2/2$）+ 并行建直方图、LightGBM 的 leaf-wise + GOSS + EFB。- 顺带：随机森林 = bagging + 特征子采样，降方差；树模型不需要归一化、对单调变换不变、不能外推。

3.5 机器学习高频补充（能写就写，5 分钟级）

```
# IoU
def iou(box_a, box_b):  # [x1,y1,x2,y2]
    ix1, iy1 = max(box_a[0], box_b[0]), max(box_a[1], box_b[1])
    ix2, iy2 = min(box_a[2], box_b[2]), min(box_a[3], box_b[3])
    iw, ih = max(0., ix2 - ix1), max(0., iy2 - iy1)
    inter = iw * ih
    area_a = max(0., box_a[2]-box_a[0]) * max(0., box_a[3]-box_a[1])
    area_b = max(0., box_b[2]-box_b[0]) * max(0., box_b[3]-box_b[1])
    union = area_a + area_b - inter
    return inter / union if union > 0 else 0.

# NMS
def nms(boxes, scores, thr=0.5):
    order = scores.argsort()[::-1]
    keep = []
    while order.size > 0:
        i = order[0]; keep.append(i)
        if order.size == 1: break
        ious = np.array([iou(boxes[i], boxes[j]) for j in order[1:]])
        order = order[1:][ious <= thr]
    return keep

# LayerNorm（手写反向是进阶题）
def layer_norm(x, gamma, beta, eps=1e-5):
    mu = x.mean(-1, keepdims=True)
    var = x.var(-1, keepdims=True)
    return gamma * (x - mu) / np.sqrt(var + eps) + beta

# BatchNorm 训练/推理差异 + 动量更新
# BN: 对 (N,H,W) 求统计量，依赖 batch（大 batch 才稳，小 batch 用 GroupNorm/LN）
# LN: 对单样本所有通道求，与 batch 无关，RNN/Transformer 标配
```

必背的还有：AUC 手算（按 score 排序后累加正样本 rank）、Top-1/Top-5、mAP、F1/Precision/Recall 与阈值、ROC vs PR 不平衡数据下的选择、L1/L2 为什么 L1 稀疏（等高线交点落在轴上/次梯度）、Dropout 训练-推理的缩放、Adam 的一阶二阶矩偏差修正、学习率 warmup+cosine 为什么有效、类别不平衡（focal loss、重采样、代价敏感）、梯度消失/爆炸（初始化、残差、LN、梯度裁剪）、双 seed 噪声地板（**这条你有实战，主动讲**）。

---

4. 手撕题——力扣部分（华为风格）

4.1 先明确考试形态

- **校招机试**：3 题 / 400 分 / 120 分钟，难度约 简单100 + 中等200 + 困难300，**ACM 模式**（自己读 stdin、自己 print），支持本地 IDE 调试。多数组只做出 200 分就能进面，**但第一题必须稳**。

- **技术面手撕**：面试官现场给题，共享屏幕写，通常 LeetCode 中等，要求能跑通 + 说复杂度。**边写边说出思路、先讲暴力再优化**，比默写满分解更受认可。
- 华为偏好：**字符串处理、模拟、DFS/BFS、DP、二叉树、数组双指针**。硬核图论/线段树极少。

4.2 ACM 模式 IO 模板（务必背，很多人挂在这）

```python
import sys

def main():
    data = sys.stdin.read().split()      # 万能：全部按空白切分
    if not data: return
    # 例：第一行 n，之后 n 个整数
    n = int(data[0])
    nums = list(map(int, data[1:1 + n]))
    print(sum(nums))

if __name__ == "__main__":
    main()
```

```python
# 多行/多组的稳健写法
import sys
lines = [l.strip() for l in sys.stdin if l.strip()]
for line in lines:
    a, b = map(int, line.split())
    print(a + b)
```

注意：华为题常有多组输入直到 EOF、以及行内用逗号/空格混排；**读进来先 split，不要相信行数**。

4.3 必刷清单（按优先级，20 + 15）

P0（闭眼能写，两天刷完）

#	题	考点
1	两数之和 / 三数之和	哈希、排序+双指针
2	无重复字符的最长子串	滑动窗口
3	最长回文子串	中心扩展（DP 次选）
4	有效括号	栈
5	字符串反转 / 翻转单词 / 大数相加	字符串模拟
6	合并两个有序链表 / 反转链表	链表指针
7	环形链表 / 相交链表	快慢指针
8	LRU 缓存	哈希 + 双向链表（**华为高频**）
9	二叉树层序遍历 / 最大深度 / 之字形	BFS/DFS
10	验证二叉搜索树	中序 or 上下界
11	岛屿数量 / 岛屿最大面积	DFS 淹没
12	腐烂的橘子	多源 BFS
13	全排列 / 子集 / 组合总和	回溯模板
14	爬楼梯 / 打家劫舍	线性 DP
15	编辑距离 / 最长公共子序列	二维 DP
16	最大子数组和	贪心/DP
17	数组中第 K 大 / 前 K 高频	堆 or 快速选择
18	搜索旋转排序数组	二分变体
19	零钱兑换	完全背包

#	题	考点
20	手写快排 / 归并 / 堆排	排序（面试常直接考）

P1（有余力）：接雨水、滑动窗口最大值（单调队列）、最小覆盖子串、最长递增子序列（O(nlogn) 贪心+二分）、01 背包、课程表（拓扑）、朋友圈（并查集）、二叉树最近公共祖先、路径总和 II、字符串解码、螺旋矩阵、复原 IP 地址。

4.4 五个模板（现场默写用）

```python
# 1) 快排（含随机化，避免有序数组退化）
import random
def quick_sort(a, l=0, r=None):
    if r is None: r = len(a) - 1
    if l >= r: return a
    p = random.randint(l, r)
    a[l], a[p] = a[p], a[l]
    pivot, i = a[l], l
    for j in range(l + 1, r + 1):
        if a[j] < pivot:
            i += 1
            a[i], a[j] = a[j], a[i]
    a[l], a[i] = a[i], a[l]
    quick_sort(a, l, i - 1); quick_sort(a, i + 1, r)
    return a

# 2) 二分（左边界，防死循环）
def lower_bound(a, x):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < x: lo = mid + 1
        else: hi = mid
    return lo

# 3) 滑动窗口（无重复最长子串）
def length_of_longest(s):
    last, left, ans = {}, 0, 0
    for i, c in enumerate(s):
        if c in last and last[c] >= left:
            left = last[c] + 1
        last[c] = i
        ans = max(ans, i - left + 1)
    return ans

# 4) 小顶堆 TopK（n log k）
import heapq
def topk(nums, k):
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k: heapq.heappop(h)
    return h

# 5) 并查集
class DSU:
    def __init__(self, n): self.p = list(range(n)); self.r = [0]*n
    def find(self, x):
        while self.p[x] != x:
            self.p[x] = self.p[self.p[x]]
            x = self.p[x]
        return x
    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb: return False
        if self.r[ra] < self.r[rb]: ra, rb = rb, ra
        self.p[rb] = ra
        if self.r[ra] == self.r[rb]: self.r[ra] += 1
        return True
```

写题时的三个习惯：**先说复杂度 → 写边界（空、单元素、全相同、溢出）→ 写完自己过一遍样例**。

5. 多模态八股速记（12 问，各 30 秒版）

> 这里是**口头速记版**；完整答案库（118 问，按面试顺序 + 追问 + 项目接口话术 + 融合调控/损失函数专章）见同目录《华为-多模态大模型-八股知识问答_20261029.docx/md/pdf》。

1. **CLIP 怎么训的**：图像/文本双塔，batch 内 N×N 相似度矩阵，对角为正样本，对称 InfoNCE（in-batch negatives）。温度 τ 可学习。零样本靠 prompt + 文本分类器权重。
2. **CLIP 的局限**：只做粗粒度全局对齐，不擅长计数/空间关系/细粒度；对比学习不考虑词序（bag-of-words 问题）。
3. **BLIP-2 的 Q-Former**：冻结图像塔 + 冻结 LLM，中间用固定数量 learnable query 通过 cross-attention 抽取视觉信息，把可变长 patch 压成固定 token（省 LLM 上下文）。两阶段：表征学习（ITC/ITM/ITG）→ 生成学习（接 LLM 做 LM loss）。
4. **LLaVA 的 projector**：两层 MLP 直接映射视觉 token，简单有效（消融显示比 Q-Former 不差）；两阶段训练=先冻结 LLM 只训 projector（对齐），再解冻部分/全量做指令微调。
5. **ViT vs CNN**：patch 化 + 全局注意力、无归纳偏置、需要大数据或强增广/蒸馏；计算 $O(N^2)$，高分辨率要用窗口/分层（Swin）。
6. **视觉 token 太多怎么办**：Q-Former/Perceiver 压缩、池化、token merge、动态分辨率、只取关键帧。**接你的经验：BEV query 重排与深度加权投影就是"几何感知的 token 组织"**。
7. **位置编码**：正弦/可学习/RoPE/ALiBi；RoPE 保相对位置、LLaMA 系标准；长上下文靠位置内插/NTK/缩放。
8. **训练三阶段**：预训练（对齐）→ SFT（指令遵循）→ 偏好对齐（RLHF/DPO）。DPO 不用 reward model，直接用偏好对做分类式损失。
9. **LoRA**：低秩分解 ΔW=BA，只训 A/B，rank 8~64、alpha 缩放、可合并推理零开销。
10. **幻觉**：视觉 token 与语言先验冲突时模型倾向语言先验（POPE 评测）；缓解=更强视觉监督、负样本、解码策略、RAG。
11. **评测**：MMBench/MMMU/SEED（综合），POPE（幻觉），RefCOCO/Visual Grounding，VideoMME（长视频）。说清"评什么、怎么算分"就够。
12. **端侧/昇腾**：CANN、ATC（ONNX→om）、AOE 调优、量化 INT8/INT4、算子不支持要替换。**你有 ATC/MDC610 真机经验，这题是送分题，务必主动引到这儿**。

---

6. 3 天冲刺表

时间	任务
D1 上午	改简历（1.1 映射表逐条过）+ 背开场 30 秒自述 + 红线
D1 下午	手撕 softmax/CE、kmeans、self-attention 单头 → 多头 mask 版（手写 3 遍不看书）
D1 晚	力扣 P0 前 10 题（限时 20 分钟/题，ACM 模式输入输出）
D2 上午	手撕 CART + 追问点复述（划分准则/剪枝/GBDT 推导）
D2 下午	力扣 P0 后 10 题 + 五个模板默写
D2 晚	多模态八股 12 问（能开口讲，不追求逐字）
D3 上午	四个 STAR 故事各讲 3 分钟并录音自听；补 CLIP/BLIP-2/LLaVA 结构图能画出来
D3 下午	全真模拟：一道力扣中等（边说边写）+ 一道机器学习手撕 + 项目深挖 15 分钟

---

7. 反问环节（3 个，显示你在想落地）

1. 团队的多模态模型主要在哪个模态组合上做（图-文 / 视频 / 点云-图），当前瓶颈在数据、结构还是推理成本?
2. 端侧和云侧的比例大概多少？模型要到什么时延/算力预算，昇腾侧是 ATC 自研链路还是 MindSpore 栈?
3. 新人前三个月最可能被派到哪一块（数据管线 / 训练 / 评测 / 部署）？

附：代码自测状态

第 3、4 章代码已落到可运行文件（与本文同一目录）：

- `华为多模态手撕题库_自测通过_20261029.py`

已跑通的断言：softmax/log_softmax（含 1e3 级大数不溢出、平移不变）；交叉熵与 `F.cross_entropy` 数值一致（含类别权重）；kmeans 从三个高斯簇正确恢复到 0.5 半径内；CART 二分类/三分类训练集 ACC=1.0 且根节点阈值落在簇间（x0≤1.362）；IoU/NMS 的期望框保留顺序；LayerNorm 零均值单位方差；力扣五个模板（快排随机化 50 组随机数组、lower_bound、滑窗、TopK、DSU）全通过。

注意：本机没有 torch 环境，**3.3 的 torch 版 MHA 是用同构 numpy 实现验证逻辑的**（单头/多头/因果 mask/padding mask 形状与 softmax 归一化均已验证），但 torch 的 `view/transpose/masked_fill/gather` 具体调用建议你在自己的环境里跑一遍再上场默写。

本文档基于你已有项目（BEV 多任务融合、OCC 数据闭环、标定与端侧部署）撰写，所有数字均来自既有记录；不构成虚构经历。

多模态大模型 八股知识问答（华为 AI 模型工程师岗）

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 与《面试速成指南》第 5 章的 12 问速记版配套：**那份是口头速记，这份是完整答案库**。 共 **118 问**，按面试出现顺序编排（S1~S7 通用八股 74 问 + S8 融合调控专章 22 问 + S9 损失函数专章 22 问）。标注说明：★★★ = 几乎必问，★★ = 大概率问，★ = 加分题。 答法原则：**先给一句结论，再给一层原因，最后给一个你亲手做过的证据**。三段式，别背长段落。

S1 视觉编码器与 token（8 问）

Q1 ★★★ ViT 是怎么工作的？一张图变成多少 token 怎么算？

A：把图像切成固定大小的 patch（如 16×16），每个 patch 展平后线性投影成一个 token，加上位置编码，送入标准 Transformer 编码器。token 数 = (H/p) × (W/p)。

现场可背的例子： - 224×224、patch 16 → 14×14 = **196 个 token** - 384×704（车载常见分辨率）、patch 16 → 24×44 = **1056 个 token** - 384×384、patch 14 → 27×27 = **729 个 token**

追问：为什么 patch 越小越好但也越贵？→ token 数平方增长，attention 是 $O(N^2)$，显存和算力都吃紧；SAM/LLaVA-1.6 用更小 patch 换细节，代价是 token 爆炸，所以要配 token 压缩。

Q2 ★★ CLS token 是干什么的？必须有吗？

A：ViT 沿用 BERT 的做法，在序列最前面加一个可学习的 `[CLS]` token，靠 self-attention 汇聚全局信息，做分类时只取它的输出。**不是必须的**：ViT 论文里也试过对 patch token 做全局平均池化（GAP），效果接近；现代多模态模型大多**不用 CLS**，直接把所有 patch token 交给 LLM（因为下游要的是每个位置的细粒度特征，不是单一向量）。

Q3 ★★ ViT 和 CNN 的本质区别？为什么大模型都用 ViT？

A：CNN 有强归纳偏置（局部性、平移等变、权重共享），小数据下更强；ViT 只有全局注意力、没有这些先验，**必须有大数据或强增广/蒸馏才能超过 CNN**，但一旦数据够，它的表达能力和"和 LLM 同构"的优势就出来了。

用 ViT 的真实原因：① 结构同构，方便和多模态 LLM 拼装；② 全局感受野，适合需要长程关系的任务；③ scaling 行为好（参数/数据同比例涨，loss 平滑下降）。

追问：Swin 怎么补回局部性？→ 窗口内注意力 + 窗口移位（shifted window）做跨窗交互 + 分层下采样，复杂度从 $O(N^2)$ 降到 O(N)。

Q4 ★★ 多模态里位置编码有哪些选择？RoPE 为什么能外推？

A：四类——正弦绝对编码、可学习绝对编码、**RoPE（旋转位置编码）**、ALiBi（按距离线性衰减 bias）。

RoPE 的做法：把 q/k 的相邻两维看成一个二维向量，按位置 m 旋转 m·θ 角度，于是内积只依赖相对位置差 (m-n)。**能外推是因为它编码的是相对距离，不是绝对下标**；但直接外推会随距离增大而失真，所以要配位置内插（PI）、NTK-aware 缩放、YaRN 这些方法扩上下文。

追问：图像 token 用 2D RoPE 怎么搞？→ 行列各一套频率（Qwen2-VL 就是 M-RoPE：时间/高/宽三套）。

Q5 ★★ 高分辨率图像怎么处理？（这是多模态工程的核心问题之一）

A：四条主流路线： 1. **固定放大**（LLaVA-1.5 用 336×336 或 448×448）——简单但细节丢失、宽高比失真。 2. **切块 + 缩略图**（LLaVA-1.6/InternVL 的 dynamic tiling）：把大图切成若干 336 块，每块独立编码，再加一张全局缩略图。token 数变成块数×(块 token)+缩略图 token。 3. **原生动态分辨率**（NaViT、Qwen-VL 的 M-RoPE）：直接按原分辨率切 patch，不缩放。 4. **像素重排/pixel shuffle**（InternVL）：把相邻 2×2 个 patch 在通道维拼起来，token 数直接降 4 倍，用空间换通道。

追问：token 变多有什么代价？→ ① LLM 的 prefill 变慢、KV cache 变大；② 图文 token 比例失衡，注意力被图像挤占。

Q6 ★★★ 视觉 token 太多怎么压缩?

A：五类手段，按"是否可学习"排： 1. **池化/重采样**（平均池化、Perceiver Resampler 用固定数量 query 做 cross-attention）——**Q-Former/Perceiver 就是这一类，把可变长 patch 压成固定 32/64 个 token**。 2. **token merge**（相邻相似 token 合并，如 Token Merging、LLaVA-PruMerge）。 3. **空间重排**（pixel shuffle，2×2 拼通道，token ÷4）。 4. **重要性剪枝**（按注意力分数保留 top-k token）。 5. **只取关键帧/区域**（视频/大图场景）。

你的接口话术："我在 BEV 里做的就是同类问题——7 个视角的 patch token 经深度分布加权投影到 65,536 个 BEV query 上，本质是几何约束下的 token 重组；换成 VLM 就是把 query 数量压到 LLM 预算内。"（**这句是本文档最有价值的一句话，务必背下来**）

Q7 ★ ViT 的预训练方式有哪些?

A：① 监督分类（ImageNet）；② **对比学习**（CLIP 图文对比、DINO 自蒸馏无标签）；③ **掩码重建**（MAE，遮 75% patch 重建像素，训练效率高）；④ 生成式（BEiT 预测离散 visual token）。多模态场景主流是 CLIP/SigLIP 这类图文对比预训练，因为它天然对齐了文本空间。

Q8 ★★ 一张 4K 图（3840×2160）patch 14 直接切，token 多少？为什么不能这么干?

A：274×154 ≈ **42,196 个 token**。对比：LLaMA-2 的上下文是 4096。所以不可能直接喂——必须缩放、切块、压缩三选一（见 Q5/Q6）。这也是"端侧多模态"最现实的约束：**分辨率是 token 的来源，token 是算力和显存的货币**。

S2 图文对齐与经典模型（12 问）

Q9 ★★★ CLIP 是怎么训练的？损失函数是什么?

A：双塔（图像塔 ViT/ResNet + 文本塔 Transformer），各自编码后投影到同一维度并 L2 归一化，得到 B×B 的相似度矩阵。**对角是正样本对，其他 2(B-1) 个都是负样本（in-batch negatives）**，损失是对称的 InfoNCE：

```
L = 1/2 · [ CE(图像→文本) + CE(文本→图像) ]
```

即图像侧做一次 softmax 分类（正确文本是 label），文本侧再做一次。温度 τ 是可学习标量（CLIP 里初始化为 0.07 的倒数形式，即 logit_scale=1/τ）。

追问：为什么必须有 in-batch negatives? → 没有负样本，模型会把所有图文映射到同一点（坍缩）。负样本的多样性直接决定表征质量，所以**大 batch 是 CLIP 的关键**（原始实现 batch 32768）。

Q10 ★★ CLIP 怎么零样本分类?

A：把每个类名套进 prompt 模板（如 "a photo of a {class}"）编码成文本向量，图像向量与所有类文本向量算余弦相似度，取最大。**prompt 工程很关键**（CLIP 论文用了 80 个模板做集成），因为文本侧的训练分布是"句子"而非"单词"。

Q11 ★★ CLIP 的局限有哪些?

A：① 只做**全局粗粒度**对齐，不擅长细粒度属性、计数、空间关系（"左边的红球"）；② 词序不敏感（bag-of-words 倾向，"dog bites man" 和 "man bites dog" 难分）；③ 需要海量数据、大 batch；④ 只能做判别式检索，不能生成；⑤ 对分布外/罕见概念泛化差。

追问：怎么补? → 加 region-level 监督（GLIP/RegionCLIP）、加 captioning loss（BLIP）、交给能生成的多模态 LLM。

Q12 ★★ SigLIP 和 CLIP 的区别?

A：把 softmax 的**多分类交叉熵换成逐对 sigmoid 二分类（pairwise sigmoid loss）**。好处：每个图文对独立判断，不再需要全 batch 归一化，**因此 batch 可以小、也能做分布式异步**，训练更省内存；效果同级甚至更好，是现在很多开源 VLM（PaliGemma、Qwen-VL 系）的视觉塔选择。

Q13 ★ BLIP-2 的 Q-Former 是干什么的？为什么要有它?

A：Q-Former 是一个小 Transformer，内含**固定数量（32/64 个）可学习的 query**，通过 cross-attention 从冻结的图像编码器抽取信息，输出固定长度的视觉 token。

它解决两个问题：① **把可变长 patch token 压成固定长度**，让 LLM 的上下文可控；② **在冻结视觉塔和冻结 LLM 的前提下做桥接**，只训这一个轻量模块，成本极低。

两阶段训练：**阶段一表征学习**（ITC 图文对比 + ITM 匹配 + ITG 生成，用 query 学对齐）；**阶段二生成学习**（把 query 输出接到冻结 LLM，用语言建模损失训）。

追问：Q-Former 和 Perceiver Resampler 有什么区别？→ 同一类东西（固定 query + cross-attention），Flamingo 的 Perceiver 是纯重采样后用 gated cross-attention 注入，BLIP-2 的 Q-Former 还要多任务监督、对齐得更强。

Q14 ★★★ LLaVA 为什么用两层 MLP 就够了？这和 Q-Former 的取舍是什么？

A：LLaVA 的消融显示：**在冻结视觉塔 + 冻结 LLM 的设定下，简单线性/MLP 投影和 Q-Former 效果接近甚至更好**，而且更简单、训练更快。原因是视觉塔已经产出了足够好的语义特征，桥接层只需要做"维度对齐 + 轻量变换"，复杂的压缩反而损失信息。

取舍：MLP 保信息但 token 多（贵）；Q-Former 压 token 但要多阶段训练、实现复杂。**决定因素是你的 token 预算**：预算宽松（图少、上下文长）用 MLP；预算紧（视频、端侧）用 query 压缩。

LLaVA 两阶段：① 冻结 LLM 只训 projector（用图文对做"描述生成"式对齐）；② 解冻 LLM 做指令微调（LLaVA-1.5 约 558K 预训练 + 665K 指令数据）。

Q15 ★★ Flamingo 的结构有什么特别？

A：三件套——**冻结的视觉编码器**（NFNet）+ **Perceiver Resampler**（把任意数量视觉特征压成固定数量）+ **在冻结 LLM 里插入 gated cross-attention 层**（用 tanh 门控初始化 0，保证训练初期不破坏原 LLM 行为，这是很漂亮的设计）。它支持交错图文（多图 + 多轮）输入，是小样本多模态能力的代表。

Q16 ★ 为什么大家都冻结视觉塔？

A：① 视觉塔参数量大（ViT-L 约 300M），冻结省显存和算力；② 预训练视觉特征已经很通用，微调反而容易过拟合到指令数据的分布；③ 冻结后视觉侧可以离线预抽特征（或者用更小的学习率）。**什么时候该解冻**：目标任务和预训练分布差很远（医学影像、红外、深度图）、或者需要极细粒度定位时，常见做法是解冻最后几层 + 小学习率。

Q17 ★ 图文匹配里常见的三种 loss 是什么？

A：**ITC**（image-text contrastive，对比学习，学全局对齐）、**ITM**（image-text matching，二分类判断是否匹配，需要融合后的细粒度特征，常用 hard negative 挖掘）、**ITG**（image-grounded text generation，给定图像生成文本，用 causal mask 防止信息泄漏）。BLIP 就是三者联合，各自的 attention mask 不同——这是它设计上的巧思。

Q18 ★ 图文对数据从哪来？质量怎么保证？

A：来源：网页 alt-text 抓取（量最大、噪声也最大）、人工标注（小但准）、合成（用强模型给图生成 caption）、业务数据（你自己的场景）。质量控制三板斧：① **过滤**（短文本、非自然语言、重复、比例异常图、水印图）；② **重写 caption**（用强 VLM 重新描述，把 alt-text 的噪声换成结构化描述，这就是 LLaVA-1.5 / ShareGPT4V 的做法）；③ **去重**（图像近重复是评测泄漏的最大来源）。

你的接口：这块你极强——数万 个场景、百万级 个关键帧的真值生产线，口径核查、版本反转 bug、双 seed 噪声地板，这些方法论完全可以直接讲成"大规模多模态数据的生产线与质量治理"。

Q19 ★★ 如果图像和文本的 token 在序列里混排，怎么区分它们？

A：靠**特殊 token + 位置/模态标识**：用 `<image>` / `<|image_pad|>` 这类特殊占位 token 标出图像区间；位置编码上，文本用文本位置，视觉 token 用 2D（或 3D）位置，或者干脆共享一个序列位置（多数实现）。更细的做法是加**模态嵌入**（modality embedding）或**视觉 token 的类型标记**，让模型知道这段是"看来的"不是"读来的"。

Q20 ★★★ 多模态里的"对齐"到底是哪几层对齐？

A：三层，面试时按层说很清晰： 1. **表征对齐**（representation）：图像和文本映射到同一语义空间——CLIP 干这个。 2. **粒度对齐**（granularity/region）：把文本里的"猫"对应到图像里的某个区域——Q-Former、grounding 数据、RefCOCO 监督干这个。 3. **行为对齐**（behavior）：让模型按人类偏好/指令回答——SFT + RLHF/DPO 干这个。

你的接口："我做的视觉-激光融合是第一层的跨模态对齐（不同传感器到统一 BEV 空间），配合几何标定做第二层的空间对齐；多模态 LLM 的第三层行为对齐是我要补的。"

S3 多模态大模型架构与训练（12 问）

Q21 ★★★ 一个多模态 LLM 由哪几部分组成？

A：三件套 + 两处细节： 1. **视觉编码器**（ViT/SigLIP）——把图变成 token； 2. **模态桥接层**（projector / Q-Former / resampler）——维度与数量对齐； 3. **语言模型**（LLM）——统一在 token 序列上做自回归。

细节：图像/文本的**特殊占位 token** 与位置编码方案；训练时的**冻结/解冻策略**。数据流是 `image → ViT → 桥接 → 与文本 token 拼接 → LLM → 文本输出`。

Q22 ★★★ VLM 的训练分几个阶段？每阶段干什么？

A：三阶段，每阶段的"目的—数据—冻结策略"都不同：

阶段	目的	典型数据量	冻结策略	损失
① 预训练/对齐	让视觉特征能被 LLM 读懂	数十万~数百万图文对（caption 形式）	冻结 ViT + LLM，**只训 projector**	语言建模（生成 caption）
② 指令微调 SFT	学会按指令回答、多轮、拒答	数十万~百万条指令数据	解冻 LLM（或加 LoRA），ViT 常冻结	只在回答部分算 LM loss
③ 偏好对齐	对齐人类偏好、减少幻觉	数万~数十万偏好对	全参或 LoRA	DPO / PPO / GRPO

关键点（加分）：SFT 时**只对 assistant 回复算 loss，用户提问部分 mask 掉**；阶段 ① 用冻结 LLM 是为了不让随机初始化的 projector 把 LLM 带崩。

Q23 ★★ 指令数据怎么造？

A：四条来源：① 人工标注（质量最高、最贵，如 LLaVA 的 GPT-4 生成 + 人工筛）；② 强模型蒸馏（GPT-4V 生成对话，注意**只用它生成文本、不用它生成图像**）；③ 现有数据集转换（VQA、检测、OCR、grounding 数据改写成对话）；④ 合成负样本/拒答样本（教会模型说"图里没有"）。质量治理：去重、任务配比、难度分层、答案长度控制、**加入纯文本数据防遗忘**。

Q24 ★★ 冻结策略怎么选？

A：三种常见配置： - **只训 projector**（最省，适合对齐阶段、大量数据但算力紧）； - **训 projector + LLM（LoRA 或全参）**（主流，SFT 阶段）； - **全解冻含视觉塔**（数据量大、领域差异大时才做，学习率要小，否则破坏预训练特征）。

判断依据一句话：**数据量小就多冻结，领域差异大就多解冻**。

Q25 ★★★ LoRA 是什么？为什么能省显存？

A：冻结原权重 W，只学低秩增量 ΔW = B · A，其中 A∈R^{r×d}、B∈R^{d×r}，r 远小于 d，前向是 `h = Wx + (alpha/r)·BAx`。

省显存的原因：**只对 A/B 求梯度、只给它们存优化器状态**（Adam 要 2 份动量，这才是显存大头），可训练参数常降到 0.1%~1%。推理时可以把 BA 加回 W，**零额外延迟**。

参数建议：rank 8~64，alpha 常取 2r，作用于 q/k/v/o 和 MLP 的全部线性层（比只加 q/v 效果好）。追问 **QLoRA**：基座量化到 4bit（NF4）+ 双重量化 + paged optimizer，可在单卡训 65B。

Q26 ★★★ 训练大模型显存都花在哪？怎么省？

A：四块（按大小排，以参数量 P、字节数计）：**参数 2P（fp16）+ 梯度 2P + 优化器状态 12P（Adam fp32 的 m/v/主权重）+ 激活**。前三个是"静态的"，激活随 batch×序列长度线性增长。

省的手段：**ZeRO-1 切优化器状态 / ZeRO-2 再切梯度 / ZeRO-3 再切参数**；**梯度检查点**（重算激活，显存换 30% 时间）；**混合精度 + bf16**（比 fp16 稳，不用 loss scaling）；**梯度累积**（小 batch 模拟大 batch）；**序列并行/FlashAttention**（省激活里的注意力矩阵）；**offload**（CPU/NVMe，慢）。

你的接口："我在 8 卡训练集群 上把显存从 约 89GB 压到 约 82GB，方法是用 32×32 混合矩阵替代 3-tap 大核，保住 batch 和分辨率——本质是'算力不变但中间张量变小'的重构思路。"（**这条极其对口，面试官会顺着问下去**）

Q27 ★★ 训练不稳定 / loss 突然爆掉 / 出 nan，怎么排查?

A：按顺序查，别乱猜： 1. **数据侧**：label 越界、空样本、异常值（我遇到的真实案例：CE 的 target 越界，参数名 `cls_num` 与实际类别数不一致，loss 直接崩）； 2. **数值侧**：fp16 溢出 → 换 bf16 或加 loss scaling；attention 里 mask 全 −inf → nan；除零； 3. **学习率**：太大 → 加 warmup、降峰值；梯度裁剪（通常 1.0）； 4. **结构侧**：新加的层初始化不当（用恒等初始化/零门控）；残差尺度； 5. **硬件侧**：CUBLAS workspace 不足、OOM 报错伪装成算子错误（我真遇到过 `CUBLAS_STATUS_ALLOC_FAILED`，根因是超大 batch 的 GEMM workspace，改 depthwise+pool 解决）。

答法价值：这段回答的分数在"你怎么定位"，而不是"你修了什么"——按"数据→数值→超参→结构→硬件"的层次说，显得有方法论。

Q28 ★ 多图/多轮对话在序列里怎么组织?

A：图像 token 按出现顺序插在对话流里，用特殊 token 界定每张图的区间；多轮时**历史轮次保留（可能要压缩）**，图像通常只在第一次出现时插入，后续轮引用"上面那张图"。工程上的三个坑：① KV cache 随轮数增长；② 图像 token 太多导致历史被挤掉，需要"图像 token 压缩/丢弃旧图"策略；③ 位置编码要能区分"第几张图"。

Q29 ★★ 宽高比差异大的图（比如 1920×200 的长条票据）怎么处理?

A：固定 resize 会严重失真（OCR 直接废掉）。方案：**按比例切块**（切成若干接近正方形的小块 + 一张缩略图，InternVL/LLaVA-1.6 的做法）；或**pad 到固定比例**；或**原生分辨率 patch 化**（NaViT）。判断标准：下游是否依赖细节（OCR/小目标 → 必须保分辨率）。

Q30 ★★ 图文 token 比例怎么控制?

A：给一个经验量化：**一张 336×336 图约 576 个 token（patch14）或 256 个（patch16 + 池化）**；LLM 上下文 4096 时，一张图能吃掉 1/7。所以工程上要定"每图 token 预算"（如 256/576/1024），按任务分配——OCR/grounding 给多，闲聊给少。视频更极端：**必须做帧采样 + 时序池化**。

Q31 ★ 模态桥接层选 linear / MLP / Q-Former / Perceiver，怎么选?

A：一句话决策表：

桥接	token 是否压缩	训练成本	什么时候用
linear	不压	最低	极简基线
2 层 MLP	不压	低	**主流选择**（LLaVA 系），token 预算宽松
Q-Former	压到固定数	中（多阶段）	token 预算紧、要多任务对齐（BLIP-2）
Perceiver	压到固定数	中	交错图文、多图（Flamingo）

Q32 ★★ 混合模态训练时，模型"忘记"纯文本能力怎么办?

A：**数据里保留纯文本数据**（通常是图文数据的一定比例，经验值 10%~30%），并且**图文数据与纯文本数据同 batch 混训**；SFT 阶段尤其要注意，否则模型会退化成"只会描述图像"。评测时同时看纯文本 benchmark，别只看多模态分。

S4 幻觉、对齐与评测（12 问）

Q33 ★★★ 多模态大模型的"幻觉"是什么？分几类?

A：模型生成了图像中不存在的信息。两类分法（面试用第二种更专业）： 1. **按对象**：物体幻觉（说有不存在的物体）、属性幻觉（颜色/数量错）、关系幻觉（空间关系错）。 2. **按来源**：**语言先验主导型**（沿用训练语料里的常见搭配，无视图像）、**视觉信息缺失型**（图里看不清/没看到，编一个）、**对齐错误型**（看到了但映射错）。

Q34 ★★★ 幻觉的根因是什么?

A：一句话：**语言先验的强度压过了视觉证据**。具体四层：① 训练数据里图文对噪声大，模型学会"不看图也能猜对"；② 自回归目标鼓励"流畅"而非"忠实"；③ 视觉 token 少（分辨率低/被压缩），证据本身就弱；④ SFT 数据里缺少"我不知道/图里没有"的样本。这条根因和 POPE 里 "adversarial" 设置的设计动机直接相关。

Q35 ★★★ 怎么缓解幻觉?

A：按训练流程分四层： - **数据**：负样本（有图的/没有的）、拒答样本、"不同→对比"样本；提高图文对质量。 - **训练**：加辅助损失（如对比/grounding 监督，强制对齐区域）、视觉 token 加权、加大视觉分辨率。 - **解码**：降低温度、对比解码（VCD，用"加噪图"和"原图"的 logits 差放大视觉贡献）、惩罚高频先验词。 - **后处理/评测**：POPE/HallusionBench 定期体检；RAG 引入外部事实。

你的接口：可以类比讲"我在占用预测里做的 fn/fp 成分解剖（no_hit/wrong_cls/early/late）——把错误拆成分项，才知道该改模型还是改标签"。这套方法论迁移到幻觉上就是"把幻觉按类型统计，而不是只看一个总分"。

Q36 ★★ SFT 数据为什么要"拒绝采样"?

A：让强模型对同一问题采样多条回答，用奖励模型/规则/人工筛掉差的，只保留高质量的那条（best-of-n）。好处：把"生成能力"和"筛选能力"解耦——**用贵的模型筛，用便宜的模型学**。这也是蒸馏型 SFT 数据的常见做法。

Q37 ★★ RLHF 的完整流程?

A：三步：① **SFT** 得到初始策略；② **训练奖励模型 RM**（用人类偏好对 (chosen, rejected) 做 Bradley-Terry 二分类）；③ **用 PPO 优化策略**，奖励 = RM 打分 − β · KL(π ‖ π_ref)（KL 项防止跑偏太远）。多模态里 RM 要吃到图像，所以 RM 本身也是个 VLM。

Q38 ★★★ DPO 和 PPO 的区别？为什么 DPO 更流行?

A：DPO 把"RM + RL"合并成一个**分类式损失**：直接用偏好对 (y_w, y_l) 优化策略，隐含的奖励是 $\beta \cdot \log(\pi(y|x)/\pi_ref(y|x))$ 。

- **PPO**：要单独训 RM、要采样、要 4 个模型（policy/value/ref/RM），工程复杂、易不稳。
- **DPO**：不需要 RM、不需要在线采样，一个 loss、两个模型（policy + ref），稳定且便宜。
- 代价：DPO 是**离线**的，没有在线探索，理论上限低于 PPO/在线方法；数据分布外表现差。

追问：GRPO 是什么？→ DeepSeek 系的做法：去掉 value model，对同一个 prompt 采样一组（group）输出，用组内相对优势做基线，省掉 critic 网络，等价于"组内归一化的策略梯度"。

Q39 ★★★ 多模态评测集都测什么？各举一个必背的。

评测集	测什么	一句必背
MMBench	综合能力（20 类能力维度）	用 CircularEval 消除选项顺序偏差
MMMU	大学水平学科推理（图表、公式、考试题）	多学科、多图、需要专业知识
SEED-Bench	生成式理解（图像+视频）	多项选择 + 人工校验
POPE	**物体幻觉**	用"是否存在某物体"的 yes/no 探测幻觉
HallusionBench	幻觉（视觉依赖 + 反事实）	专测"看图说话 vs 靠先验编"
MM-Vet	综合能力（识别+推理+OCR 混合）	用 GPT-4 打分，能力维度交叉
RefCOCO/RefCOCO+/gRefCOCO	指代定位 grounding	给文本描述输出框，准确率/IoU
VideoMME / LongVideoBench	视频理解（短/中/长，有无字幕）	长视频显式考察时序与检索
TextVQA / DocVQA / ChartQA	OCR 与文档/图表理解	工业落地最常用

Q40 ★★ POPE 的三个设置是什么？为什么这么设计？

A：**random**（随机选不存在物体）、**popular**（选数据集中高频但图里没有的物体）、**adversarial**（选和图中物体共现频率高、最容易骗过模型的物体）。指标是 Accuracy / Precision / Recall / **F1** 和 yes 比例。设计动机：随机负样本太简单，会上报虚高分数，**adversarial 才是真考验"是否真在看图"**。

Q41 ★★ MMBench 的 CircularEval 是什么？为什么需要？

A：把一道多选题的选项**循环移位多次**，每次让模型回答，只有全部正确才算对。因为 VLM 对选项位置有强烈偏好（比如总选 A/B），不移位会高估能力。这是"评测口径"的经典例子——**面试时讲这个，比背指标更能证明你读过评测细节**。

Q42 ★★ 评测里常见的坑有哪些？

A：五个：① **数据泄漏**（训练集和评测集图像近重复，尤其 CLIP 系预训练数据几乎覆盖全网）；② **prompt 敏感**（换个模板差 5 个点）；③ **选项偏差**（要 CircularEval / 打乱）；④ **指标口径不一致**（BLEU/CIDEr 对生成式评测不可靠，所以现代评测多用 GPT 打分或人工，但 GPT 打分本身有偏差）；⑤ **只报总分不报分项**（真实能力要看分项，比如 OCR 强但推理弱）。

Q43 ★★ grounding 任务怎么评测？

A：输入"图中的白色小狗"，模型输出框或 mask，和 GT 比 **IoU / 点命中**。指标：RefCOCO 用 Acc@0.5（IoU>0.5 的比例），也常用**点级准确率**（输出点落在 GT mask 内的比例，更宽容）。难点：**指代歧义**（多只同类物体）、**否定/关系描述**（"不在桌上的那个"）。

Q44 ★ 为什么要关心"纯文本能力不退化"？

A：多模态模型最终是个 LLM，用户会问纯文本问题。若 SFT 只喂图文数据，模型的语言能力、指令遵循、长文组织都会退化（catastrophic forgetting）。做法：混入纯文本指令数据、评测时同时跑文本 benchmark、必要时用 LoRA 限制参数漂移。

S5 视频理解与生成（10 问）

Q45 ★★★ 视频的 token 数量为什么是灾难？怎么压？

A：算一下就明白：一帧 224×224/patch16 = 196 token，**32 帧就是 6,272 token**，还没算文本；1 分钟 30fps 有 1800 帧 → 35 万 token。**必须压**。

四条路：① **帧采样**（均匀/关键帧/内容自适应，常见 8~64 帧）；② **空间压缩**（每帧先池化/pixel shuffle）；③ **时间压缩**（相邻帧特征做时序池化、3D VAE、token merge）；④ **分层+检索**（长视频先摘要再检索片段，LongVA/Video-LLaVA 那类）。工程口诀：**"先降帧率，再降分辨率，最后压在时序维上"**。

Q46 ★★ 视频里的时序信息怎么建模？

A：三种主流：① **逐帧独立编码 + 序列拼接**（最简单，靠 LLM 自己学时序，需要时序位置编码区分帧）；② **时空联合 patch**（3D ViT / VideoMAE，把 t 维也切 patch）；③ **时空聚合模块**（在视觉塔里做帧间 attention 或 pooling，输出固定帧数）。位置编码上常用 **3D RoPE（时间+高+宽）**，Qwen2-VL 就是 M-RoPE。

Q47 ★★ 长视频（1 小时以上）怎么做？

A：核心思想是**不要把所有帧塞进上下文**，而是"摘要 + 检索 + 分层"： 1. 关键帧/场景切分（用变化检测或聚类）； 2. 每段单独编码成紧凑描述（caption/embedding）； 3. 提问时先检索相关段（embedding 检索或轻量模型打分），只把相关段的细节 token 送进 LLM； 4. 需要全局理解时用分层摘要（段→章）。

你的接口：你的 scene-aligned 时序切分（避免跨场景污染）+ 时序历史缓存控制，和这个"分段+检索"思路同构，可以顺势讲。

Q48 ★★ 视频理解评测看什么？

A：**VideoMME**（短/中/长视频 × 有无字幕，显式暴露长视频退化）、**LongVideoBench**（长上下文）、**MVBench**（时序动作）、**EgoSchema**（第一视角长视频）。关键不是分数而是**分档看**：短视频好、长视频崩，是普遍的现状——所以要报"分时长档"的结果。

Q49 ★★ 扩散模型的前向和反向过程各是什么?

A：- **前向**（加噪，无参数）：$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon$，T 步（典型 1000 步）把数据变成纯噪声，马尔可夫链。- **反向**（去噪，学网络）：训练一个网络 ε_θ(x_t, t) 预测噪声，损失是 $||\varepsilon - \varepsilon_\theta(x_t, t)||^2$（也可以是预测 x_0 或 v）。采样时从纯噪声逐步去噪。

一句话本质：**扩散模型 = 学一个"从噪声回到数据"的分数函数（score）**。

Q50 ★★ DDPM 和 DDIM 的区别?

A：DDPM 的反向是**随机**马尔可夫过程（每步加噪），要 1000 步；DDIM 构造一个**确定性**（或可控随机）的非马尔可夫采样过程，**可以用 20~50 步得到近似结果**，且同一噪声种子结果可复现。后来的 DPM-Solver、uniPC 把它做成 ODE 求解器，10 步左右就够。**工程意义：采样步数是生图/生视频的延迟主因**。

Q51 ★★★ CFG（Classifier-Free Guidance）是什么?

A：训练时把条件（文本）以一定概率置空，让同一个网络同时学"有条件"和"无条件"；采样时把两者外推：

```
ε = ε_uncond + s · (ε_cond − ε_uncond)
```

`s` 就是 guidance scale（常用 5~9）。**s 大 → 更贴合文本但多样性下降、易过饱和；s=1 → 等于纯条件采样。**

追问：为什么叫 classifier-free? → 早期用单独训的分类器梯度做引导（Classifier Guidance），后来改成同一个网络的条件/无条件差，不需要分类器。

Q52 ★★ Latent Diffusion（Stable Diffusion）的关键设计?

A：不在像素空间扩散，而是先用 VAE 把图压到**低维 latent**（如 512×512×3 → 64×64×4，压缩 48 倍），在 latent 上做扩散，最后 VAE 解码回像素。好处：算力下降一个量级以上，让消费级卡能训。代价：VAE 是瓶颈（细节/文字渲染差，所以后来的 SD3/Flux 改 VAE 或换架构）。条件注入用 **cross-attention**（文本 token 作为 K/V）——**这里和 LLM 的多模态融合是同一个机制，可以顺势讲**。

Q53 ★ DiT 是什么？统一理解与生成为什么难?

A：DiT 把扩散的 U-Net 换成 **Transformer**（patch 化 + adaLN 注入时间/条件），scaling 行为更好，是 SD3/Flux/Sora 系的基础。统一模型的难点：**理解要判别式语义特征，生成要像素级重建细节**，两者目标冲突；路线有"共享 Transformer + 不同 head"（Transfusion、Emu3 用离散 token 统一）、"生成用扩散头、理解用 LM 头"。目前没有公认最优解——**答"这是开放问题"比硬编答案好**。

Q54 ★★ 视频生成比图像生成难在哪?

A：四条：① **时序一致性**（闪烁、形变、物体消失）；② **数据**（高质量视频+文本对远少于图像）；③ **算力/显存**（3D VAE + 时空注意力，长度线性涨）；④ **运动可控性**（镜头运动 vs 物体运动要分开控制，常用轨迹/光流/相机参数条件）。工程上还有"首尾帧控制、参考图一致性"这类产品化需求。

S6 推理部署与工程（10 问）

Q55 ★★★ 推理时 KV cache 怎么算？为什么它是瓶颈?

A：每层每个 token 要缓存 K 和 V：

```
显存 ≈ 2 × n_layers × n_kv_heads × d_head × seq_len × batch × 精度字节
```

例：32 层、32 头、d_head=128、fp16、seq 2048、batch 1 → 2×32×32×128×2048×2 ≈ **1.07 GB**，batch 和长度都线性叠上去。

它是瓶颈的原因：**decode 阶段每一步都要读一遍整个 cache**，是**访存密集**而不是计算密集，所以长上下文时 GPU 利用率很低。解法：GQA/MQA（`n_kv_heads` 变小）、KV 量化（INT8/INT4）、PagedAttention（消除碎片）、滑动窗口/丢弃远距离、prefix cache 复用。

Q56 ★★★ Prefill 和 Decode 两个阶段的差异?

A:

	Prefill（读 prompt）	**Decode（逐 token 生成）**
并行度	整段并行，**计算密集**（大 GEMM）	每步 1 个 token，**访存密集**
瓶颈	算力 / FLOPS	HBM 带宽（要读全部权重+KV cache）
优化	FlashAttention、大 batch、融合算子	KV cache 压缩、投机解码、连续批处理、量化

面试价值："端侧延迟主要花在 prefill（图 token 多）还是 decode（输出长）要分开看"——这句话能立刻显出你有实测概念。**你的真实数据**：MDC610 上做算子解耦+向量化把误检从 约 5% 降到约 1.4%，可以顺势讲"我做过 prefill 侧（视觉塔+投影）的算子级优化"。

Q57 ★★ PagedAttention / vLLM 解决什么问题?

A：传统实现给每个请求预分配连续 KV 显存，长度未知导致**大量碎片和浪费（实测浪费可到 60%+）**。PagedAttention 把 KV cache 切成固定大小的 block，用页表映射，**按需分配、可共享（同一 prompt 的多个采样共享前缀）**，配合 **continuous batching**（请求完成就换新请求进 batch，不等最慢的）把吞吐提升数倍。

Q58 ★★ 投机解码（speculative decoding）是什么?

A：用一个小模型（或 n-gram/检索）先草拟 k 个 token，大模型**并行验证**这 k 个，接受前缀匹配的部分、拒绝第一个不匹配处。因为验证是并行的（一次 forward 出 k 个位置的 logits），**在不改变输出分布的前提下加速**（严格等价，这是它的理论保证）。典型加速 1.5~3×，取决于草稿命中率。

Q59 ★★ 量化怎么选? INT8 / INT4 各有什么代价?

A：权重 INT8 基本无损（W8A8/W8A16 常用）；**INT4 权重会出现明显精度损失**，需要算法补偿：**GPTQ**（逐层量化 + Hessian 补偿）、**AWQ**（保护重要通道，激活感知）、**SmoothQuant**（把激活的难度迁到权重上）。还有 KV cache 量化（长上下文更划算）。判断标准：先看任务对长尾/数值敏感的程度（OCR、数学推理对量化更敏感）。

Q60 ★★ 蒸馏在多模态里怎么做?

A：三条路：① **logit 蒸馏**（学生对齐教师的输出分布，加温度 T）；② **特征蒸馏**（对齐中间层/token 表示，多模态常用"对齐教师看同一张图后的视觉 embedding"）；③ **数据蒸馏**（用教师生成的问答对训学生，最常用、最省事）。端侧落地常见组合：**小视觉塔 + 小 LLM + 教师蒸馏数据 + 量化**。

Q61 ★★ 端侧多模态模型最难的三件事?

A： 1. **分辨率 vs 算力**：端侧要看清（OCR/小目标）就得高分辨率，而分辨率直接变成 token 和算力——这是根本矛盾。 2. **显存/内存**：手机/车机的可用内存是 GB 级，KV cache + 权重 + 激活都要算进去。 3. **算子与后端**：不是所有算子都能落到 NPU/GPU（attention 的动态形状、动态分辨率、nms/topk），要**改写算子或换等价语义**。

你的接口（这是你最强的差异化）："我在 MDC610 上做过完整的 ONNX→ATC/AOE→ `.om` 链路，对不支持算子做了解耦 + OpenCV 向量化重写，端侧误检 约 5%→约 1.4%。所以端侧多模态我也会先问三个问题：**目标 SoC 的算子白名单、内存预算、以及是否允许动态分辨率**。"

Q62 ★★★ 昇腾（Ascend）生态你了解多少? ——送分题，必须答满

A：分层说： - **硬件**：Ascend 310/910B（推理/训练），达芬奇架构（Cube 矩阵单元 + Vector 单元）。 - **软件栈**：**CANN**（异构计算架构，对标 CUDA）→ **ATC**（模型转换：ONNX/PB → `.om` 离线模型）→ **AOE**（自动调优，选 tiling/算子实现）→ **AscendCL**（推理 API）。 - **框架**：**MindSpore**（自研）/ PyTorch 通过 torch_npu 适配。 - **工程坑**：算子不支持要拆或替换；动态 shape 支持有限，常需分档固定 shape；精度对齐要做（fp16 溢出、算子融合顺序差异）；内存要手工规划（模型/工作空间）。

话术："我做过完整的 ONNX(opset11/16)→ATC/AOE→MDC610 `.om` 落地，对 AscendCL 的输入输出 binding、内存对齐、算子不支持时的等价改写都有实操。"→ **这是华为面试里最能加分的一段，因为大部分算法候选人只会写 train.py。**

Q63 ★ 端侧时延怎么拆？怎么定位瓶颈？

A：拆成四段并分别打点：① **预处理**（resize/归一化/拼接，CPU 上很贵，别忽略）→ ② **视觉塔**（ViT 前向，可用 batch/半精度）→ ③ **projector + prefill**（LLM 一次前向处理所有图文 token）→ ④ **decode**（每 token 一次，按输出长度线性）。

定位方法：先测各段独立耗时 + 用 nsys/推理框架的 profile 看算子占比；常见结论是 **prefill 被图像 token 拖累、decode 被 KV cache 带宽拖累、预处理被 Python/CPU 拷贝拖累（用 pinnedMemory/零拷贝可省几十毫秒）**。

Q64 ★ 多模态推理服务的吞吐怎么提？

A：五个手段：① **统一 batch 内的 shape**（动态分辨率会让 batch 里 shape 不一，要么 padding 要么分桶）；② **连续批处理**（不等最长序列）；③ **prefix/图像特征缓存**（同一张图多轮问答，视觉塔的结果缓存重用，这是多模态特有的收益）；④ **量化 + 投机解码**；⑤ **视觉塔与 LLM 异构并行**（视觉塔放另一张卡/另一个流，与 decode 重叠）。

S7 把你的项目接进多模态叙事 + 临场策略（10 问）

Q65 ★★★ 「你做的是感知，凭什么做多模态大模型？」怎么答？

A：三段式，**不要说"我可以学"**： 1. **做过的同构问题**："我做的视觉+激光融合本质是跨模态对齐：两种异构传感器（图像是稠密透视、激光是稀疏度量）在统一 BEV 网格上做门控融合，处理的核心难点和 VLM 里的图文对齐同源——**都是把不同分布、不同粒度的表征映射到一个可比较的空间**。" 2. **可迁移的工程能力**："大规模训练调优（8 卡训练集群、显存墙下的结构重构、loss 崩溃的三连排查）、数据生产线与评测口径（25 万场景/百万关键帧、双 seed 噪声地板）、端侧部署（ONNX→ATC→MDC610）。这三件事恰好是 VLM 落地最缺的能力——**模型结构论文里都有，但数据、训练稳定性、部署是真功夫**。" 3. **明确承认缺口 + 已做的补课**："VLM 的 SFT/偏好对齐我没在生产里做过，所以我按 LLaVA 两阶段在自己的数据上复现过、也读了 Qwen-VL/InternVL 的报告，重点看它们的 token 压缩和数据配比。"

Q66 ★★★ 你的视觉-激光融合 vs 多模态的 cross-attention，有什么关系？

A："我的融合是**结构化的跨模态交互**：两路特征在 BEV 网格上对齐后，用 SE 通道门控做重加权，再由卷积融合。VLM 里的 cross-attention 是**用注意力动态决定'从另一模态取什么'**，表达力更强但计算更贵（$O(N^2)$）。

如果让我改造，我会把静态门控换成 **query-based cross-attention**：BEV query 作 Q、图像特征作 K/V，这样融合权重是按位置和内容动态生成的——**这正好是 BEVFormer 和 Flamingo/Q-Former 的共同思路**。"

（**这段是全场最高分的回答，把两个领域用同一个机制连起来了**）

Q67 ★★ 你的 BEV query 和 Q-Former 的 query 是一回事吗？

A："形式上同类：都是**固定数量、可学习、通过 cross-attention 从高冗余输入里抽信息**。差别在约束：BEV query 有**强几何先验**（每个 query 对应一个固定的世界坐标格子，靠相机内外参做投影），所以 token 的位置是"物理正确"的；Q-Former 的 query 没有几何约束，纯靠数据学。

前者适合需要**度量精度**的任务（占用、检测），后者适合**语义检索**（"图里有什么"）。如果要给 VLM 加空间精度，用带几何先验的 query（3D grounding、SpatialVLM 一类工作）是自然方向。"

Q68 ★★ 你的时序建模经验怎么迁移到视频多模态？

A："我做过两件相关的事：按 scene 对齐切时序段（避免跨场景污染）、维护时序历史缓存（在 mIoU 与显存之间做权衡）。换成视频理解，对应的问题是**帧采样策略**和**历史 token 预算**：帧采样的关键是'别把同一场景的冗余帧重复喂进去'，历史 token 的关键是'压缩方式要与任务相关'——我倾向于先做场景切分 + 关键帧，再做时序池化，这和我在 OCC 上的做法同构。"

Q69 ★★ 数据生产线经验怎么讲成 VLM 的加分项？

A："多模态模型的上限基本由数据决定，而我在'数据-评测-归因'这条链上有完整实操：① **大规模生产**（数万个场景 / 百万级关键帧，含三态标注）；② **口径治理**（发现过转换器两个方向相反的 filter 版本导致 mask 体积只有 约 0.64 倍、mIoU 降约 3 个点 的隐性 bug，用逐元素对比 + IoU=1.0 锁定）；③ **评测设计**（万级帧协议 + 双 seed 噪声地板，把'模型变好'和'评测抖动'分开）。

迁到 VLM：数据配比、去重、caption 重写、幻觉负样本构造、以及'分项评测而不是只看总分'，方法论完全一致。"

Q70 ★★ 端侧部署经验怎么讲?

A：见 Q61/Q62。核心三句：**ONNX→ATC/AOE→ `.om` 全链路实操过；不支持算子做等价改写（Decode/TopK/NMS 解耦 + OpenCV 向量化）；有实测指标（ROI 超分误检 约 5%→约 1.4%）。**

Q71 ★ 具身智能 / VLA / 空间智能你了解吗?

A：一句话版：**VLA = 视觉语言模型 + 动作输出头**（RT-2、OpenVLA、π0 这类），把"看图 + 听指令 → 输出动作 token/轨迹"。关键难点：动作数据的稀缺与异构（不同机器人本体）、实时性（要 10~100Hz 控制频率）、空间精度（需要 3D 表征而不是 2D token）。

你的优势：**你是所有候选人里少数真做过 3D 几何、占有栅格、标定的人**——"具身落地最缺的就是 3D 空间理解与传感器标定，这两块是我的强项"。这句话会让面试官记住你。

Q72 ★★★ 遇到完全不会的问题怎么答？（临场策略，价值极高）

A：三步，别硬编： 1. **确认范围**："您问的是训练侧还是推理侧？"（把开放问题变窄，争取思考时间）； 2. **给出你确定的相邻知识**："这块我没在生产里做过，但我知道它的相邻问题是这样解决的……"（展示知识网而不是空白）； 3. **给出验证路径**："如果让我做，我会先在小规模上做一个对照实验，用 XX 指标卡住，再放大。"（展示方法论）。

绝对不要：瞎编数字、把"听说过"说成"做过"、被追问就沉默。**面试官容忍"不会"，不容忍"编"。**

Q73 ★★ 反问环节问什么?

A：三个显示你在想落地的问题： 1. 团队的多模态模型主要做哪个模态组合（图文/视频/点云-图），当前瓶颈在数据、结构还是推理成本？ 2. 端侧和云侧的比例大概多少？时延/算力预算是多少，昇腾侧是自研 ATC 链路还是 MindSpore 栈？ 3. 新人前三个月最可能被派到哪一块（数据管线 / 训练 / 评测 / 部署）？

Q74 ★★★ 红线清单（宁可少说，不可说错）

1. **不说自己训过 LLM/多模态大模型**。说"做过同构的跨模态对齐与大规模训练调优，VLM 侧在系统补课"。
2. **不报参数量**（百万级参数量 这种会被"这也叫大模型"直接打穿）。改说结构设计意图与单变量控制。
3. **不说论文已发表**。
4. **不说做过 SLAM**（只说视觉几何、位姿对齐、ICP、4-DOF 同位变换）。
5. **不跨协议比指标**。被问延迟/精度，先报口径（帧数、平台、是否含预处理）再报数。
6. **不把"读过论文"说成"复现过"**。可以说"读过并做过最小复现"。
7. **不确定的数字给量级**（"约 4 亿图文对"而不是"400,132,000 对"）。

S8 多模态融合的调控（雷达 / 激光 + 图像）

> 这一章是**技术专章**：把"融合"从一句口号拆成**六个可以分别调、分别验证的旋钮**。面试官问"你们怎么融合的"，能按这六个旋钮回答，就已经超过 90% 的候选人。

融合调控的六个旋钮（总览）

旋钮	要决定什么	出问题时的症状
① **在哪融**（层级/位置）	数据级 / 特征级 / 决策级；backbone 前 / 中 / 后；图像平面 or BEV 空间	融合后反而比单模态差
② **拿什么融**（表征）	raw point / BEV 特征 / 体素 / object proposal / occupancy	信息被稀释，或两路高度冗余
③ **怎么配权**（权重）	静态可学习权重 / 动态门控 / 不确定性加权 / 可靠性路由	一路主导，另一路"形同虚设"
④ **怎么对齐**（坐标与时间）	标定精度 / 时间戳选择 / 自车运动补偿 / 尺度匹配	融合出现鬼影、错位、拖影
⑤ **怎么防退化**（训练策略）	模态 dropout / 失效增广 / 融合层 warmup / 梯度平衡	雨天、夜间、镜头污损就崩
⑥ **怎么证明**（评测归因）	单模态基线 / 模态消融 / 门控可视化 / 逐类逐距归因	说不清融合到底贡献了什么

一句话心法：融合不是"把两个特征拼起来"，而是**先回答"这两个模态在什么空间、以什么权重、在什么条件下可信"三个问题**。

Q75 ★★★ 融合有哪几个层级？各自适合什么任务？

A：三层 + 一个混合式：

层级	做法	优点	缺点
数据级（早期）	点云投影到图像取 ROI，或图像反投影成点带颜色	简单、直观、信息最原始	强依赖标定精度；破坏点云几何结构；投影后稀疏
特征级（中期，**主流**）	两路各自编码成 BEV 特征 / 体素特征，在统一网格上融合	保住几何、可端到端、表达力强	需要空间对齐；计算量最大
决策级（晚期）	各自出检测框，再做关联（匈牙利匹配）与置信度融合	鲁棒、可解释、模块可替换	丢细粒度信息，早融合的增益拿不到
混合式	训练时用单模态辅助头做蒸馏，推理只走融合	兼顾收敛与精度	实现复杂

面试答法："我们走的是特征级、在 BEV 空间融合——因为感知任务要的是**度量精度**，图像平面上的融合丢了深度和尺度；决策级虽然鲁棒但拿不到'用激光的几何去校正视觉的语义'这种细粒度收益。"

追问：为什么 BEV 是融合的"公共语言"？→ 因为它是唯一同时满足"有度量、可向各传感器投影、能承载时序"的空间。相机靠深度估计投上去，激光/雷达靠标定投上去。

Q76 ★★★ 相机、激光雷达、毫米波雷达三者的特性怎么互补？

A：面试必备表格：

维度	相机	激光雷达（LiDAR）	毫米波雷达（Radar）
数据类型	稠密透视图像	稠密 3D 点（万~十万级）	极稀疏 3D+速度（千~万级）
语义/纹理	**最强**（颜色、文字、细分类）	弱（反射强度）	几乎无
几何精度	间接（靠深度估计）	**最强**（cm 级测距）	中等（角分辨率差，距离准）
速度测量	需多帧估计	需多帧估计	**直接测速**（多普勒）
恶劣天气	差（雨雾、逆光、夜间）	差（雨雾散射、衰减）	**最强**（穿透雨雾尘）
成本	低	高（机械式尤甚）	**最低**
高度信息	有（透视）	有	传统雷达弱（4D 成像雷达才补上俯仰）

一句话总结："相机给语义、激光给几何、雷达给全天候和速度——**融合的本质是让某一路的短板刚好被另一路覆盖**。"

追问：既然雷达全天候，为什么不只用雷达？→ 稀疏 + 角分辨率低 + 无类别语义，必须靠视觉补语义、靠激光补几何。

Q77 ★★ 毫米波雷达-图像融合 和 激光雷达-图像融合，做法有什么区别？

A：差别来自**数据密度和信息维度**：

	激光-图像	毫米波雷达-图像
点密度	万~十万级	千~万级（4D 成像雷达可达数万）
融合前提	可体素化/BEV 稠密化后卷积	**不能直接稠密化**（太稀疏，卷积核被空洞稀释）
主流做法	BEV 特征 concat + 门控 / cross-attention	**pillar 化 + 稀疏卷积**，或**点级 query**，或**先多帧累积再加稠密**
独有信息	高度、精确轮廓	**多普勒速度通道**（单独当一路特征）、RCS 强度
时序依赖	中等	**强**（单帧点太少，几乎必须累积多帧）
对齐难点	标定精度、稀疏投影空洞	**角分辨率差**（远处方位误差大）、多径杂波

关键差异一句话："激光可以'先稠密化再卷积'，雷达不行——雷达必须**先累积时序、再用稀疏算子**，否则融合模块学到的主要是空洞。"

追问：多普勒速度怎么用？→ 三种用法：① 作为额外通道拼进点特征；② 用来做**静止点剔除/自车速度估计**（点云配准的前置）；③ 作为独立的一路"速度图"（BEV 上按距离环统计），和图像的车道/语义特征交叉验证。

Q78 ★★★ 特征级融合有哪四种典型结构？各自代价？

A：

1. **Concat + 卷积**（基线）：`f = Conv([f_img ; f_lidar])`。最便宜，但**没有配权能力**，容易一路主导。
2. **门控/注意力加权**：`f = g⊙f_img + (1−g)⊙f_lidar`，`g` 由 sigmoid 产生（SE / CBAM / per-pixel gate / FiLM）。**性价比最高**，工业界主流。
3. **Cross-attention**：一个模态作 Q、另一个作 K/V，做非对称的"查询—取用"。表达力最强，代价 $O(N^2)$，需要 mask（视锥裁切）。
4. **MoE / 混合专家**：多个融合专家 + 路由网络，按场景（高速/城区/地库）分配。参数量涨、路由可能退化（负载不均）。

选择口诀：**算力紧、模态稳定 → 门控；需要细粒度互查 → attention；场景类型差异大 → MoE；先有 concat 基线再往上加**。

Q79 ★★★ 门控具体怎么设计？（这是"调控融合"最核心的一块）

A：四种门控，从粗到细：

```
① SE（通道级）：  z = GAP(f) → FC → FC → σ → 乘回每个通道
② CBAM（通道+空间）：SE 之后接一个 7×7 卷积出空间注意力图
③ per-pixel gate（逐位置）：g = σ(Conv1x1(f))，f = g⊙f_a + (1−g)⊙f_b
④ FiLM（条件调制）：用一路特征生成 γ,β，去调制另一路：  f_a' = γ⊙f_a + β
```

三个必须讲的实现细节：

- **门控初始化**：把最后卷积的 bias 初始化成较大正值（如 4.0），使 sigmoid ≈ 1，融合层初始化时**近似恒等**，不破坏预训练分支。这就是 Flamingo 的 gated cross-attention 思想（用 `tanh` 门控 + 零初始化）。
- **别让门控退化成常数**：如果门控对几乎所有输入都输出 0.5，等于静态加权。检查手段：打印门控值的直方图与空间分布。
- **跨模态归一化**：两路特征的均值方差往往差一个量级（激光强度分布 vs 图像 BN 后分布），直接 concat 会让一路吃满梯度。做法：**每路一个 modality-specific BN，融合前再统一 LayerNorm**。这一步经常比换融合结构更有效。

追问：为什么 per-pixel gate 比 SE 好？→ SE 只有通道维度，无法表达"近处信激光、远处信相机"这种空间依赖；per-pixel 加空间维，代价只是几个 1×1 卷积。

Q80 ★★ 用 cross-attention 做融合时，Q/K/V 怎么分配？为什么？

A：**Q 来自"目标空间"，K/V 来自"信息源"**。三种分配：

配置	Q	K/V	用途
BEV query ← 图像	BEV 网格（固定数量、有几何坐标）	多视角图像特征	主流：几何可解释、token 数可控（BEVFormer 式）
图像 query ← 雷达/激光	图像 patch	点云/雷达 BEV 特征	图像侧增强（加深度/遮挡信息）
双向	互相	互相	co-attention，表达力强但双倍代价

为什么 BEV query 作 Q 更好：① query 数与分辨率解耦，可压缩到预算内；② 每个 query 有明确世界坐标，**位置编码可以直接用 3D 坐标**（不是学出来的）；③ 视锥 mask 天然可算——把不在任何相机视锥内的 query 屏蔽掉，避免"用不存在的观测去猜"。

追问：mask 怎么做？→ 用内外参把每个 BEV query 投到每个相机，判断是否落在图像有效区域内且深度为正；对不可见的 (query, camera) 对屏蔽。

Q81 ★★ 稀疏模态（雷达/点云）怎么进融合网络?

A：四条路，按数据密度选：

1. **体素/pillar 化 + 稀疏卷积**：只在实际有点的体素上计算，空洞不参与。适合激光（万级点）和 4D 雷达。
2. **稠密化（Pixor 式）**：把 z 维折进通道，压成 BEV 图。简单，但点少时 99% 是零，卷积核被稀释。
3. **点级 query**：把每个点当 token 做 attention（Point Transformer 式）。表达力强，点数多时贵。
4. **先时序累积再稠密化**：雷达专用——累积 5~10 帧把点密度提上来，再进 BEV 卷积。

关键工程点：如果坚持稠密化，**必须加"占用 mask + 归一化"**——只在有观测的位置计算卷积响应，否则学出来的是"哪里有空洞"而不是"哪里有目标"。

Q82 ★★ 什么时候用"轻量调制"（FiLM/MoE/标量权重）而不是 attention?

A：一个判断表：

情况	推荐	理由
模态可靠性稳定、算力紧	门控 / FiLM	参数少、几乎不加延迟
两路特征语义已对齐，只需"调强弱"	标量或通道权重	attention 属于杀鸡用牛刀
需要"按位置取信息"（如遮挡区）	per-pixel gate / attention	空间级交互必须有权重图
场景类型差异极大	MoE	路由能按场景切换专家
端侧部署	**FiLM / 门控**	attention 的动态 shape 在 NPU 上难落地

端侧那句话（对你特别有用）："端侧我优先选能静态展开的融合（门控/FiLM），因为 attention 的动态 shape 和 gather 在 NPU 上要么不支持、要么要拆算子。"

Q83 ★★★ 模态权重怎么调控？四类方法怎么选?

A：四类，从"不学到强先验"：

1. **静态可学习标量**：`f = w₁ · f_a + w₂ · f_b`，`w` 经 softmax 归一化。最简单，但全场景一个权重。
2. **动态门控（内容相关）**：`w = σ(MLP([f_a, f_b]))`，逐位置/逐通道。主流。
3. **不确定性加权**：每个模态学一个观测噪声 σ，权重自动变成 $1/(2\sigma^2)$（详见 Q84）。适合"某模态在某条件下整体不可靠"。
4. **可靠性路由（外部信号驱动）**：用**非学习信号**调权重——能见度/雨量、雷达点云密度与 RCS 分布、图像清晰度（拉普拉斯方差）、激光回波强度统计。规则可写死，也可作为门控的额外输入。

选型：`动态门控` 打底（解决空间/内容差异）+ `可靠性路由` 兜底（解决天气/失效这类学习难以泛化的情况）。**两者叠加是工业界常见组合**。

追问：为什么不能只靠学习？→ 雨雾、镜头污损这类分布外条件在训练集里占比低，模型学不到"该降权"，必须用先验规则兜住。

Q84 ★★★ 不确定性加权怎么推导?

A：Kendall 多任务不确定性加权，假设每个模态的预测服从高斯，噪声为 σ：

```
L = Σ_i [ (1/2σ_i²) · L_i + log σ_i ]
```

- `L_i` 是第 i 个模态（或任务）的损失；`log σ_i` 是正则项，防止 σ 无限增大（否则模型靠"说自己不准"来逃避损失）。
- 直观理解：**σ 大 → 该模态权重大幅下降**；但 `log σ` 罚它不能太躺平。
- 实现：把 $\log \sigma^2$ 作为可学习参数（**不是 σ 本身**，避免除零和负值），用 $\exp(-\log \sigma^2) \cdot L + \log \sigma^2$ 计算。
- 坑：① 需要对 $\log \sigma^2$ 做 clamp（如 [-6, 6]）；② 初始化 0；③ 与门控不同——它是**全局标量**，表达不了"近处信激光、远处信相机"。

一句话区分：**不确定性加权调"整体信谁"，门控调"每个位置信谁"**。两者可以叠：门控输出再乘上不确定性权重。

Q85 ★★ 退化/失效感知的融合怎么做？（雨天、夜间、镜头污损、外参漂移）

A：分四类信号 + 四类响应：

失效类型	可检测信号	响应策略
光照/能见度差	图像平均亮度、对比度、梯度能量	降图像权重、升雷达权重（规则门控）
雨雾对激光衰减	点数骤降、远距离回波缺失、强度分布右移	降激光权重、缩短有效距离、改用近场几何
镜头污损/遮挡	局部拉普拉斯方差低 + 固定位置模糊块	屏蔽该相机、靠其他视角补
外参漂移	跨模态投影残差变大、对齐一致性下降	在线标定（路面/杆状物拟合）、降权该视角

做法要点：把上述信号做成一个**小的可靠性向量**（4~6 维），拼进融合门控的输入——这样"学习 + 规则"结合，既有泛化又能覆盖长尾。

追问：外参漂移怎么在线估？→ 用图像上可检测的结构（车道线、杆、路沿）与点云/雷达的对应关系，做小自由度优化（4-DOF：yaw + 3 平移），用残差监控；失败就退回离线标定值。

Q86 ★★ 门控学歪了怎么发现、怎么纠？

A：三个诊断 + 三个纠正：

诊断：1. **门控值直方图**：若集中在 0.5 附近 → 退化成静态权重；若恒为 0 或 1 → 学成了硬选择（可能是对的，也可能一路被忽略）。2. **空间分布**：画出"近处 vs 远处"、"晴天 vs 雨天" 的平均门控值，看是否符合物理直觉（远处应更信雷达/激光的几何？其实远处激光返点少，应更信相机语义 + 雷达速度）。3. **单模态消融**：推理时把某一路置零看掉多少分。**掉分很少 = 这一路没被用上**。

纠正：1. **模态 dropout**（Q91）：训练时随机屏蔽一路，强制模型学会两条通路。2. **辅助单模态损失**：训练时给每路各加一个轻量预测头（推理丢弃），保证每路特征都有判别力。3. **门控初始化偏置 + 正则**：把门控初始化到接近 1（恒等），再让它学下降；必要时加"门控方差正则"惩罚常数门控。

面试话术："我判断融合是否真的生效，不看总分，看**模态消融的掉分幅度**和**门控值的空间分布**——如果屏蔽一路几乎不掉分，说明这一路在偷懒。"

Q87 ★★★ 空间对齐有哪些坑？（融合失败最常见的根因）

A：六个坑，按踩中概率排：

1. **外参角度误差被距离放大**：1° 的旋转误差在 50m 处约 **0.87m** 横向偏移。所以远距离目标的"错位"往往是标定问题，不是模型问题。
2. **时间不同步放大空间误差**：100km/h 下 50ms 的时间差 ≈ **1.4m** 位移。**这是"融合鬼影"的头号原因**，比标定更常见。
3. **视锥裁切缺失**：把投影到图像外的点当成有效观测 → 污染背景。必须按内外参 + 图像尺寸 + 深度>0 三重条件裁。
4. **畸变未校正**：广角/鱼眼相机不做去畸变，边缘区域偏差可达数十像素。
5. **自车运动未补偿**：多帧点云/雷达累积时，不做 ego-motion 变换会出现"拖影"。
6. **网格分辨率与角分辨率不匹配**：BEV 0.6m/格 vs 雷达 1° 方位分辨率（50m ≈ 0.87m），远处一个格子里塞进好几个雷达量测方向。

一句话："融合精度上限 = 对齐精度上限。所以我会先做'投影可视化 + 残差统计'这一步，确认对齐没问题再调模型。"

Q88 ★★★ 时间对齐怎么做？（异步传感器的核心工程）

A：四步：

1. **统一时间基准**：所有传感器用**硬件同步时间戳**（PTP/GPS 秒脉冲），不要用主机接收时间（差几十 ms 且抖动大）。这是第一步也是最容易被忽略的一步。
2. **选参考帧**：一般以视觉帧为参考（频率最高、语义最强），把雷达/激光量测通过 **ego-motion** 变换到该时刻。`text p_ref = T_ego(Δt) · T_ext · p_sensor` Δt 来自时间戳差，`T_ego` 用 IMU/轮速/里程计积分（或用上一帧位姿 + 速度模型近似）。
3. **插值与累积**：雷达 10~20Hz、相机 30Hz，通常不插值（雷达点插值没有物理意义），而是**取最近帧 + 多帧累积**；激光可做运动补偿去畸变（按点时间戳逐点变换）。

4. **验证**：用静止目标（杆、墙、停车车辆）做残差检查——静止目标在多帧融合后应该是"一个点"而不是"一条线"。

追问：时间戳经常差几十 ms 怎么办？→ ① 先查是否用了软件时间（改硬件同步）；② 用路面/静止结构做在线时延估计（扫一个 Δt 范围让对齐残差最小）。

Q89 ★★ 尺度/分辨率失配怎么处理?

A：三种手段：

- **模糊化对齐**：把高分辨率一路先做高斯/平均池化到低分辨率再融合（避免"锐利的一路压过模糊的一路"）。
- **由粗到细**：先低分辨率融合（稳定），再用高分辨率特征做精修（detail refinement）。
- **多尺度金字塔融合**：在 2~3 个尺度上分别融合（类似 FPN 的双向融合）。

注意：不要指望一个融合模块同时处理不同尺度——**尺度差 2 倍以上时，同一组卷积核的感受野语义就不一样了**，必须分开融。

Q90 ★★ 跨模态一致性损失怎么加?

A：三种，从易到难：

1. **单模态辅助头**（最简单）：每路特征各接一个轻量预测头，训练时算损失、推理丢弃。作用是**保证每路特征都有判别力**，避免一路退化成"旁路"。
2. **一致性正则**：两路各自出预测，让它们互相一致（KL / L2 / 预测级蒸馏），或让融合输出同时逼近两路（防融合"两头不靠"）。
3. **跨模态对比**（最重）：同一目标的图像特征与雷达/激光特征在共享空间拉近，不同目标推远（InfoNCE）。需要目标级对应关系，标注成本高。

注意坑：一致性损失太强会让两路趋同、失去互补性（"融合两个一样的模态"）。实践中给个小权重（0.1~0.3）并监控"两路特征相似度"。

Q91 ★★★ 模态 dropout / 传感器失效增广怎么做?

A：这是"防退化"里性价比最高的一招。

- **模态 dropout**：训练时以概率 `p` 把整路输入置零（或替换成可学习的 "missing embedding"，比置零更好）。建议 `p = 0.1~0.3`，**单模态预热之后再开**，否则早期训练太不稳定。
- **关键细节**：置零时**同步跳过融合层的对应权重、并保持归一化统计**——否则 BN/LN 的统计量会被污染，推理时对不上。
- **失效增广**（模拟真实传感器退化）：
- 点云：随机丢弃远处点（模拟衰减）、按距离衰减点数、加高斯噪声
- 雷达：随机丢点、加多径杂波
- 图像：模糊、亮度/对比度扰动、随机遮挡块（模拟污损）、JPEG 压缩
- 外参：±0.3°~0.5° 随机微扰（模拟标定漂移）
- **训练目标**：让"任意单模态 + 任意退化程度"都有可接受的下限表现，而不是只在"两路都完美"时最好。

面试话术："我们不追求实验室里的峰值，而是要保证雨天、夜晚、镜头被泥糊住时还有兜底——所以模态 dropout 和失效增广是标配，模型要为'某只眼睛瞎了'做好训练。"

Q92 ★★ 融合层怎么 warmup? 渐进训练怎么排?

A：三步走：

1. **单模态预热**：先让每一路分支单独训到可用（不要一开始就开融合，否则随机初始化的融合层会打断两路的学习）。
2. **零初始化融合**：融合层的输出投影/sigmoid 偏置初始化为"恒等"（输出≈原特征），这样开融合的瞬间不改变原有行为，然后逐步学出增益。
3. **分层学习率**：新加的融合层用较大 lr，预训练分支用较小 lr（1/10）；必要时先冻结分支只训融合，再联合微调。

追问：怎么判断 warmup 够了？→ 看"单模态 loss 是否已进入平台期"和"融合后 loss 是否立刻上扬"（上扬说明融合层初始化太激进）。

Q93 ★★ 两路梯度失衡怎么办？

A：症状：某一路的梯度范数小一个数量级 → 该路学不动 → 最终被忽略。

四个手段： 1. **模态特定归一化**（先做这个，成本最低）：每路独立 BN/LN，让两路输出尺度可比，梯度量级自然接近。 2. **学习率分组**：给梯度小的那一路更大的 lr。 3. **GradNorm**：按各任务的梯度范数与收敛速度动态调整权重，让它们同步收敛。 4. **PCGrad（梯度手术）**：当两路梯度方向冲突（余弦相似度为负）时，把冲突分量投影掉。

实测建议：绝大多数"融合无效"的案例，**先查归一化和特征尺度**，能解决一半；剩下才考虑梯度手术。

Q94 ★★ 模型学会了只用一路（单模态惰性）怎么发现和纠正？

A：这是融合里最隐蔽的问题——训练指标很好，实际只用了一路。

发现： - 模态消融：屏蔽某一路看指标掉多少（**掉 <1% 基本等于没用上**） - 门控分布：若某路权重恒接近 0 - 遮挡实验：把图像关键区域抠掉，性能不变 → 说明没在看图

纠正： 1. 模态 dropout（最有效） 2. 辅助单模态损失 3. **切断单模态直通路径**：让两路都必须经过融合层才能到检测头（否则强的一路会绕过融合） 4. 降低强模态的输入信息量（如降低分辨率）做对照实验，看是否依赖

话术："我判断多模态有没有'真融合'，做的是**遮挡/屏蔽实验**——把一个模态的信息破坏掉，性能应该明显下降；如果不下降，说明融合是装饰。"

Q95 ★★★ 怎么证明融合真的起作用？（避免自欺）

A：五层验证，缺一层都可能自欺：

1. **单模态基线**：同 backbone、同数据、同 schedule、同增广，只去掉融合。**这是唯一有说服力的对照组。**
2. **融合消融**：concat / 门控 / attention 各跑一遍，说明"为什么选这个"。
3. **模态消融（推理时）**：屏蔽任一路，报出各自的退化曲线。
4. **双 seed 噪声地板**：同一配置跑两个种子，**差值就是噪声地板**；融合增益必须显著大于它（不然是随机波动）。
5. **逐类 + 逐距离归因**：把 fp/fn 拆成 no_hit / wrong_cls / early-late（位置偏差）等成分，看融合到底改善了哪一类错误——如果主要改善的是"远处小目标"，和物理直觉一致，结论就站得住。

面试价值：这五条一说，面试官会认为你**做过真实的 A/B，而不是只在论文里见过融合**。

Q96 ★★★ 融合后效果反而变差，怎么定位？

A：按"对齐 → 配权 → 信息"三段查：

症状	大概率原因	怎么验证
出现鬼影、目标位置偏移	**对齐错**（时间戳/外参/畸变）	静止目标残差检查；投影可视化；扫 Δt 看残差曲线
某些类别掉分、某些涨	**配权错**（一路主导）	门控分布 + 单模态消融；看是否"强模态压制弱模态的有效信息"
整体小幅掉分	融合层初始化太激进 / 学习率不对	关融合只训分支，再零初始化开融合
近处变好、远处变差	**尺度/分辨率失配**	按距离分段统计指标
晴天好、雨天崩	**缺失效增广**	造退化测试集；加模态 dropout + 退化增广
训练集好、验证集差	融合层过拟合小数据	减少融合参数（门控替 attention）、加正则

一句话收尾："融合出问题，我第一步不是改结构，而是**把对齐和配权这两件事分别量化**——因为 80% 的'融合无效'其实是这两件事没做对。"

S9 损失函数：从单任务选型到多任务配平

这一章按四个层次组织：**① 单任务怎么选**（Q97-Q101）→ **② 占据/3D 任务专用**（Q102-Q106）→ **③ 几何任务**（Q107-Q110）→ **④ 多任务配平与调试**（Q111-Q118）。 面试时最容易被问的不是"你知道哪些 loss"，而是**"为什么选这个、怎么配平、怎么证明它有用"**——后三层才是分水岭。

损失设计的四个层次（总览）

层次	要回答的问题	典型错误
① 单任务选型	分类/回归/几何各用什么?	分类用 MSE；框回归只用 L1
② 专用结构	占用三态、SDF/TSDF、边界判据怎么设?	unknown 当负样本；TSDF 不做截断
③ 几何一致性	深度/视差/时序怎么约束?	单目深度直接 L1（尺度歧义）
④ 多任务配平	各项权重怎么定？怎么证明有效?	拍脑袋加权；只看 loss 不看指标

Q97 ★★★ 分类/分割为什么用交叉熵而不是 MSE？CE 有哪些变体?

A：三个层次回答：

1. **梯度层面**：CE + softmax 的梯度是 `p − y`，误差越大梯度越大，**收敛快**；MSE + softmax 的梯度多一个 `p(1−p)` 因子，在 softmax 饱和区（预测很自信但错）梯度趋近 0——**恰恰在最该纠正的时候不动**。
2. **概率层面**：CE 是负对数似然，优化它等价于做最大似然估计；MSE 假设高斯噪声，对"概率分布"这个目标不合适（非凸、非归一化）。
3. **性质层面**：CE 对错误自信的惩罚趋于无穷（`−log p → ∞`），天然抑制过度自信。

四个必须会讲的变体：

变体	公式要点	什么时候用
Label Smoothing	target 从 one-hot 变 `(1−ε)·onehot + ε/C`	防过度自信、正则化、提升校准；ε 常 0.1
Focal Loss	`−α(1−p_t)^γ · log p_t`，γ 常 2、α 0.25	极端类别不平衡、大量易分负样本（检测、占用）
Soft-target CE	直接用教师分布算交叉熵	蒸馏、多标注者标签
OHEM / hard example mining	只对 loss 最大的前 k 个样本回传	难例主导的场景；注意别让噪声样本霸占

追问：focal 的 γ 起什么作用？→ 调节"难易样本的相对权重"：γ=0 退化为 CE；γ 越大，易分样本的权重被压得越狠（`(1−p_t)^γ`），难例占比上升。坑：γ 过大 + 学习率大容易训崩，且对**标签噪声**更敏感（噪声样本天然"难"）。

Q98 ★★★ 回归损失怎么选？L1 / L2 / SmoothL1 的取舍?

A：

损失	梯度	特点	适用
L2 (MSE)	与误差成正比	对大误差极敏感，**异常值主导梯度**	噪声近似高斯、无离群点
L1 (MAE)	恒定（±1）	对离群点鲁棒，但零点不可导、小误差时梯度不衰减 → 收敛慢、易抖	有离群点、需要鲁棒
SmoothL1 / Huber	小误差 L2、大误差 L1	兼顾两者，**检测框回归的事实标准**	绝大多数回归
log 变换后 L1	—	处理跨数量级目标（深度 0.5~100m）	深度/视差/距离

关键细节（易被追问）： - SmoothL1 的拐点 β（PyTorch 默认 1.0）：β 太大会退化成 L2，太小退化成 L1；对归一化后的坐标通常取 1/9（对应 IoU 意义上的 3 像素）。 - **回归前先做目标归一化**（除以均值或 std），否则不同任务（0.1m 的高度误差 vs

10m 的距离误差）梯度量级完全不同——这是多任务配平的第一道门槛。 - 对**方向/角度**回归不要用 L1，用 `sin/cos` 双通道 + CE，或用周期化角度损失（否则 ±π 处不连续）。

Q99 ★★★ 检测框回归为什么用 GIoU/DIoU/CIoU 而不是纯 L1?

A：L1 把 `(x, y, w, h)` 当四个**独立标量**优化，但评测指标是 **IoU**——代理损失与目标不一致。而且： - 两个框 L1 距离相同，IoU 可能差很多（尺度不同的框）； - 框不相交时 IoU 恒为 0，**没有梯度**。

演进线（面试按顺序说）：

```
IoU      : 1 − IoU，不相交时梯度为 0
GIoU     : 加最小闭包框，解决不相交的问题（L_GIoU = 1 − IoU + |C \ (A∪B)|/|C|）
DIoU     : 再加中心点距离/对角线，收敛更快
CIoU     : 再加长宽比一致性项（长宽比用 arctan 避免不可导）
```

实践结论：工业检测器**通常是 L1 + IoU 类联合**（L1 提供稳定的独立坐标监督，IoU 提供与指标一致的尺度不变项），而不是只用一个。3D 检测还要处理 yaw 角（用 `sin/cos` 或 bin-based 分类）和尺度差异（不同类别用不同权重）。

Q100 ★★ 类别不平衡有哪些处理手段？各自什么时候失效?

A：五类手段 + 各自的坑：

手段	做法	失效/风险
class weight	按频率取倒数，或有效样本数 `1/(1−β^n)`	极端长尾时权重爆炸，梯度不稳
Focal Loss	降低易分样本权重	对标签噪声敏感（噪声=难例）
OHEM	取 loss top-k	前几轮全是噪声样本，训练偏斜
重采样（含 CBGS）	按类平衡分组采样	**遇到异常帧会引入偏置**（我们的真实踩坑：CBGS 被个别异常数据扰动，采样后分布跑偏，需要先做数据清洗再加权）
损失组合	CE + Dice / Tversky	权重不当时 Dice 项主导，分类精度掉

重要认知：改采样就会改**训练分布**，而评测分布没变——所以**必须分类别报指标**，否则"整体涨了"可能只是采样偏向的假象。

追问：Tversky 怎么调漏检/误检偏好？→ `Tversky = TP/(TP + α·FP + β·FN)`。`β > α` 惩罚漏检（提高召回），`α > β` 惩罚误检（提高精度），是"按业务偏好调 F1 家族"的旋钮。

Q101 ★★ 分割任务什么时候用 Dice / Lovasz / Tversky，而不是纯 CE?

A：

损失	优化目标	适用场景	缺点
CE	逐像素独立分类	通用基线	类别不平衡时易被多数类主导
Dice	区域级重叠	小目标、前景占比极低（医学、占用）	训练不稳定（梯度方差大）、对小错误惩罚不线性
Lovasz	**直接优化 IoU（mIoU 的凸代理）**	目标是 mIoU 的场景	实现复杂、对噪声敏感
Tversky	可调 FP/FN 权重	有明确漏检/误检偏好	超参敏感

实践建议：`CE（主） + Dice/Lovasz（辅，权重 0.2~0.5）` 是最稳的组合；纯 Dice 训练早期很容易震荡。

追问（对口你的场景）：占用预测的评测是 **RayIoU**（沿射线判命中），而训练损失是体素级 CE——**两者并不一致**。所以必须用"逐类 fn/fp 归因"（no_hit / wrong_cls / early-late 位置偏差）来桥接：知道 CE 下降主要改善了哪类错误，才能判断它是否真的对 RayIoU 有利。

Q102 ★★★ 三态占用（free / occupied / unknown）的损失怎么设?

A：核心原则一句话：**unknown 绝不能当负样本**。

三种常见组织方式：

1. **二分类 + mask**：`occupied` 用 BCE，`unknown` 区域用 `ignore_index`/mask 掉。最常用、最稳。
2. **三分类 CE**：free/occupied/unknown 三类，注意各类占比极端不均（free 极多、occupied 少、unknown 中等），要配 class weight 或 focal。
3. **两路输出**：一路预测 `occupied`（BCE），另一路预测 `observed`（是否被观测到，BCE），`free = observed ∧ ¬occupied`。这种解耦的好处是**可解释、可以分别调**——观测不到（unknown）和空（free）在损失里彻底分开。

为什么 unknown 不能当负样本：传感器看不到的地方既不是空也不是占用，把它当"空"会让模型学会"看不到就判空"——在遮挡严重、远距离场景会系统性误判（而且这个错误在训练集上看起来没惩罚）。

追问：free 标签可靠吗？→ 由 **ray-casting** 生成的 free 标签（射线未被命中段）比 occupied 更可靠（因为 occupied 受遮挡、标注口径影响）。所以实践中可以给 free/occupied **不同的权重**，或对 occupied 用更大的权重（因为它是下游真正关心的）。

Q103 ★★★ 语义 + 占用联合预测，损失怎么组织？

A：两种范式：

- **多任务（多 head 共享特征）**：`L = λ₁·L_det + λ₂·L_sem + λ₃·L_occ`。优点是共享几何表征、互相正则；缺点是**任务冲突**（分类要判别性、占用要几何精度）。
- **级联**：先检测/分割，再把结果喂给占用头。优点是职责清晰；缺点是无法端到端、误差累积。

我们采用的是多任务（检测 / 语义 / 三态占用三头共用 BEV 特征），三个头各自的损失按"量纲归一化 + 手工权重"配平，具体权重靠**逐类 fn/fp 归因反推**——先看哪个头拖了整体指标，再调它的权重，而不是网格搜。

追问：多任务里的分类头为什么常配 focal 而不是纯 CE？→ 因为 3D 目标极度稀疏（体素里 99% 是背景），纯 CE 会让梯度被背景淹没。

Q104 ★★ ray-based / depth-aware 占用损失是什么？

A：体素级 CE 有个根本问题：**表面内部（被遮挡实体的内部）到底该判 occupied 还是 unknown，标签本身有歧义**。ray-based 损失从渲染思路借过来：

- 对每根相机射线采样若干点，网络给每个点的"占用概率"；
- 用渲染公式把沿射线的占用概率合成终止分布（第一个命中的期望深度）；
- 与 GT 深度（或 GT 射线终止位置）算损失。

好处：① 直接对齐"看到什么"这件事，天然处理遮挡；② 与下游基于深度的评测更一致。

我们的做法（可以直接讲的四件套）： 1. **终止分布 CE**：沿射线的离散终止分布做交叉熵（监督"第一个表面在哪"）； 2. **前沿线空 BCE**：把"表面之前"的采样点约束为空（抑制前置虚警）； 3. **面后占用抑制 BCE**：把"表面之后"的采样点压向非占用（抑制"背后有实体"的误判）； 4. **终止点语义 CE**：在终止点（表面）上做语义分类。

一句话总结："这四件套把'深度上的第一性'和'语义上的类别'分开监督，比单纯体素 CE 更贴合可见性。"

Q105 ★★ SDF / TSDF 的监督损失怎么设计？

A：四件套：

1. **主损失：截断 L1** `text L = |clip(f(x), -δ, +δ) - clip(sdf_gt(x), -δ, +δ)|` **必须截断**：远处符号（内/外）不影响表面，且不截断会让网络把算力浪费在远离表面的区域。δ 常取 0.05~0.2m。
2. **Eikonal 正则**：`(|∇f| - 1)²`，强制输出满足距离场性质（梯度模长为 1）。这是 SDF 类方法的标配，能让几何更"干净"、表面更平滑。
3. **表面附近采样**：均匀采样 + **表面附近密集采样**（按距离分桶），否则大部分样本的梯度信息量极低。
4. **符号/内外一致性**：多个视角/多个截断区域拼起来时，符号必须自洽（同一实体内部统一为正），否则融合后出现"半实半虚"。

TSDF 的差异：TSDF 天然带截断（±1 归一化），用截断 L1 即可，且**越靠近表面权重越高**（因为远端的 TSDF 值本就不精确），常按 `exp(-|d|/σ)` 或线性衰减加权。

追问：SDF 和 TSDF 哪个更适合做表面重建/判据？→ SDF 连续可微、能算梯度求表面法向（Eikonal 天然），适合优化与法向估计；TSDF 是加权平均、抗噪好、适合多帧融合。**判据类任务（表面/边界判定）通常用 SDF 值 + 阈值，用 TSDF 做几何兜底**。

Q106 ★★ 表面化判据 / 边界 / 前沿线这类任务用什么损失?

A：这是"极稀疏二分类"问题，纯 BCE 不行：

- **主损失 BCE + focal**：正样本占比极低，focal（γ=2）或正样本加权（pos_weight = 负/正比）是基线。
- **distance transform 加权**：按每个点到 GT 表面的距离给权重（越近权重越高，如 $w = \exp(-d^2/2\sigma^2)$），让"差一点就是表面"的区域获得更大梯度——这是边界类任务最有效的一招。
- **软标签 / 模糊带**：把标注本身的不确定性建模成一个带宽（表面附近 1~2 体素的软标签），避免在标注歧义区强行拉满。
- **Dice / Tversky 辅助**：直接优化区域重叠，对小目标边界更敏感。

面试可讲的点："边界类任务的瓶颈通常不在损失函数本身，而在**标注一致性**——表面到底算在哪个体素，标注者之间差 1 个体素就足以让 mIoU 抖几个点。所以我会先做双人/双次标注一致性统计，再谈损失调优。"

Q107 ★★ 立体匹配 / 深度估计的损失怎么设计?

A：三部分组合：

1. **主监督：Smooth L1 on 视差/深度**（EPE 指标对应的就是它），只对有效视差（非遮挡、非无效）计算——`mask` 必须做，否则无效区域会污染梯度。
2. **多尺度序列监督**（迭代式方法如 RAFT-Stereo）：每次迭代都算损失，权重随迭代递增（`γ^(N−i)`，γ 常 0.9），让粗到细都受监督。
3. **无监督/自监督分支**（有图像对时可用）：photometric（把右图按预测视差 warp 到左图，算 SSIM + L1）+ **左右一致性**（左右视差互为参考）+ **平滑项**（二阶/TV，边缘处放松）。

关键细节：- **遮挡/无纹理区域要 mask**：这些区域本来就没有匹配信息，强行监督会把噪声学进网络。- **置信度分支**：用"预测值与 GT 的差"作为置信度监督（差越小置信度越高），推理时可用来滤除不可靠深度。- **视差归一化**：不同数据集视差范围差异大，先除以均值或做 scale-shift invariant（SSI）归一化。

Q108 ★★ 单目深度的尺度歧义怎么处理?

A：单目没有绝对尺度，直接 L1 会让网络在"整体缩放"上纠缠。三种处理：

1. **尺度不变损失（SSI）**：先对预测和 GT 做最小二乘对齐（scale a、shift b），再算 L1：`text (a*, b*) = argmin || a·d_pred + b − d_gt ||² 然后 L = |a*·d_pred + b* − d_gt|` 这就是 MiDaS 系列的经典做法（位移不变 + 尺度不变）。
2. **对数空间**：`|log d_pred − log d_gt|`，等价于相对误差，对小深度不惩罚过重。
3. **相对深度 / 排序**：只监督相对关系（ordinal regression、pairwise ranking），适合没有绝对 GT 的数据。

追问：什么时候必须用绝对尺度？→ 下游要做度量判断（占用、可通行、测距）时必须；纯视觉展示类任务可以用相对深度，再用少量带尺度标注的数据或传感器做**尺度对齐（scale calibration）**。

Q109 ★ 点云/重建类任务的损失有哪些?

A：四种常用：

- **Chamfer Distance**：正反两个方向的最近点距离之和。快，但**对密度不敏感**（点聚在一起也能低分）。
- **Earth Mover's Distance**：最优传输距离，考虑密度，更准但 $O(n^3)$ 级别贵（一般用近似）。
- **法向一致性**：`1 − cos(n_pred, n_gt)`，几何质量的关键指标。
- **隐式场损失**（我们用的 SDF/TSDF）：在采样点上监督距离值，见 Q105——现在主流已经从"生成点"转向"生成场再提取表面"，因为场更连续、更容易加 Eikonal 约束。

Q110 ★ 时序一致性损失怎么加?

A：三个层面，从模型到损失：

1. **权重层面（最省）**：EMA 教师模型 + 自蒸馏，或用"上一帧权重"做约束——我们用的是 **EMA（12/24 epoch）**，本质是免费的时序平滑。
2. **预测层面**：用自车运动把上一帧预测 warp 到当前帧，与当前预测算一致性（`|P_t − warp(P_{t−1})|`），只在静态区域计算（动态目标要用光流或 mask 掉）。
3. **特征层面**：时序历史特征与当前特征做一致性/门控（注意别让历史信息"抄近路"导致当前帧不学）。

坑：一致性损失对**对齐误差**极敏感——自车运动估计不准时，一致项会把误差"学成规律"。所以要先确认位姿/时间同步质量（同 S8 的对齐原则）。

Q111 ★★★ 多任务 loss 权重怎么定？（这是面试最爱问的）

A：五类方法，按落地顺序：

方法	做法	优点	缺点
① 量纲/量级归一化	各项先除以初始值或滑动均值，再给权重	**成本最低、最有效**	依赖初始值选择
② 手工权重 + 网格/贝叶斯搜索	按经验设比例如 1:1:0.2	可控	任务多了搜不动
③ 不确定性加权（Kendall）	每项学 $\log \sigma^2$，$L = \Sigma \exp(-\log \sigma^2) \cdot L_i + \log \sigma^2$	自动、有理论	σ 会漂，要 clamp
④ GradNorm	按各任务梯度范数动态调整，使收敛速度同步	针对"学得慢的任务"	要算梯度范数，开销大
⑤ DWA（动态权重平均）	按最近窗口的 loss 下降比率调权	简单有效	权重可能震荡，要平滑

我们的实际做法（能直接讲）：先用 ① + ② 打底（多尺度深监督的粗头权重 0.2，各头量纲先归一化），再用**逐类 fn/fp 成分解剖**反推要不要调——比如发现某类主要是 `wrong_cls`（分类错），就加分类头的权重；发现是 `no_hit`（根本没预测到），就加召回相关的项（focal 的 α 或正样本权重）。**这一套比网格搜索快，也更容易解释。**

追问：权重需要随训练阶段变吗？→ 需要。常见做法是辅助头权重**随 epoch 衰减**（前期帮助收敛、后期让主任务主导），主任务权重保持稳定。

Q112 ★★★ 深监督 / 辅助头怎么加权?

A：三条经验：

1. **辅助权重小**：典型 0.1~0.4。太大 → 辅助任务主导，主任务学偏；太小 → 没作用。我们用的 **ASPP 粗头权重 0.2**，收敛期主 loss 降了约一个量级。
2. **随训练衰减**：`w(t) = w₀ · (1 − t/T)` 或按平台期衰减，让后期主任务独占梯度。
3. **辅助头设在"信息瓶颈之前"**：放在较浅/较粗尺度的强任务（如粗粒度语义），能提供更密集的梯度；放在深层反而容易和主头冲突。

坑：辅助头如果用同一份标签（就是主任务的多尺度版本），要注意**标签分辨率与预测分辨率一致**（下采样标签时用最近邻而非插值，否则边界变模糊，辅助头学的是假梯度）。

Q113 ★★ 梯度冲突 / 负迁移怎么处理?

A：四步：

1. **先判断是否真的冲突**：算两个任务梯度的余弦相似度，长期为负才是真冲突。
2. **PCGrad（梯度手术）**：冲突时把一方梯度在另一方的正交方向上投影掉，消除方向冲突分量。
3. **参数隔离**：冲突严重的任务不共享全部 backbone（共享浅层、任务专属深层，或多分支头）。
4. **训练顺序**：先各自单任务预热，再联合微调（避免早期强任务压制弱任务）。

实测建议：优先级是 **归一化 > 权重调整 > 参数隔离 > PCGrad**。绝大多数"多任务训不好"是前两项没做对，GradNorm/PCGrad 是最后的补丁。

Q114 ★★ 训练中动态调权要注意什么？

A：三条纪律：

- **必须平滑**：DWA 这类按窗口算比率的方法，对单个 batch 的噪声极敏感，要做 EMA 平滑（否则权重在高频震荡）。
- **必须有上下界**：任何动态权重都做 clamp（如 [0.05, 5]），否则某项会一路涨到"吃掉整个训练"。
- **必须监控**：把各项 loss 和权重分别画曲线，权重剧烈变化的时刻往往对应数据分布问题（异常帧、某个传感器数据缺失），而不是模型问题。

Q115 ★★★ loss 一直在降，但指标不涨，怎么查？

A：五个方向，按概率排：

1. **代理与指标不一致**：loss 优化的是体素 CE/L1，评测指标是 RayIoU/mAP——两者不同步是常态。**做法：把评测指标的错误拆成分项**（no_hit / wrong_cls / 位置偏差），看 loss 下降改善了哪一项。如果改善的是评测不关心的部分（如远处海量 free 体素），那 loss 降而指标不动完全是合理的。
2. **评测噪声地板**：同一配置跑两个 seed，差值就是噪声。指标变化小于噪声地板时，"不涨"可能是假象——我们做双 seed 噪声地板的直接动机就是这个。
3. **过拟合/数据泄漏**：训练集 loss 降、验证集指标不动；或验证集和训练集有近重复帧。
4. **类别不均衡掩盖**：整体 loss 被多数类主导，少数类的恶化被冲掉。必须分类看。
5. **后处理/解码不一致**：训练用 CE 优化概率，解码用阈值 0.5 或 top-k，阈值没调过的话指标自然不涨。

面试话术："我判断损失项是否有效，不看 loss 数值，看**指标的分项归因**和**双 seed 噪声地板**——先确认指标变化超出噪声，再确认改善落在错误的哪个成分上。"

Q116 ★★★ 某一类永远学不会，怎么定位？

A：六步查，从数据到结构：

1. **标签**：数量是否极少？有没有越界/映射错？（我们的真实案例：CE 的 target 越界，根因是 `cls_num` 参数与实际类别数不一致——loss 直接崩，而不是"学不会"）
2. **是否被 ignore/mask 掉**：某些类在特定距离/视角下被裁掉或标为 unknown。
3. **权重**：class weight 是否让这类几乎不产生梯度。
4. **分辨率/感受野**：小目标在 BEV 0.6m/格 下可能只占 1~2 格，必须有对应尺度的特征（多尺度/高分辨率头）。
5. **特征可分性**：可视化该类的特征分布，看是否和相近类糊在一起（→ 需要更强监督：边界、法向、时序）。
6. **评测口径**：RayIoU 的 `no_hit` 高说明"根本没预测到"，`wrong_cls` 高说明"预测到了但类错"——两条路的解法完全不同。

Q117 ★★ loss 出现尖峰 / 爆掉，处理顺序？

A：严格按这个顺序（我们踩过三连，已知最高效的路径）：

1. **数据**：label 越界、空样本、异常帧（先做数据完整性断言：类别 id 范围、尺寸一致、无 NaN）；
2. **数值**：bf16/fp16 溢出 → 换 bf16 或加 loss scaling；log/exp 未在数值稳定域计算（softmax/CE 没减 max）；
3. **超参**：学习率过大、缺 warmup、梯度裁剪（通常 clip 1.0）；
4. **结构**：新加层初始化不当（零初始化/恒等初始化）、残差尺度、归一化层 eps；
5. **硬件/框架**：GEMM workspace 不足、显存碎片、分布式通信超时（这类报错常伪装成算子错误，例如 `CUBLAS_STATUS_ALLOC_FAILED`）。

一句话："先证伪数据，再证伪数值，最后才怀疑模型——顺序错了会浪费一整天。"

Q118 ★★ 怎么证明某个损失项真的有效?

A：四层证据：

1. **消融**：关掉该项，其余完全不变（同 seed、同 schedule），看指标变化是否超过噪声地板。
2. **曲线分离**：画出"有/无该项"的**评测指标**曲线（不只是 loss 曲线）——很多项能让 loss 降但指标不变。
3. **分项归因**：确认改善落在预期的错误成分上（如加了面后抑制项，应主要降低"背后虚警"而不是近处漏检）。
4. **正交性检查**：两项一起加的增益 ≈ 各自增益之和，说明它们解决的是不同问题；若一条加完另一条就没用了，说明二者冗余（面试官很吃这一套分析）。

我们在当前任务中的实际损失配置（可直接讲）

> 结构信息，不含数据集规模与指标数值。

任务头	损失	关键设置
3D 检测	分类 focal + 框回归平滑 L1 + IoU 类项	分类配 focal 抗背景稀疏；yaw 用 sin/cos 双通道
语义分割（BEV）	CE（+ 类别权重）	逐类权重按频率倒数、clamp 上下界
三态占用	free/occupied BCE + **unknown 用 mask 排除**	free/occupied 分权重（occupied 是下游真正关心的）
地面高度回归	带 mask 的 L1	只在地面类有效处计算
细粒度二分类（表面/边界）	BCE（含焦点项）	正样本极少，配距离加权
多尺度深监督	ASPP 粗头辅助损失，**权重 0.2**	收敛期主损失降约一个量级；辅助损失标签用最近邻下采样
SDF 类任务	截断 L1 + Eikonal 正则	表面附近密集采样；δ 取厘米级
立体/深度	平滑 L1（有效视差 mask）+ 多尺度序列监督	迭代步间权重 γ^i；遮挡区 mask 掉
多任务配平	量纲归一化 + 手工权重，用**逐类 fn/fp 归因反推**	不靠网格搜索；辅助项随训练衰减
时序稳定	EMA（12/24 epoch）	免费的一致性收益

三条踩过的坑（讲这些比讲配置更有说服力）： 1. **CE target 越界**：参数名与实际类别数不一致，loss 直接崩——所以现在数据加载加了 id 范围断言。 2. **CBGS 被异常帧扰动**：class-balanced 采样一旦遇到脏数据，训练分布跑偏，整体指标虚高而单类别变差——所以采样前必须先清洗。 3. **深监督标签下采样用插值**：导致边界模糊、辅助头学假梯度，改成最近邻后曲线才对。

附：按 30 秒精度划分的优先级（复习用）

优先级	题目	说明
必须脱口而出（24 题）	Q1 Q6 Q9 Q14 Q20 Q21 Q22 Q25 Q26 Q34 Q39 Q40 Q55 Q56 Q62 **Q75 Q76 Q83 Q87 Q91 Q97 Q102 Q111 Q115**	这些答不上来基本判定"没做过多模态"
要能讲 1 分钟（42 题）	Q3 Q4 Q5 Q10 Q11 Q12 Q13 Q15 Q23 Q24 Q27 Q30 Q33 Q35 Q37 Q38 Q41 Q42 Q45 Q48 Q49 Q51 Q57 Q59 Q63 **Q77 Q78 Q79 Q80 Q84 Q85 Q88 Q95 Q96 Q98 Q99 Q100 Q104 Q105 Q107 Q112 Q116**	讲到"原因 + 一个例子"即可
能接住就行（其余）	剩下全部	听懂了、能接一句，不要把不确定的说满

复习节奏建议：面试前 3 天每天过一遍"必须脱口而出 24 题"；面试前 1 小时只看这一页的表格。

视觉-文本大模型 + 机器学习 / 卷积网络 八股问答

> 与《八股知识问答 118 问》的分工：那份是**多模态总览 + 项目接口话术**，这份是**VLM 深度知识 + 机器学习算法原理推导**，两份不重复、可对照看。 编号用 A（视觉-文本大模型 + 损失/架构/LoRA，77 问）、B（机器学习算法，45 问）、C（图像处理与卷积网络 + 视觉基础模型 + 单目深度 + 激活函数，88 问），共 **210 问**，避免与 Q1~Q118 冲突。 **§ B6 附原理验算代码**（47 项断言，面向 Python 语法不熟者逐行注释）。 难度：★★★ 几乎必问 · ★★ 大概率问 · ★ 加分题。

Part A 视觉-文本大模型（VLM）

A1 模型族谱与架构（A1–A12）

A1 ★★★ VLM 有哪几种架构范式？各自的代表模型和取舍？

A：四种范式，按"视觉怎么进语言模型"分：

范式	结构	代表	取舍
① 双塔对比（encoder-only）	图像塔 + 文本塔，各自编码后算相似度	CLIP、ALIGN、SigLIP	检索/零样本分类强，**不能生成**，只学全局对齐
② 融合编码器	图文进同一个 Transformer（cross-attention 交互），输出融合表征	ALBEF、BLIP、ViLT	适合匹配/理解类任务，参数量比双塔小，但仍不生成自由文本
③ 编码器-解码器桥接（主流 VLM）	冻结/半冻结视觉塔 → 桥接层 → LLM 自回归生成	BLIP-2、LLaVA、Flamingo、Qwen-VL、InternVL	**能力最强、可指令化**，代价是 token 多、训练分阶段
④ 原生/早期融合（Early fusion）	一开始就把图像 patch 与文本 token 拼进同一个序列，统一自回归	Chameleon、Emu3、Fuyu	结构最简洁、天然支持交错图文，但需从零训或大改，工程门槛高

面试答法："主流是第 ③ 种——因为它的训练成本最低（可冻结双塔）、能力上限又最高；第 ④ 种是方向但需要从头训，数据门槛太高。"

追问：Fuyu 有什么特别？→ **没有独立视觉编码器**，直接把图像 patch 线性投影成 token 送进语言模型，省掉视觉塔、也省掉桥接层，代价是需要更强数据、分辨率灵活性靠 patch 切分。

A2 ★★★ LLaVA、BLIP-2、Flamingo 三者的桥接设计有什么本质区别？

A：

	桥接方式	视觉 token 数量	训练成本	核心思想
LLaVA	2 层 MLP（线性投影）	**不压缩**（与 patch 数相同）	最低（只训 MLP）	视觉塔特征已够好，简单映射即可
BLIP-2	**Q-Former**（32/64 个可学习 query + cross-attention）	**固定**（32/64）	中（两阶段多任务）	压缩 + 强对齐
Flamingo	Perceiver Resampler + **LLM 里插 gated cross-attention**	固定（resampler 输出）	中（插入新层）	不改 LLM 主干权重，只加"旁路"

本质区别一句话：LLaVA 是"**改维度**"，BLIP-2 是"**压数量 + 强对齐**"，Flamingo 是"**在 LLM 内部做融合**"。

追问：为什么 LLaVA 敢用最笨的 MLP？→ 因为它冻结双塔、数据量大（数十万~百万级指令），MLP 已经足够；而且论文消融显示，在冻结设定下复杂桥接的收益不显著。**这是"简单方案 + 足够数据"胜过"精巧结构"的典型案例。**

A3 ★★★ 视觉塔为什么大多用 CLIP/SigLIP 而不是 ImageNet 预训练的 ResNet/ViT?

A：三条理由：

1. **语义空间已经和文本对齐**：CLIP 系在 4 亿级图文对上做过对比学习，特征里已经含"可被语言描述"的语义；ImageNet 特征是判别 1000 类的，语义粒度不匹配。
2. **冻结就能用**：CLIP 特征在冻结条件下零样本泛化好，适合 VLM"冻结视觉塔"的训练范式。
3. **ViT 结构同构**：patch token 序列可以直接送进 LLM，不需要额外适配。

什么时候不用 CLIP：需要**密集预测**（分割/深度/占用）时，CLIP 的全局对齐特征反而丢失细粒度空间信息，这时 DINOv2（自蒸馏，保留空间结构）或纯监督 ViT 更合适。**混合做法**：CLIP 塔给语义 + DINOv2 给结构，双塔拼接。

追问：SigLIP 比 CLIP 好在哪？→ 用 pairwise sigmoid 替代 softmax 交叉熵，**不需要全 batch 归一化**，因此 batch 可以小、能分布式异步训练，更省内存；效果同级甚至更好，是现在很多开源 VLM 的默认视觉塔。

A4 ★★ BLIP 的 CapFilt 是干什么的?

A：网页图文对（alt-text）噪声大——图里是"一只狗在草地上"，alt-text 却是"点击购买"。CapFilt = **Captioner + Filter**：

1. **Captioner**：用图像生成 caption（合成描述）；
2. **Filter**：用 ITM 头判断"图文是否匹配"，把原 alt-text 和合成 caption 里不匹配的过滤掉。

本质是用模型给自己造更干净的监督（bootstrapping）。这条和现在"用强 VLM 重新 caption 训练数据"（ShareGPT4V 一类）是同一思想。

A5 ★★ ALBEF 的三个训练目标是什么？为什么需要 ITM?

A：**ITC（图文对比）+ ITM（图文匹配）+ MLM（掩码语言建模）**。

- **ITC** 学全局粗对齐（快、但只能到"整体像不像"）；
- **ITM** 用融合后的特征做二分类（细粒度、但要负样本，靠 ITC 的相似度做 **hard negative 挖掘**）；
- **MLM** 让模型学词级与图像区域的对应（补细粒度）。

为什么需要 ITM：ITC 用的是各自独立的编码（没有跨模态交互），学不到"这个词对应这块区域"；ITM 让图文进同一个编码器交互。**代价是推理时每对都要过一次融合编码器，慢——所以 BLIP-2 之后就用生成式目标替代了它。**

A6 ★★ CoCa 和 BLIP 的思路差异?

A：CoCa 把"**对比学习**（对齐）"和"**caption 生成**（生成）"放在**一个模型的两个头**上，共享图像编码器：一个头做图文对比（学判别式表征），另一个头做自回归 caption（学生成能力）。

意义：**证明对齐与生成可以共存且互补**（对比目标提供稳定的语义空间，生成目标提供细粒度语言监督）。现在很多 VLM 训练都是这个混合思路的延伸。

A7 ★★★ 视觉 token 到底有多少？为什么它是 VLM 的"预算中枢"?

A：算一遍就有体感： - 336×336、patch14 → 24×24 = **576 token** - 448×448、patch14 → 32×32 = **1024 token** - LLaVA-1.6 的 anyres（4 切片 + 1 缩略图）→ 约 **2880 token** - InternVL 的 pixel shuffle（2×2 拼通道）→ token **÷4**

而一张 4096 上下文的 LLM，一张高分辨率图就能吃掉 70%。所以：**分辨率（看得清）↔ token 数（算力/显存）↔ 上下文（能塞几张图/多轮）三者是硬约束的三角**。

工程手段：pixel shuffle、池化、Q-Former 压缩、动态分辨率（按需给 token）、只取关键帧（视频）。

A8 ★★ Qwen-VL 的 M-RoPE 解决什么问题?

A：普通 RoPE 是 1D 的（按序列位置旋转），但视觉 token 有 **时间（帧）+ 高 + 宽** 三维结构。M-RoPE（Multimodal RoPE）把旋转维度切成三份，分别编码 t/h/w，于是： - 同一帧内相邻 patch 的相对位置正确； - 不同帧之间有时间相对关系； - 图像与文本混排时位置语义不乱。

意义：让一个模型同时处理**图像、多图、视频**而不需要改结构——这是"原生多模态位置编码"的代表做法。

A9 ★ InternVL 的 pixel shuffle 为什么能省 token?

A：把相邻 2×2 个 patch 在**通道维**拼起来：原本 4 个 patch 各 `d` 维 → 合成 1 个 patch `4d` 维。token 数 ÷4，再用一个 MLP 把 `4d` 投回 `d`。

代价：空间分辨率信息被"压缩进通道"，细粒度定位会略损；但配合**动态分辨率**（大图切块）能补回来。这是"用通道换序列长度"的典型取舍。

A10 ★★ MiniGPT-4 和 LLaVA 的差别是什么?

A：结构上 MiniGPT-4 用 **Q-Former + 单层线性投影**（沿用 BLIP-2 的视觉侧），LLaVA 用 **CLIP ViT + 2 层 MLP**。数据上 MiniGPT-4 用短 caption 预训练 + 高质量长对话微调（用 GPT-4 生成、人工清洗），LLaVA 用 GPT-4 生成的视觉指令数据。

历史意义：MiniGPT-4 证明了"冻结 LLM + 轻量桥接"能获得对话式视觉能力；LLaVA 把这条路线做到了极致简化 + 规模化，成为开源 VLM 的事实标准。

A11 ★ 原生多模态（Early fusion）和"桥接式"到底差在哪?

A：差在**融合发生的位置**和**训练方式**：

- 桥接式：视觉塔独立编码（可能冻结）→ 投影 → 拼进 LLM 序列。融合发生在 LLM 的第一层之后（视觉 token 已经是"外来词"）。
- 原生式：图像 patch 直接当 token，与文本 token 从第一层起就在同一序列里做 attention。**模型从一开始就不知道"这是图像"还是"这是文本"**，全靠数据学。

代价：原生式通常要从头预训练（或大规模继续预训练），数据门槛极高；好处是**交错图文（多图穿插在长文档里）**天然支持，且不会出现"视觉塔与 LLM 表征错配"。

A12 ★ 视觉塔参数量要不要大？为什么很多模型用 ViT-L 而不是 ViT-H/G?

A：这是**性价比**问题。视觉塔只在 prefill 时跑一次，但它的计算量与**分辨率平方 × 参数量**挂钩；而 LLM 的算力主要在 decode（逐 token）。所以在"视觉塔够用"的前提下，加参数到视觉塔的边际收益远低于加在 LLM 上。

实践：ViT-L（约 300M）是主流；追求更强视觉理解时才上 ViT-H/SoViT；端侧反过来会用 ViT-S/B 配蒸馏。

A2 训练范式与数据（A13–A22）

A13 ★★★ VLM 训练为什么必须分阶段？一次性端到端训行不行?

A：能训，但**代价大且不稳**，原因三条：

1. **初始化错配**：随机初始化的桥接层输出是噪声，若同时解冻 LLM，这些噪声会通过梯度污染 LLM 的预训练能力（灾难性遗忘）。
2. **学习率不可共享**：视觉塔需要小 lr（保护预训练特征），桥接层需要大 lr（从零学），LLM 需要中 lr。分阶段可以分别设。
3. **数据配比不同**：对齐阶段用海量短 caption（学"图上有啥"），指令阶段用少量高质量对话（学"怎么答"）。混在一起训，模型会偏向数据量大的那类。

标准做法：阶段一冻结双塔只训桥接（对齐）→ 阶段二解冻 LLM 做指令微调 → 阶段三（可选）偏好对齐。

A14 ★★★ 对齐阶段（stage-1）为什么要冻结 LLM?

A：三个原因：

1. **保护语言能力**：桥接层初始输出没有语义，让 LLM 去适配它 = 让 LLM 学"如何理解噪声"，会破坏语言先验。
2. **训练信号更干净**：冻结 LLM 后，梯度只能改桥接层，等于在解"如何把视觉特征翻译成 LLM 能懂的词"这个子问题。
3. **成本**：只训几百万参数，单卡就能跑。

追问：那为什么有些模型（如 InternVL）在阶段一就解冻部分 LLM? → 数据量足够大时（亿级图文对），直接联合训练能获得更强的对齐；小数据下必须冻结，否则一定崩。

A15 ★★ 指令微调数据怎么构造？有哪些来源？

A：五类来源，按质量排：

1. **人工标注**（最贵）：真实用户的问答、专家标注。
2. **强模型蒸馏**：用 GPT-4V/Claude 对图像生成多轮对话，再人工抽检。**注意只让它生成文本，不要用它生成图像**（版权与分布问题）。
3. **现有数据集改造**：VQA、检测（转成"图里有什么/在哪"）、OCR、grounding、文档理解，全部改写成对话格式。
4. **合成负样本**：故意问图中没有的东西，教模型说"没有"——**这是抑制幻觉最有效的数据手段**（对应 POPE 的 adversarial 设置）。
5. **纯文本数据混入**：防语言能力退化（见 A21）。

质量控制：去重（图像近重复是评测泄漏的头号来源）、答案长度控制（过长的回答会教模型"啰嗦"）、任务配比（OCR/推理/描述/grounding 要有比例）、难度分层。

A16 ★★ 怎么定图文 token 比例 / 数据配比？

A：没有普适公式，但有三条经验：

1. **按任务定 token 预算**：OCR/文档/grounding 需要高分辨率（给多 token），日常对话可以压缩（少 token）。
2. **按能力短板调**：评测发现哪块弱就加哪块数据（不是加权采样，而是补数据）。
3. **按阶段定比例**：对齐阶段图文对占绝对多数（可以 100%）；指令阶段通常图文 : 纯文本 ≈ 7:3 或 8:2。

监控手段：分别报"纯文本 benchmark"和"多模态 benchmark"，看是否出现"多模态涨、纯文本掉"的跷跷板。

A17 ★★ 分辨率策略有哪几种？各自代价？

A：四种：

策略	做法	token 代价	代表
固定分辨率	统一 resize 到 336/448	固定（576/1024）	LLaVA-1.5
动态切块（anyres）	大图切 N 块 + 1 张缩略图	N×576 + 576	LLaVA-1.6、InternVL
原生分辨率	按原分辨率切 patch，不缩放	与像素数成正比	NaViT、Fuyu
像素重排	2×2 patch 拼通道	÷4	InternVL

关键工程点：切块时块与块之间要**留重叠**（避免边界目标被切断），并做**位置编码的连续性处理**（否则模型不知道第 2 块在右边）。

A18 ★★ 多图 / 多轮对话在训练时怎么组织？

A：三条：

1. **特殊 token 标界**：每张图用 `<image>` / `<|image_pad|>` 包裹，模型靠这个知道"这里换了一张图"。
2. **图像只在首次出现时插入**：多轮里后续轮次引用"上面那张图"，不重复插 token（否则上下文爆炸）。
3. **loss mask**：只在 assistant 回复上算损失，用户提问和历史回复 mask 掉。

追问：多轮时历史图像 token 要不要丢？→ 上下文够就保留；不够时的常见策略是"旧图只留缩略图 token"或"旧图换成语义摘要"，本质是**压缩历史**。

A19 ★ 课程学习（curriculum）在 VLM 训练里怎么用？

A：由易到难排数据：短 caption → 长描述 → 单轮 QA → 多轮对话 → 需要推理/多图对比的难题。好处是**收敛更稳**（早期不被难题的噪声主导）、**避免早期过拟合到长答案格式**。

实践中最常见的"隐式课程"就是：对齐阶段全是短 caption（简单），指令阶段才上复杂任务。

A20 ★★ 合成数据有什么风险？

A：四个风险：

1. **分布坍缩**：全用强模型生成，学生会继承教师的偏好和错误（教师也可能幻觉）。
2. **风格单一**：句式、长度、格式趋同，模型失去多样性（用户会觉得"答得像机器人"）。
3. **事实污染**：教师对图像看错的地方，被当作"标准答案"教下去。
4. **评测污染**：合成时如果不小心用到评测集图像，指标虚高。

缓解：合成 + 人工抽检、保留一部分真实数据、拒绝采样（best-of-n 后人工/规则筛）、对合成数据降权。

A21 ★★ 混合模态训练时，怎么防止纯文本能力退化？

A：三条措施：

1. **数据层面保留纯文本**：图文:纯文本通常 7:3 ~ 8:2；SFT 阶段尤其重要。
2. **参数层面限制漂移**：用 LoRA 或只解冻部分层，减少对语言能力的破坏。
3. **评测层面同时盯**：每次报多模态分的同时报纯文本分，出现"跷跷板"就调整配比。

机制解释：图文数据的答案分布（描述性、短句）与文本数据（推理、长文）差异大，模型会向数据多的那侧偏——这叫**模态偏置**，不是简单的"遗忘"。

A22 ★★ 训练时视觉塔解冻有什么风险？什么时候该解冻？

A：**风险**：① 视觉塔参数量大，解冻后显存和算力翻倍；② 小数据下会破坏预训练特征（退化成"适配指令数据的私有特征"）；③ 与 LLM 的学习率不匹配时容易震荡。

该解冻的情况：领域差异极大（医学影像、红外、深度图、遥感）、需要极细粒度定位、数据量足够大（百万级以上）。**通常做法**：只解冻最后 1-2 层 + 小学习率（1/10），或加 LoRA。

A3 能力边界与失效模式（A23–A30）

A23 ★★★ VLM 的能力短板主要集中在哪五处？

A：

短板	表现	根因
细粒度计数	数错人数/物体数	对比学习不建模数量；token 聚合丢个体
空间关系	"左边的红球"搞错	全局对齐不建模相对位置；位置编码在视觉侧弱
精确 OCR	小字、密集文本读错	分辨率不足 + patch 太粗
视觉推理链	多步推理靠语言先验猜	训练数据里缺"看图推理"的中间步骤
grounding 精度	框偏、指代歧义	缺区域级监督；token 与像素的对应不唯一

面试加分：每说一个短板，配一句"所以工业上怎么绕"——如 OCR 靠 dynamic tiling + 专门 OCR 数据，空间关系靠显式 3D/2D 坐标输入（SpatialVLM 一类）。

A24 ★★★ 幻觉的三种成因在模型结构上分别对应什么？

A：

成因	结构层面的解释	缓解手段
语言先验主导	LLM 占了绝大部分参数，视觉 token 在序列里是"少数派"，attention 容易忽略它们	提高视觉 token 权重/数量、对比解码、加大视觉分辨率

成因	结构层面的解释	缓解手段
视觉信息不足	压缩过度（Q-Former/池化）、分辨率低 → 视觉证据本身弱	减少压缩、dynamic tiling、多图交叉验证
对齐错位	桥接层是"翻译器"，翻译错了 LLM 就会基于错误信息自信作答	对齐阶段加更多数据/多任务监督、区域级对齐

追问：为什么"加更多指令数据"不能根治幻觉？→ 指令数据教的是"怎么答"，不改变"看不看得清"；幻觉主要是**视觉证据强度**问题，得从分辨率/token/监督入手。

A25 ★★ 为什么 VLM 在计数上特别差？

A：四个原因叠加：

1. **训练目标不鼓励逐个体建模**：对比学习/生成目标都没有"必须数对"的压力。
2. **token 聚合丢失个体性**：patch 特征池化成全局语义时，5 个人和 6 个人的差异被平均掉。
3. **位置编码不显式表示"第几个"**。
4. **数据偏差**：训练数据里计数类问答少且答案分布偏（多半是"1 个""2 个"）。

能改善的做法：专门的计数数据 + 让模型输出检测框/点（"先定位再数"）+ 高分辨率。

A26 ★★ 空间关系理解为什么难？

A：因为 VLM 的语言侧对"左右上下"极敏感（文本里这两词很常见），但视觉侧**没有显式坐标系**——patch 序列的位置编码是 2D RoPE 或可学习编码，与"图像平面坐标系"不是一回事。于是模型倾向于**根据语言先验猜**（"盘子通常在下"），而不是看图。

工业解法：把坐标显式输入（"物体 A 在 (x1,y1)-(x2,y2)"）、加 3D/2D 位置监督数据、或干脆让模型先输出框再回答。

A27 ★★ OCR 能力怎么系统性提升？

A：四件事，按性价比排：

1. **分辨率**（最关键）：dynamic tiling 把长文档切成条带，每块单独编码。
2. **数据**：大量文档/表格/截图类 QA（DocVQA、ChartQA、TextVQA 一类），以及纯 OCR 转录任务。
3. **视觉塔**：patch 更小或专用 OCR 前端。
4. **后处理**：让模型输出结构化结果（JSON/表格），便于校验。

A28 ★★ VLM 做视觉 grounding 的三种输出形式？

A：① **文本坐标**（"`<box>(x1,y1,x2,y2)</box>`"）——最通用，但精度受 token 化影响（数字被切成多个 token）；② **特殊 token / 离散 bin**（把坐标量化成 bin 分类）——精度可控；③ **专用 head**（接一个检测头输出框/mask）——精度最高，但破坏了"纯语言模型"的统一性。

趋势：主流通用模型用 ①，专门的方向（指代分割、GUI agent）用 ②③。

A29 ★ 视频理解比图像多出来的难点是什么？（三条）

A：① **token 爆炸**（帧数 × 每帧 token）；② **时序建模**（帧间关系、事件先后、因果）；③ **长视频检索**（1 小时视频不可能全塞，必须"摘要 + 检索"）。附加难题：视频数据标注昂贵、评测集难做（时序问答的答案歧义大）。

A30 ★★ 为什么说"VLM 的数学推理很多时候是在做语言推理"？

A：因为它可以从题目文本推出答案（哪怕图没看对）。诊断方法很直接：**把图像遮挡/替换成无关图像，看答案是否改变**——不变就说明它没用图。这也是 HallusionBench 这类"视觉依赖测试"的设计思想。

A4 评测体系（A31–A38）

A31 ★★★ 主流评测集各自测什么？给一张能背的表。

评测集	测什么	需要记住的细节
MMBench	综合能力（20 个能力维度）	**CircularEval**：选项循环移位，全对才算对，消除位置偏置
MMMU	大学水平学科推理（图、表、公式、考试题）	多学科多图，考专业知识，人类专家也只有 ~80%
SEED-Bench	生成式理解（图+视频）	多项选择，人工校验过
POPE	**物体幻觉**	random / popular / **adversarial** 三设置；报 F1 和 yes 比例
HallusionBench	幻觉（视觉依赖 + 反事实）	专测"看图 vs 靠先验编"
MM-Vet	综合（识别+推理+OCR 交叉）	GPT-4 打分，看能力维度组合
RefCOCO/+/g	指代定位 grounding	Acc@0.5（IoU>0.5）或点命中率
TextVQA / DocVQA / ChartQA / OCRBench	OCR 与文档	工业落地最常看这四个
MathVista	数学视觉推理	图表+几何+公式
VideoMME / LongVideoBench	视频（短/中/长，有无字幕）	必须**分时长档**报，长视频普遍崩

A32 ★★★ CircularEval 是什么？为什么必须有？

A：把一道多选题的选项**循环移位 N 次**（N = 选项数），每次让模型作答，**N 次全对才算答对**。

原因：VLM 对选项位置有强烈偏置（训练数据里 A/B 出现频率高、或 token 化后某些位置的概率先验高），不移位会系统性高估能力——有研究报告这个偏差能贡献 10+ 个点的虚高。

面试意义：讲得出 CircularEval，等于告诉面试官"我读过评测实现，不只看排行榜数字"。

A33 ★★ POPE 的三个设置为什么要区分？

A：因为负样本的难度决定评测的可信度：

- **random**：随机挑不存在物体 → 太简单，模型靠"这词常见"就能答对；
- **popular**：挑数据集中高频但图里没有的物体 → 中等；
- **adversarial**：挑**与图中物体共现频率高**的物体（有椅子就问"有桌子吗"）→ 真考验"是否在看图"。

只看 random 分数会严重虚高，三个设置一起报才有意义。

A34 ★★ 评测里最容易自欺的四个地方？

A：① **数据泄漏**（训练图和评测图近重复，CLIP 系预训练数据几乎覆盖全网）；② **prompt 敏感**（换模板差几个点，不报 prompt 就是不完整）；③ **选项偏置**（不做 CircularEval）；④ **只报总分**（不报分项，OCR 强推理弱看不出来）。

加分补一句：还要报 **解码配置**（贪心/beam/温度）和**是否 few-shot**——这些不写，结果不可复现。

A35 ★★ 幻觉怎么量化？除了 POPE 还有什么办法？

A：四类方法：

1. **判别式探测**（POPE）：问"有没有 X"，看 yes/no 是否正确 → 简单、可自动化；
2. **生成式一致性**：同一张图问多次/换问法，答案不一致即为不稳定（self-consistency 视角）；
3. **视觉依赖测试**：把图替换成空白/无关图，答案不该变（该变才说明用了图）；
4. **人工/强模型打分**：MM-Vet 式，成本高但是最终裁判。

A36 ★★ grounding 的评测为什么比检测的 mAP 更"宽容"?

A：因为 grounding 的目标是"**指代唯一性**"而不是"检测全面性"：输入是一句描述，只要求找到**那个**目标，不惩罚漏检其他物体。指标常用 Acc@0.5 或点命中（点落在 GT mask 内），后者更宽容（不要求框贴合）。

难点在指代歧义：图里有三只狗，"那只左边的狗"需要理解关系，而不是靠类别先验。

A37 ★ 长视频评测为什么必须分档报?

A：短视频（<2 分钟）靠帧采样就能覆盖，模型表现接近图像；中长视频开始依赖时序推理与检索，性能断崖式下降。**只报平均分会掩盖这个断崖**——所以 VideoMME 明确分短/中/长三档，并区分有无字幕（有字幕时模型可能"读字幕"而不看画面，这也是要看分项的原因）。

A38 ★★ 怎么评测"模型有没有真在看图"？设计一个实验。

A：三个可落地实验：

1. **图像替换**：换成同类无关图 → 答案应显著改变（不变 = 没用图）。
2. **局部遮挡**：遮住关键区域 → 答案应变差（不变 = 没依赖该区域）。
3. **反事实图像**：把"红色苹果"改成蓝色 → 答案应跟着变（不变 = 靠先验）。

这三个也是 HallusionBench 的核心思想。**面试时能讲出实验设计，比背分数有用。**

A5 推理与部署（A39–A45）

A39 ★★★ VLM 推理的延迟构成？哪一段是瓶颈?

A：四段：① **预处理**（resize/切块/归一化，CPU 上常被忽略但很贵）；② **视觉塔前向**（与分辨率平方相关）；③ **prefill**（把图文 token 一次前向，计算密集）；④ **decode**（逐 token，访存密集，与输出长度线性）。

瓶颈判断： - 高分辨率 + 短回答 → 卡在 ②③（视觉塔 + prefill）； - 低分辨率 + 长回答 → 卡在 ④（decode，KV cache 带宽）。

这句话很值分："端侧 VLM 要先分清是 prefill-bound 还是 decode-bound，两者的优化手段完全不同。"

A40 ★★ VLM 的 KV cache 和纯文本 LLM 有什么不同?

A：机制一样（缓存每层 K/V），但 VLM 有两处特殊：

1. **视觉 token 的 KV 是固定的**：同一张图多轮问答时，视觉部分的 KV 可以**跨轮复用**（prefix cache 思路）——这是多模态特有的优化收益。
2. **视觉 token 数量大且不等于文本长度**：一张 anyres 图可能 2880 token，KV cache 直接翻几倍，长多轮时显存爆得比纯文本快得多。

A41 ★★ 图像 token 太多，有哪些工程压缩手段?

A：五类：① 视觉侧压缩（pixel shuffle、池化）；② 桥接层压缩（Q-Former/resampler）；③ token 剪枝（按注意力重要性保留 top-k）；④ 动态分辨率（按任务给 token）；⑤ 缓存复用（同一图的多轮只算一次）。

面试提醒：压缩一定损精度，所以要说清"在哪一层压、损失什么"——通道压缩损细节，序列压缩损定位，两者可叠加但要注意叠加放大误差。

A42 ★★ VLM 量化有什么特别的坑?

A：三个：① **视觉塔对量化更敏感**（输入分布极端、动态范围大），常见做法是视觉塔保 FP16、只量化 LLM；② **视觉 token 的离群值**（ViT 输出常有较大激活）会破坏 INT8，需要 per-channel 或对视觉通路单独处理（类似 AWQ 保护重要通道）；③ **KV cache 量化**在多轮/长上下文里收益最大，但要单独校准。

A43 ★ 投机解码能用在 VLM 上吗?

A：能，但草稿模型的输出分布要覆盖**图文场景**——用纯文本小模型做草稿，遇到需要"看细节"的问题命中率会掉。实践中：① 用同系列的小 VLM 作草稿；② 或者只在"回答格式固定/重复性高"的场景（如结构化抽取）用。

A44 ★★ 端侧 VLM 的现实约束是什么?

A：四条硬约束：① **内存**（权重 + KV cache + 激活，手机级别只有几 GB 可用）；② **算子支持**（动态形状、动态分辨率、attention 变体在 NPU 上常常要改）；③ **功耗/发热**（长时间推理会降频）；④ **预处理开销**（切块/resize 在 CPU 上可能比模型还慢）。

你的接口（可直接讲）："我在 MDC610 上做过 ONNX→ATC/AOE 的完整链路，所以端侧 VLM 我会先问三件事——**目标 SoC 的算子白名单、内存预算、是否允许动态分辨率**。"

A45 ★ 昇腾/国产芯片上部署 VLM，主要工作量在哪?

A：四块：① **算子适配**（不支持的算子拆解或等价重写，例如动态 attention、top-k、动态 shape）；② **shape 固定化**（把动态分辨率按档位枚举，分桶推理）；③ **精度对齐**（fp16 溢出、算子融合顺序导致的数值差异，要逐层比对）；④ **内存规划**（模型段/工作空间手工分配，KVCache 动态增长需要预分配）。

A6 VLM 的损失函数与评测指标（A46–A57）

> 一句话前提：**VLM 的损失 90% 就是"文本侧的下一个 token 交叉熵"**，其他都是辅助。指标则完全分任务走。

A46 ★★★ VLM 训练里最主要的损失是什么？为什么不是 MSE?

A：**自回归的 token 级交叉熵（next-token CE / NLL）**：

```
L = −(1/T) Σ_t log p_θ( y_t | y_<t , 视觉token , 指令 )
```

为什么不用 MSE：1. 语言是**离散分布**，MSE 假设高斯连续误差，与概率建模不匹配（见 B13/B97）； 2. CE 对"自信但错"的惩罚趋于无穷，天然抑制过度自信； 3. 生成任务的目标是**分布匹配**（每个位置的概率分布），不是回归某个数。

注意：图像**生成**（扩散）才用 MSE（噪声预测），这与 VLM 的文本生成是两套目标（A55）。

A47 ★★★ 对齐阶段（stage-1）常见的损失组合有哪些?

A：四种流派，取决于模型结构：

流派	损失组合	代表
生成式对齐	next-token CE（冻结 LLM，只训 projector）	LLaVA 两阶段的第一阶段
对比 + 匹配 + 生成	ITC（InfoNCE）+ ITM（BCE）+ ITG（CE/MLM）	BLIP、ALBEF
纯对比（双塔）	对称 InfoNCE（或 SigLIP 的 pairwise sigmoid）	CLIP、ALIGN、SigLIP
多任务 query 对齐	ITC + ITM + ITG，Q-Former 输出接冻结 LLM 做 LM loss	BLIP-2 两阶段

关键差别：**对比损失（ITC）学"全局是否匹配"，生成损失（CE）学"细节怎么描述"**。BLIP/ALBEF 两者都要，是为了兼得；LLaVA 只保留生成，是因为它有海量指令数据 + 强视觉塔，不需要靠对比来逼对齐。

A48 ★★ 为什么 ITC 用 InfoNCE 而不是 MSE?

A：三层原因：

1. **目标不同**：ITC 要的是"**相对排序**"——同一 batch 里，正确配对的相似度要高于其他配对。InfoNCE 是 softmax 分类（正确配对是标签），天然表达"区分度"；MSE 只要求"数值接近"，学不出"相对更近"。
2. **梯度行为不同**：InfoNCE 的梯度是 `p − y`，在"把错误配对排到前面"时梯度大；MSE 在大 batch 的 B×B 矩阵上会被大量"本来就对的负样本对"稀释。

3. **信息论视角**：InfoNCE 是**互信息的下界**（这也是"多模态对齐 = 最大化互信息"这个说法的来源）。

追问：负样本从哪来？→ **in-batch negatives**（batch 内其他样本的图/文），所以大 batch 是 CLIP 的命门；SigLIP 改成 pairwise sigmoid 后就不再需要全 batch 归一化。

A49 ★★ ITM（图文匹配）为什么必须有负样本？怎么挖 hard negative？

A：ITM 是二分类（匹配/不匹配），**没有负样本就无法训练**。

hard negative 的挖法（ALBEF/BLIP 的做法）： - 用 ITC 算 batch 内相似度矩阵，**取"相似度高但不是正确配对"的样本**作为 hard negative（如把 A 的图配 B 的相似文本）； - 这些样本在语义上接近、但确实不匹配，最能教会模型细粒度区分。

为什么需要 hard negative：随机负样本太容易（"猫的图 + 汽车的描述"），模型学不到细粒度；只有"猫的图 + 狗的相似描述"才能逼模型去看细节。

A50 ★★★ SFT 阶段的 loss mask 怎么做？为什么只对 assistant 算？

A：做法：把序列中 user 的提问、system prompt、以及（多轮里的）历史 assistant 回复都贴 label = `-100`（PyTorch 的 `ignore_index`），只有**当前轮的 assistant 回复参与 loss**。

三个原因： 1. **目标任务明确**：我们训练的是"看到指令后如何回答"，不是"复述指令"。 2. **避免模型学会"自问自答"**：如果对 user 也算 loss，模型会倾向于生成用户的问题（尤其在多轮里出现"接着提问"的退化）。 3. **梯度更集中**：user 部分通常比回答长（尤其检索增强的长上下文），不过滤会浪费大量算力。

工程细节：多轮对话里，历史 assistant 回答**通常也 mask 掉**（只算最后一轮），否则模型会被"重复自己说过的话"带偏；但有些实现会算全部轮次（相当于更多训练信号），两种都有人用——面试时能说出这个取舍很加分。

A51 ★★ RLHF 的两个损失分别是什么？

A：两段：

① 奖励模型的 Bradley-Terry 损失（给偏好对 chosen / rejected）：

```
L_RM = −log σ( r(x, y_w) − r(x, y_l) )
```

② PPO 的 clip 目标（策略优化）：

```
ratio = π_θ(a|s) / π_old(a|s)
L_clip = −E[ min( ratio · A ,  clip(ratio, 1−ε, 1+ε) · A ) ]
总奖励 R = r_RM − β · KL(π_θ ‖ π_ref)
```

三个关键设计： - **clip** 限制单步更新幅度（防止策略崩）； - **KL 惩罚** 防止模型为了拿高分而跑偏成"奇怪的表达"； - **GAE**（广义优势估计）把稀疏奖励分摊到每个 token。

多模态的特殊性：RM 本身要是个 VLM（得看图），所以 RLHF 在多模态里成本约为纯文本的 2~3 倍。

A52 ★★ DPO 损失是怎么来的？实现要点？

A：DPO 把"RM + RL"合并成一个分类式损失，**直接用偏好对优化策略**：

```
L_DPO = −log σ( β · [ log(π_θ(y_w|x)/π_ref(y_w|x)) − log(π_θ(y_l|x)/π_ref(y_l|x)) ] )
```

推导要点（能说清这段很值钱）：在 KL 约束下最优策略有闭式解 `π* ∝ π_ref · exp(r/β)`，反解出 `r = β · log(π*/π_ref) + const`；把它代回 Bradley-Terry 损失，就得到上式——**奖励被"吸收"进策略里了**，所以不需要单独训 RM。

实现要点： - 需要**两个前向**（policy 和 ref）算 4 个 logprob（chosen/rejected × policy/ref）； - `β` 常取 0.1~0.5，越大越保守； - 用 **logprob 之和**而不是平均（长度归一化方式会影响效果，有论文专门讨论）。

与 PPO 的取舍：DPO 便宜稳定但离线（无探索），PPO 能在线探索但工程复杂、需要 4 个模型。

A53 ★ GRPO 与 PPO 在损失层面的差别？

A：GRPO **去掉了 value network（critic）**：对同一个 prompt 采样一组（G 个）输出，用**组内相对优势**当基线：

```
A_i = (r_i − mean(r_1..r_G)) / std(r_1..r_G)
L = − E[ min(ratio · A_i, clip(ratio,1−ε,1+ε) · A_i) ] + β · KL
```

好处：省掉 critic 网络（显存和调参都省一大块），且组内归一化自带方差缩减。**代价**：需要同一 prompt 多次采样（推理成本上升），组太小或奖励方差为零时信号很弱。

A54 ★ 带任务头时（grounding / 分割 / OCR）损失怎么加?

A：分两种情况：

① 纯文本输出坐标（主流通用 VLM）：把坐标当字符串生成 → **还是 next-token CE**，但要注意坐标数值被 tokenizer 切成多个 token，导致精度受限（见 A28）。可以加**坐标回归辅助头**补精度。

② 专用头（分割/检测）：

任务	损失	注意
检测/grounding	分类 focal + 框 SmoothL1 + GIoU/DIoU	与 S9 一致；框回归要归一化
指代分割	BCE（主）+ Dice（辅）	正样本极少，用 focal 或权重
OCR / 转录	token 级 CE	长文本要注意长度归一化
密集预测（深度/占用）	任务专用（见 S9/C8）	与文本损失量纲差几个数量级**必须归一化**

关键工程点：文本 CE 的量级通常是 1~3，而框回归/深度损失常是 0.01~0.1，**直接相加会让文本损失主导**——必须做量纲归一化（S9 Q111）。

A55 ★ 图像生成侧的损失和 VLM 的文本损失有什么本质区别?

A：

	VLM 文本生成	扩散图像生成
目标	离散 token 的**概率分布**	连续噪声的**回归**
损失	交叉熵（逐 token）	**MSE**（预测噪声 ε 或 v 或速度场）
输出	每步一个 token 的分类分布	每步一个连续张量
采样	逐 token 自回归	多步去噪（DDIM 20~50 步）

统一视角：两者都是"从噪声/前缀恢复数据"，但 VLM 在**离散序列空间**做分类，扩散在**连续空间**做回归。新一代"统一模型"（Chameleon/Emu3 用离散 token、Transfusion 用双头）就是在试图把这两套目标合到一起——这也是当前的开放问题（A53）。

A56 ★★★ VLM 的评测指标分哪几类？各举必背的

A：

类别	指标	说明
综合理解	MMBench / MMMU / MM-Vet / SEED 的**准确率**	多为多项选择 → acc；MM-Vet 用 GPT-4 打分
VQA	VQA accuracy（`min(#同答/3, 1)`）	VQA v2 的老口径，10 个标注者投票
幻觉	POPE 的 **Accuracy / F1 / yes 比例**；HallusionBench 的 acc	见 A33
定位	RefCOCO 的 **Acc@0.5** 或点命中率	见 A36
OCR/文档	**ANLS**（DocVQA）、精确匹配（OCRBench）	ANLS 对编辑距离做归一化，比精确匹配宽容
生成（描述）	CIDEr / BLEU / METEOR	与人类判断相关性弱，现在多用 GPT 打分
生成（图像）	FID / IS / CLIPScore	见 C49
检索	Recall@K / mAP	双塔模型的看家指标
效率	TTFT（首 token 延迟）、TPS（每秒 token）、峰值显存	落地必看

必背的三条"口径提醒"：① MMBench 必须 CircularEval；② POPE 必须报三设置的 adversarial；③ 生成类指标（CIDEr/FID）与人类判断弱相关。

A57 ★★ 为什么"损失降、指标不涨"在 VLM 上特别常见?

A：四个 VLM 特有的原因：

1. **损失是 token 级，指标是回答级**：一个回答 50 个 token，错 1 个 token（比如数字写错）loss 只升一点，但整个答案判错 → **准确率是"全对才算对"，比 loss 严格得多**。
2. **格式比内容更容易被 loss 奖励**：模型学会"先复述问题再答""用 markdown 列表"这类高概率模式就能显著降 loss，但指标不变。
3. **训练分布与评测分布不同**：SFT 数据的问法、长度、语言与评测集差异大（评测集多是短问答 + 多选）。
4. **有截断/格式不匹配**：评测要抽取答案，格式不对就判错（明明答对了）。

诊断手段：把 loss 拆成"格式 token（标点、连接词）"和"内容 token（实体、数字）"分别看——如果降的是格式 token，那指标不涨就是合理的。**这个做法很实用，面试可以直接讲。**

A7 Encoder / Decoder 的辨析（A58–A65）

> 这个词在一句话里可能有三种含义，**先问清是哪一种**，否则答得越多越偏。

A58 ★★★ 先分清三种 "encode/decode"

A：

语境	encode	decode	例子
Transformer 架构	双向注意力，输出表征	因果注意力，输出下一 token	BERT vs GPT
VLM 角色	视觉编码器 / 文本编码器（理解）	语言模型（生成）	ViT + LLaMA
工程压缩	把数据压成潜变量	把潜变量还原	VAE encoder/decoder、视频编解码
序列化	字符串 → token id	token id → 字符串	tokenizer

面试时第一句就说："您指的是 Transformer 的 encoder/decoder，还是 VLM 里视觉编码器 vs 语言模型？"——**这一问就能避免 80% 的误解**。

A59 ★★★ Transformer 里 encoder 和 decoder 的本质区别是什么?

A：**一处核心差异 + 三处衍生**：

```
核心：注意力 mask
   Encoder：全注意力（每个位置看所有位置）          → 双向上下文
   Decoder：因果 mask（位置 i 只看 ⩽ i）           → 单向上下文（自回归）
```

	Encoder	Decoder
注意力	双向（无 mask）	因果 mask（下三角）
输出用途	**表征**（给下游分类/检索/匹配）	**下一 token 的分布**（生成）
训练目标	MLM / 对比 / 回归	next-token CE
训练并行	可并行	**也可并行**（见 A60）
推理	一次前向	必须串行（每步依赖前一步输出）

一个常考细节：encoder 里通常**没有** cross-attention，decoder 里**可以有** cross-attention（看 encoder 的输出，如 T5/BART）；decoder-only 模型（GPT/LLaMA）则完全没有 cross-attention 层。

A60 ★★★ 为什么 decoder 训练能并行、推理必须串行?

A：

训练能并行：靠 **teacher forcing + causal mask**。把完整目标序列一次性喂进去，mask 保证位置 i 只能看到 ≤ i 的真实 token，所以**所有位置的预测可以同时算**（一次前向得到 T 个位置的 loss）。

推理必须串行：因为位置 t 的输入是模型自己刚生成的 t−1 个 token，**没有 GT 可喂**，只能逐步生成。

KV cache 的意义：串行时每步都要重算全部历史，代价 $O(L^2)$；缓存历史 K/V 后每步只算 1 个新 token 的 K/V + 与全部历史的注意力，**每步 O(L)**，总 $O(L^2)$ 但常数小得多。

一句话："训练时把已知答案当输入（可以并行），推理时答案要自己产生（只能串行）——**这就是为什么大模型推理是 memory-bound**。"

A61 ★★ encoder-decoder 架构（T5 / BART）和 decoder-only 有什么不同?

A：

	encoder-decoder	decoder-only
结构	encoder（双向）+ decoder（因果 + cross-attention）	单塔因果
典型	T5、BART、Whisper、部分多模态（BLIP）	GPT、LLaMA、LLaVA
优势	输入理解更深（双向）；输入输出分离清晰	结构简单、权重可复用、few-shot 能力好
劣势	参数与工程更复杂；难做"输入输出同分布"的任务	输入也走单向，理论上理解力弱一点

关键差别在"输入能否双向编码"：encoder-decoder 让输入被完整双向理解后再生成；decoder-only 的输入 token 也只能看到自己之前的 token（虽然对生成式任务影响不大，但对"精确检索长上下文"这类任务理论上吃亏）。

A62 ★★★ 在 VLM 里谁是 encoder、谁是 decoder?

A：

```
视觉侧：ViT/SigLIP  →  encoder（双向，输出 patch token）
桥接：  projector / Q-Former → 既不是典型 encoder 也不是 decoder（对齐模块）
语言侧：LLM → 主流是 decoder-only（因果）
```

所以一个典型 VLM（LLaVA 类）是 **"encoder(视觉) + decoder-only(语言)"** 的异构组合，**不是 encoder-decoder 架构**。这一点很多人会说错。

例外：BLIP 用的是 encoder-decoder（BERT 式文本编码器 + 解码器）；CLIP 是**双 encoder**（两个都是 encoder，都没有 decoder）。

A63 ★★★ 为什么主流 VLM 选 decoder-only?

A：五个理由：

1. **生态与权重复用**：直接拿现成 LLM（LLaMA/Qwen）当底座，省掉从零训。
2. **任务统一**：所有任务都变成"下一个 token 预测"——视觉问答、描述、grounding（输出坐标字符串）都是同一个接口，不需要为每个任务设计 head 和 loss。
3. **数据效率**：指令数据天然是"对话续写"的形式，decoder-only 的 SFT 非常直接（A50 的 mask 策略）。
4. **few-shot / in-context 能力强**：这是 decoder-only 的固有优势（自回归建模带来的分布内插）。
5. **推理框架成熟**：KV cache、continuous batching、投机解码都是围绕 decoder-only 成熟的。

代价（见 A64）：视觉与文本的交互是**单向**的（视觉 token 只被文本"看到"，视觉不反向查询文本）；另外多图/长文档时，输入侧也只能单向处理。

A64 ★★ decoder-only 做 VLM 有什么代价?

A：四条：

1. **单向融合**：视觉 token 无法"回看"文本（不像 cross-attention 可以让视觉查询文本），所以**细粒度指代**（"第三个按钮"）依赖 LLM 自己学会用位置信息。
2. **输入不能双向编码**：长文档/多图场景下，前面的信息只能通过单向注意力传下去。

3. **token 预算紧张**：视觉 token 和文本 token 共享同一个上下文窗口（decoder-only 没有独立的 encoder 侧），所以 token 压缩格外重要。
4. **推理延迟**：图像 token 全在 prefill 里，长图会显著抬高 TTFT（首 token 延迟）。

取舍结论：这些代价被"生态 + 数据效率"的优势覆盖，所以工业上**几乎没有例外地选 decoder-only**；需要双向融合时，用 **Q-Former / 双向编码的视觉塔 / 专门的 cross-attention 层（Flamingo）**来补。

A65 ★ encoder 的双向注意力对多模态融合有什么优势？（开放题）

A：三条优势 + 一条代价：

优势： 1. **区域与词的细粒度对应更强**：双向注意力让每个图像 patch 都能"看到"全部文本，从而学到"这个词对应这块区域"——这是 ALBEF/BLIP 里 ITM 头的能力来源。 2. **检索/匹配更准**：双向编码的融合表征适合"判断是否匹配"，比单向拼接更强。 3. **适合做** grounding/指代这类"输入侧就要求对齐"的任务。

代价：不能直接生成自由文本（需要解码器）。

所以在工程上的常见组合是：双向 encoder 做融合/对齐（用于检索、匹配、定位），decoder-only 做生成（用于问答、对话）——这也是"双塔 + 生成塔"混合模型（如 CoCa 的对比 + caption 双头）的思路。

A8 LoRA 与 PEFT 微调（A66–A77）

> LoRA 是大模型微调的**事实标准**，也是面试的高频点。这一节把"为什么有效、省了什么、怎么选参、怎么避坑"讲透，并给出一套能直接报出来的数字。

A66 ★★★ LoRA 的数学形式是什么？为什么它有效？

A：冻结原权重 $W \in R^{d_{out}\times d_{in}}$，只学一个低秩增量：

$$W' = W + \Delta W = W + B \cdot A \qquad A \in R^{r\times d_{in}},\ B \in R^{d_{out}\times r},\ r \ll \min(d_{in}, d_{out})$$

$$\text{前向：}\ h = Wx + (\alpha/r)\cdot B \cdot A \cdot x$$

有效的原因（三层）： 1. **微调的增量本身就是低秩的**：已有研究（Aghajanyan 等）表明，预训练模型适配下游任务所需的"内在维度"远小于参数量——**ΔW 的秩很低**，所以低秩近似几乎不损失信息。 2. **不破坏预训练知识**：W 完全冻结，只是加一个"旁路修正"，天然避免了灾难性遗忘。 3. **优化更简单**：可训练参数少 2~3 个数量级，优化器状态和梯度都小，等效于强正则。

一句话："LoRA 不是压缩，而是**假设微调只需要在少数方向上修正**。"

A67 ★★★ A 和 B 为什么要这样初始化？α/r 是干什么的？

A：

初始化：**A 用高斯随机、B 全零** → 训练开始时 $\Delta W = B \cdot A = 0$，模型的输出与原始模型**完全一致**。

为什么必须这样：如果 A、B 都随机，初始的 ΔW 就是一个随机扰动，相当于给已经训好的模型"注入噪声"，训练早期会出现 loss 尖峰甚至破坏原有能力。**这和我们做多任务时用"零初始化残余分支"保安全的思路是同一件事**（恒等初始化 → 开新分支不影响原行为）。

α/r 的作用：缩放因子。`alpha` 是"学习率的另一种表达"——

$$h = Wx + (\alpha/r)\cdot BAx$$

- 固定 α 而增大 r 时，α/r 变小，抵消了"参数变多导致的有效步长变大"，**所以改 r 时不必大改学习率**；
- 实践惯例：**α = r 或 α = 2r**（如 r=16、α=32）；rsLoRA 提出改用 $\alpha/\sqrt{r}$，让不同 rank 下的更新尺度更一致。

附：r 与 α 到底各管什么（实测验证）

分工一句话：r 管"能改多少"（容量），α 管"改得多用力"（幅度），两者一起决定有效更新量。

① r = 秩 = 容量的上限（$\Delta W = BA$ 的秩最多是 r）

r	参数量 r(d_in+d_out)（d=256 示例）	相对原始权重	实测 rank(BA)
1	512	0.78%	1
4	2,048	3.12%	4
16	8,192	12.50%	**16**
64	32,768	50.00%	64

含义：**r 就是"允许模型在多少个方向上修正原权重"**。r 太小 → 装不下任务需要的改动（欠拟合）；r 太大 → 参数量涨、过拟合风险上升、adapter 文件变大。

② ΔW 的幅度随 √r 增长（这才是 α 存在的理由）

r	‖BA‖_F	‖BA‖_F / √r	乘 α/r（α=32）后	乘 α/√r 后
4	0.0510	0.0255	0.4080	0.8160
16	0.1026	0.0257	0.2053	0.8211
64	0.2050	0.0256	0.1025	0.8200
256	0.4092	0.0256	0.0512	0.8185

读三件事： 1. ‖BA‖_F / √r 几乎恒定（0.0256）→ **同样初始化下，r 越大 LoRA 分支的输出越"响"**（幅度 ∝ √r）； 2. 用 **α/r** 缩放时，r 越大有效幅度越小（∝ 1/√r）→ **大 rank 会学不动**，这是 α/r 的缺陷； 3. 用 **α/√r** 缩放时幅度与 r 无关（0.82 恒定）→ **这正是 rsLoRA 的动机**。

③ r=16、α=32 具体意味着什么

```
α/r = 32/16 = 2      → LoRA 分支的输出被放大 2 倍后加到原输出上
可训练参数（7B、全线性层）= 39.98M = 0.57% 的模型参数
adapter 文件（fp16）≈ 80 MB
```

- 缩放 = 2 是"略微主动"的设置（α=r 时缩放为 1，更保守；α=4r 则更激进）；
- 这个组合是社区最常见的默认值之一（LLaMA-Factory / PEFT 示例里大量出现），**但它不是普适最优**——任务差异大时该往上调 rank。

④ 三条实践规则 1. **改 r 就按比例改 α**（保持 α/r 不变），否则学习率的实际效果会意外变化； 2. **α 太小**（缩放 < 0.5）→ 学不动、loss 平；**α 太大**（缩放 > 4）→ 训练初期输出被扰动，loss 尖峰甚至破坏原模型（本质是"学习率太大"）； 3. rank 开到 **64 以上**时，改用 α/√r（rsLoRA 式）避免信号衰减。

A68 ★★★ LoRA 到底省了什么显存？（算一遍账）

A：以 **LLaMA-7B** 为例（hidden 4096、FFN 11008、32 层）：

配置	可训练参数	占 7B	LoRA 侧显存（bf16 参数+梯度+Adam）
全参微调	7B	100%	权重 14GB + 梯度 14GB + 优化器 **84GB** = **约 112GB**（不含激活）
LoRA r=8，全部线性层	19.99M	**0.29%**	约 0.28 GB
LoRA r=16，全部线性层	39.98M	0.57%	约 0.56 GB
LoRA r=32，全部线性层	79.95M	1.14%	约 1.12 GB
LoRA r=16，**仅 q/v**	12.58M	0.18%	约 0.18 GB

关键认知（面试常被追问）： - LoRA 省掉的是 **① 梯度 ② 优化器状态（Adam 的 m/v/主权重 = 12 字节/参数）**； - **不省** ① 冻结权重本身（7B 的 bf16 仍占 14GB）、② **激活**（随 batch×序列长度增长，长上下文仍是 OOM 主因）。 - 所以"LoRA 之后还 OOM → 该上梯度检查点 / FlashAttention / 减小序列，而不是调 rank"。

adapter 文件大小：r=16 全线性层约 **80MB**（fp16），这就是你能随手分享、多任务切换的物理基础。

A69 ★★★ rank 和 target modules 怎么选?

A:

rank：- 起步 **8~16**，实测不够再翻倍（16→32→64）；- **判断够不够**：训 loss 与目标任务指标在更大 rank 下**不再改善** = 已饱和；- 规律：**任务越"远离"预训练分布（新语言、新领域、代码/数学），需要越大 rank**；纯格式/风格适配用小 rank 即可。

target modules（加到哪些层）：- **最小可用**：`q_proj, v_proj`（原论文的做法）；- **推荐**：`q, k, v, o` + **MLP 的 gate/up/down**（全线性层）——实测通常比只加 q/v 好 1~3 个点，参数只多 3 倍（相对 7B 仍是 0.5% 量级）；- **注意**：不同框架的层名不一样（LLaMA 系是 `q_proj/k_proj/v_proj/o_proj/gate_proj/up_proj/down_proj`，Qwen 系可能是 `c_attn/w1/w2`），配错名字会静默地"什么都没训"——**必须打印可训练参数数量校验**。

A70 ★★ 训练完怎么合并？有哪些坑?

A:

```
推理前合并：  W_merged = W + (α/r)·B·A      ← 合并后无任何额外延迟
不合并（在线）：h = Wx + (α/r)·BAx          ← 多一次小矩阵乘，约 1~3% 延迟
```

四个坑：1. **精度损失**：在 fp16 下 $W + \Delta W$ 会因舍入丢信息，尤其 ΔW 很小时。**做法：把 W 升到 fp32 合并后再转回 bf16/fp16**。2. **合并后不可逆**：想换 adapter 就得保留原始权重（PEFT 的做法是保存 adapter + 基座指针）。3. **多 adapter 不能直接相加**：两个在不同任务上训的 ΔW 相加，会得到"什么都会一点、什么都不精"的结果；正确做法是**切换**（加载不同 adapter）或重新联合训练。4. **量化基座上的合并**：QLoRA 训完若直接合回 4bit，精度掉得厉害；**标准做法是先在 fp16 合并，再重新量化**（或干脆不合并、推理时动态加）。

A71 ★★★ QLoRA 的三个关键技术是什么?

A:

1. **NF4（4-bit NormalFloat）**：一种**按分位数**设计的 4bit 数据类型，对"近似正态分布"的权重是最优的信息论编码——比普通 INT4 精度更高。
2. **双重量化（Double Quantization）**：量化本身需要"量化常数"（每个 block 一个 scale），这些常数也占空间；**把量化常数再量化一次**，平均每参数省约 0.37 bit（65B 模型约省 3GB）。
3. **分页优化器（Paged Optimizers）**：用 NVIDIA 的统一内存，把优化器状态在显存/内存间按页换入换出，**避免长序列时突然 OOM**。

效果：论文在**单张 48GB GPU 上微调 65B 模型**。7B 的 4bit 基座约 **3.5GB**，加 LoRA 与激活，单张 24GB 消费级卡就能训。

代价：① 训练速度变慢（反量化开销）；② 4bit 有精度损失（通常比 LoRA 略差）；③ 合并麻烦（见 A70 第 4 条）。

A72 ★★ DoRA / rsLoRA / LoRA+ 各改了什么?

A:

变体	改了什么	直觉
DoRA（Weight-Decomposed）	把权重分解成 **幅度 m × 方向 V/‖V‖**，LoRA 只训方向、幅度单独训	微调主要改"方向"，改幅度反而破坏知识 → 小 rank 下明显更稳、更准
rsLoRA（rank-stabilized）	缩放改成 **$\alpha/\sqrt{r}$**	大 rank 时 α/r 衰减太快导致学不动，$\sqrt{r}$ 让不同 rank 的更新尺度一致
LoRA+	**A 和 B 用不同学习率**（B 用大 lr，如 16×）	B 初始为零、A 是随机，两者梯度量级不同，同步 lr 会浪费一半
AdaLoRA	用 SVD 形式**自适应分配 rank**（重要的层给大 rank）	层间重要性不同
PiSSA	用**权重的 SVD 主成分**初始化 A/B	从"最重要的方向"开始学，收敛更快

面试建议：默认用 LoRA；小 rank + 要更好效果时试 DoRA；rank 开大（>64）时用 rsLoRA。

A73 ★★ LoRA 和其他 PEFT 方法怎么对比？

A：

方法	做法	推理延迟	效果	备注
LoRA	低秩旁路 ΔW	**可合并为零**	强	主流
Adapter	插入小 MLP 模块（串行）	**有**（除非剪枝）	强	破坏并行、层数变深
Prefix / P-Tuning	学一段虚拟 token 前缀	占上下文	中（对生成任务偏弱）	省参数但不是"改权重"
IA3	学三个缩放向量（k/v/FFN）	可合并	中	参数极少（万级）
BitFit	只训 bias	零	中偏弱	极简基线
全参微调	训全部权重	零	**上限最高**	显存 ~16× 参数

一句话："LoRA 赢在**无推理延迟 + 效果接近全参 + adapter 可切换**这三点的组合，其他方法都缺一项。"

A74 ★★★ VLM 微调时 LoRA 加在哪里？

A：三个部位，优先级不同：

部位	是否加 LoRA	理由
projector / 桥接层	**通常全参训练**（不加 LoRA）	它本来就是新加的、参数量只有几 M，加 LoRA 反而限制了学习
LLM	**加 LoRA**（q/k/v/o + MLP）	主力，参数量占 99%
视觉塔	**默认冻结**；领域差异大时加 LoRA（只加最后几层）	见 A22；加 LoRA 在视觉塔上时通常用更小的 rank（4~8）

典型配置（可直接讲）：

```
视觉塔：冻结（或最后 2 个 block 加 rank=4 的 LoRA）
projector：全参
LLM：LoRA r=16, alpha=32, target=q,k,v,o,gate,up,down
学习率：LLM 的 LoRA 用 1e-4 ~ 2e-4（比全参微调大 10 倍，因为可训参数少）
```

一个坑：VLM 里如果只给 LLM 加 LoRA、projector 也冻结，模型几乎学不动（projector 是随机初始化的话更糟）——**projector 必须可训**。

A75 ★★ 多任务多 adapter 怎么管理？切换代价是什么？

A：

方式：1. **每个任务一个 adapter**（共享同一基座）：存储省（每个几十 MB）、切换快；2. **一个 adapter 混训多任务**：更省显存、但任务间会互相干扰（小 rank 下尤其明显）；3. **MoE-LoRA**（每个任务一个 LoRA 专家 + 路由）：灵活但要额外训练路由。

切换代价：加载 adapter 需要把 BA 加进（或替换掉）当前权重——**如果合并过就必须重载基座**（代价大）；**如果保持分离，只需换 adapter 指针**（毫秒级，但推理时有小量额外算力）。

实践建议：训练时保持分离（方便切换），部署时对每个任务分别合并（拿零延迟）。

A76 ★★ LoRA 微调的五个常见坑

A：

1. **target modules 名字写错** → 静默地什么都没训（loss 看起来还在降，因为训的是 projector/bias）。**必须打印可训练参数数量**。
2. **rank 太小却指望大改能力**（比如给中文模型加英文任务）→ 欠拟合，表现为"loss 降不下去、输出风格不变"。
3. **改了 r 没改 α** → 有效学习率意外变化（见 A67）。
4. **合并精度损失** → fp16 下合并后效果比"不合并"差；用 fp32 合并。
5. **以为 LoRA 能解决长上下文 OOM** → LoRA 不省激活，长序列仍要靠梯度检查点/FlashAttention。

一个隐蔽的坑：`eval` 时忘了启用 adapter（PEFT 的 `disable_adapter()` 或加载时未指定），会用**基座模型**去评测，结果是"训了半天没效果"。

A77 ★★ 怎么判断一个 LoRA 训好了？

A：四层检查，从便宜到贵：

1. **参数校验**：可训练参数占总参数的比例合理（7B + r=16 全线性层 ≈ 0.5%），且非零。
2. **loss 曲线**：训练 loss 应平稳下降、无尖峰；**但 loss 下降不等于能力提升**（见 A57）。
3. **目标任务指标**：在**留出的验证集**上看任务指标（准确率/ROUGE/CIDEr），而不是看 loss。
4. **通用能力回归**：跑一个小的通用评测（如常识问答、指令跟随），确认没有灾难性遗忘——**这是 LoRA 相对全参微调的最大优势，必须验证它真的保住了**。

加分做法：扫 `rank ∈ {8,16,32}` 与 `target ∈ {q/v, 全线性}`，看指标是否饱和——**这能证明你的超参不是拍脑袋的**。

Part B 机器学习算法原理

> 这一部分**不写代码**（代码在手撕两册里），只讲**定义、推导、性质、对比、失效场景**——面试官问"为什么"的时候用。

B1 Softmax 家族（B1–B8）

B1 ★★★ 写出 softmax，它的雅可比矩阵是什么？

A：

```
softmax(z)_i = exp(z_i) / Σ_j exp(z_j)
```

雅可比（这个能推出来很加分）：

```
∂p_i/∂z_j = p_i · (δ_ij − p_j)      即   J = diag(p) − p · pᵀ
```

含义：**对角线是 p_i(1−p_i)（自身对自身的敏感度），非对角线是 −p_i p_j（此消彼长）**，因为输出必须和为 1，抬高一个必然压低其他。

两个关键性质：1. **平移不变**：`softmax(z + c) = softmax(z)`（分子分母同乘 e^c）。 2. **和为 1 且恒正**：输出可解释为概率单纯形上的一点。

B2 ★★★ 为什么要减 max？给出严格的数值论证。

A：`exp(x)` 在 float32 下 x > 88.7 就上溢成 `inf`，`inf/inf = nan`。减 max 后：

```
softmax(z)_i = exp(z_i − m) / Σ_j exp(z_j − m)，其中 m = max_j z_j
```

- **数学等价**：分子分母同乘 `e^{−m}`，结果不变（这是"等价"，不是近似）。
- **数值安全**：减完后最大项是 `exp(0)=1`，其余 ≤ 1，不会上溢；小项下溢成 0 也无害（softmax 里 0 就是"几乎不可能"）。
- **但下溢在 log 域有害**：如果先算 softmax 再取 log，下溢成 0 的地方会变 `−inf`。所以**交叉熵要直接在对数域算**（log_softmax / logsumexp），这是工程上的标准做法。

追问：全 −inf 的一行会怎样？→ 分母 0 → NaN。所以 mask 用 `−1e9` 而不是 `−inf`，或对全屏蔽行单独补 0。

B3 ★★★ 推导 softmax + 交叉熵的梯度为什么是 `p − y`。

A：设 `L = −log p_y = −z_y + log Σ_j exp(z_j)`。

对 `z_i` 求导，分两种情况：

```
∂L/∂z_i = −δ(i=y) + exp(z_i)/Σ_j exp(z_j) = p_i − δ(i=y)
```

即 $\partial L/\partial z_i = p_i - y_i$（y 是 one-hot）。

这个结果的意义：1. 梯度**不含 softmax 的雅可比**——两者精确抵消了（这也是框架把 softmax+CE 合成一个算子（如 `F.cross_entropy`）的原因：更快、更稳）。2. 误差越大梯度越大（预测 0.1、真值 1 → 梯度 −0.9），**没有饱和问题**；3. 对比 MSE + softmax：后者梯度多一个 $p(1-p)$ 因子，在"预测 0.99 但真值是 0"这种最该纠错的情况下梯度趋于 0。

B4 ★★ softmax 的温度参数有什么作用？

A：$\text{softmax}(z/T)$：

- $T = 1$ 标准；
- $T > 1$ → 分布更平（熵更大），用于**知识蒸馏**（软标签含"类间相似性"信息）、**采样多样性**、**LLM 生成**；
- $T < 1$ → 分布更尖（接近 argmax），用于确定性输出；
- $T \to 0$ → 退化成 one-hot（hard max）；$T \to \infty$ → 均匀分布。

为什么蒸馏要用 T：教师模型的原始 logits 里，"猫"和"狗"的相似性会被 softmax 压成接近 0 和 1，T 放大后这部分"暗知识"才显现出来，学生才能学到。

B5 ★★ softmax 存在"过参数化"，怎么理解？

A：$\text{softmax}(z + c \cdot 1) = \text{softmax}(z)$——**所有 logit 同加一个常数，输出不变**。所以：- C 类分类实际只有 **C−1 个自由度**，参数矩阵存在一整个方向的冗余（解不唯一）；- 导致 Hessian 奇异、优化不唯一、某些正则化行为异常。

三种处理：① 固定某一类的 logit 为 0（等价于用 C−1 组参数，这在数学上恰好等价于 sigmoid 二分类的推广）；② 加 L2 正则（把解拉向唯一）；③ 干脆不管（深度学习里因为正则/优化噪声，实践中不影响）。

B6 ★★ softmax 和 sigmoid 是什么关系？

A：**二分类时两者等价**（差一个常数偏移）：

```
softmax([z, 0])_1 = e^z/(e^z+1) = σ(z)
```

所以"softmax + CE"在 C=2 时与"sigmoid + BCE"给出相同的决策边界和梯度形式。

关键区别在多标签：如果一个样本可以同时属于多个类（不互斥），必须用**逐元素 sigmoid + BCE**，因为 softmax 强制各类概率和为 1（互斥假设）。

B7 ★★ log_softmax 与 logsumexp 技巧是什么？

A：

```
log_softmax(z)_i = z_i − logsumexp(z),   logsumexp(z) = m + log Σ_j exp(z_j − m)
```

- 直接算 $\log(\text{softmax}(z))$ 会先下溢再取 log → $-\text{inf}$；log_softmax 全程在对数域，**数值稳定**。
- logsumexp 是核心恒等式：**先减最大值，算完再加回来**。它出现在交叉熵、CRF、变分推断、混合模型里。
- 工程上 torch.logsumexp 已内置该实现；手写时**务必自己减 max**。

B8 ★ softmax 在什么情况下梯度会消失？怎么缓解？

A：当某个 logit 远大于其他（$z_{max} - z_{second} \gg 0$）时，softmax 输出接近 one-hot，$p_i(1-p_i) \to 0$，**梯度饱和**。这在训练早期 logit 尺度过大时容易出现（初始化不好、学习率过大、输入未归一化）。

缓解：① 输出层用小初始化（或用温度缩放）；② 加 label smoothing（防止一路把概率推到 1）；③ 用 focal loss（主动放大难例权重）；④ 检查输入特征尺度与归一化。**这也是"训练初期 loss 不动"的常见原因之一。**

B2 信息熵与信息论（B9–B16）

B9 ★★★ 信息量和熵的定义与直觉?

A:

- **自信息**：$I(x) = -\log p(x)$。概率越小，事件发生带来的"信息量"越大（小概率事件更有信息）。
- **熵**（信息熵）：$H(X) = -\Sigma_x p(x) \cdot \log p(x) = E[-\log p(X)]$，即**平均信息量 / 不确定度**。
- 单位：log 以 2 为底是 bit，以 e 为底是 nat（深度学习多用 nat）。
- **极值**：均匀分布熵最大（最不确定）；确定性分布熵为 0。

一句话直觉：熵 = "描述这个随机变量平均需要多少比特" = "我猜它有多难"。

面试常配的追问：为什么熵是不确定度的合理度量？→ 三条公理（连续性、均匀分布最大、独立可加）唯一确定出 $-\Sigma p \log p$ 的形式（Shannon 唯一性定理）。

B10 ★★★ 联合熵、条件熵、链式法则?

A:

```
联合熵    H(X,Y) = −ΣΣ p(x,y) log p(x,y)
条件熵    H(Y|X) = H(X,Y) − H(X)        （给定 X 后 Y 的剩余不确定度）
链式法则  H(X,Y) = H(X) + H(Y|X)
```

重要性质：- $H(Y|X) \leqslant H(Y)$：知道 X **不会增加** Y 的不确定度（等号当且仅当独立）。- $H(X,Y) \leqslant H(X) + H(Y)$：等号当且仅当独立。

在树模型里怎么用：ID3 的信息增益本质就是 $H(D) - H(D|A)$，即"用特征 A 划分后，标签的不确定度减少了多少"。

B11 ★★★ 互信息是什么? 有什么意义?

A:

```
I(X;Y) = H(X) − H(X|Y) = H(Y) − H(Y|X) = H(X) + H(Y) − H(X,Y)
```

含义：知道 Y 能减少多少关于 X 的不确定性（对称的"相关性"度量）。

三个用途（面试可举）：1. **决策树的信息增益** = $I(标签; 特征)$；2. **特征选择**：互信息高 = 该特征与标签相关性强；3. **多模态对齐的另一种视角**：CLIP 的 InfoNCE 本质在最大化**图像与文本的互信息下界**（这是理解对比学习的信息论视角）。

B12 ★★★ KL 散度的定义与三个性质?

A:

```
D_KL(P||Q) = Σ_x p(x) · log( p(x)/q(x) )
```

三个性质：1. **非负**：$D_{KL} \geqslant 0$，等号当且仅当 P=Q（由 Jensen 不等式或 Gibbs 不等式）。2. **非对称**：$D_{KL}(P||Q) \neq D_{KL}(Q||P)$，所以**不是距离**（不满足三角不等式）。3. **不度量"距离"而度量"代价"**：$D_{KL}(P||Q)$ 是"用 Q 去编码 P 的额外代价"。

非对称的实际后果（这个例子很值分）：设 P 是双峰分布，Q 是单峰高斯：- **前向 $KL(P||Q)$**（最小化 Q 覆盖 P 的所有模式）→ Q 会摊平成宽分布，**mass-covering**；- **反向 $KL(Q||P)$**（Q 避开 P 概率为 0 的地方）→ Q 会塌到某个峰，**mode-seeking**。这就是变分推断（前向）与 GAN/反向 KL 行为差异的根源。

B13 ★★★ 交叉熵和 KL 散度的关系? 为什么分类要用交叉熵?

A:

```
H(P,Q) = −Σ p log q = H(P) + D_KL(P||Q)
```

即 **交叉熵 = 真实分布的熵 + KL 散度**。因为 $H(P)$ 与模型参数无关（数据固定），所以：

最小化交叉熵 ≡ 最小化 KL 散度，即让预测分布尽量逼近真实分布。

用交叉熵而不用 KL 的工程原因：**不需要知道 H(P)**（P 是 one-hot 时 H(P)=0，直接就是 −log q_y），计算更简单。

一句话：交叉熵是"用模型分布去编码真实标签的代价"，最小化它就是最大似然估计。

B14 ★★★ 信息增益怎么算？它偏向什么？

A：

```
Gain(D, A) = H(D) − Σ_v (|D_v|/|D|)·H(D_v) = H(D) − H(D|A) = I(D; A)
```

即"用特征 A 划分后标签熵的下降量"，也就是互信息。

偏向多值特征：特征取值越多，划分得越细，`H(D|A)` 越容易小（极端情况：用 ID 号划分，每个样本一组，增益最大但毫无泛化）——**这是 ID3 的根本缺陷**，被 C4.5 的增益率修正。

B15 ★★ 信息增益率与固有值是什么？为什么能修正偏向？

A：

```
增益率 = Gain(D, A) / IV(A),   IV(A) = −Σ_v (|D_v|/|D|)·log(|D_v|/|D|)
```

`IV(A)` 是特征 A 自身的**固有值/分裂信息**（把样本按 A 分成 v 组的"熵"），取值越多 IV 越大，于是增益率被惩罚。

但增益率也有反向偏向（偏向取值少的特征），所以 C4.5 用的是**启发式**：先找出增益高于平均的特征，再在其中选增益率最高的。

B16 ★★ 基尼指数和熵是什么关系？为什么 CART 用基尼？

A：

```
基尼    Gini = 1 − Σ_c p_c²   = Σ_{c≠c'} p_c p_{c'}     （随机取两个样本、类别不同的概率）
熵      H = −Σ_c p_c log p_c
```

- **形状高度相似**：都在均匀分布处取最大，都在纯节点处取 0；二分类下 `Gini = 2p(1−p)`，与熵的曲线几乎重合。
- **基尼是熵的近似**（在均匀分布附近可用低阶展开近似），所以划分结果基本一致。
- **用基尼的原因**：只有乘法加法、**没有 log**，计算快（大数据下差异明显）；且梯度更"平缓"，对噪声略鲁棒。

追问：两者的最优划分会不同吗？→ 可能不同，但实践差异很小；论文级对比显示 mIoU/准确率差异通常在 1% 内。

补充：还有 **最大熵原理**（在已知约束下选熵最大的分布，最"不假设"）和 **困惑度**（`perplexity = exp(H)`，语言模型常用），面试顺口提到很加分。

B3 分类树（B17–B28）

B17 ★★★ ID3 / C4.5 / CART 三种算法怎么对比？

A：

	ID3	C4.5	CART
划分准则	信息增益	**信息增益率**	**基尼指数**（分类）/ MSE（回归）
分裂方式	**多叉**（每个取值一支）	多叉	**二叉**
连续特征	不支持	支持（排序+二分）	支持
缺失值	不支持	支持（按权重分配到各分支）	支持（代理分裂 surrogate split）
剪枝	无	悲观剪枝（PEP）	**代价复杂度剪枝（CCP）**
输出	分类	分类	**分类 + 回归**

一句话："ID3 是原型，C4.5 补工程（连续/缺失/剪枝），CART 补统一性（二叉 + 回归）——sklearn 和所有 GBDT 家族用的都是 CART。"

B18 ★★★ 为什么 CART 用二叉树?

A：四个理由：

1. **避免碎片化**：多叉会让样本被迅速切碎（取值多的特征尤其），每个分支样本太少 → 过拟合且统计不可靠。
2. **表达力更强**：二叉允许**同一特征被多次使用**（"年龄<30 且 年龄>20"），而多叉一个特征只能用一次。
3. **计算高效**：每次只需找最优二分点，比多路划分的组合搜索简单得多。
4. **统一处理类别型特征**：把"取值集合"按 one-vs-rest 二分（对类别特征做分组），天然支持"把 {红,蓝} 归一边"。

B19 ★★★ 连续特征怎么处理？为什么只需枚举相邻中点?

A：做法：该特征的所有取值**排序去重** → 只在**相邻不同取值的中点**试切分。

为什么只需相邻中点：对同一特征，若两个阈值落在同一对相邻取值之间，**划分出的样本集合完全相同**（因为区间内没有样本），所以只有 n−1 个"有效"候选阈值（n = 去重后取值数），彼此等价。

复杂度：排序 $O(n \log n)$ + 一次扫描 $O(n)$（用前缀计数增量更新左右子集的不纯度），而不是"每个阈值重扫一遍"的 $O(n^2)$。

追问：为什么要去重？→ 相同取值的样本任何阈值都分不开，切开是无意义分裂。

B20 ★★★ 剪枝的三种方式？代价复杂度剪枝怎么工作?

A：

方式	做法	优缺点
预剪枝	提前停：深度上限、叶最小样本、增益阈值、纯度阈值	快；但**贪心视野短**，可能过早停止（当前增益小、后续收益大）
后剪枝	先长满，再自底向上判断"是否合并叶子"	更准（看到全树后再决策），但训练成本高
CCP（代价复杂度）	定义 $R_\alpha(T) = R(T) + \alpha \cdot \lvert T \rvert$（R 是训练误差，	T

CCP 的机制：α=0 时最优是全树；α 增大时叶子的"惩罚"变重，逐步剪枝 → 得到一串嵌套子树 → 用验证集选最好的 α（sklearn 的 `ccp_alpha`）。

面试话术："预剪枝省时但短视，后剪枝准但贵；工业上常见组合：先用预剪枝控深度（防止爆内存），再用后剪枝微调。"

B21 ★★ 缺失值怎么处理?

A：两种主流：

- **C4.5**：把缺失样本**按权重分到所有分支**（权重 = 各分支样本比例），增益计算时乘上"有效样本比例"的折扣因子。
- **CART（sklearn 不支持，但理论上有）**：**代理分裂（surrogate split）**——为每个主分裂再学若干"代理特征"，推理时若主特征缺失，用与它最相似的代理特征代替。

工程实践：多数框架直接要求先填充缺失值（均值/中位数/模型预测），再建树——因为树分裂的代理机制实现复杂且收益不稳定。

B22 ★★ 回归树怎么建？叶子输出什么?

A：和分类树完全同构，只换两处：

1. **划分准则**：从基尼换成 **MSE 下降**（$MSE = (1/n)\Sigma(y-\bar{y})^2$）。
2. **叶子输出**：从"多数类"换成 **均值**（最小化平方误差的最优常数就是均值；若损失是 MAE，则取**中位数**）。

注意：回归树的预测是**分段常数**（一个叶子一个值），所以它**不能外推**（超出训练范围的值只能给边界值），且对目标做单调变换会改变结果（因为 MSE 不等变）。

B23 ★★★ 特征重要性怎么算？有什么偏差？

A：两种：

1. **不纯度重要性（Gini/MSE importance，sklearn 默认）**：把"该特征每次分裂带来的加权不纯度下降 × 样本数"累加后归一化。
2. **置换重要性（permutation importance）**：打乱该特征的值（破坏其与标签的关系），看验证指标下降多少。**更可靠、更贵**。

三种已知偏差：- **偏高基数/多值特征**（分裂机会多，累积下降自然大）；- **对相关特征会"分摊"**（两个高度相关的特征各拿一半，看起来都不重要）；- **在不纯度重要性里，训练集上计算会偏向能拟合噪声的特征**。

面试加分："所以我一般用置换重要性做最终判断，不纯度重要性只用来看相对排序。"

B24 ★★ 决策树为什么不稳定？为什么要集成？

A：**不稳定**的原因：树的分裂是**贪心的离散选择**——数据微扰（换一批样本、加几个噪声点）可能让最优切分点改变，后面的整棵子树结构随之变化，预测方差大。

偏差-方差视角：单棵树**偏差低（可以拟合很复杂的边界）、方差高**。所以：- **Bagging / 随机森林**：并行多棵 + 平均 → **降方差**（不显著降偏差）；- **Boosting / GBDT**：串行拟合残差 → **降偏差**（同时要控制方差，所以用小树 + 学习率）。

B25 ★★★ RF / GBDT / XGBoost / LightGBM / CatBoost 怎么串起来？

A：

	集成方式	优化目标	关键创新
随机森林	Bagging（样本 + 特征双采样，并行，投票/平均）	降方差	双重随机性让树之间去相关
GBDT	Boosting（串行，每棵树拟合**负梯度/残差**）	降偏差	用树做梯度下降
XGBoost	GBDT + 工程	二阶泰勒 + 显式正则	二阶信息、正则化（$\gamma T + \lambda w^2/2$）、稀疏感知、特征级并行、近似分位点
LightGBM	GBDT + 效率	同 XGB	**leaf-wise 生长**（同一叶数下更低 loss，但易过拟合，需控 `num_leaves`）、直方图分桶、**GOSS**（保留梯度大的样本）、**EFB**（互斥特征捆绑）
CatBoost	GBDT + 类别特征	同 XGB	**对称树（oblivious）**、**ordered boosting**（解决目标泄漏）、类别特征的有序目标编码

一句话链："RF 是并行降方差，GBDT 是串行降偏差；XGBoost 给 GBDT 加了二阶和正则，LightGBM 把速度做到极致，CatBoost 专治类别特征。"

B26 ★★ XGBoost 相对传统 GBDT 的关键改进？

A：四件事：

1. **二阶泰勒展开**：目标函数用一阶梯度 g + 二阶 h 近似，**最优叶权重有闭式解** $w^* = -\Sigma g/(\Sigma h+\lambda)$，分裂增益也有解析式。这比只用一阶梯度收敛更快、更准。
2. **显式正则**：$\Omega = \gamma T + (\lambda/2)\Sigma w^2$，直接在目标函数里惩罚树复杂度（叶数 T 和叶权重 w），而不是靠早停/剪枝。
3. **稀疏感知**：为缺失值学一个默认方向（分裂时把缺失样本统一分到增益大的一侧）。
4. **工程**：特征预排序（block 结构）、特征级并行、近似分位点（`approx_hist`）、缓存优化。

B27 ★★ LightGBM 的 leaf-wise、GOSS、EFB 分别解决什么？

A：

- **leaf-wise（按叶生长）**：每次选"增益最大的叶子"继续分裂，而不是按层。同样的叶数下 loss 更低（效率高），但**容易长出深的不平衡树 → 过拟合**，所以要用 `num_leaves` 和 `max_depth` 双重约束。

- **GOSS（Gradient-based One-Side Sampling）**：梯度大的样本对增益贡献大，**保留全部大梯度样本 + 随机采样一部分小梯度样本**（并放大权重），用少量样本近似全量增益。
- **EFB（Exclusive Feature Bundling）**：稀疏场景下很多特征互斥（不会同时非零），把它们**捆绑成一个特征**，降低特征维度 → 减少直方图构建开销。

B28 ★ 什么时候选树模型，什么时候选线性/神经网络?

A:

场景	选树	选线性/NN
表格数据、中小规模、需要可解释	✅树（尤其 GBDT，仍是表格任务的王者）	—
高维稀疏（文本 TF-IDF）	—	✅线性（快、可解释）
特征有强交互、非线性	✅树	✅NN（需更多数据）
需要外推（预测超出训练范围）	❌树不能外推	✅线性/NN
需要平滑输出	❌分段常数	✅
图像/语音/文本原始信号	—	✅深度模型

一句话："树赢在'不需要特征工程 + 自动做交互 + 可解释'，输在'不能外推、不平滑、不稳定'。"

B4 K-Means（B29–B36）

B29 ★★★ KMeans 的算法与目标函数?

A:

目标函数（inertia / 簇内平方和）：

```
J(C, μ) = Σ_i min_j ||x_i − μ_j||²
```

算法（交替优化 / 坐标下降）：

```
1. 初始化 k 个中心
2. E 步（固定中心）：把每个点分给最近的中心
3. M 步（固定分配）：把每个中心更新为其簇内所有点的均值
4. 重复 2-3 直到中心移动量 < tol 或达到迭代上限
```

为什么 M 步是均值：固定分配后，`J = Σ_j Σ_{i∈C_j} ||x_i − μ_j||²` 对 μ_j 求导置零，得 `μ_j = (1/|C_j|)Σ x_i`——即**均值是最小化平方误差的最优常数**。

B30 ★★★ 为什么 KMeans 一定收敛？为什么只是局部最优?

A:

收敛：E 步（在固定中心下，把点分给最近中心）不会增大 J；M 步（固定分配下取均值）也不会增大 J。J 单调不增且有下界 0；而划分方案是**有限个**（n 个点分到 k 个簇，最多 k^n 种，实际远少），所以不会无限循环 → **有限步内收敛到一个驻点**。

局部最优：因为 J 非凸，交替优化只保证到达**坐标下降的局部极小**。初始化不同 → 收敛到不同的局部解（我实测一组数据：20 个随机种子里有 7 个没能正确分出三个簇，最差的只对上 125/150）。

工程对策：`n_init` 多次随机初始化取 inertia 最小；或 k-means++ 初始化。

B31 ★★★ k-means++ 的原理？为什么有效?

A：**D^2 采样**：

```
1. 第一个中心均匀随机选一个点
2. 之后每个新中心按概率 ∝ D(x)² 采样（D(x) = 到最近已有中心的距离）
3. 重复到选出 k 个
```

即"**离已有中心越远的点，越容易被选为新中心**"。

为什么有效：随机初始化最坏情况可能把两个中心都放在同一个真实簇里（剩下一个簇分不到中心），D^2 采样让中心天然分散，规避了这种退化。

理论保证：k-means++ 的期望代价 ≤ **O(log k)** 倍最优解（对比随机初始化的任意差）。代价只是初始化时多算几次距离。

B32 ★★★ k 怎么选？（四种方法）

A：

1. **肘部法（elbow）**：画 inertia-k 曲线，找拐点（边际收益骤降处）。缺点是"肘"常常不明显。
2. **轮廓系数（silhouette）**：`s = (b − a)/max(a, b)`，a = 到本簇其他点的平均距离，b = 到最近其他簇的平均距离。范围 [−1,1]，越大越好，取 s 平均最大的 k。
3. **Gap statistic**：比较 log(inertia) 与"零假设下（均匀分布）的期望值"的差，差距最大处为最优 k。
4. **稳定性 / 下游指标**：多次子采样聚类的一致性；或直接用下游任务指标（最实际）。

面试加分："我更看重**稳定性**——k 稍微变化时簇结构是否稳定；如果 k=4 和 k=5 的划分看起来完全不同，说明数据本身没有清晰的簇结构。"

B33 ★★ KMeans 的复杂度？怎么加速？

A：**O(n · k · d · t)**（n 样本、k 簇、d 维、t 迭代轮数）。空间 O(n · d + k · d)。

加速手段：1. **MiniBatch KMeans**：每步只用一个小 batch 更新中心（`O(b · k · d · t)`），大规模数据首选。 2. **Elkan 算法**：用三角不等式剪枝，跳过不可能更近的中心计算。 3. **kd-tree / ball-tree**：低维（d < 20）时加速最近邻搜索。 4. **降维**（PCA/随机投影）后再聚类。 5. **初始化用 k-means++ + n_init 小**（减少迭代次数）。

B34 ★★ KMeans 的假设与失效场景？

A：四个隐含假设：

1. **簇是凸的、球形的**（欧氏距离隐含等向性）→ 遇到月牙形/环形数据直接失效（该用 DBSCAN/谱聚类）。
2. **各簇方差相近、大小相近** → 一个大簇会被劈成两半去迁就小簇。
3. **欧氏距离有意义** → 高维下距离集中（所有点距离都差不多），聚类失效。
4. **对异常值敏感** → 均值被拉走（可用 k-medoids / 截断损失）。

另外：它只给**硬分配**，不能表达"某个点 60% 属于簇 A"（要软分配用 GMM）。

B35 ★★ KMeans / GMM / DBSCAN / 层次 / 谱聚类怎么选？

A：

方法	核心思想	优点	缺点
KMeans	中心 + 硬分配	快、简单、可扩展	球形假设、要指定 k、异常敏感
GMM（EM）	高斯混合 + 软分配	概率输出、允许不同协方差（椭圆簇）	要指定 k、对初始化敏感、高斯假设
DBSCAN	密度可达	不用指定 k、能找任意形状、能识别噪声	对 eps/minPts 敏感、密度差异大时失效、高维差
层次聚类	自底向上合并 / 自顶向下分裂	不用指定 k、给树状结构、可解释	$O(n^2)$/$O(n^3)$、不可回溯（贪心合并）
谱聚类	图拉普拉斯特征分解	能处理非凸簇	要指定 k、大图贵（需稀疏近似）

KMeans 与 GMM 的关系（高频追问）：KMeans 是 GMM 的**特例**——所有簇共享各向同性协方差 $\sigma^2 I$ 且 $\sigma \to 0$ 时的硬分配极限。EM 的 E 步是软分配（后验概率），M 步更新均值/协方差/权重。

B36 ★ 空簇和初始化有哪些实际陷阱？

A：三个：

1. **空簇**：某轮没有点分给它 → 均值无定义（除零）。处理：保留旧中心、或把它重置到"离其他中心最远的点"、或直接删掉该簇（k 减一）。
2. **初始中心重复**：随机抽到同一个点（k 接近 n 时概率不低）→ 退化。用 `replace=False` 抽样或 k-means++。
3. **用整数存中心**：均值可能是小数，用 int 会累积误差甚至震荡不收敛（这是实现 bug，但很常见）。

收敛判据的选择：用"中心移动量 < tol"而不是"标签不再变化"——后者在某些数据上会出现**标签循环震荡**（目标函数不变但分配在两组等价解之间跳）。

B5 自注意力（B37–B45）

B37 ★★★ 写出 self-attention 的完整定义，并说明每一步的形状。

A：

```
Q = X · W_Q,  K = X · W_K,  V = X · W_V          X: [L, d],  W: [d, d]
A = softmax( Q · Kᵀ / √d_k + mask , dim=-1)  Q · Kᵀ: [L, L]
O = A · V                                  [L,L]×[L,d] → [L, d]
```

三步的语义：1. **打分**：`Q · Kᵀ` 得到任意两 token 的相似度（**全局**、与距离无关）；2. **归一化**：逐行 softmax → 每个 query 对所有 key 的注意力分布（和为 1）；3. **聚合**：用注意力权重对 V 加权求和 → 每个位置得到"按相关性混合"的新表示。

三个矩阵的不同角色：Q 是"我要找什么"，K 是"我能被什么匹配"，V 是"匹配上之后提供什么内容"。

B38 ★★★ 为什么除以 √d_k？给出方差论证。

A：设 `q, k` 各维独立同分布、均值 0、方差 1，则点积：

```
q · k = Σ_{i=1}^{d_k} q_i · k_i
E[q · k] = Σ E[q_i]E[k_i] = 0
Var[q · k] = Σ Var(q_i k_i) = Σ E[q_i²] · E[k_i²] = d_k       (独立性 + 零均值)
```

所以点积的**标准差是 √d_k**——d_k=64 时量级约 8，d_k=256 时约 16。softmax 的输入尺度一大，就会进入**饱和区**（最大项 ≈1、其余≈0），此时雅可比 `diag(p)−ppᵀ ≈ 0`，**梯度消失**。

除以 √d_k 后方差回到 1，softmax 落在梯度良好的区间。**这也是 d_k 越大越需要缩放的原因。**

B39 ★★★ self-attention 的复杂度？优化方向有哪些？

A：

```
时间: O(L² · d)      (打分 L²d + 加权 L²d)
显存: O(L² + L · d)  (要存注意力矩阵 L²)
```

L 是序列长度，所以**长上下文是平方级代价**。五个优化方向：

1. **IO 优化**：**FlashAttention**——分块（tiling）+ 在线 softmax，不把 L×L 矩阵落到显存，**省显存和 HBM 读写，不减少 FLOPs**。
2. **稀疏/滑窗**：Longformer（滑窗 + 全局 token）、BigBird（随机 + 窗口 + 全局）。
3. **低秩/核近似**：Linformer（对 K/V 降维）、Performer（核函数近似 softmax）。
4. **共享 KV**：MQA/GQA（主要省推理 KV cache）。
5. **换范式**：线性注意力、状态空间模型（Mamba，线性复杂度）。

B40 ★★★ 多头的作用是什么？参数量怎么算？

A：把 `d` 维按**列**切成 h 份，每份 `d_k = d/h`，各自做 attention，再把 h 个 `[L, d_k]` 输出**按列拼回** `[L, d]`，最后过输出投影 `W_O`。

作用：不同头在**不同的子空间**学不同的依赖模式（相邻、句法、指代、跨模态对应），类似 CNN 的多通道分工。**计算量几乎不变**（$h \cdot L^2 \cdot (d/h) = L^2 \cdot d$），只是注意力的"视角"变多。

参数量：$4d^2$（Q/K/V/O 各 d×d）+ bias $4d$，**与 h 无关**（这是常被考的常识）。

B41 ★★★ 为什么需要位置编码？四种方案的取舍？

A：**self-attention 是置换等变的**（打乱输入只是打乱输出行），本身不含顺序信息，所以必须显式注入位置。

方案	做法	优点	缺点
正弦绝对编码	不同频率的 sin/cos	无需学习、可外推到更长	表达力弱、外推质量差
可学习绝对编码	每个位置一个向量	简单、效果好	**不能外推**（超过训练长度没有参数）
RoPE（旋转）	把 q/k 的相邻两维按位置旋转	编码**相对**位置、可外推、无额外参数	外推需插值/NTK 调整
ALiBi	注意力分数按距离加线性负 bias	外推好、实现极简、无参数	表达力略弱于 RoPE

RoPE 为什么能外推：因为 $q_m \cdot k_n$ 只依赖相对距离 $(m-n)$，所以理论上长度无关；但直接外推到远超训练的尺度会失真，所以实践要配位置内插（PI）、NTK-aware 缩放、YaRN。

多模态的扩展：图像/视频需要 2D/3D 位置（高、宽、时间），如 Qwen-VL 的 **M-RoPE**（把旋转维度分给 t/h/w）。

B42 ★★ attention 里的 mask 有哪几种？实现要注意什么？

A：三种：

1. **causal mask**（因果/自回归）：位置 i 只能看到 ⩽ i 的 key → 上三角置 $-\infty$。
2. **padding mask**：一个 batch 里不同的有效长度，key 侧的 pad 位置屏蔽。
3. **结构/跨模态 mask**：如视锥裁切（BEV query 看不到的相机）、跨样本隔离。

实现注意：- 用加性 mask（$scores + (-1e9 \text{ 或 } -inf)$）或 $masked_fill$，**统一语义**（True 是屏蔽还是保留，PyTorch 的 SDPA 里 bool mask 是 $True = 允许$，与 $masked_fill$ 相反）。- **一行全被屏蔽 → 分母为 0 → NaN**：要么保证每行至少一个有效位，要么对全屏蔽行单独处理。

B43 ★★ "置换等变"具体是什么意思？有什么后果？

A：设 P 是置换矩阵，则 $Attention(PX) = P \cdot Attention(X)$——**输入换顺序，输出只是跟着换顺序，内容不变**。

两个直接后果：1. **必须加位置编码**（否则模型分不清"猫追狗"和"狗追猫"）；2. **注意力的权重矩阵不含"距离"概念**（不像卷积的局部性）——所以它对"相邻"这件事没有先验，全靠学。

顺带一个面试题：为什么 Transformer 能表达卷积？→ 可以用特定的注意力模式（固定的相对位置偏置）模拟卷积核的行为，即"Transformer 的表达力包含卷积"。

B44 ★★ MQA / GQA / FlashAttention / 线性注意力 / Mamba 各解决什么？

A：

方法	解决什么	代价
MQA（Multi-Query）	所有头共享 1 组 K/V → KV cache ÷h	表达力略降
GQA（Grouped-Query）	h 个头分 g 组共享 K/V → ÷(h/g)	折中，主流选择
FlashAttention	分块 + 在线 softmax，不落 L^2 矩阵 → **省显存与 HBM 读写**	算法等价，不改 FLOPs
线性注意力（Performer/Linformer）	核近似/低秩 → **O(L) 复杂度**	近似误差、长程依赖略弱
Mamba / SSM	选择性状态空间 → 线性复杂度、推理 O(1) 每步	表达力待验证、生态不成熟

一句话："MQA/GQA 省的是**推理时的 KV cache**；FlashAttention 省的是**显存和带宽**；线性注意力/Mamba 改的是**复杂度阶**——三者解决的是不同层面的问题。"

B45 ★ self-attention 与 CNN / RNN 的本质区别?

A:

	CNN	RNN	Self-Attention
感受野	局部（靠堆叠扩大）	理论上全局（实际衰减）	**每层全局**
并行性	高	**低**（时序依赖）	高
归纳偏置	强（局部性、平移等变）	中（序列性）	**弱**（几乎无先验）
复杂度	O(L · k^2)（与长度线性）	O(L)（串行）	**$O(L^2)$**
长依赖	需深层	梯度消失/爆炸	直接建模（但可能"注意力分散"）

取舍结论："**归纳偏置越强，小数据越好、大数据越受限**。CNN 在数据少时更强，Transformer 在数据够时上限更高——这也是 ViT 需要大数据/强增广/蒸馏才超过 CNN 的原因。"

B6 用代码验证原理（Python 语法不熟也能跑）

> 这一节的目的不是"实现算法"，而是**用几行代码把抽象结论变成看得见的数字**。 例如"softmax+CE 的梯度是 p−y"——写 5 行数值微分就能验出来，比背公式记得牢。
>
> 完整可运行文件： `ml_principles_verify.py` （47 项断言，全部实测通过），运行 `python3 ml_principles_verify.py` 。 下面每个片段都是那个文件里的精简版，**可以直接复制到 Python 里跑**。

B6.0 先补 12 个语法点（这一节只用得到这些）

你只写 C++ 的话，下面这些是"看得懂这段代码"的最小集：

#	语法	说明	等价 C++
1	`np.array([1,2,3])`	列表 → 数组	`vector<int>{1,2,3}`
2	`x.max(axis=-1, keepdims=True)`	沿最后一维取最大；`keepdims` 保住维度	手写循环 + 记录维度
3	`a * b` / `a / c`	**逐元素**运算，自动广播	循环逐元素
4	`y[mask]`	**布尔索引**：取出 mask 为 True 的元素	`if (mask[i]) v.push_back(...)`
5	`~mask`	布尔取反（**不是 `!`**）	`!mask`
6	`a[:, j]`	逗号左边是行、右边是列；`:` 表示全部	手写循环取第 j 列
7	`a[::-1]`	反向切片	`reverse`
8	`np.unique(a)`	去重并排序	`sort` + `erase(unique)`
9	`np.bincount(y)`	统计每个整数出现几次	`int cnt[k]={0}` 累加
10	`f"{x:.4f}"`	f-string 格式化，`.4f` 是 4 位小数	`printf("%.4f", x)`
11	`[... for x in xs]`	列表推导式	`for` + `push_back`
12	`np.allclose(a, b, atol=1e-6)`	浮点近似相等（**自检必用**）	`fabs(a-b)<eps`

一个提醒：Python 的 `=` 是**引用**， `b = a` 不会复制数组，要复制写 `b = a.copy()` 。这在本节里出现过好几次，是最容易踩的坑。

B6.1 验证 softmax 的三条结论

① 雅可比矩阵就是 $\mathrm{diag}(p) - p \cdot p^T$

```
import numpy as np, math

def softmax(z):
    z = np.asarray(z, dtype=np.float64)        # ← 语法点：list 转数组
    z = z - z.max(keepdims=True)               # 减最大值（数值稳定，见下一条）
    e = np.exp(z)
    return e / e.sum(keepdims=True)

z = np.array([0.5, -0.2, 1.1])
p = softmax(z)

J_analytic = np.diag(p) - np.outer(p, p)       # ← 语法点：np.outer 求外积 → 3×3

eps = 1e-6
J_numeric = np.zeros((3, 3))
for j in range(3):                             # ← 语法点：range(3) = 0,1,2
    z2 = z.copy()                              # ← 语法点：必须 copy，否则改的是同一个数组
    z2[j] += eps
    J_numeric[:, j] = (softmax(z2) - p) / eps  # ← 语法点：[:, j] 取第 j 列

print(np.allclose(J_analytic, J_numeric, atol=1e-5))     # True
print(J_analytic.sum(axis=1))                            # [0, 0, 0]
```

跑出来的两件事：1. 解析雅可比与数值微分**完全一致** → 公式 $\partial p_i/\partial z_j = p_i(\delta_{ij} - p_j)$ 是对的；2. 雅可比**每行和为 0** → 这就是"输出必须和为 1，所以所有 logit 同加常数不变"的数学体现。

② 不减 max 会直接得到 nan

```
z = np.array([1000.0, 1000.0, 1000.0])
naive = np.exp(z) / np.exp(z).sum()     # RuntimeWarning: overflow → nan
safe  = softmax(z)                       # [1/3, 1/3, 1/3]
```

观察：`np.exp(1000)` 超 float64 上限 → `inf`，`inf/inf = nan`。减 max 后最大项是 `exp(0)=1`，永不溢出。

③ 交叉熵的梯度就是 $p - y$

```
z = np.array([0.3, 1.2, -0.7]); y = 1

def loss(z):
    return -math.log(softmax(z)[y])         # ← 语法点：函数里可以直接用外部变量 y

p = softmax(z)
grad_analytic = p.copy(); grad_analytic[y] -= 1          # ← p − y

eps = 1e-6
grad_numeric = np.zeros(3)
for i in range(3):
    zp, zm = z.copy(), z.copy()
    zp[i] += eps; zm[i] -= eps
    grad_numeric[i] = (loss(zp) - loss(zm)) / (2 * eps)   # 中心差分

print(np.allclose(grad_analytic, grad_numeric, atol=1e-6))   # True
```

这就是"框架为什么把 softmax 和 CE 合成一个算子"的实证：两者的雅可比精确抵消，$p - y$ 既简单又不饱和。

顺带两个小验证（一行代码的结论）

```
# 温度：T 越大分布越平
for T in (0.5, 1.0, 3.0):
    print(T, np.round(softmax(z / T), 4))

# 二分类时 softmax([z,0]) == sigmoid(z)
print(softmax([1.7, 0.0])[0], 1 / (1 + math.exp(-1.7)))    # 0.8455 0.8455
```

B6.2 验证信息熵的四条结论

① 链式法则：`H(X,Y) = H(X) + H(Y|X)`

```
def entropy(p):
    p = np.asarray(p, dtype=np.float64)
    p = p[p > 0]                              # ← 语法点：布尔索引筛掉 0（0 · log0 记为 0）
    return -(p * np.log(p)).sum()

Pxy = np.array([[0.20, 0.05, 0.00],
                [0.05, 0.30, 0.05],
                [0.00, 0.05, 0.30]])
Px = Pxy.sum(axis=1)                          # 按行求和 = X 的边缘分布
Hxy = entropy(Pxy.ravel())                    # ← 语法点：ravel 把矩阵摊平
Hx = entropy(Px)
Hy_given_x = sum(Px[i] * entropy(Pxy[i] / Px[i]) for i in range(3))
#            ^ ↑ 语法点：生成器表达式放进 sum()，等价于一个 for 循环累加
print(abs(Hxy - (Hx + Hy_given_x)) < 1e-12)   # True
```

为什么值得跑一遍：条件熵的定义式 `H(Y|X) = Σ_x p(x)·H(Y|X=x)` 里"**每一项都要先归一化**（`Pxy[i] / Px[i]`）"这个细节，光看公式容易漏；跑一次就记住了。

② KL 散度非对称

```
P = np.array([0.5, 0.5])
Q = np.array([0.9, 0.1])

def kl(a, b):
    m = a > 0
    return float((a[m] * np.log(a[m] / b[m])).sum())

print(kl(P, Q), kl(Q, P))      # 0.5108  1.2451  ← 差别很大
```

观察到的两个数：`KL(P||Q)≈0.51`、`KL(Q||P)≈1.25`——这直接说明它**不是距离**（不满足对称性），也解释了变分推断里"前向 KL 会让 Q 摊平覆盖 P 的所有峰，反向 KL 会塌到某个峰"。

③ 交叉熵 = 熵 + KL

```
ce = -float((P * np.log(Q)).sum())
h  = entropy(P)
kl_v = float((P * np.log(P / Q)).sum())
print(abs(ce - (h + kl_v)) < 1e-12)     # True
```

结论：分类时 `H(P)` 是常数，所以**最小化交叉熵 ≡ 最小化 KL 散度**——这就是"为什么用交叉熵而不是直接算 KL"（省掉算 H(P)）。

④ 信息增益偏向多值特征（最重要的实验）

```
y = np.array([1, 1, 1, 0, 0, 0])          # 3 正 3 负
A = np.array([0, 0, 0, 1, 1, 1])          # 好特征：2 个取值，每类纯
ID = np.arange(6)                          # 坏特征：每个样本一个取值

def cond_entropy(feat, y):
    total = 0.0
    for v in np.unique(feat):              # ← 语法点：np.unique 去重
        mask = feat == v
        w = mask.sum() / len(feat)
        total += w * entropy(np.bincount(y[mask], minlength=2) / mask.sum())
    return total

H_parent = entropy([0.5, 0.5])                        # 0.6931
gain_A  = H_parent - cond_entropy(A,  y)              # 0.6931  ← 完全分开
gain_ID = H_parent - cond_entropy(ID, y)              # 0.6931  ← 也"完全分开"!
```

这就是 ID3 缺陷的现场证据：用"样本 ID"这种毫无泛化能力的特征，信息增益同样是最大的。接着验证增益率能区分它们：

```
def iv(feat):
    return entropy(np.array([(feat == v).sum() / len(feat) for v in np.unique(feat)]))

print(gain_A / iv(A), gain_ID / iv(ID))    # 1.0000   0.3869
```

结论：增益率把 ID 号特征从 1.0 压到 0.387——因为它的"分裂信息 IV"更大（6 个取值 vs 2 个取值）。

B6.3 验证分类树的两条结论

① 手写 CART 能拟合到 100%（用来确认实现没错）

```
def gini_counts(y, K):
    p = np.bincount(y, minlength=K) / len(y)      # ← 语法点：bincount 统计每类次数
    return 1.0 - (p ** 2).sum()

def build_tree(X, y, depth=0, max_depth=4, min_leaf=2):
    node = {"label": int(np.bincount(y, minlength=2).argmax())}   # ← 用 dict 当节点
    if depth >= max_depth or len(np.unique(y)) == 1 or len(y) < 2 * min_leaf:
        return node
    best = (1e-9, None)
    parent = gini_counts(y, 2)
    for f in range(X.shape[1]):
        vals = np.unique(X[:, f])
        for t in (vals[:-1] + vals[1:]) / 2:       # ← 候选阈值 = 相邻中点
            m = X[:, f] <= t
            if m.sum() < min_leaf or (~m).sum() < min_leaf:
                continue
            g = parent - (m.sum() * gini_counts(y[m], 2)
                          + (~m).sum() * gini_counts(y[~m], 2)) / len(y)
            if g > best[0]:
                best = (g, (f, t, m))
    if best[1] is None:
        return node
    f, t, m = best[1]
    node.update({"feat": f, "thr": float(t),
                 "left":  build_tree(X[m],  y[m],  depth + 1, max_depth, min_leaf),
                 "right": build_tree(X[~m], y[~m], depth + 1, max_depth, min_leaf)})
    return node
```

注意三个 Python 语法点：- `{"label": ...}` 用**字典**表示节点——Python 没有指针，字典/对象都是"引用传递"，所以 `node["left"] = build_tree(...)` 天然就能建树（等价于 C++ 的 `node->left = ...`，但不用管内存）。- `~m` 是布尔取反（`!` 在 numpy 里不适用）。- `vals[:-1] + vals[1:]` 这个切片加法一次就得到所有相邻对，C++ 里要显式循环。

实测：两簇数据训练集准确率 **1.000**，根节点阈值切在两簇之间。

② 连续特征的候选阈值只有 n−1 个

```
col = np.array([0.1, 0.5, 0.5, 0.9, 1.7])
vals = np.unique(col)                       # [0.1, 0.5, 0.9, 1.7] —— 5 个点去重成 4 个
thrs = (vals[:-1] + vals[1:]) / 2           # [0.3, 0.7, 1.3] —— 3 个候选
print(len(thrs) == len(vals) - 1)           # True
```

说明：原始 5 个样本、去重后 4 个不同值 → 只有 **3 个有效候选阈值**。两个 0.5 之间切不开，任何落在同一区间的阈值划分结果完全相同。

B6.4 验证 K-Means 的三条结论

① inertia 单调不增（收敛性的证据）

```
def kmeans_simple(X, k, iters=50, seed=0, record=False):
    rng = np.random.default_rng(seed)                    # ← 语法点：新式随机数生成器
    C = X[rng.choice(len(X), k, replace=False)].copy()   # ← 选 k 个不同下标
    hist = []
    for _ in range(iters):                               # ← 语法点：_ 表示"用不到的循环变量"
        d = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)   # [N,k] 距离平方（广播）
        lab = d.argmin(1)                                # 每行最小值的下标
        for j in range(k):
            m = lab == j
            if m.any():                                  # ← 语法点：.any() 有 True 就 True
                C[j] = X[m].mean(0)                      # 布尔索引取子集再求均值
        hist.append(((X - C[lab]) ** 2).sum())           # C[lab] = 按标签取中心
    return lab, C, hist
```

实测输出：首轮 inertia ≈ 890 → 末轮 ≈ 98，**每轮都不增**（`np.diff(hist) <= 0` 全为真）。这就是 B30 说的"E 步和 M 步都不会增大目标函数"的实证。

② 局部最优：20 个种子差 55 倍

```
inertias = [kmeans_simple(X3, 3, seed=s)[2][-1] for s in range(20)]
inertias = sorted(inertias)
print(f"min={inertias[0]:.1f} 中位={inertias[10]:.1f} max={inertias[-1]:.1f}")
```

实测：`min=22.7 / 中位=22.7 / max=1247.2`——最差的种子比最好的**差 55 倍**，因为它把两个真实簇合并、另一个簇劈开了。

这句话可以直接讲给面试官："同一份数据、同一个算法，只换初始化，inertia 能差两个数量级——所以 `n_init` 或 k-means++ 不是优化项，是必需品。"

③ 球形假设失效：同心环上完全分不开

```
t = np.linspace(0, 2 * math.pi, 100, endpoint=False)     # ← 语法点：等距数列
ring1 = np.column_stack([np.cos(t), np.sin(t)]) * 1.0     # ← 按列拼成 (100,2)
ring2 = np.column_stack([np.cos(t), np.sin(t)]) * 3.0
X = np.vstack([ring1, ring2])                             # ← 语法点：vstack 竖着拼
```

实测：kmeans(k=2) 在同心环上的"按环划分纯度"只有 **100/200**（等于完全没分开：每个簇里两个环各占一半）。这就验证了"kmeans 只能找凸的、球形的簇"——这种数据该用 DBSCAN 或谱聚类。

顺带实测了 k 的选择：

k	inertia	轮廓系数均值
1	887.8	—
2	**98.3**	**0.778**
3	81.4	0.548
4	68.3	0.566

读法：inertia 从 k=1 到 k=2 掉了一大截（887→98），之后平缓 → **肘部在 k=2**；轮廓系数也在 k=2 最高。两种方法给出同一答案，这才是可信的 k。

B6.5 验证自注意力的四条结论

① 手算 attention，打印每个中间量

```
def attention_manual(X, Wq, Wk, Wv, mask=None):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv          # ← @ 就是矩阵乘
    dk = Q.shape[-1]                          # 最后一维长度
    S = Q @ K.T / math.sqrt(dk)               # [L,L] 相似度
    if mask is not None:
        S = np.where(mask, -1e9, S)           # ← 逐元素三目
    A = softmax(S)                            # 逐行归一化
    return A @ V, A
```

取 `X=[[1,0],[0,1],[1,1]]`、权重为单位阵，实测中间量：

```
Q·Kᵀ     = [[1, 0, 1],        scores = Q·Kᵀ/√2 = [[0.707, 0,    0.707],
            [0, 1, 1],                            [0,     0.707,0.707],
            [1, 1, 2]]                            [0.707, 0.707,1.414]]
attn     = [[0.4011, 0.1978, 0.4011],   行和 = [1, 1, 1]
            [0.1978, 0.4011, 0.4011],
            [0.2483, 0.2483, 0.5035]]
```

看懂这张表：第 3 行（token 是 [1,1]）和所有 token 的相似度都高，所以注意力更"平均"；第 1、2 行各自只跟自己像，于是对角权重最大。**这就是"注意力按相关性分配"的直观样子**。

② 为什么除以 √dk：实测标准差

```
for dk in (8, 64, 256):
    q = rng.normal(size=(2000, dk)); k = rng.normal(size=(2000, dk))
    dots = (q * k).sum(1)                    # 逐行点积
    print(dk, dots.std(), (dots / math.sqrt(dk)).std())
```

实测：

dk	未缩放 std	缩放后 std
8	2.76（≈√8=2.83）	0.98
64	7.94（≈√64=8）	0.99
256	16.15（≈√256=16）	1.01

这就是 B38 方差论证的实证：`Var[q·k] = d_k`、`std = √d_k`；除以 √dk 后所有维度的 scores 都回到 std≈1，softmax 不会饱和。**面试时能报出这三个数字，比只背公式强得多。**

③ 置换等变：打乱输入，输出同序打乱

```
out,      _ = attention_manual(X, W, W, W)
perm = np.array([2, 0, 3, 1])                # ← 语法点：用数组当下标做重排
out_perm, _ = attention_manual(X[perm], W, W, W)
print(np.allclose(out_perm, out[perm]))       # True
```

结论：attention 对输入顺序**完全无感**——所以必须加位置编码（B41）。这是"为什么非要 RoPE/正弦编码"的最硬证据。

④ 因果 mask 的实际权重矩阵

```
mask = np.triu(np.ones((4, 4), dtype=bool), k=1)   # ← 语法点：np.triu 取上三角
```

实测输出：

```
attn = [[1.000, 0.000, 0.000, 0.000],
        [0.001, 0.999, 0.000, 0.000],
        [0.109, 0.651, 0.240, 0.000],
        [0.080, 0.517, 0.172, 0.231]]     每行和都 = 1
```

读法：上三角全 0（看不到未来），下三角按相关性分配；每行仍然和为 1（归一化只在可见范围内做）。顺便能看到"越靠后的 token 分给第一个 token 的权重越小"，这是因为 softmax 重新归一化后的自然结果。

B6.6 这一节能给你什么（面试用法）

1. **被问"你怎么确认自己理解对了"**：答"我习惯用数值微分/小数据把公式跑出来验证，比如 softmax 的雅可比我用数值微分对过，交叉熵的梯度确实是 p−y"。
2. **被问"为什么除 √dk"**：直接报实测——"dk 从 8 到 256，点积标准差从 2.76 涨到 16.15，正好是 √dk"。
3. **被问"kmeans 为什么只是局部最优"**：报"同一数据 20 个初始化，inertia 从 22.7 到 1247，差 55 倍"。
4. **被问"信息增益有什么问题"**：现场写三行——用 ID 号特征算出和信息增益一样大的结果，再报增益率把它从 1.0 压到 0.387。

这四句话的共同点：都是"我跑过"而不是"我记得"。

Part C 图像处理与卷积网络

> 这一部分补 CV 基本功：**Part C1 图像处理**（C1–C12）→ **C2 CNN 组件与计算**（C13–C26）→ **C3 架构演进**（C27–C36）→ **C4 训练调优**（C37–C44）→ **C5 指标与损失**（C45–C50）→ **C6 与传统/多模态的接口**（C51–C54）。 特点：**凡是能算的都给出算式和具体数字**，面试时现场算一遍比背结论有说服力。

C1 图像处理基础（C1–C12）

C1 ★★ 一张图像在计算机里怎么表示？常见色彩空间各适合什么？

A：表示：`H×W×C` 的张量，`dtype` 常为 uint8（0–255）或 float32（归一化到 0–1 或 z-score）。

色彩空间	含义	适合什么
RGB	红绿蓝加色	显示、深度学习默认输入（但三通道高度相关）
HSV / HSL	色相、饱和度、明度	**颜色分割、颜色增广**（改色相不影响亮度）
YUV / YCbCr	亮度 + 两路色度	**视频编码/压缩**（人眼对亮度更敏感，色度可下采样 4:2:0）
Lab / CIELAB	接近人眼感知均匀	颜色差计算（ΔE）、跨光照的稳定性
灰度	单通道亮度	边缘/纹理/OCR 预处理

面试考点：为什么视频压缩用 YUV？→ 因为可以**对色度下采样**（4:2:0 直接省 50% 数据）而人眼几乎无感，RGB 做不到这一点（三通道同等重要）。

C2 ★★★ resize 用了什么插值？对下游有什么影响？

A：三种主流插值：

方法	做法	特点
最近邻	取最近的源像素	最快、无新值；**放大有锯齿，缩小会丢信息（欠采样别名）**
双线性	4 邻域加权（按距离）	快、平滑；**缩小会模糊**（相当于低通）
双三次	16 邻域（三次核）	质量最好、边缘更锐；稍慢

工程要点（面试常问）：- **缩小图像要先抗锯齿**（area 插值或先高斯模糊再采样），否则出现摩尔纹/别名——`cv2.INTER_AREA` 就是干这个的。- **深度学习里 resize 用双线性居多**（与 backbone 的平滑性匹配），**语义分割最好用双线性/双三次**（最近邻会让 mask 边界错位）。- **全卷积网络对分辨率敏感**：训练用 512、测试用 1024 会掉点，因为感受野/统计量变了。

C3 ★★★ 常见的图像滤波有哪些？各自适合去什么噪声？

A：

滤波器	原理	适合	代价/缺点
均值滤波	邻域平均	高斯噪声	边缘模糊严重
高斯滤波	按距离加权（σ 控制平滑度）	高斯噪声、平滑	边缘也糊；σ 越大越糊
中值滤波	邻域中位数	**椒盐噪声**（脉冲噪声）	慢（要排序）、对高斯噪声一般
双边滤波	空间距离 × **灰度差**加权	保边去噪	慢（非可分离）、对强噪声无解
引导滤波	用引导图（如原图）做局部线性模型	保边、可做**边缘保持上采样**	需要引导图
非局部均值（NLM）	按图像块相似度加权	强去噪、纹理保留好	很慢

面试加分：**为什么高斯核可分离？** → $G(x,y) = G(x)\cdot G(y)$，二维卷积可拆成两次一维卷积，计算量从 K^2 降到 $2K$（K=7 时从 49 次乘加到 14 次）。

3σ 原则：高斯核半径通常取 3σ，超出部分的权重 < 0.3%，可以截断。

C4 ★★★ Canny 边缘检测的四步是什么？为什么需要非极大值抑制？

A：四步：

1. **高斯平滑**（去噪，避免噪声被误判成边缘）；
2. **计算梯度**（Sobel 算 gx、gy → 幅值 $\sqrt{(gx^2+gy^2)}$ 和方向 $atan2(gy,gx)$）；
3. **非极大值抑制（NMS）**：沿**梯度方向**看相邻两个像素，只有当前点幅值最大才保留 → 把"粗边缘"细化成**单像素宽**；

4. **双阈值 + 滞后连接**：高阈值确定"强边缘"，低阈值给出"弱边缘"候选，只保留与强边缘**连通**的弱边缘 → 抑制孤立噪声点，同时保住边缘的连续性。

为什么需要 NMS：梯度幅值在边缘附近是一个"脊"（几个像素宽都很大），不抑制就会得到粗边缘、后续做轮廓/直线拟合会重复计数。

追问：双阈值的两个值怎么定？→ 经验是高:低 ≈ 2:1 或 3:1；也有自适应方法（按梯度幅值分位数）。

C5 ★★ 直方图均衡化 / CLAHE 在做什么？为什么工业上常用 CLAHE?

A：**直方图均衡**把灰度分布映射成近似均匀分布（累积分布函数 CDF 作为映射函数），从而**提升对比度**。

问题：全局均衡会**过度放大平坦区域的噪声**（比如大片天空的细微噪声被拉到可见），且图中有强亮/暗区时局部仍然看不清。

CLAHE（限制对比度的自适应直方图均衡）：把图分块（如 8×8）各自均衡，并用 `clipLimit` **截断直方图的峰值**（超出部分均匀分回），再做块间**双线性插值**消除块边界。

为什么工业界爱用：夜间/逆光/雨雾场景下能显著提升视觉可见性，且参数少、可复现、无学习成本。**我们做低光照预处理时它就是首选基线**。

C6 ★★ 相机畸变怎么建模和校正?

A：两类畸变：

> 径向畸变（镜头形状）： $x_d = x(1 + k1 \cdot r^2 + k2 \cdot r^4 + k3 \cdot r^6)$， $r^2 = x^2+y^2$
> 切向畸变（装配不平行）： $x_d = x + [2 \cdot p1 \cdot x \cdot y + p2 \cdot (r^2+2x^2)]$， y 方向对称

- k1 < 0 通常是**桶形**（广角/鱼眼），k1 > 0 是**枕形**。
- 校正：用内参 K（fx, fy, cx, cy）+ 畸变系数（k1,k2,p1,p2,k3），通过 `undistort` 反算 → 实际实现是**反向映射 + 插值**（因为正向映射会有空洞）。
- **鱼眼**要用专门的模型（等距投影 $r = f \cdot \theta$），普通针孔模型拟合不了。

面试加分：畸变系数怎么标定？→ 张正友标定法（多张棋盘格，用单应性求解内参初值，再非线性优化最小化重投影误差）；评估标准是**重投影误差**（RMS，好的标定 < 0.5 px）。

C7 ★★ 仿射变换和透视变换的区别？怎么求变换矩阵?

A：

	仿射（Affine）	**透视（Homography）**
自由度	**6**（保持平行性）	**8**（平行线可交于消失点）
矩阵	2×3（最后一行 [0,0,1]）	3×3（差一个尺度，8 自由度）
需要点对	**3 对**不共线点	**4 对**（任意三点不共线）
用途	旋转/缩放/平移/剪切	车道线 BEV、文档矫正、平面单应

求解：把对应点代入方程得到线性方程组，用 **DLT（直接线性变换）** 求最小二乘解（4 对点以上用 SVD），再用 RANSAC 剔除外点。

追问：为什么 BEV 变换常用单应而不是仿射？→ 因为相机是斜视地面的（透视投影），仿射保持不了"近大远小"，只有地面**平面**的单应才是精确的——**这也是为什么单应只对地面平面前提成立**（非平面区域会被拉变形）。

C8 ★★ 图像金字塔有什么用？拉普拉斯金字塔在做什么?

A：- **高斯金字塔**：逐层高斯模糊 + 下采样（每层尺寸减半），用于多尺度检测、特征匹配（SIFT 的尺度空间）。 - **拉普拉斯金字塔**：`L_i = G_i − upsample(G_{i+1})`，即"本层减去上层的上采样"，保存的是**各频段的细节**。 - **用途**：图像融合（把两张图的不同频段拼接，如多曝光融合/多聚焦融合）、图像压缩、超分。

为什么有用：不同尺度关注的语义不同（低分辨率看整体、高分辨率看细节），检测里的 FPN、分割里的多尺度特征都是这个思想的延伸。

C9 ★★ PSNR / SSIM / LPIPS 分别在衡量什么？

A：

```
MSE  = (1/N)Σ(I − K)²
PSNR = 10 · log10(MAX² / MSE)        （dB，越大越好；但与人眼感知相关性弱）
SSIM = (2μxμy + C1)(2σxy + C2) / [(μx²+μy²+C1)(σx²+σy²+C2)]    （结构相似性，[−1,1]）
```

- **PSNR** 只看逐像素误差，对"整体亮度偏移"极敏感、对"结构错乱"不敏感 → 常出现 PSNR 高但肉眼更差的情况。
- **SSIM** 从**亮度、对比度、结构**三项比较，更贴近人眼（局部窗口计算后取均值）。
- **LPIPS** 用预训练网络（VGG/AlexNet）的**深层特征距离**，与人类判断相关性最高，是现在超分/生成的默认指标。

面试话术："指标要和任务匹配：去噪看 PSNR/SSIM，超分/生成看 LPIPS + 感知质量，检测分割还是看任务的 IoU/mAP。"

C10 ★ 形态学操作有哪些？各自的作用？

A：四个基本操作：

操作	定义	作用
腐蚀	结构元覆盖区域内**取最小**	缩小前景、断开细连接、去小噪点
膨胀	取**最大**	扩大前景、填补小孔
开运算	先腐蚀后膨胀	**去小噪点/毛刺**，面积不变
闭运算	先膨胀后腐蚀	**填小孔/缝隙**，面积不变

进阶：顶帽（`原图 − 开运算`，提取小亮目标）、黑帽（`闭运算 − 原图`，提取小暗目标）、形态学梯度（`膨胀 − 腐蚀`，粗边缘）。

用途：二值 mask 的后处理（去孤立点、填洞）、OCR 字符粘连分离、工业缺陷检测的预处理。

C11 ★★ 图像数据增广有哪些？哪些任务不能用？

A：按类型：

- **几何**：翻转、随机裁剪、旋转、缩放、仿射/透视扰动。
- **颜色**：亮度/对比度/饱和度抖动、色调偏移、gamma、JPEG 压缩、模糊、噪声。
- **遮挡/混合**：Random Erasing、Cutout、**Mixup**（两张图线性混合 + 标签混合）、**CutMix**（贴一块区域 + 标签按面积混合）。
- **策略搜索**：AutoAugment、**RandAugment**（随机选 N 种变换 + 统一强度 M，超参只有两个）。
- **多尺度**：随机分辨率训练（尺度抖动）。

哪些不能用（面试高频）：- **OCR/文档**：颜色抖动、翻转、强裁剪都会破坏语义（文字方向、颜色对含义有影响）。- **检测/分割**：裁剪后必须**同步变换标签**（框/掩码），否则标签错位。- **VLM 指令微调**：增广会让"描述内容"与图不一致（颜色变了但 caption 没变），**制造假监督**。- **医学/工业**：翻转可能违反解剖/工艺先验（左右心、螺纹方向）。

C12 ★ 频域方法（傅里叶/小波）在图像里有什么用？

A：三条实际用途：

1. **滤波设计**：频域里做理想/巴特沃斯低通（但会振铃），高斯低通无振铃（因为高斯是自傅里叶变换）。
2. **周期性噪声去除**：在频谱图上找到亮点（周期噪声的频点）挖掉，再逆变换——纺织/扫描件的经典手法。
3. **小波**：多分辨率分解（低频近似 + 高频细节），用于图像压缩（JPEG2000）、去噪（小波阈值）、医学融合。

深度学习里的对应：频域卷积（FFT 加速大核卷积）、频域域适应（把低频（风格）与高频（结构）分离做风格迁移/域泛化）。

C2 CNN 组件与计算（C13–C26）

C13 ★★★ 卷积的输出尺寸怎么算？给一个具体例子。

A：

```
O = floor( (I + 2P − K) / S ) + 1
```

（I 输入边长、P padding、K 核大小、S 步长）

带空洞卷积时：`K_eff = D·(K−1) + 1`，再代入上式。

例子（面试常现场问）：- 输入 224，K=7，S=2，P=3 → `(224 + 6 − 7)/2 + 1 = 112`（这正是 ResNet 的 stem：7×7/s2 → 再 maxpool 3×3/s2 → 56） - 输入 56，K=3，S=2，P=1 → `(56 + 2 − 3)/2 + 1 = 28`（降一半） - 输入 32，K=3，S=1，P=0 → `30`（**same 尺寸必须 P=(K−1)/2，K 为奇数**）

追问：为什么核大小通常是奇数？→ ① 有中心像素（便于定义"以谁为中心"）；② same padding 时 `P=(K−1)/2` 是整数。

C14 ★★★ 感受野怎么算？三个 3×3 堆叠等于一个 7×7 吗？

A：递推公式（从浅到深）：

```
RF_l = RF_{l−1} + (K_l − 1) · Π_{i<l} S_i        初值 RF_0 = 1
```

三个 3×3（stride 1）：`1 → 1+2=3 → 3+2·1=5 → 5+2·1=7` → **感受野 7，等于一个 7×7**。

但不等价于一个 7×7 卷积：

	3 个 3×3	1 个 7×7
感受野	7	7
参数	$3·(9C^2) = 27C^2$	$49C^2$
非线性	**3 层**（表达能力更强）	1 层
计算量	$3·9HWC^2$	$49HWC^2$

结论："感受野相同，但堆叠版的**参数只有 55%、非线性多两层**——这就是 VGG 全用 3×3 的原因。"

追问：有效感受野（ERF）是什么？→ 理论感受野内每个像素的贡献并不均等，实际"起作用"的区域是**高斯状**的，中心大、边缘小（可用梯度/扰动实验测）。**所以理论感受野 7 不代表边缘信息真被用上了**。

C15 ★★★ 参数量和 FLOPs 怎么算？举一个对比。

A：

```
标准卷积参数 = (Cin · K · K + 1) · Cout          (+1 是 bias)
标准卷积 FLOPs（近似 MACs）= H_out · W_out · Cin · K · K · Cout
```

具体例子（Cin=Cout=256, K=3, H=W=56）：

	参数	MACs
标准卷积	$(256·9+1)·256 = 589{,}824$	$56·56·256·9·256 \approx 1.85\ G$
深度可分离（DW+PW）	$256·9 + 256·256 = 67{,}840$（**约 1/8.7**）	$56·56·(256·9 + 256·256) \approx 0.21\ G$（**约 1/8.7**）

深度可分离卷积 = 逐通道卷积（每个通道一个 K×K，输出通道数不变）+ 逐点卷积（1×1，跨通道混合）：

```
参数 = Cin · K · K + Cin · Cout        压缩比 ≈ 1/Cout + 1/K²
```

面试提醒：报"参数量"和"FLOPs"要区分**是否含 bias、是否算乘加为 2 次**（有的论文算 2×MACs），否则数字对不上。

C16 ★★★ 1×1 卷积有什么用？为什么它能"降维"？

A：四个作用：

1. **跨通道信息融合**：本质是"在通道维度做全连接"（每个位置独立的线性组合）。
2. **降维/升维**：把 C 通道变成 C'，`参数 = C·C'`，极便宜。ResNet 的 bottleneck（256→64→64→256）就是靠它把计算量压到 1/4~1/8。
3. **改变通道数而不改空间尺寸**：投影到目标维度。
4. **加非线性**：1×1 conv + ReLU 等价于"逐位置的两层 MLP"。

为什么能降维：它不聚合邻域信息（K=1），只在通道方向做线性映射，所以是"通道上的矩阵乘"。

面试延伸：ViT 里的 patch embedding 也是用 `K=patch_size, S=patch_size` 的卷积实现的（等价于切块 + 线性投影）。

C17 ★★ 池化 vs 步长卷积，怎么选？

A：

	最大池化	平均池化	步长卷积
作用	保留最强响应、扩大感受野	平滑、保留整体能量	降采样 + **可学习**
优点	无参数、对位置小变化鲁棒	无参数、梯度均匀	学得到更好的下采样方式
缺点	丢位置信息、梯度只在最大处	弱响应被平均掉	有参数、可能引入棋盘/混叠

趋势：现代架构（ResNet-D、MobileNet、ConvNeXt）**更倾向步长卷积或 avgpool + 1×1**，因为 maxpool 丢掉的信息无法恢复；而"下采样要配合抗混叠（anti-aliasing）"（BlurPool）也是常考点。

C18 ★★★ BN / LN / IN / GN 的区别？CV 里为什么默认 BN？

A：

归一化	统计量的维度	依赖 batch?	场景
BN	每个通道，跨 (N, H, W)	**是**	CNN 标配（大 batch）
LN	每个样本，跨 (C, H, W)	否	Transformer / RNN
IN	每个样本每个通道，(H, W)	否	风格迁移（抹掉逐图风格差异）
GN	每个样本分成 G 组，组内 (C/G,H,W)	否	**小 batch 检测/分割**（BN 失效时的替代）

> BN：$y = \gamma \cdot (x - \mu_B)/\sqrt{(\sigma_B^2 + \varepsilon)} + \beta$　　训练用 batch 统计量，推理用滑动平均

CV 里默认 BN 的原因：卷积核共享 → 同一通道的统计量在空间上稳定，跨 batch 估计更可靠；同时 BN 平滑了损失曲面（允许更大学习率）。

BN 的坑（必答三个）：① **小 batch 时统计量噪声大**（检测/分割 batch=1~4 时掉点严重）；② **训练/推理不一致**（忘记切换 eval 模式 = 经典 bug）；③ **分布差异大的任务**（多域、风格差异）统计量不匹配 → 用 GN/LN 或 AdaBN。

C19 ★★★ 激活函数怎么演进？为什么 ReLU 取代了 sigmoid？

A：

激活	特点	问题
sigmoid / tanh	平滑、有界	**两端饱和 → 梯度消失**（导数最大仅 0.25）
ReLU `max(0,x)`	正区间梯度恒 1、计算极快、稀疏激活	**死亡 ReLU**（负区间梯度 0，学习率大时神经元永久失活）
LeakyReLU / PReLU	负区间给一个小斜率（可学习）	多一个超参
ELU / SELU	负区间平滑趋近 −1	含 exp，稍慢
GELU	`x·Φ(x)`（高斯 CDF 加权）	稍慢；Transformer/ViT 默认

激活	特点	问题
SiLU / Swish	$x \cdot \sigma(x)$	主流大模型常用（平滑、非单调）

ReLU 取代 sigmoid 的三个原因：① 正区间**梯度不衰减**（解决深层网络梯度消失）；② 计算只有一次比较（比 exp 快得多）；③ 稀疏性（约一半神经元输出 0）→ 表征更解耦。

追问：SiLU/GELU 为什么更好？→ 在 0 附近**平滑且非单调**（有个小负值区），梯度更连续、对优化更友好；代价是计算量。

C20 ★★★ 残差连接为什么能缓解梯度消失？给个推导。

A：设残差块 $x_{l+1} = x_l + F(x_l)$，展开到第 L 层：

$$x_L = x_l + \Sigma_{i=l}^{L-1} F(x_i)$$

对 x_l 求导：

$$\partial L_{loss}/\partial x_l = \partial L_{loss}/\partial x_L \cdot \left(1 + \partial/\partial x_l \Sigma F(x_i) \right)$$

关键在于那个 1：即使后面的 ∂F 项因连乘而趋近 0，梯度仍能通过**恒等路径**无损回传。这是"梯度高速公路"。

附带好处：① 恒等映射让"加深至少不变差"成为可能（优化上的 ease）；② 特征复用/集成视角（ResNet 可看作多路径的隐式集成）。

追问：为什么是"加法"而不是"拼接"（DenseNet）？→ 加法**不增加通道数**（计算量恒定、可无限堆叠），拼接会增加通道数（后续层更贵，但特征复用更彻底）。

C21 ★★ 权重初始化怎么选？Xavier 和 He 的区别？

A：目标是**保持各层激活的方差稳定**（避免逐层放大/缩小）。

```
Xavier (Glorot) : Var(W) = 2/(n_in + n_out)      适合 tanh/sigmoid（对称激活）
He (Kaiming) :    Var(W) = 2/n_in                适合 ReLU（因为 ReLU 把一半激活置零，方差减半，所以要加倍）
```

推导思路：$Var(y) = n_in \cdot Var(W) \cdot Var(x)$，要 $Var(y)=Var(x)$ 就得 $Var(W)=1/n_in$；ReLU 输出只保留一半方差 → 补一个 2 倍。

实践经验：① 现代框架的默认初始化基本可用；② **BN 存在时初始化就没那么敏感**（BN 会重新标准化）；③ 太小的初始化 → 信号逐层衰减；太大 → 爆炸。

C22 ★★ 反卷积（转置卷积）为什么会有棋盘格伪影？怎么避免？

A：转置卷积实现上采样时，输出像素被覆盖的次数**不均匀**（核重叠次数随位置变化），导致某些位置值偏大 → 网格状纹路。

四种避免方法：1. 用 $stride=1$ 的卷积 + 前面的**插值上采样**（最常用、最稳，如 $upsample(bilinear) + conv3x3$）。2. 用 $stride$ 整除 $kernel$（如 S=2,K=4 或 S=2,K=2），让重叠均匀。3. **Pixel Shuffle**（亚像素卷积）：先卷积出 $C \cdot r^2$ 通道，再 reshape 成高分辨率（无重叠问题，超分里常用）。4. 用 resize + conv 替代（语义分割的常见做法）。

追问：转置卷积和"真反卷积"是一回事吗？→ **不是**。数学上的反卷积是卷积的逆运算；深度学习说的"反卷积"其实是**转置卷积**（梯度形式的卷积），不保证能还原输入。

C23 ★★ 空洞卷积的作用与副作用？怎么缓解？

A：**作用**：在不增加参数、不降分辨率的前提下**成倍扩大感受野**（$K_eff = D \cdot (K-1)+1$），DeepLab 系列用它替代池化。

副作用（网格效应 / gridding）：连续堆叠相同 dilation 的空洞卷积时，**采样点只落在固定格点上**，中间像素永远不参与计算 → 小目标/细节信息被跳过。

缓解：① **混合 dilation**（如 1,2,5 交替，HDC 设计）；② 串联不同 dilation 的空洞空间金字塔（ASPP）；③ 与普通卷积/上采样混用。

面试加分："我们的多任务模型里用 ASPP 做粗尺度辅助头，本质就是用多个 dilation 覆盖不同感受野，让粗头的监督更全局——这个辅助头给了 0.2 的权重。"（**这条把 C23 和你的实际配置连起来了**）

C24 ★★ 为什么卷积有"平移等变性"，而它有局限？

A：卷积 + 池化带来**平移等变（translation equivariance）**：输入平移，输出跟着平移。这是极强先验，也是 CNN 数据效率高的原因。

局限（三个）： 1. **下采样破坏等变性**：stride/pooling 会让等变只在采样格点上成立（亚像素平移不保持），这就是"CNN 对小平移不够鲁棒"的原因之一。 2. **边界效应**：padding 让边缘位置的等变性不成立。 3. **全局等变 vs 位置敏感**：检测里"同一物体在不同位置应有不同输出"，所以需要位置编码/坐标监督来**打破**纯等变。

顺带：ViT 因为几乎没有这个先验，才需要海量数据；ConvNeXt 用大核 + LN + GELU 把架构现代化，证明"CNN 的归纳偏置仍有效"。

C25 ★★ 混合精度、channels_last、融合算子各优化什么？

A：

技术	优化什么	注意
AMP（fp16/bf16）	显存减半、TensorCore 提速	fp16 要 loss scaling（防下溢）；**bf16 不需要**（动态范围大，但精度低）
channels_last（NHWC）	访存连续性（GPU 上 NHWC 对 1×1 卷积/BN 更快）	与某些算子不兼容，需要全链路统一
融合算子（conv+BN+ReLU 折叠）	减少 kernel launch 与访存	推理时常见；**训练时 BN 统计量要小心**
梯度检查点	显存换时间（重算激活）	约 30% 时间代价
torch.compile / TensorRT	图优化 + 算子融合	动态 shape 支持有限

C26 ★★ 卷积和全连接的关系？为什么现代网络几乎不用大 FC 层？

A：**全连接是卷积的特例**（K = 输入尺寸、感受野覆盖全图）；反过来，把 FC 的权重 reshape 就是卷积核。

不用大 FC 的原因：① 参数量爆炸（`H·W·C → N` 的 FC 是灾难级参数，如 VGG 第一个 FC 就有 1 亿参数）；② 丢失空间结构（不做归纳偏置）；③ 不能处理变尺寸输入。

现代做法：**全局平均池化（GAP）+ 一层小 FC**（ResNet 的做法），参数量从亿级降到 `C·N`。

C3 经典架构演进（C27–C36）

C27 ★★★ 串一遍经典 CNN 的演进线，每一步解决了什么？

A：

模型	年份	关键贡献	解决的痛点
LeNet-5	1998	conv + pool + FC 的雏形	手写数字识别
AlexNet	2012	ReLU、Dropout、GPU 训练、数据增广	深网络能训起来了（sigmoid 饱和被 ReLU 解决）
VGG	2014	**全 3×3 堆叠**、模块化	证明"深度 + 小核"有效（参数量爆炸是代价）
GoogLeNet/Inception	2014	**多尺度并行分支 + 1×1 降维 + GAP**	用宽度换深度、把参数从亿级降到 500 万
ResNet	2015	**残差连接 + BN + bottleneck**	解决深层退化（56 层比 20 层差）与梯度消失
DenseNet	2017	密集连接（每层接收前面所有层）	特征复用、参数更少；代价是显存
MobileNet/ShuffleNet	2017-18	深度可分离、通道混洗	端侧算力约束
EfficientNet	2019	**复合缩放**（深度/宽度/分辨率联合）	缩放策略从"拍脑袋"变"可搜索"
ConvNeXt	2022	把 Transformer 的设计（大核、LN、GELU、倒瓶颈）搬回 CNN	证明 CNN 的归纳偏置仍有价值

一句话串起来："AlexNet 证明能训深，VGG 证明深有效，Inception 证明要宽且省，ResNet 证明还能更深，后面的都在**效率**上做文章，ConvNeXt 是回头看'CNN 到底输在哪'。"

C28 ★★ Inception 模块的核心思想？为什么必须用 1×1 卷积？

A：**多尺度并行**：同一层输入同时过 1×1、3×3、5×5、maxpool 四条支路，再在通道维拼接 → 一个模块同时看不同感受野。

为什么必须 1×1 降维：如果直接对 256 通道做 5×5 卷积，参数是 $25 \cdot 256 \cdot 256 \approx 1.6M$；先用 1×1 把 256 降到 32 再 5×5，参数 $256 \cdot 32 + 25 \cdot 32 \cdot 32 \approx 34k$——**降了近 50 倍**。没有 1×1，这个模块根本装不下。

追问：Inception 和"多分支 ResNet（ResNeXt）"的区别？→ Inception 是**手工设计**的分支拓扑，ResNeXt 把分支变成**同构分组卷积**（组数可调），更规整、更好扩展。

C29 ★★★ ResNet 的 bottleneck 为什么是 1×1 → 3×3 → 1×1？

A：以 256→256 为例：

方案	参数量
直接 3×3：$9 \cdot 256 \cdot 256$	**589,824**
bottleneck：$1\times1\ 256\to64$ + $3\times3\ 64\to64$ + $1\times1\ 64\to256$	$16{,}384 + 36{,}864 + 16{,}384 = 69{,}632$（**约 1/8.5**）

逻辑：先用 1×1 **压缩通道**（降维）→ 在低维空间做 3×3（省算力）→ 再 1×1 **恢复通道**（升维）。这就是"bottleneck"命名的由来（中间窄、两头宽）。

深层设计原因：ResNet-50 之前用 basic block（两个 3×3），到 50 层以上必须用 bottleneck，否则算力下不来。

追问：为什么中间是 3×3 而不是更大的核？→ 3×3 是"最小的能同时感知上下左右"的核，且堆叠便宜（见 C14），是精度/效率的最优点。

C30 ★★ DenseNet 和 ResNet 的本质区别？

A：

	ResNet	DenseNet
连接	$x_{l+1} = x_l + F(x_l)$（**相加**）	$x_{l+1} = concat(x_l, F(x_l))$（**拼接**）
通道数	不变（可无限堆叠）	**线性增长**（第 l 层的输入通道 = $k_0 + k \cdot l$）
参数	普通	**更少**（因为每层只需输出很小的 k=32 通道）
显存	较低	**很高**（所有中间特征都要保存，供后续拼接）
特征复用	隐式（多路径集成视角）	**显式**（每层直接看到原始输入）

面试话术："ResNet 是加法式的恒等路径（省计算、能很深），DenseNet 是拼接式的全连接复用（参数效率高、但显存爆炸）。工业界主流是 ResNet 系，因为它好部署。"

C31 ★★★ 轻量化网络的三条路线？

A：

1. **减少卷积计算量**： - **MobileNet v1**：深度可分离卷积（DW + PW），算力降到约 1/8~1/9（见 C15）。 - **ShuffleNet**：分组卷积（group conv）省算力，但组间信息不流通 → **channel shuffle**（把通道打乱重排）恢复通信。 - **MobileNet v2**：**倒残差（inverted residual）**——先 1×1 升维 6 倍 → DW 3×3 → 1×1 降维，配 **线性瓶颈**（最后一层不用 ReLU，防止低维信息被截断）。
2. **减少冗余分支/结构搜索**：NAS（MnasNet）、EfficientNet（复合缩放：深度 ∝ α^φ、宽度 ∝ β^φ、分辨率 ∝ γ^φ，约束 $\alpha\beta^2\gamma^2 \approx 2$）。
3. **蒸馏/量化/剪枝**：结构不变，靠训练与压缩技术。

面试加分："端侧我会先算三件事——**参数（内存）、MACs（算力）、访存（带宽）**。很多轻量模型参数量很小但仍快不起来，原因是逐层访存（depthwise 的访存效率低、GPU 上不友好）。"

C32 ★★★ SE 和 CBAM 各是什么？SE 为什么能提点还不贵？

A：

- **SE（Squeeze-and-Excitation）**：`GAP`（把 H×W 压成 1×1）→ `FC → ReLU → FC → sigmoid` 得到每通道一个权重 → **逐通道缩放**特征图。本质是**通道注意力**。
- **CBAM**：**通道注意力 + 空间注意力**串联（通道分支用 avg+max 双池化，空间分支用一个 7×7 卷积生成 H×W 的权重图）。

SE 便宜的原因：两个 FC 之间有降维比 r（通常 16），所以参数量只有 $2 \cdot C^2/r$（C=256, r=16 → 8k），相比卷积的 590k 可以忽略。而收益是稳定的（SE-ResNet50 在 ImageNet top-1 上约 +1 个点量级）。

追问：SE 和"注意力机制"（Transformer 的 attention）是一回事吗？→ 不同。SE 是**通道级重新加权**（软特征选择），没有 query-key 的成对建模；Transformer 的 attention 是**位置间的动态聚合**。**但思想的共同点是"用全局上下文来调制局部响应"**。

C33 ★★ FPN 为什么要"自顶向下 + 横向连接"？

A：目标检测的核心矛盾：**深层特征语义强但分辨率低**（定位粗），**浅层特征分辨率高但语义弱**（分类差）。FPN 用两条路把它们结合起来：

1. **自底向上**（backbone 的正常前向）：得到 P3/P4/P5 不同尺度的特征；
2. **横向连接**：每个层级用 1×1 卷积把通道数统一（如 256）；
3. **自顶向下**：把深层特征**上采样**后与同层横向特征**相加** → 浅层也获得强语义。

结果：每个尺度都有"高语义 + 高分辨率"，小目标检测显著提升。**这也是分割（DeepLab/UNet）共用的思想。**

C34 ★★★ 分割网络的三条技术路线（UNet / DeepLab / HRNet）？

A：

路线	代表	核心	强项	弱项
编码-解码 + 跳连	**UNet**、SegNet	下采样提语义、上采样恢复分辨率，**skip 把浅层细节直接送回去**	医学/小数据、边界精细	参数量大（UNet 对称结构）
空洞卷积保分辨率	**DeepLab v1/v2/v3+**	不做激进下采样，用空洞扩大感受野；**ASPP**（多 dilation 并行 + 全局池化）捕捉多尺度	大场景语义分割	高分辨率下显存/算力大、空洞网格效应
全程高分辨率	**HRNet**	不做"先降后升"，多分辨率分支**并行 + 反复融合**	关键点/姿态/精细分割	计算量高

面试一句话："UNet 靠跳连找回细节，DeepLab 靠空洞不丢分辨率，HRNet 干脆全程不降——本质都是在解决'语义与分辨率的矛盾'。"

追问：ASPP 和 SPP 什么关系？→ ASPP 是 SPP 的空洞版（用多个 dilation 的并行卷积替代多尺度池化），都为了多尺度上下文。

C35 ★★★ 卷积和 Transformer 的本质取舍？ConvNeXt 做了什么？

A：

	CNN	ViT
归纳偏置	**强**（局部性、平移等变、权重共享）	**弱**（几乎无先验）
数据效率	小数据也能训	**需要大数据/强增广/蒸馏**
长程依赖	需要堆叠扩大感受野	**每层全局**
计算	$O(HWC^2K^2)$，与分辨率线性	$O(N^2)$，与分辨率平方
高分辨率	友好	贵（需窗口/稀疏注意力）

ConvNeXt 的结论：把 ViT 的现代设计搬回纯 CNN（**7×7 depthwise 大核**、**LN 替代 BN**、**GELU**、**倒瓶颈**、**更少的激活/归一化层**、LayerScale），就能追平 Swin Transformer —— 说明**当年"Transformer 更强"有一部分其实是"训练技巧与结构细节"的差异**，不全是自注意力的功劳。

面试话术："我会按落地约束选：数据少/要端侧 → CNN 或混合；数据多/要长程建模 → ViT；实际大都是混合（CNN 提局部 + attention 提全局）。"

C36 ★ 多模态模型为什么偏爱 ViT，但这个选择有什么代价？

A：**偏爱 ViT 的原因**：① patch token 序列**天然适配 LLM 的 token 接口**（不需要额外的空间池化/展平策略）；② 全局注意力对"整图关系"建模更强（对描述/推理类任务友好）；③ 与 CLIP 系预训练权重配套（直接可用）。

代价（这是加分点）： 1. **密集预测弱**：ViT 的全局注意力会把空间细节"抹匀"，做分割/深度/占用时要额外加 positional 信息或多尺度结构——**这正是我做 3D 占用时的体会：纯 ViT 特征对细粒度几何不够，需要 FPN/多尺度头来补**。 2. **高分辨率贵**：$O(N^2)$，所以要靠窗口注意力或 token 压缩。 3. **小数据难训**：必须依赖大规模图文预训练。

C4 训练与调优（C37–C44）

C37 ★★★ 学习率策略有哪些？为什么需要 warmup？

A：四种主流：

策略	形状	适用
Step / MultiStep	阶梯下降（每 N 轮 ×0.1）	经典 CNN（ResNet 的标准配方）
Cosine 退火	余弦曲线平滑降到 0	现代默认，配合 warmup
One-Cycle	先线性升到峰值再降到很低	训练轮数固定时收敛最快
Plateau	指标不涨就降	不确定总轮数时

为什么需要 warmup：训练初期参数随机、梯度方向噪声大（尤其 BN 统计量还没稳定），直接用大学习率会导致**震荡甚至发散**；warmup 让学习率从很小逐步升到目标值，给 BN 统计量和早期表征一个"稳定期"。Transformer/ViT 尤其依赖 warmup。

追问：余弦退火为什么比 step 好？→ 平滑下降避免"降学习率时的指标抖动"，且后期小学习率有利于精细收敛；另外 cosine 的周期性重启（SGDR）能帮助跳出局部极小。

C38 ★★★ SGD 和 Adam 怎么选？为什么经典 CV 模型用 SGD？

A：

	SGD + Momentum	Adam / AdamW
更新	用一阶动量平滑梯度，**所有参数同一学习率**	一阶 + 二阶矩，**逐参数自适应步长**
收敛速度	慢（要精调 lr 与 schedule）	**快**（对 lr 不敏感、易上手）
泛化	通常**更好**（尤其中小数据、CNN）	常常略差（有"尖锐极小"倾向）
显存	小（只存动量）	大（存 m、v，约 3 倍参数显存）

经典 CV 用 SGD 的原因：① 大量实验表明同样的超参预算下 SGD+动量 的**最终泛化**更好；② Adam 的自适应步长在卷积网络上容易收敛到"尖锐"的解；③ 传统 pipeline（长训练 + cosine + 大 batch）已经和 SGD 调得很顺。

但现在：Transformer/ViT/大模型基本都用 **AdamW**（解耦 weight decay），因为 LN + 自适应优化更匹配。

C39 ★★★ AdamW 和 Adam 的区别？为什么 weight decay 要"解耦"？

A：**Adam 的实现里，L2 正则被加进梯度**，然后被自适应学习率缩放：$\theta \leftarrow \theta - lr\cdot(\hat{g}+\lambda\theta)/\sqrt{\hat{v}}$。结果是**梯度大的参数被正则得弱、梯度小的被正则得强**（因为除以 $\sqrt{v}$），与"均匀惩罚大权重"的初衷不符。

AdamW 解耦：$\theta \leftarrow \theta - lr\cdot\hat{g}/\sqrt{\hat{v}} - lr\cdot\lambda\cdot\theta$，正则项**不经过二阶矩缩放**，直接按 lr 衰减权重。

效果：同样的 λ 下正则更有效、更稳定；这是现在大模型训练的标准做法。

面试延伸：这也是为什么"Adam 上直接调 weight_decay 经常没效果"——因为它被自适应缩放吃掉了。

C40 ★★ 正则化手段有哪些？分别在防什么？

A：

手段	机制	备注
weight decay / L2	惩罚大权重	与 BN 一起时要注意耦合（AdamW 解耦）
Dropout	随机置零 + 推理时缩放（或训练时 inverted dropout）	CNN 里现在少用（BN 已正则），FC 层仍常用
DropPath / Stochastic Depth	随机"跳过整个残差块"	**深网络/ViT 标配**，比 Dropout 更适合残差结构
Label Smoothing	软标签	防过度自信、提升校准
Mixup / CutMix	混合样本与标签	强增广，小数据提点明显；**训练更慢、对检测/分割需适配**
早停	验证指标不涨就停	最朴素也最有效
数据增广	见 C11	最"正统"的正则（不改模型）

追问：Dropout 和 BN 一起用为什么有时会掉点？→ 两者都在做"噪声/尺度"层面的正则，叠加后训练/推理的方差估计不一致（Dropout 改变激活方差，BN 统计量随之变化），实践中常改为只用其一。

C41 ★★ 迁移学习怎么微调？冻结哪些层？

A：按"数据量 + 领域差异"四象限决策：

	数据少	**数据多**
领域相近	只训分类头（冻结 backbone）	解冻全部，小学习率微调
领域差异大	训部分深层 + 头（浅层通用特征保留）	全部解冻（甚至从头训）

关键细节（面试常问）： - **BN 层**：微调时常见做法是**冻结 BN 的统计量**（保持预训练的 running mean/var，只训 γ/β），尤其小 batch 时——否则统计量会被目标任务的小 batch 带偏。 - **分层学习率**：浅层小（如 1/10）、深层大。 - **先冻后解冻**：先只训头几轮（稳定），再解冻微调（避免早期大梯度破坏预训练特征）。

C42 ★★ 训练 loss 正常但验证指标不涨，和过拟合怎么区分？

A：看**两条曲线的组合**：

现象	判断	处理
train loss ↓、val loss ↓ 但指标不涨	**指标与损失口径不一致**（见 S9 Q115）	拆指标成分、检查解码/阈值
train loss ↓、val loss ↑	**过拟合**	加正则/增广、减容量、早停
train loss 不降	**欠拟合或优化问题**	加容量、查 lr/初始化/数据管线
train loss ↓、val loss 平稳且指标差	**分布差异**（train/val 不同源）	查数据划分与预处理一致性

最常见也最隐蔽的一条：**训练和推理的预处理不一致**（归一化参数不同、resize 插值不同、通道顺序不同）——这种 bug 会让"训练很好、验证很差"，且极难从 loss 曲线看出。

C43 ★ 梯度累积和分布式训练的注意点？

A：

- **梯度累积**：`loss/N` 累加 N 步再更新，模拟大 batch。**注意 BN 的统计量仍按真实 batch 算**（不等价于真的大 batch，除非换 SyncBN 或 GN）。
- **DDP（DistributedDataParallel）**：多进程、每卡一个模型副本，梯度 all-reduce 同步——**推荐**。

- **DP（DataParallel）**：单进程多线程，主卡瓶颈严重——**已不推荐**。
- **SyncBN**：跨卡统计 BN，使"多卡小 batch"等价于"单卡大 batch"（检测/分割必备）。

C44 ★ 训练不稳定/掉点，排查顺序？

A：和 S9 Q117 同源，但 CV 特有两条**放在最前面**：

1. **数据管线一致性**（最常翻车）：训练/验证的 resize、归一化、通道顺序、色彩空间（BGR vs RGB）是否一致；
2. **BN 的 train/eval 模式**（忘记 `model.eval()` 是最经典 bug）；
3. 之后才是：label 越界 → 数值溢出（fp16）→ 学习率/warmup → 初始化/结构 → 硬件/框架。

C5 图像任务的评价指标（C45–C50）

C45 ★★★ 分类/检测/分割的核心指标各是什么？给一张能背的表。

A：

任务	主指标	定义要点	常见陷阱
分类	top-1 / top-5 acc	预测概率最高的类别是否命中 / 前 5 是否命中	类别不平衡时看 **macro-F1** 而非 acc
	AUC	排序质量，与阈值无关	极不平衡时 AUC 会虚高
检测	**mAP**	各 IoU 阈值下的 AP 平均	口径差异极大（见 C46）
	AP50 / AP75	IoU=0.5 / 0.75 的 AP	单一阈值不够全面
分割	**mIoU**	各类 IoU 的平均	小目标被大类淹没；边界不计
	Dice / F1	`2TP/(2TP+FP+FN)`	与 IoU 单调相关但不相等
	Boundary IoU	只算边界附近像素	对边界质量更敏感
深度	EPE / δ<1.25^k / bad pixel	平均端点误差 / 相对误差阈值 / 超阈值比例	尺度对齐方式不同

C46 ★★★ mAP 的口径差异有多大？VOC 和 COCO 怎么区分？

A：mAP 是"最容易被口径坑"的指标：

	VOC（2007/2012）	COCO
IoU 阈值	**单一 0.5**	**0.5 : 0.05 : 0.95 共 10 个**，再平均
PR 曲线积分	2007 用 11 点插值；2012 起用全点插值	全点插值（101 点采样）
类别平均	所有类 AP 的均值	同，但另有 APs/APm/APl（按面积）与 AR（召回）

结论：**COCO AP 的数值通常比 VOC mAP@0.5 低 15~25 个点**，两者不能直接比较。面试报指标必须说清"哪个口径、哪张表"。

追问：AP 的"插值"是什么？→ PR 曲线是锯齿状的，用"右侧最大值包络"（$p_{interp}(r) = \max_{r' \geq r} p(r')$）消除锯齿后再积分。

C47 ★★ mIoU 和 Dice 什么关系？为什么分割常报 mIoU 又报 Dice？

A：两者单调相关，有闭式关系：

$$Dice = 2 \cdot |A \cap B| \,/\, (|A| + |B|) = 2 \cdot IoU \,/\, (1 + IoU)$$

关系：IoU 0.5 ↔ Dice 0.667；IoU 0.8 ↔ Dice 0.889。所以 Dice 永远比 IoU 大。

为什么都报：① 领域习惯（医学爱用 Dice，自动驾驶爱用 mIoU）；② 两者对小目标的敏感度不同（Dice 对小目标更"宽容"，因为分母不含 TN）；③ 比赛/论文的口径要求。

Boundary IoU 补什么：mIoU 对"整体面积对、边界糊"不敏感；Boundary IoU 只在边界带内计算 IoU，专门衡量**边界质量**（这对占用/分割的精细度量很关键）。

C48 ★★ 深度/立体匹配的评价指标?

A:

- **EPE（End-Point Error）**：预测与 GT 的平均欧氏距离（视差或深度空间），最常用。
- **δ < 1.25^k（k=1,2,3）**：`max(pred/gt, gt/pred) < 1.25^k` 的像素比例（相对误差阈值），越大越好，是单目深度的标准指标。
- **Bad pixel（bad 1.0/2.0/3.0）**：误差超过阈值（如 1px 视差、或相对 3%）的像素占比。
- **AbsRel / RMSE / log10**：相对误差、均方根误差、对数误差。

坑：单目深度要看是否做过**尺度对齐**（median scaling / 最小二乘对齐）——不做对齐的 AbsRel 没有可比性（见 B 部分 Q108 尺度歧义）。

C49 ★ 生成质量的 FID/IS/CLIPScore 各有什么局限?

A:

指标	衡量	局限
IS	生成图的类别可分性	只在 ImageNet 类别上有意义、可被"每类各生成一张"刷高
FID	生成分布与真实分布在 Inception 特征空间的距离	依赖 Inception 特征（对纹理敏感）；**小样本时估计有偏**；对"语义错误但纹理像"不敏感
CLIPScore	图文一致性（CLIP 相似度）	只测粗粒度对齐（不擅长计数/空间关系）
人工评估	最终裁判	贵、主观、需要一致性设计

面试话术："FID 只能说明'像不像那个分布'，不能说明'对不对'——所以生成任务我都会配一个人工抽检或任务级指标。"

C50 ★ 图像任务的损失怎么选？（与 S9 交叉）

A：一句话汇总（详细见 S9）：

- **分类**：CE + label smoothing（不平衡用 focal）；
- **框回归**：SmoothL1 + IoU 族（GIoU/DIoU/CIoU）；
- **分割**：CE（主）+ Dice/Lovasz（辅）；
- **深度/视差**：SmoothL1 on 有效区域（+ 尺度不变项、+ photometric）；
- **去噪/超分**：L1（比 L2 更锐）+ 感知损失（LPIPS/VGG 特征）+ GAN 损失；
- **风格化**：内容损失 + 风格损失（Gram 矩阵）。

C6 与传统流程、多模态的接口（C51–C54）

C51 ★★ 预处理细节对 VLM/下游的影响有多大?

A：影响极大，且是最常见的"训练/推理不一致"来源。四个必须对齐的点：

1. **归一化参数**：CLIP 系用 `mean=(0.4815,0.4578,0.4082)`、`std=(0.2686,0.2613,0.2758)`（OpenAI 的口径），用 ImageNet 的 mean/std 会**直接掉点**。
2. **resize 策略**：短边 resize + 中心裁剪（分类）vs 直接拉伸（会变形，OCR 类任务致命）vs 保持长宽比 + pad。
3. **通道/色彩**：OpenCV 默认 **BGR**、PIL 是 RGB——混用会让红蓝互换（模型仍能跑，但性能莫名下降）。
4. **插值方式**：双线性 vs 双三次 vs area，在缩小倍数大时差异明显。

面试话术："我会把预处理写成一个**唯一函数**被训练和推理共用，并加一条断言校验（形状、dtype、均值范围），避免两边各写一套。"

C52 ★★ 图像增广在 VLM 微调里为什么有风险?

A：因为**标签是自然语言**，增广会破坏"图文一致性"：

- 颜色抖动 → 图像从"红色苹果"变"橙色苹果"，但 caption 还说红色 → **制造假监督**；
- 裁剪 → 目标被裁掉，但回答还在描述它；
- 翻转 → 文字反了、左右关系错了（"左边的车"变"右边"）；
- Mixup → 语义变成"两个物体的叠加"，而文本没有这样的描述方式。

安全做法：只做**不改变语义**的增广（轻微的缩放/亮度/JPEG 压缩/小角度旋转），或者用**同步生成的修改后文本**（成本高）。这也是"VLM 的数据增广比 CV 保守得多"的原因。

C53 ★ 深度学习时代，传统图像处理还有用吗?

A：有用，而且在工业里**不可替代**。四个位置：

1. **预处理**：去畸变、白平衡、CLAHE、去噪（这些是几何/物理校正面，学习做不好也不该学）。
2. **几何校正**：标定、单应、极线校正（立体匹配必须先做）。
3. **后处理**：形态学去噪、连通域分析、轮廓拟合、NMS（很多是确定性算子，比神经网更可控）。
4. **数据生产**：标注工具、自动标注（用传统几何生成伪标签）、数据质量检查（模糊/过曝/重复检测）。

一句话："神经网络负责**语义理解**，传统方法负责**几何正确性和确定性约束**——两者的分工在工业落地里很清晰。"

C54 ★★ 一个完整工业视觉链路的典型顺序?

A：从传感器到输出的标准流程（面试画出来很加分）：

```
采集 → 时间同步/去畸变（标定）→ 色彩/亮度校正（CLAHE、白平衡、去噪）
     → 几何变换（resize / 切块 / 单应到 BEV）→ 归一化 → 网络推理
     → 后处理（解码、NMS、阈值、形态学）→ 跟踪/融合 → 业务规则
```

每一环都可能成为瓶颈：采集（曝光）、同步（时间戳）、校正（标定漂移）、预处理（CPU 耗时）、推理（prefill/decode）、后处理（NMS 在 CPU 上很慢）。

面试收尾："所以我判断一个 CV 系统的问题，习惯先看**链路哪一段的输入输出不对**，而不是直接怀疑模型——80% 的线上问题在模型外面。"

C7 视觉基础模型：DINO 家族与 DPT（C55–C68）

> 这一节补两个在面试里高频、但极容易被答混的东西：**DINO（有两个完全不同的算法同名）** 和 **DPT（ViT 做密集预测的经典改造）**。它们分别对应两条真实需求：**选视觉塔**（DINO/DINOv2）和 **做像素级输出**（DPT，深度/分割）。

C55 ★★★ DINO 有两个完全不同的算法，怎么区分?

A：这是面试的高频陷阱，**同名但毫无关系**：

	DINO（自监督）	**DINO（检测器）**
全称	**DI**stillation with **NO** labels	**D**ETR with **I**mproved de**NO**ising anchor boxes
出处	Meta / Inria，2021（Caron 等）	IDEA，2022（Zhang 等）
任务	**自监督预训练**（不需要标签，学表征）	**目标检测**（有监督，出框）
输出	视觉特征 / 注意力图	检测框 + 类别
关键词	teacher-student 自蒸馏、EMA、multi-crop、防坍塌	3D 可变形注意力、对比去噪、混合 query 选择

	DINO（自监督）	DINO（检测器）
在 COCO 的地位	—	首个在 COCO 上刷到 63+ AP 的 DETR 系模型

答法："您说的 DINO 如果是视觉塔那个，是自监督的 DINO/DINOv2；如果是在检测榜单上那个，是 DETR 系的 DINO 检测器——两者只是缩写撞名了。"（**这一句能立刻分清你在哪一层**）

C56 ★★★ 自监督 DINO 怎么训练？为什么不会坍塌？

A：**自蒸馏（self-distillation）**：

```
同一张图做两个不同视角（crop/增广）→ student 看视角 A、teacher 看视角 B
teacher 的输出经过 centering + sharpening 后作为"软标签"
student 输出与之算交叉熵；teacher 的权重 = student 权重的 EMA（不参与梯度）
```

三个防坍塌的机制（必答）：

1. **EMA teacher**：teacher 更新慢（动量 0.99+），提供稳定的目标，避免 student 和自己同步跑偏。
2. **Centering**（减均值）：把 teacher 输出的均值减掉，**防止某一维主导**（否则所有样本都预测成同一个 one-hot）。
3. **Sharpening**（低温 softmax，τ≈0.04）：让 teacher 分布变尖，**防止输出塌成均匀分布**。

关键理解：centering 和 sharpening 的作用**方向相反**（一个鼓励均匀、一个鼓励尖锐），两者**必须配对**——只做 centering 会塌成均匀，只做 sharpening 会塌成 one-hot。

与对比学习的区别：DINO **不需要负样本、不需要超大 batch**（SimCLR/MoCo 靠负样本防坍塌），这是它工程上更友好的原因。

C57 ★★ 多裁剪（multi-crop）策略起什么作用？

A：每张图取 **2 个 global crop（大视野，如 224）+ 若干个 local crop（小视野，如 96）**：

- **global crop 之间**互相比（学全局语义、物体级不变性）；
- **local crop 只和 global 比**（学局部到全局的对应，即"这块局部属于哪个整体"）。

收益：在不增加 batch 的情况下大幅增加"同一图像的视角数"，等价于廉价的强增广；而且逼模型学"局部—全局"关系（这也是 DINO 的 attention map 能自然对应物体区域的原因之一）。

追问：local crop 之间为什么不互相比？→ 两个小局部之间可能本来就不属于同一个物体（甚至没有重叠），强行对齐会引入错误监督。

C58 ★★★ DINOv2 相对 DINO 的关键改进？

A：五个层面：

层面	改进
数据	自建 **LVD-142M**（约 1.4 亿图文/图像）——用自监督检索 + 去重从公开数据里"挑"出与目标分布相似的子集（而不是直接堆量）
训练目标	自蒸馏 loss **+ 掩码图像建模（iBOT 式）** 双目标：前者学不变性，后者学局部细节
防坍塌	**Sinkhorn-Knopp centering**（替代简单减均值）+ **KoLeo 正则**（鼓励特征在空间里分散，防止挤成一团）
规模	从 ViT-S 一路到 **ViT-g（约 11 亿参数）**，且**蒸馏出小模型**（大模型当 teacher，小模型学）
使用方式	明确主打"**冻结特征直接可用**"：linear probe / k-NN 就接近微调效果

一句话："DINOv2 = DINO 的自蒸馏 + iBOT 的掩码建模 + 更大的精挑数据 + 更好的防坍塌与正则。"

C59 ★★ DINOv2 的特征和 CLIP 特征有什么本质差别？

A：

	CLIP	DINOv2
监督信号	**图文对**（语言监督）	**纯视觉自监督**（无语言）
特征倾向	**语义**（"这是什么"、可被语言描述）	**结构与实例**（形状、部件、同类区分）
优势任务	零样本分类、检索、图文对齐	分割、深度、对应、检测（**密集任务**）
弱点	细粒度空间与实例区分较弱	没有语言接口（不能做零样本文本查询）

为什么会这样：语言监督会把特征"压缩成语义类别"（猫的照片和猫的简笔画在 CLIP 空间接近）；而无语言监督必须靠**实例区分**（这只猫 vs 那只猫），所以保留了更多结构与个体信息。

面试加分："所以做 VLM 时，如果任务是'描述/问答'，用 CLIP 塔；如果要'精准定位/细粒度属性'，DINOv2 更合适；两者特征拼接（concat）也是常见做法——一个管语义、一个管结构。"

C60 ★★ 什么时候用 DINOv2 做视觉塔，什么时候用 CLIP?

A：决策表：

任务特征	选
图文检索、零样本分类、描述生成	**CLIP/SigLIP**（有语言对齐）
需要细粒度定位（grounding、OCR 边界、计数）	**DINOv2 打底**，或在 CLIP 塔上再加 DINOv2 分支
密集预测（分割/深度/占用）	**DINOv2**（结构信息丰富）
端侧/预算紧	CLIP 系成熟小模型（ViT-B）或蒸馏后的 DINOv2 小模型

坑（值得提）：DINOv2 的 patch 是 14，输入分辨率要求是 14 的倍数（如 224/336/518），不能随便 resize；而且它的特征维度（ViT-g 1536 维）比 CLIP 大，接 LLM 时投影层要更宽。

C61 ★★ 顺便问：DINO 检测器（同名那个）的四个关键技术?

A：简要四个（如果面试官问的是检测）：

1. **3D 可变形注意力**：参考点 + 采样偏移，只关注周围少量关键位置（比全局注意力省，且适合多尺度）。
2. **对比去噪（contrastive denoising）**：给 GT 加噪声生成正负 query（正：小噪声；负：大噪声），让模型学会"区分接近 GT 的框和干扰框"，加速收敛。
3. **混合 query 选择（mixed query selection）**：位置部分用 encoder 的 top-k 输出初始化，内容部分用可学习 query（因为 encoder 输出的内容信息不可靠）。
4. **look forward twice**：梯度回传时用上一层的梯度信息修正下一层的参数更新（"看两次"），提升精度。

一句话："DINO 检测器的三个关键词是**去噪、混合 query、两次前向**——它解决的是 DETR 收敛慢的问题。"

C62 ★★★ DPT 是什么？为什么 ViT 不能直接做密集预测?

A：**DPT（Dense Prediction Transformer）** 是把 ViT 改造成"能输出像素级结果"的经典架构（用于单目深度估计、语义分割）。

ViT 直接做不了密集预测的三个原因：1. **只有单一低分辨率**：patch 16 的 384 输入只有 24×24 个 token，输出的"特征图"太小，无法恢复边界；2. **没有多尺度特征**（不像 CNN 有 P3/P4/P5 的层级），而密集预测恰恰依赖多尺度；3. **没有 skip 连接**：UNet 的精度来自"深层语义 + 浅层细节"，ViT 缺这一半。

DPT 的解法：从 ViT 的**多个中间层**取 token → **reassemble** 成空间特征图 → **逐级上采样 + 卷积融合**（把深层的全局语义与浅层的细节结合）→ 输出全分辨率预测。

一句话："DPT 就是给 ViT 补上 CNN 的'金字塔 + skip'这两件事，同时保留 ViT 的全局感受野。"

C63 ★★★ DPT 的 reassemble 和 fusion 具体做了什么?

A：

Reassemble（把 token 变回图像）：

```
输入：某一层 ViT 的 (N+1, D) token（含 CLS）
步骤：① 丢掉 CLS，把剩余 N 个 token reshape 成 (D, H/p, W/p)
      ② read：把相邻 2×2 个 token 在通道维拼起来（类似 pixel shuffle），空间减半、通道 ×4
      ③ project：1×1 卷积把通道投到目标维度（如 256）
      ④ 再双线性上采样到 patches 分辨率的 4 倍（即 H/4）
```

Fusion（逐级融合，从深到浅）：

```
从最深的 reassemble 特征开始：
  上采样 2× → 与下一层的 reassemble 特征 concat → 两层 3×3 卷积（通道减半）
  重复到最高分辨率
最后：Refine 模块（stride 2 下采样再上采样，抑制上采样的棋盘/伪影）→ 1×1 conv 输出 1 通道深度
```

为什么 reassemble 要"2×2 拼通道"：ViT 的 token 之间是**无空间邻接信息**的（只是一维序列），把 2×2 邻居拼进通道，等于**显式地把空间邻接关系还原**，比单纯 reshape 更能保住局部结构。

追问：为什么要用多个中间层而不是只用最后一层？→ 最后一层语义最强但空间细节最弱；**中间层（如第 6、9、12 层）兼有细节与语义**，这正是"ViT 版的 FPN"。

C64 ★★ 为什么 DPT 的深度边界比 CNN 更锐利？

A：两个原因：

1. **全局注意力**：每个 token 都能"看到"整幅图，所以它能判断"这块区域和远处那块是同一个平面还是不同物体"，而 CNN 受局部感受野限制、容易在边界处模糊。
2. **多尺度融合时的高分辨率通路**：reassemble 保留了 patch 级的空间信息，配合逐级上采样，边界不会被过度平滑。

代价：ViT 的 token 少（384 输入只有 576 个），细节恢复依赖上采样卷积——所以在"极细的边界"（如铁丝网、远处小物体）上反而不如高分辨率 CNN；这也是后来 Depth Anything 等用**更大输入分辨率 + 更强数据**来补的原因。

C65 ★★★ DPT 的训练损失为什么必须"尺度/位移不变"？

A：因为**单目深度有尺度歧义**：同一张图，预测 2 米和 4 米可能只差一个全局缩放，而"形状结构"完全正确。如果用普通 L1，模型会把大量梯度花在"整体缩放"这个本就不确定的方向上。

DPT 用 **scale-and-shift-invariant loss**：

```
先对预测做最小二乘对齐：d* = a · d_pred + b
其中 a、b 由最小二乘闭式解得到（与 GT 对齐）
然后 L = (1/n) Σ |d* − d_gt|
```

这正好对应 B 部分 Q108 的 SSI（尺度不变损失）——两者是同一个思想：**先消除不确定的全局尺度，再比较结构**。

追问：什么时候不能用尺度不变损失？→ 下游需要**绝对度量**时（占用、可通行区域、测距）。此时要靠传感器（激光/双目）提供绝对尺度，或用少量带尺度标注的数据做校准。

C66 ★★ DPT vs UNet vs SegFormer 的 MLP 头，怎么选？

A：

方案	结构	优势	劣势
UNet	CNN 编码-解码 + skip	边界精细、成熟、数据效率高	全局上下文弱（要靠深层堆叠）
DPT	ViT 多层 + reassemble + 融合	全局一致性好、边界锐利	参数量大、需要大输入分辨率才发挥
SegFormer 头	多尺度特征统一 MLP + 融合（无卷积）	极轻量、快	精细边界略弱

选型口诀："数据少、要精细边界 → UNet；要全局一致性（大面积深度/平面）→ DPT；要端侧速度 → 轻量 MLP 头。"

C67 ★★ Depth Anything 和 DPT 是什么关系？

A：**Depth Anything 用 DPT 作为解码头**，核心创新在**数据与训练策略**，不在网络：

1. **数据规模化**：用 6200 万张无标注图像做自监督（教师模型生成伪深度，学生学），相当于把 DPT 放进"数据引擎"里；
2. **语义先验**：利用 DINOv2 初始化视觉编码器（**注意：DINOv2 + DPT 是当前单目深度的主流组合**）；

3. **强增广**：对无标注数据做强扰动（颜色、模糊），逼模型对扰动不变；
4. **V2 的改进**：合成数据 + 更大教师 + 分阶段训练，并解决了"深度图有幻觉细节"的问题。

面试话术："Depth Anything 的贡献不是结构，而是把 DPT 这类头放进了'大规模数据 + 强增广 + 教师蒸馏'的流水线——这正好说明在视觉任务里，**数据和训练策略的收益常常大于结构创新**。"

C68 ★ 如果面试官说"我们想用 DINO 做视觉塔"，你会怎么回应？

A：这是个开放题，分三步答（展示你能落地）：

1. **先确认是哪个 DINO**："您指的是自监督的 DINO/DINOv2，还是检测的 DINO？"
2. **如果确定是 DINOv2 做视觉塔**，指出两个必须解决的对接问题： - **没有语言对齐**：DINOv2 特征不能直接零样本查询，需要靠指令微调把语义"教"进去（或与 CLIP 特征融合）； - **分辨率约束**：patch 14，输入必须是 14 的倍数；token 数与分辨率挂钩，要注意预算。
3. **给出建议**："我倾向 DINOv2 做**密集任务**（定位/深度/占用）的特征源，语义问答类仍用 CLIP/SigLIP；如果两者都要，就双塔拼接后过投影层——代价是 token 数和显存翻倍。"

这道题的价值：它同时考"你分不分得清同名模型"、"你知不知道 DINOv2 的局限"、"你能不能给出取舍"。

C8 Depth Anything V2 详解（C69–C78）

> Depth Anything 是当前**单目深度估计（MDE）的事实基线**，V2（2024.06）是零样本泛化的最强开源方案之一。 这一节把它的"为什么这么做"讲透——它是"数据策略胜过结构创新"的教科书案例。

C69 ★★★ V2 相对 V1 的核心改变是什么？（一句话能答）

A：**换掉了教师模型的训练数据来源**——V1 用"有标注真实图像"训教师，V2 改用"**合成图像**"训教师。

```
V1: 1.5M 有标注真实图 → 教师(ViT-L) → 给 62M 无标注真实图打伪标签 → 训学生
V2: 595K 合成图（Hypersim + Virtual KITTI 2）→ 教师(DINOv2-G) → 给 62M 无标注真实图打伪标签 → 训学生
                    ↑ 关键：教师换成"精确深度的合成数据"训练，规模也从 ViT-L 提到 ViT-G
```

为什么这个改动有用：伪标签的质量决定学生上限。真实数据集虽然有标注，但标注本身有噪声、覆盖窄、且**深度边界不精确**（标注来自传感器/立体匹配）；合成数据的深度是**渲染出来的真值，边界精确到像素**。用精确数据训教师，它给出的伪标签也就更干净。

C70 ★★★ 为什么"用合成数据训教师"反而更好？（最反直觉的一点）

A：三层理由：

1. **合成数据的深度是"精确的"，真实标注的深度是"近似的"**。室内数据集（如 NYU）的深度来自结构光/Kinect，边界处误差大；自动驾驶数据集来自激光，稀疏且需插值。而合成场景的深度由渲染管线直接给出，**每个像素都精确**。
2. **教师只需要"相对深度关系正确"，不需要覆盖所有场景**——它之后要在 62M 真实图上做预测，**分布多样性由被标注的数据提供，不需要教师见过**。这就是"精确性"比"多样性"更重要的原因。
3. **合成数据的分布窄是可以被接受甚至有益的**：场景简单、纹理干净 → 教师不会学到"纹理强的区域深度变化大"这种错误先验（V1 的幻觉细节正是这么来的）。

一句话："教师的任务是'给出精确的相对深度'，不是'认识所有场景'——所以用精确但窄的合成数据训，比用多样但带噪的真实标注训更合适。"

C71 ★★ 完整的三阶段训练流程？

A：

```
Stage 1｜教师微调
  DINOv2-G (ViT-g, ~1.1B) + DPT 头 → 在 595K 合成图上微调（编码器极小 lr，解码头较大 lr）
  → 得到一个"在合成域上精确"的教师

Stage 2｜伪标签生成
  用教师在 62M 无标注真实图上推理 → 得到伪深度
  → 质量筛选：去掉天空区域、去掉不可靠区域（透明/反光/极端光照）

Stage 3｜学生训练
  学生 = ViT-S/B/L + DPT 头 → 在 62M 伪标签图上训练（全参数）
  训练时用强增广（颜色抖动、高斯模糊、CutMix 等）逼学生学"与外观无关的几何" → 提升泛化
```

出三个尺寸的学生：ViT-S（24.8M）、ViT-B（97.5M）、ViT-L（335.3M）——**ViT-S 就已经接近 V1 的 ViT-L，但快约 10 倍**（这是最有说服力的工程收益）。

C72 ★★ 为什么教师要用 DINOv2-G（1.1B）？大教师对小学生的价值在哪？

A：两个层面的原因：

1. **DINOv2 的特征本身适合密集任务**（无语言监督 → 保留结构与实例信息，见 C59），适合做深度的几何基础。
2. **教师越大，伪标签越准**——论文观察到：即使大教师在**小合成集上已经过拟合**，它产出的伪标签仍然更好。原因很关键：**伪标签需要的是"相对深度关系正确"（谁近谁远），而不是合成域上的绝对精度**。大模型的表征能力强，在真实图上泛化出的深度关系更可靠。

面试话术："这是个很典型的'教师-学生规模解耦'案例——教师可以又大又会过拟合，只要它的伪标签在真实分布上关系正确；学生小、推理快，能力靠教师蒸馏过来。"

C73 ★★ 伪标签怎么筛选？为什么必须筛？

A：主要是**天空掩码 + 不可靠区域剔除**：

- **天空**：天空的深度在物理上无定义（无限远），标成任何值都是错的。V1/V2 都会把天空区域 mask 掉（或用语义分割/启发式检测）。
- **透明/反光物体**（玻璃、水面、镜子）：深度本身有歧义，容易产生假的深度跳变。
- **极端光照/过曝区域**：纹理信息缺失，模型只能靠语义猜。

为什么必须筛：伪标签是被当作"GT"用的，**错误伪标签会被学生当真理学进去**——这就是 V1 细节幻觉的根源。

C74 ★★★ V2 用的损失函数是什么？为什么必须尺度不变？

A：**scale-shift invariant loss（尺度-位移不变损失）**，与 V1 一致：

```
对"预测"和"GT"各自做同一套归一化（两边都要做，尺度/平移才被同时消除）：
    d^ = (d  − t(d))  / s(d)      t(d) = median(d),  s(d) = mean(|d − t(d)|)
    d* = (d* − t(d*)) / s(d*)
然后 L = (1/n) Σ |d^− d*|
```

三步走的实质：① 用**中位数**做平移对齐；② 用**平均绝对偏差**做尺度对齐；③ 在归一化空间里算 L1。 **为什么两个统计量选 median 和 MAE 而不是 mean/std**：中位数和平均绝对偏差对离群值（天空、极远点、遮挡空洞）更鲁棒，不会被少数极端深度带偏。

为什么必须：单目深度只有**相对**意义（仿射歧义）——预测 2 米和 4 米可能只差一个全局缩放。直接 L1 会让模型把梯度花在"整体缩放"这个本就不确定的方向上。

注意：这个损失**只在训练时**消除尺度；推理时输出的是"相对深度"，要用到度量任务必须另做尺度对齐（见 C77）。

C75 ★★ DA-2K 是什么？为什么要新造一个评测集？

A：**DA-2K** = 2000 张图，**人工标注的成对深度关系**（"A 比 B 更近还是更远"），用于评测相对深度的**边界/细节精度**。

为什么需要：现有 MDE 评测集有三个问题：① 深度 GT 本身有噪声（传感器/立体匹配），"接近 GT"不等于"对"；② 不同数据集用不同指标（AbsRel/RMSE/δ），口径不一致；③ **无法反映"细粒度边界是否准"**（V1 的幻觉细节在旧指标上分数很好）。

DA-2K 的做法：标注像素对之间的相对关系（人眼很容易判断），再用"判断正确的比例"作为指标。并在标注中结合了强 VLM 辅助（如 Qwen-VL / GPT-4o 先给初标、人工校验），提高了效率。

意义：这是"**指标比模型更容易骗人**"的正面案例——换成成对判断后，V1 的幻觉细节问题立刻暴露，V2 的提升也变得可量化。

DA-2K 的指标具体怎么算（这是一个纯粹的排序判断，没有"损失"）：

```
标注：一张图上标注若干像素对（i，j），标签是"哪个更近"（深度值更小 = 更近）
预测：模型输出深度图 d
判断：pred_label = (d[j] < d[i])  →  判为"j 更近"
指标：accuracy = 判断正确的对数 / 总对数
```

注意三点：① 它**只依赖相对顺序**，所以对全局尺度/平移完全免疫；② 它**不看绝对值**，所以度量错误、细节幻觉都会以"关系错"的形式暴露；③ DA-2K 特意在**边界/细结构等挑战区域加密标注**，否则随机像素对会被大片正确区域稀释。

我用一个 64×64 的合成例子实测了三个指标的行为（代码见 `demo_da2k_metrics.py`）：

情况	AbsRel	RMSE	δ<1.25	SSI 损失	成对准确率
A 完美预测	0.000	0.000	1.000	0.0000	1.000
B **只差一个仿射变换**（×2 + 3）	**2.558**	4.945	**0.000**	**0.0000**	**1.000**
C **细节幻觉**（局部区域关系反了）	**0.013**	0.161	**0.976**	0.1380	0.948

这张表把两件事一次说清了： - **B 行**：模型其实**完全正确**（只差全局尺度），但 AbsRel=2.56、δ=0 会把它判成"彻底失败" → 旧指标**冤枉**尺度歧义； - **C 行**：模型有细节幻觉，但 AbsRel=0.013、δ=0.976 **看起来非常好** → 旧指标**看不出**局部错误。

把 C 的成对准确率按区域拆开看，问题更清楚：

```
随机像素对   : 0.971   ← 大片区域是对的，把错误稀释了
边界/细节对 : 0.917   ← 细节幻觉在这里暴露
```

这就是 DA-2K 的意义：指标必须**对"关系错"敏感、对"尺度差"免疫**，才能测出 V1 那种"整体对、细节编"的问题。

C76 ★★ V1 的"细节幻觉"（hallucinated details）到底是什么问题?

A：V1 在以下场景会**凭空生成看起来合理但错误的深度细节**：

- **透明物体**（玻璃杯、窗户）：模型画出不存在的物体轮廓；
- **细结构**（毛发、铁丝网、细线）：出现假的深度跳变/条纹；
- **纹理丰富的平面**（草地、格子布）：把纹理误当成几何起伏；
- **天空/远景**：给出不一致的深度块。

根因：V1 的教师用**有标注真实数据 + 大量网络无标注图**训练，网络图像本身带有**贴纸、水印、纹理噪声**，教师从中学到了"纹理强 = 深度变化"的错误关联，再通过伪标签传给学生。

V2 怎么解决的：改用干净、精确的合成数据训教师 → 切断了这个错误关联的来源。**这也是"数据质量 > 数据规模"的直接证据**。

C77 ★★ Depth Anything V2 能不能直接用于度量任务（绝对深度）？

A：**原版不能**，它是仿射不变的相对深度。要用在度量任务上有三条路：

1. **Metric 版本**：官方发布的 `Depth-Anything-V2-Metric-Indoor / Outdoor`，在带绝对尺度的合成数据（Hypersim/VKITTI2 的 metric 标注）上微调，直接输出米制深度。**前提是场景类型匹配**（室内用它训室内、室外用室外），跨域会掉。
2. **用传感器标定尺度**：拿激光/双目/雷达的稀疏深度，对 DA-V2 的相对深度做 **scale-shift 对齐**（最小二乘解 a、b），再用于度量判断——**这是工业落地最常见、最稳的做法**。
3. **只用"关系"不用"数值"**：做排序、可通行区域（比阈值）、遮挡判断这类任务，相对深度就够。

面试话术（对你特别有用）："我会用 DA-V2 做稠密的几何先验、用激光/双目做尺度锚点，两者做一次最小二乘对齐——这样既有单目的稠密性，又有传感器的度量正确性。"

C78 ★ 在你自己的感知任务里怎么用它？（三条可落地）

A：

1. **补激光稀疏**：远距离/低线束区域激光点稀疏，DA-V2 提供稠密相对深度，用近处激光点拟合 scale-shift 后插值补远——**这是单目深度最主流的工程用法**。
2. **作为占用/可通行预测的几何先验**：把 DA-V2 的深度（或它的倒数 = 视差图）作为额外输入通道送进 BEV 网络；或用它做"自由空间的初始猜测"，让占用头只学修正量（类似残差预测）。
3. **训练期作为辅助监督**：给没有激光的场景提供深度自监督信号（配合光度一致性），缓解 GT 覆盖率不足的问题。

三个必须注意的坑（能说出来说明你真跑过）：- **逐帧推理会闪烁**（视频时序不一致），要加时序平滑或滑窗中值；- **DA-V2 在天空/透明物体上不可靠**，用作几何先验时要 mask 掉（V2 自己也 mask）；- **强增广训练出来的模型对"外观"不敏感，但对"极端光照"仍会崩**，夜间/雨雾要么单独训，要么只用它做近场。

C9 激活函数专题（C79–C88）

> 激活函数考的是"**为什么选它、它有什么病、怎么治**"。这一节把每条都给出实测数字或具体机制，不只列公式。

C79 ★★★ 为什么必须有非线性激活？全用线性会怎样？

A：因为**多层线性叠加仍等价于一层线性**：

```
W₂(W₁x) = (W₂W₁)x = Wx        ← 三个矩阵乘等于一个矩阵乘
```

所以没有激活函数，无论堆多少层，模型的表达能力都只有"一个线性变换"——**连 XOR 都学不会**（XOR 不可线性分割）。

一句话：激活函数是深度的**唯一意义来源**；它把网络从"线性回归"变成"通用函数逼近器"。

追问：神经网络为什么能逼近任意连续函数？→ 万能逼近定理（Universal Approximation）：**单隐层 + 非线性激活 + 足够宽的神经元**可以以任意精度逼近紧集上的连续函数。注意它只说"存在"，不说"能训出来"——所以实际要靠深度 + 优化技巧。

C80 ★★★ sigmoid 为什么被淘汰？（三个具体原因 + 一个实测数字）

A：

原因	说明
① 梯度消失（最致命）	导数 $\sigma'(x) = \sigma(1-\sigma) \leqslant 0.25$。**实测最大导数正好 0.25**。10 层连乘就是 $0.25^{10} \approx 1e\text{-}6$——梯度彻底消失。**tanh 的导数上界是 1.0、ReLU 是 1.0**，所以活下来了。
② 输出非零中心	$\sigma(x) > 0$ 恒成立，导致下一层的输入全为正 → 梯度更新方向被"锁"在同一个象限 → 出现 zigzag 震荡，收敛慢。tanh 是零中心的，所以在 RNN/生成模型里仍有一席之地。
③ 计算贵	含 $\exp$。实测 1e6 个元素：sigmoid 5.9ms vs ReLU 3.2ms（差约 2 倍，在 NPU 上差距更大）。

注意：sigmoid **没有消失**，它还有两个正当用途：**二分类输出层**（概率意义）、**门控机制**（LSTM 的遗忘门、GLU 的门）。

C81 ★★★ ReLU 好在哪？有什么问题？

A：

```
ReLU(x) = max(0, x)        导数 = 1 (x>0) 或 0 (x<0)
```

三个优点：1. **正区间梯度恒为 1** → 不衰减，深网络能训（这是它取代 sigmoid 的核心原因）；2. **计算只有一次比较**（实测比 sigmoid 快约 2 倍，且 NPU/GPU 有专门硬件）；3. **稀疏激活**（约一半神经元输出 0）→ 表征更解耦、有效计算量下降。

三个问题：1. **死亡 ReLU**（最严重，见 C82）；2. **非零中心**（和 sigmoid 一样的毛病）；3. **在 0 处不可微**——实践中无所谓（框架取次梯度 0 或 1），理论分析时要注意。

C82 ★★★ 死亡 ReLU 怎么产生？怎么检测？怎么治？

A：

产生机制：如果某个神经元的输入被推到恒负区间（例如学习率过大、初始 bias 太负、或者数据分布偏移），它的梯度就**恒为 0**——**而且再也不会恢复**，因为它没有任何方式把输入拉回正区间。这就是"永久失活"。

实测演示（把一个神经元的 bias 人为设成 −10，训练 30 步）：

```
ReLU        参数总位移 = 0.000000    ← 完全不动，已经死亡
LeakyReLU   参数总位移 = 0.053267    ← 负区间还有 0.01 的斜率，能慢慢恢复
```

检测方法：统计每层激活的**零值比例**（前向时记录），健康网络通常 30%~60%；如果某层 **> 90% 且持续上升**，说明大量神经元死亡。

四个治法：1. **换 LeakyReLU / PReLU / ELU**（负区间给斜率，最主要的手段）；2. **He 初始化 + BN**（保证初始输入分布合理，`Var(W) = 2/n_in`）；3. **降学习率 / 减小 weight decay**（大 lr 是头号诱因）；4. **避免大的负 bias 初始化**（bias 用 0 或小的正数）。

C83 ★★ LeakyReLU / PReLU / ELU / SELU 各改了什么？

A：

激活	公式（负区间）	改了什么	代价
LeakyReLU	`αx`（α=0.01 常见）	给负区间一个固定小斜率 → 不死	多一个超参 α
PReLU	`αx`，**α 可学习**	让网络自己决定斜率	每通道一组参数，小数据易过拟合
ELU	`α(e^x − 1)`（→ −α 饱和）	负区间**平滑且饱和**，输出均值更接近 0 → 收敛更稳	含 exp（慢）；实测负区间梯度 0.022
SELU	特化的 ELU（λ、α 固定）	**自归一化**：配合 `lecun_normal` 初始化，能让每层输出保持 均值 0 方差 1	对初始化和网络结构极敏感，实际很少有人用

实测的"负区间平均梯度"（x < −2）：ReLU **0**、LeakyReLU 0.010、ELU 0.022、GELU **−0.008**、SiLU **−0.039**。

注意最后两个是负的——这说明 GELU/SiLU **非单调**（有小负值区），它们的"恢复能力"不是靠斜率，而是靠形状本身。

C84 ★★★ GELU 是什么？为什么 Transformer/ViT 都用它？

A：

```
GELU(x) = x · Φ(x)          Φ 是标准正态的累积分布函数
工程近似：0.5x(1 + tanh(√(2/π)(x + 0.044715x³)))     ← 实测最大误差仅 0.00041
```

直觉：`Φ(x)` 是"x 有多大概率大于 0"，所以 GELU ≈ **"按概率保留输入"**——可以看作 ReLU 的**平滑随机版本**（等价于对 dropout 做确定性期望）。

为什么 Transformer/ViT/BERT/GPT 用它：1. **平滑**：处处可导，梯度连续，对深层优化友好；2. **非单调**：在 x ≈ −0.17 附近有轻微负值（实测导数最小值 −0.0076），让"弱负激活"也能传递一点信号——比 ReLU 的硬截断更细腻；3. **经验效果好**：BERT/GPT/ViT 用它，社区跟随；而且实验上比 ReLU 略好（幅度不大，1 个点以内）。

实测导数上界：GELU **1.129**、SiLU **1.100**、ReLU/tanh **1.0**、sigmoid **0.25**——前两者的导数略大于 1，深层叠加时要注意梯度放大（配合 LN 使用即可）。

C85 ★★★ SwiGLU / GeGLU 是什么？为什么大模型都用？参数量怎么补齐？

A：GLU 家族是**门控**结构：

```
GLU(a, b)  = a ⊗ σ(b)          ← 一路做内容，一路做"闸门"
SwiGLU(a,b)= a ⊗ SiLU(b)       ← LLaMA 系用它
GeGLU(a,b)= a ⊗ GELU(b)        ← PaLM 系用它
```

因为要**两个投影 + 一个输出投影**（共 3 个矩阵），比标准 FFN（2 个矩阵）多 50% 参数：

```
标准 FFN:      2 × (4096 × 11008) = 90.2 M
SwiGLU(同 F):  3 × (4096 × 11008) = 135.3 M   ← 多 50%
SwiGLU(F'=2/3F): 3 × (4096 × 7338) = 90.2 M   ← 基本持平 ✓
```

这就是为什么 LLaMA 的 FFN 维度是 `8/3·d`（而不是 GPT 的 `4·d`）——取 2/3 的 4d，正好抵消第三个矩阵带来的增量。这个数字推导面试很值分。

为什么大模型偏爱 GLU：乘法门控提供了**二阶交互**（`a·σ(b)` 能表达"当 b 满足条件时放大 a"），比逐元素非线性表达力更强；多篇论文报告在同样参数量下 perplexity 更低。

C86 ★★ 视觉和语言怎么选？端侧有什么特殊考虑？

A：

场景	首选	理由
CNN 检测/分割（追求速度）	**ReLU**	最省算力、NPU/GPU 硬件友好
CNN 追求精度/现代化	**SiLU 或 GELU**	ConvNeXt 用 GELU、YOLO 系新版用 SiLU
ViT / Transformer 视觉塔	**GELU**	与 BERT 系一致、平滑
LLM 的 FFN	**SwiGLU / GeGLU**	门控更强，LLaMA/PaLM 系标准
RNN / 生成模型隐层	**tanh**	零中心、有界（利于稳定性）
二分类输出	**sigmoid**	有概率意义
多分类输出	**softmax**（不是逐元素激活）	归一化到概率单纯形

端侧的三条特殊考虑（你面华为会用到）： 1. **ReLU 对量化最友好**（INT8 可直接做，且 NPU 有专门单元）；**GELU/SiLU 通常要用查表（LUT）或分段多项式近似**，会引入误差且可能不被算子支持； 2. **SiLU 比 GELU 便宜**（实测 6.8ms vs 23.9ms，因为 SiLU 只要一个 sigmoid）； 3. 端侧模型经常把 **GELU 换成 ReLU6 或 H-Swish**（MobileNetV3 的做法），用一点精度换可部署性。

C87 ★★ 激活函数和归一化的区别？（最常被混淆的一对）

A：

	激活函数	归一化（BN/LN）
作用	**引入非线性**	**调整分布（均值/方差）**
是否可学习	ReLU 无参数；PReLU 有	γ、β 可学习
是否改变表达力	**是**（没有它网络退化成线性）	否（只是仿射重参数化，但影响优化）
能否互相替代	**不能**	**不能**

常见误解： - "BN 也有非线性"——**错**，BN 是仿射变换，纯线性； - "用了 LN 就不需要激活"——**错**，LN 之后仍要有激活（Transformer 的 FFN 是 `LN → Linear → GELU → Linear`）；
- "激活函数负责把值压到 0~1"——只有 sigmoid 是，ReLU/GELU **不归一化**。

C88 ★★ 五个常见误区与面试陷阱

A：

误区	正确说法
"softmax 是激活函数"	**它是归一化**（跨类做，输出和为 1）；逐元素激活是另一回事。混淆这一点在面试里很致命
"tanh 比 sigmoid 好，所以都该用 tanh"	tanh 只是解决了零中心，**梯度上界仍是 1（实测）**，深层仍会衰减；真解决是 ReLU 系
"ReLU 不可微所以有问题"	实践中框架取次梯度，**没有实际影响**；真要平滑用 GELU/SiLU
"非零中心只是理论问题"	实际会让收敛变慢（zigzag），是 sigmoid 被淘汰的第二大原因
"激活函数越大越好（导数 > 1）"	GELU/SiLU 的导数略大于 1（1.13/1.10），**深层会放大梯度**，必须配归一化用

最后一个陷阱最值得记：很多人看到"GELU 导数上界 1.13"会觉得"更好"——**实际上这会让残差堆叠时梯度指数放大**，所以现代架构都要配 LN 或 LayerScale（ConvNeXt 的 LayerScale 初始值 1e-6 就是防这个）。

附：复习优先级（A/B/C 三部分）

优先级	题目
必须能讲（A 部分 17 题）	A1 A2 A3 A7 A13 A14 A23 A24 A31 A32 A33 A39 **A46 A58 A62 A66 A68**
必须能推（B 部分 14 题）	B2 B3 B9 B12 B13 B14 B17 B18 B20 B29 B30 B37 B38 B40
必须能分清/讲清（C 部分模型篇 6 题）	**C55**（两个 DINO 同名不同物）**C56**（自蒸馏为什么防坍塌）**C58**（DINOv2 改进）**C62**（ViT 为什么不能直接做密集预测）**C63**（DPT reassemble+fusion）**C65**（尺度不变损失）**C69**（DA-V2 核心改动）**C71**（三阶段流程）
必须能讲（C 部分激活 4 题）	**C80**（sigmoid 为什么死：导数 0.25）**C82**（死亡 ReLU 三治）**C84**（GELU 非单调）**C85**（SwiGLU 的 2/3 补齐）
必须能算（C 部分 10 题）	**C13**（输出尺寸）**C14**（感受野）**C15**（参数量/FLOPs）**C18**（BN/LN/GN）**C20**（残差梯度）**C29**（bottleneck 参数）**C38**（SGD vs Adam）**C45/C46**（指标口径）**C51**（预处理一致性）
能讲 1 分钟（A 其余）	A4–A6 A8–A12 A15–A22 A25–A30 A34–A38 A40–A45
能讲 1 分钟（B 其余）	B1 B4–B8 B10 B11 B15 B16 B19 B21–B28 B31–B36 B39 B41–B45
能讲 1 分钟（C 其余）	C1–C12 C16 C17 C19 C21–C28 C30–C37 C39–C44 C47–C50 C52–C54 C57 C59–C61 C64 C66–C68

现场最值钱的七个"能算"（面试官会让你当场推）

1. **卷积输出尺寸** `O = floor((I + 2P − K)/S) + 1`（C13）
2. **感受野递推** `RF_l = RF_{l−1} + (K_l−1)·Π S_i`，三个 3×3 = 一个 7×7 的感受野，但参数只有 55%（C14）
3. **参数量对比** 标准 3×3(256→256) = 589,824 vs 深度可分离 = 67,840（约 1/8.7）（C15）
4. **bottleneck 省算力** 589,824 → 69,632（约 1/8.5）（C29）
5. **softmax+CE 的梯度** `∂L/∂z = p − y`（B3）
6. **为什么除 √d_k** `Var[q·k] = d_k`（B38）
7. **残差为什么救梯度** `∂L/∂x_l = ∂L/∂x_L·(1 + ∂ΣF/∂x_l)`，那个 1 是恒等路径（C20）

B/C 两部分最容易被追问的三个"为什么"

- softmax 为什么减 max（B2）→ 上溢成 inf 再除得 nan，数学等价
- kmeans 为什么只是局部最优（B30）→ 非凸 + 坐标下降，附实测 20 个种子 7 次失败
- BN 为什么 CV 默认用（C18）→ 卷积核共享让通道统计量稳定，同时平滑损失曲面

手撕 AI 题详解 —— C++ 版

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 面向：**C++ 强、Python 弱的候选人**。全文所有代码都是 C++，可直接编译运行，已通过 26 项断言。 配套可运行文件：`ai_interview_cpp.cpp`（同目录），`g++ -O2 -std=c++17 ai_interview_cpp.cpp -o ai && ./ai`。 阅读方式：第 0、1 章必读（决定你现场怎么写）；第 2 章每题先背"口述脚本"，再背代码骨架，最后扫一遍追问。

---

0. 先解决策略问题：手撕 AI 题到底用什么语言

0.1 结论：用 C++，并且是优势，不是妥协

三个理由，你可以直接说给面试官听：

1. **公式看得见**。`np.dot` 一行看不出你到底懂不懂，而 C++ 里 `s += Q[i*D+t] * K[j*D+t]` 三次循环把"q 与 k 做点积、第 t 维累加"写在脸上——面试官要判断的正是这个。
2. **不依赖环境**。现场共享屏幕，对方可能只给你一个空白编辑器。C++ 只要 `#include <bits/stdc++.h>`，纯 STL 就能跑；torch 版你还要祈祷对方环境里有 GPU/正确版本。
3. **和你的机试语言统一**。华为机试 3 题你本来就用 C++，两套语言切换是额外风险。

开场话术（记住，一句话）：

> "手撕我用 C++ 写，纯 STL，这样每一层矩阵乘和 softmax 的维度我都能显式展开，方便您看我理解得对不对。"

这句话把"我不会 Python"翻译成了"我选择更透明的实现"。绝大多数面试官会说"可以，随你"。

0.2 三个风险点与应对（务必心里有数）

风险	触发场景	应对
面试官坚持要 PyTorch 版	对方团队全是 python 栈	用 4.2 节那三段"可背版"（MHA / softmax+CE / kmeans 向量化），只说"API 层面的封装我知道怎么写，核心逻辑和刚才 C++ 里一模一样"
C++ 写太长，15 分钟写不完	你没提前压缩代码	用第 1 章的"扁平化 + matmul 工具函数"，把 MHA 压到 45 行内；CART 压到 45 行内
边界写崩（数组越界/维度算错）	现场紧张	每道题开头先写 3 行尺寸变量（`int L, D, dk;`），再写循环，最后心里过一遍 `[i*cols+j]` 有没有越界

0.3 谁会被问哪道题（按岗位猜）

题	被问概率	谁爱问
softmax / 交叉熵	极高（几乎是开场题）	所有
self-attention / MHA	极高（多模态岗的核心）	所有
kmeans	中高（聚类、数据清洗、量化里都用）	数据侧 / 端侧
分类树 CART	中（传统 ML 基础题）	偏算法基础的老面试官
IoU / NMS	中高（你有检测背景，必被顺手问）	感知方向
LayerNorm / BN	中（问 Transformer 时连带）	大模型方向

---

1. 通用脚手架：写一次，六道题都复用它

1.1 两个约定（这是你比 Python 选手省时间的地方）

约定 A：矩阵一律用一维 `vector<float>` 扁平存，行主序，`A(i,j) = A[i*cols + j]`。

为什么不用 `vector<vector<float>>`： - 内存连续，cache 友好（讲解时可以说这句，显得懂工程） - 传参只传一个对象 + 两个尺寸，不会写出 `A[i][j]` 忘了 `.size()` - 面试现场最省时间：一行 `vector<float> A(n*m)` 就建好了

约定 B：所有尺寸用 `int` 显式传参，绝不在循环里现算。

例：`matmul(A, B, n, k, m)` 里的 n/k/m 就是 A 的行、A 的列(=B 的行)、B 的列。写代码时先把这三个数在纸上（或注释里）标出来，再动笔。

1.2 工具箱函数 matmul（六道题里四道要用）

```
// A: n x k, B: k x m  ->  C: n x m
vector<float> matmul(const vector<float>& A, const vector<float>& B, int n, int k, int m) {
    vector<float> C(n * m, 0.f);
    for (int i = 0; i < n; ++i)
        for (int t = 0; t < k; ++t) {
            float a = A[i * k + t];
            if (a == 0.f) continue;
            for (int j = 0; j < m; ++j) C[i * m + j] += a * B[t * m + j];
        }
    return C;
}
```

逐行拆解（面试官常让你边写边讲，这段照读就行）：

- `C(n*m, 0.f)`：结果矩阵先全零。**注意必须先初始化**，否则 `+=` 会累加到垃圾值上——这是最常见的现场事故。
- 三层循环 `i → t → j`：C 的每个元素 `C[i][j] = Σ_t A[i][t]·B[t][j]`。**为什么要 i-t-j 而不是 i-j-t**：i-t-j 顺序下，内层 `j` 循环访问的 `C[i*m+j]` 和 `B[t*m+j]` 都是连续内存，缓存命中高；i-j-t 顺序会把 B 每隔 m 个跳着读。面试时补一句"这里我选择 i-t-j 以保持访存连续"，是加分项。
- `if (a == 0.f) continue;`：稀疏跳零，同时避免 `0 * NaN`。
- 索引 `A[i*k+t]`、`B[t*m+j]`、`C[i*m+j]`：**这三个式子写错一个就全错**，背口诀"自己那行的步长是别人列数"。

1.3 三个救命小工具（面试不写，但你本地要练）

```
void dump(const vector<float>& M, int rows, int cols, const char* name) {
    printf("%s [%dx%d]:\n", name, rows, cols);
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) printf("%8.4f", M[i * cols + j]);
        printf("\n");
    }
}
// 随机填充 / 求和校验
vector<float> rnd(int n, unsigned seed);
float sumAll(const vector<float>& v);
```

本地练习法：每次写完一题，`dump` 出来看一眼形状对不对——比肉眼数下标快十倍。现场没有 dump 就口头报形状："Q 是 L×D，S 是 L×L，输出 L×D"。

---

2. 逐题详解

> 每题的固定六段：**题目原话 → 30 秒口述脚本 → 公式 → 代码 → 逐行拆解 → 追问 Q&A → 现场最易错 3 点**。 时间紧就先背"口述脚本"和"最易错 3 点"，这两块决定面试观感。

2.1 Softmax（开场题，必须 3 分钟写完）

面试官原话："手写一个 softmax。"（可能加一句"注意数值稳定"）

30 秒口述脚本：

> "softmax 是三件事：先减最大值防 exp 溢出，再 exp，最后除以总和归一化。减 max 不改变结果，因为分子分母同时乘了 e^{-max}。向量版一遍循环就够，矩阵版逐行调用。"

公式：

```
softmax(x)_i = e^x_i / Σ_j e^x_j = e^(x_i - m) / Σ_j e^(x_j - m),   其中 m = max_j x_j
```

代码（原地版 + 返回版，两个都准备）：

```cpp
// 对 x[off .. off+len-1] 原地做 softmax
void softmaxInPlace(vector<float>& x, int off, int len) {
    float m = -1e30f;
    for (int i = 0; i < len; ++i) m = max(m, x[off + i]);   // 1) 找最大值
    float s = 0.f;
    for (int i = 0; i < len; ++i) {                          // 2) exp(x-m) 并累加
        x[off + i] = expf(x[off + i] - m);
        s += x[off + i];
    }
    for (int i = 0; i < len; ++i) x[off + i] /= s;           // 3) 归一化
}

vector<float> softmax(const vector<float>& x) {
    vector<float> y = x;
    softmaxInPlace(y, 0, (int)y.size());
    return y;
}
```

逐行拆解：

行	在干什么	为什么这么写
`float m = -1e30f;`	用一个足够小的初值	**不要用 `-FLT_MAX`**，因为 `-FLT_MAX - (-FLT_MAX)` 可能出 NaN；初值取 -1e30 是工业惯例
`m = max(m, x[off+i])`	一遍扫出最大值	`std::max_element` 也行，但带偏移量的循环更通用（矩阵逐行调用时直接传 off）
`expf(x-m)`	减 max 后取指数	若 x 全为 1e3，不减 max 会 `exp(1000)` → `inf`，再除 → `nan`
`s += ...`	累加分母	用 `float` 够；若 logits 动态范围极大，可换 `double` 累加（补一句显得严谨）
`/= s`	归一化	结果和恒为 1（浮点误差内）
`off, len` 参数	支持矩阵逐行	这样 MHA 里 `softmaxInPlace(S, i*L, L)` 一行搞定，不用另写矩阵版

追问 Q&A（必背）：

- **Q：为什么减 max 不改变结果？** A：分子分母同乘 e^(-m) 就约掉了：e^(x_i-m) / Σ_j e^(x_j-m) = [e^x_i · e^(-m)] / [Σ_j e^x_j · e^(-m)] = e^x_i / Σ_j e^x_j。
- **Q：减 max 足够吗？还有什么更稳的？** A：减 max 解决上溢；**下溢**（某些项 exp 后为 0）在 softmax 里无害，但在 log 域有害，所以交叉熵要在 log 域用 logsumexp 算（见 2.2）。
- **Q：softmax 的温度 T？** A：指数里除以 T，即 e^(x_i / T)。T>1 分布变平（蒸馏、采样多样性）；T<1 变尖（接近 argmax）；T→0 退化为 one-hot。
- **Q：带 mask 的 softmax 怎么写？** A：把屏蔽位置置成 `-1e30`（C++）或 `float('-inf')`（Python）。**坑**：如果一行全被屏蔽，分母为 0 → NaN。工业实现会给被屏蔽行补 0 权重或直接跳过计算。
- **Q：softmax 和 sigmoid 的关系？** A：二分类时 softmax 两输出等价于 sigmoid（相差一个常数偏移）；多标签（不互斥）用逐元素 sigmoid，对应 BCEWithLogits。
- **Q：softmax 加交叉熵的梯度是什么？**（高频延伸） A：对 logit 求导就是 p_i - y_i，即"预测减标签"，极其干净——这也是为什么分类头后面直接接 CE。

现场最易错 3 点：1. 忘了初始化累加变量（`float s;` 而不是 `float s = 0.f;`）。2. 用 `int s` 累加概率 → 全变 0。3. 减 max 写在除法之后（顺序错）。

2.2 交叉熵 / log_softmax（和 softmax 连着问）

面试官原话："交叉熵损失怎么写？为什么分类任务不直接用 MSE？"

30 秒口述脚本：

> "单样本交叉熵就是 logsumexp(logits) 减掉正确类的 logit。我不先算 softmax 再取 log，而是直接在对数域算，避免概率下溢。数学上是 $-\log p_y$，梯度是 $p - y$。"

公式：

```
L = -log( e^z_y / Σ_j e^z_j ) = logsumexp(z) - z_y,   其中 logsumexp(z) = log Σ_j e^z_j
```

代码：

```cpp
float crossEntropy(const vector<float>& logits, int target) {
    float m = *max_element(logits.begin(), logits.end());
    double s = 0;
    for (float v : logits) s += exp((double)v - m);
    float logsumexp = m + (float)log(s);
    return logsumexp - logits[target];      // = logsumexp(logits) - logits[target]
}
```

逐行拆解：- `m`：再减一次 max，保证 `exp` 不溢出（同 softmax）。- `double s`：累加用 double，减少长类别数下的精度损失（一句话加分项）。- `logsumexp = m + log(s)`：这是恒等式 log Σ e^z_j = m + log Σ e^(z_j - m)，**背下来**，面试官问"logsumexp 技巧"多半就是这句。- `- logits[target]`：分子是正确类，-log(e^z_y / Z) = log Z - z_y。- **没有除以 N**：这是我故意写单样本版，批量版本在外面 `sum / N` 或按类别权重加权。

追问 Q&A：- **Q：为什么不用 MSE 做分类？** A：① 交叉熵配 softmax 的梯度是 `p - y`，误差大时梯度大，收敛快；MSE 配 softmax 的梯度含 p(1-p) 因子，饱和区梯度消失。② MSE 对概率分布不是合适的距离（非凸、非概率性）。- **Q：标签平滑怎么改？** A：target 从 one-hot 换成 (1-ε) 的 one-hot + ε/C 均匀分布；等价的简化实现是 L = (1-ε)·CE + ε·mean(-z)。- **Q：多标签 / 带类别权重？** A：多标签用逐元素 `log(1+e^(-z)) - y·z`（BCEWithLogits）；带权重按类别频率取倒数或 sqrt(1/f)，在分子求和后按权重归一。- **Q：你真实用过的坑？**（这是你的强项，主动讲）A：把"CE target 越界"讲出来——参数名 `cls_num` 与实际类别数不一致导致 target 超出范围、loss 直接崩，属于"配置口径"问题，不是模型问题。

现场最易错 3 点：忘了减 `m`；用 `log(p)` 二次计算（p 可能为 0 → -inf）；偏移 target（写成 0-based/1-based 混淆）。

2.3 LayerNorm（问 Transformer 必连带）

面试官原话："写个 LayerNorm" / "BN 和 LN 的区别，为什么 Transformer 用 LN？"

30 秒口述脚本：

> "LayerNorm 对**单样本的特征维**做归一化：先算这一行所有元素的均值和方差，标准化后再乘可学习的 γ、加 β。和 BatchNorm 的区别是统计量在哪个维度求——BN 跨 batch 求，LN 在样本内求，所以 LN 不受 batch 大小影响，也能用在变长序列上。"

公式：

```
y = γ · (x - μ) / sqrt(σ² + ε) + β,   μ、σ² 是这一行（最后一个维度）的均值方差
```

代码：

```cpp
void layerNorm(vector<float>& x, const vector<float>& g, const vector<float>& b, float eps = 1e-5f) {
    int D = (int)g.size();
    for (size_t row = 0; row + D <= x.size(); row += D) {
        float mu = 0;
        for (int i = 0; i < D; ++i) mu += x[row + i];
        mu /= D;
        float var = 0;
        for (int i = 0; i < D; ++i) { float d = x[row + i] - mu; var += d * d; }
        var /= D;
        float inv = 1.0f / sqrtf(var + eps);
        for (int i = 0; i < D; ++i) x[row + i] = g[i] * (x[row + i] - mu) * inv + b[i];
    }
}
```

逐行拆解：- `for (row = 0; row + D <= x.size(); row += D)`：把扁平数组按行切分。**注意终止条件是 `row + D <= size`**，不是 `row < size`，否则最后一轮越界——这是 C++ 手写最常见的崩法。- `mu /= D` / `var /= D`：用总体方差（除 D），不是无偏方差（除 D-1）。**框架实现是除 D**，答错会被追一句"你确定？"。- `inv = 1/sqrt(var + eps)`：先算倒数再乘，比每维做一次除法快（这是工程细节加分项）。- `eps=1e-5`：防 var=0 时除零。**注意 eps 在 sqrt 里面**，不是外面。- γ、β 是可学习参数：γ 初始化为 1、β 初始化为 0 时，LN 是恒等变换（这条对"为什么 LN 不破坏表达力"很好用）。

追问 Q&A：- **Q：LN vs BN？** A：BN 对每个通道跨 (N,H,W) 求统计量 → 依赖 batch，小 batch 不稳，推理时要用滑动平均；LN 对单样本所有通道求 → 与 batch 无关，适合变长序列/RNN/Transformer。CV 里小 batch 的替代是 GroupNorm。- **Q：为什么 Transformer 用 Pre-LN 而不是 Post-LN？** A：Post-LN（原始）残差后再归一，深层时梯度需要穿过 LN，必须 warmup 才稳；Pre-LN 把 LN 放在子层输入侧，残差路径是恒等的，梯度直通，可去 warmup，这是现在的主流。- **Q：RMSNorm 是什么？** A：去掉均值中心化，只除 RMS：$y = \gamma \cdot x / \sqrt{\text{mean}(x^2) + \varepsilon}$。少一次求均值和减法，更快，LLaMA 系在用。- **Q：BN 推理时怎么算？** A：用训练期累积的滑动平均 μ、σ^2（momentum 一般 0.1），不再用当前 batch 统计量。这也是"训练/推理不一致"的经典 bug 来源。

现场最易错 3 点：除 D-1；eps 放错位置；`row + D <= size` 写成 `row < size`。

2.4 IoU + NMS（你有检测背景，顺手就问）

面试官原话："写个 IoU" → "那 NMS 呢" → "NMS 有什么问题，怎么改进？"

30 秒口述脚本：

> "IoU 是交集面积除并集面积，注意并集要减掉一次交集，宽高要 clamp 到 0 以上防负数框。NMS 是按分数降序，贪心保留当前最高分，把它 IoU 超阈值的框全部压掉，循环到没有框。"

代码：

```
struct Box { float x1, y1, x2, y2, score; };

float iou(const Box& a, const Box& b) {
    float ix1 = max(a.x1, b.x1), iy1 = max(a.y1, b.y1);
    float ix2 = min(a.x2, b.x2), iy2 = min(a.y2, b.y2);
    float iw = max(0.f, ix2 - ix1), ih = max(0.f, iy2 - iy1);
    float inter = iw * ih;
    float ua = (a.x2 - a.x1) * (a.y2 - a.y1) + (b.x2 - b.x1) * (b.y2 - b.y1) - inter;
    return ua > 0 ? inter / ua : 0.f;
}

vector<int> nms(vector<Box> bs, float thr) {
    sort(bs.begin(), bs.end(), [](const Box& p, const Box& q) { return p.score > q.score; });
    vector<int> keep;
    vector<char> dead(bs.size(), 0);
    for (size_t i = 0; i < bs.size(); ++i) {
        if (dead[i]) continue;
        keep.push_back((int)i);                      // 当前分最高的留
        for (size_t j = i + 1; j < bs.size(); ++j)
            if (!dead[j] && iou(bs[i], bs[j]) > thr) dead[j] = 1;
    }
    return keep;
}
```

逐行拆解：- 交点坐标 = `max(左上) , min(右下)`——两个框的左上取大的、右下取小的，交集才存在。- `max(0.f, ix2-ix1)`：两框不相交时 `ix2-ix1` 为负，不 clamp 会算出负面积 → 负 IoU。**这是最经典的错**。- `ua = area_a + area_b - inter`：并集容斥，**别忘减 inter**。- `ua > 0 ? ... : 0.f`：两个退化框（面积为 0）时防除零。- NMS 用 `dead` 标记数组而非 erase：避免 O(n^2) 的搬移，且保留下标与输入的对应关系（面试官会问"返回的下标怎么对上原数组"）。- `sort` 的 lambda 用 `>` 实现降序；**比较函数必须是严格弱序**（不能写 `>=`，会崩）。

追问 Q&A：- **Q：NMS 的复杂度？** A：朴素 O(n^2)；实际用按 score 排序后只跟 top-k 比，或按空间分桶/网格先把候选缩到邻域内，接近 O(n log n)。- **Q：NMS 有什么缺陷？** A：① 密集场景（人群、并排车辆）会误压真实目标；② 阈值敏感；③ 不可导、不能端到端训练。- **Q：改进方案？（必背三个）** A：**Soft-NMS**（不直接删，按 IoU 高斯/线性衰减分数，保留重叠目标）；**IoU-aware / GIoU-DIoU 类 NMS**（用 IoU 预测质量修正分数）；**可学习的 NMS / set prediction**（DETR 用匈牙利匹配 + 二分图匹配，彻底不用 NMS）。- **Q：IoU 的局限？** A：不相交时 IoU 恒为 0，无梯度 → GIoU（加最小闭包面积）、DIoU（加中心距离）、CIoU（再加速度一致性），这是检测损失的标准演进线。

现场最易错 3 点：忘 clamp；并集忘减交集；`sort` 比较函数写 `>=`。

2.5 K-Means（考"你会不会讲算法缺陷"多于考代码）

面试官原话："手写一个 kmeans" → "k 怎么选" → "初始化有什么讲究" → "它一定会收敛吗？"

30 秒口述脚本：

> "kmeans 是 EM 的特例：E 步把每个点分给最近的中心，M 步把中心更新成簇内均值，交替到中心移动量小于阈值。复杂度 O(n · k · d · iters)。它只能收敛到局部最优，对初始化和异常值敏感，所以工程上要跑多次初始化取 inertia 最小的，或者用 k-means++。"

代码：

```
vector<int> kmeans(const vector<float>& X, int n, int d, int k,
                   int iters = 100, float tol = 1e-4f, unsigned seed = 42) {
    mt19937 rng(seed);
    vector<int> idx(n);
    iota(idx.begin(), idx.end(), 0);
    shuffle(idx.begin(), idx.end(), rng);
    vector<float> C(k * d);
    for (int j = 0; j < k; ++j)
        for (int t = 0; t < d; ++t) C[j * d + t] = X[idx[j] * d + t];   // 随机选 k 个样本当初始中心

    vector<int> lab(n, 0), cnt(k, 0);
    for (int it = 0; it < iters; ++it) {
        for (int i = 0; i < n; ++i) {                    // E 步：每个点归到最近中心
            float best = 1e30f;
            int bj = 0;
            for (int j = 0; j < k; ++j) {
                float dist = 0;
                for (int t = 0; t < d; ++t) { float df = X[i * d + t] - C[j * d + t]; dist += df * df; }
                if (dist < best) { best = dist; bj = j; }
            }
            lab[i] = bj;
        }
        vector<float> S(k * d, 0.f);
        fill(cnt.begin(), cnt.end(), 0);
        for (int i = 0; i < n; ++i) {                    // M 步：重新算中心
            int j = lab[i];
            ++cnt[j];
            for (int t = 0; t < d; ++t) S[j * d + t] += X[i * d + t];
        }
        float shift = 0;
        for (int j = 0; j < k; ++j) {
            if (cnt[j] == 0) continue;                   // 空簇：保留旧中心
            for (int t = 0; t < d; ++t) {
                float nc = S[j * d + t] / cnt[j];
                float df = nc - C[j * d + t];
                shift += df * df;
                C[j * d + t] = nc;
            }
        }
        if (sqrtf(shift) < tol) break;                   // 收敛判据：中心移动量
    }
    return lab;
}
```

逐行拆解：- `iota` + `shuffle`：造 0..n-1 的下标再打乱，取前 k 个当初始中心。**注意是在样本里抽，不是随机生成坐标**（随机生成坐标可能离数据很远，收敛慢）。- E 步用**距离平方**比较：不用开方，省一个 sqrt，且 argmin 结果相同（这句是加分点）。- M 步用**累加求和再除计数**，而不是"每次平均"——避免除法的累积误差，也避免空簇除零。- `if (cnt[j] == 0) continue;`：**空簇处理**，保留旧中心。这是面试官最爱问的实现细节。- 收敛判据用**中心移动平方和**（shift），不是标签是否变化（标签判据可能出现标签循环震荡）。- `iters=100` 只是保险，正常迭代次数远小于它。

追问 Q&A：- **Q：k 怎么选？** A：肘部法（inertia 随 k 的拐点）、轮廓系数 silhouette（−1~1，越大越好）、Gap statistic；实际业务常直接用下游指标定 k。- **Q：kmeans++ 是什么？** A：初始化时第一个中心随机，之后每个新中心按"到已有中心的最近距离平方"成比例采样——离得远的点更容易被选，直接把随机初始化最坏情况压下去。- **Q：一定会收敛吗？收敛到全局最优吗？** A：目标函数（inertia）每步单调不增且有下界，所以一定收敛；但**只保证局部最优**。**这条我有实测数据**：在我自测的三高斯簇

（150 点）上，seed=7 单次初始化只对上 125/150，换 20 个初始化取最好就 150/150——可以直接把这个数字讲给面试官，说服力极强。 - **Q：kmeans 的假设/局限？** A：簇是凸的、各向同性、方差相近、大小相近；对异常值敏感（均值被拉走，可用 k-medoids 或剪枝）；只给硬分配（软分配要 GMM/EM）；高维失效（距离集中现象）。 - **Q：复杂度与加速？** A：O(n · k · d · iters)；加速：MiniBatch kmeans（每批采样更新）、Elkan 三角不等式剪枝、kd-tree/球树（低维）、先 PCA 降维。 - **Q：kmeans 和 GMM 的关系？** A：GMM 用 EM 做软分配 + 各簇自己的协方差；kmeans 是"等协方差 + 硬分配"的特例。

现场最易错 3 点：用 `vector<vector<float>>` 存中心导致下标混乱；空簇除零；收敛判据用标签导致死循环。

2.6 Self-Attention（多模态岗最核心的一题，占分最高）

面试官原话："手写一下 self-attention" → 通常紧跟三个追问：为什么要除以 √d、复杂度多少、mask 怎么加。

30 秒口述脚本（先说，让对方知道你要写什么）：

> "self-attention 是四步：X 分别乘 Wq/Wk/Wv 得到 Q、K、V；Q 乘 K 的转置得到 L×L 的相似度矩阵，除以 √d 缩放；遮掉 mask 位置后逐行 softmax 得到注意力权重；最后权重乘 V 得到输出。输出形状和输入一样是 L×D。"

公式：

```
Attention(Q, K, V) = softmax( Q·K^T / sqrt(d_k) + mask ) · V
    Q = X·Wq   K = X·Wk   V = X·Wv      (X: L×D, W*: D×D)
    Q·K^T : L×L        softmax 按行做        结果 : L×D
```

代码（这段是完整可跑版，来自 `ai_interview_cpp.cpp`）：

```cpp
// X: L x D, Wq/Wk/Wv: D x D, mask: L x L (true=屏蔽) 可为空
vector<float> selfAttention(const vector<float>& X, int L, int D,
                            const vector<float>& Wq, const vector<float>& Wk,
                            const vector<float>& Wv, const vector<float>& mask,
                            vector<float>* attnOut = nullptr) {
    auto Q = matmul(X, Wq, L, D, D);            // [L,D]
    auto K = matmul(X, Wk, L, D, D);
    auto V = matmul(X, Wv, L, D, D);

    float scale = 1.0f / sqrtf((float)D);       // 缩放，防止点积过大把 softmax 推饱和
    vector<float> S(L * L, 0.f);
    for (int i = 0; i < L; ++i)
        for (int j = 0; j < L; ++j) {
            float s = 0;
            for (int t = 0; t < D; ++t) s += Q[i * D + t] * K[j * D + t];
            S[i * L + j] = s * scale;           // = q_i · k_j / sqrt(d)
        }
    if (!mask.empty())
        for (int i = 0; i < L * L; ++i)
            if (mask[i]) S[i] = -1e30f;         // 屏蔽位：用一个极大的负数，exp 后为 0
    for (int i = 0; i < L; ++i) softmaxInPlace(S, i * L, L);   // 逐行 softmax
    auto O = matmul(S, V, L, L, D);             // [L,L] x [L,D] -> [L,D]
    if (attnOut) *attnOut = S;
    return O;
}
```

逐行拆解（这段一定要能讲，面试官几乎必问）：

步骤	代码	讲法
1. 三次投影	`matmul(X, Wq, L, D, D)`	X 是 L 个 token、每个 D 维；乘 D×D 权重后还是 L×D。**三个投影矩阵不共享**，这是 self-attention 有"三种角色"的原因
2. 打分	三层循环 `Q[i*D+t] * K[j*D+t]`	取 Q 的第 i 行、K 的第 j 行，逐维乘加 = 点积。注意是 `K[j*D+t]` 而不是 `K[t*D+j]` ——**这里等价于 Q · K^T**，因为 K 的第 j 行就是 K^T 的第 j 列
3. 缩放	`* scale`	见下面追问，方差论证
4. mask	`S[i] = -1e30f`	加性 mask：加一个极负的数，exp 后趋 0，相当于"看不到"
5. 逐行 softmax	`softmaxInPlace(S, i*L, L)`	每一行（一个 query 对所有 key 的注意力）单独归一化。**这正是我 2.1 把 off/len 抽出来的原因**

步骤	代码	讲法
6. 加权求和	`matmul(S, V, L, L, D)`	L×L 的权重乘 L×D 的 V → L×D，第 i 行输出 = Σ_j attn[i][j] · V[j]

追问 Q&A（按被问概率排序）：

- **Q：为什么除以 √d_k?** （必问，答不出直接掉档） A：假设 q、k 各维独立、均值 0 方差 1，则点积 `q·k = Σ_t q_t k_t` 的方差是 d_k，标准差是 √d_k。d_k 大（比如 64、128）时点积量级到 10 以上，softmax 会被推到饱和区：最大项接近 1、其余接近 0，梯度几乎为 0（softmax 饱和）。除以 √d_k 把方差拉回 1，softmax 落在梯度良好的区间。
- **Q：复杂度?** A：时间 $O(L^2 \cdot D)$（打分和加权各一次），显存 $O(L^2)$（要存注意力矩阵）。L 是序列长度，所以长上下文是平方级代价。
- **Q：$O(L^2)$ 怎么省?** （必答，直接连到你熟悉的领域） A：四条线——① **FlashAttention**：分块（tiling）+ 在线 softmax，把 L×L 矩阵切块流式计算，**省的是显存和 HBM 读写，不减少 FLOPs**；② **稀疏/滑窗注意力**（Longformer、BigBird）；③ **低秩/线性注意力**（Linformer、Performer 用核函数近似）；④ **MQA/GQA**（共享 K/V 头，主要省推理时的 KV cache）；⑤ 状态空间模型 Mamba（线性复杂度，我做过多任务模型里的时序模块，可以顺势讲）。
- **Q：mask 有哪几种?** A：① **causal mask**（下三角可见，上三角屏蔽）用于自回归生成；② **padding mask**（按 key 的有效长度屏蔽 pad）；③ **结构 mask**（跨模态/跨样本隔离）。实现统一是加 −∞ 或 −1e30。
- **Q：softmax 里一行全被 mask 会怎样?** A：分母为 0 → NaN。工程上要么保证每行至少一个有效位，要么对全屏蔽行单独置 0。
- **Q：self-attention 和 cross-attention 区别?** A：Q 的来源。self 是 Q/K/V 同源；cross 是 Q 来自解码器（或 query token），K/V 来自编码器（或另一种模态）。**多模态融合就是 cross-attention 的典型**——图像 token 当 K/V、文本 token 当 Q（或反过来）。这条一定主动讲，能把你简历里的视觉-激光融合接上。
- **Q：为什么要有位置编码?** A：self-attention 对 token 顺序是置换不变的（打乱输入只是打乱输出行），本身不含位置信息。方案：正弦/可学习绝对编码、RoPE（旋转位置编码，保相对位置、可外推，LLaMA 系）、ALiBi（按距离线性衰减 bias）。
- **Q：多头和单头在计算量上一样吗?** A：**几乎一样**。总维度 D 被切成 h 份，每份 d_k=D/h，注意力计算量 $h \cdot L^2 \cdot (D/h) = L^2 \cdot D$，与单头相同；差别只在"分成 h 个子空间分别学依赖"。
- **Q：KV cache 占多少显存?** A：每层每 token 要存 K 和 V 各 D 个值，即 2·D；总显存 ≈ 2 · 层数 · D · 序列长度 · batch · 精度字节。这就是长上下文推理的瓶颈，也是 MQA/GQA 的动机。

现场最易错 3 点： 1. 写成 `K[t*D+j]`（那是 K 而不是 K^T 的取法，维数甚至对不上）； 2. 忘了逐行 softmax 直接对整块做（会把整个 L×L 归一化成和 1）； 3. mask 用 `0` 表示屏蔽（0 加进去等于没屏蔽，exp(0)=1）。

2.7 Multi-Head Attention（承接上题，通常让你"改成多头"）

面试官原话："在上面基础上改成多头" / "多头的 Q、K、V 是怎么切分的？"

30 秒口述脚本：

> "多头就是把 D 维按**列**切成 h 份，每份 d_k = D/h。第 i 个头用 Wq/Wk/Wv 的第 i 个列块做投影，得到 L×d_k 的 Q/K/V，各自算一遍 attention，然后把 h 个 L×d_k 的输出按列拼回 L×D，最后过一遍输出投影 Wo。参数量是 $4D^2$。"

公式：

```
head_i = Attention(X · Wq[:, i · dk:(i+1) · dk], X · Wk[:, 同列块], X · Wv[:, 同列块])
MHA(X) = Concat(head_0, ..., head_{h-1}) · Wo
```

代码（完整可跑，来自验证文件）：

```
vector<float> multiHeadAttention(const vector<float>& X, int L, int D, int h,
                                 const vector<float>& Wq, const vector<float>& Wk,
                                 const vector<float>& Wv, const vector<float>& Wo,
                                 const vector<float>& mask,
                                 vector<float>* attnOut = nullptr) {
    int dk = D / h;
    vector<float> Ocat(L * D, 0.f);
    vector<float> attnAll(h * L * L, 0.f);
```

```
    for (int hi = 0; hi < h; ++hi) {
        // 第 hi 个头只取权重的第 [hi*dk, hi*dk+dk) 列
        auto headW = [&](const vector<float>& W) {
            vector<float> Ws(D * dk);
            for (int r = 0; r < D; ++r)
                for (int c = 0; c < dk; ++c) Ws[r * dk + c] = W[r * D + hi * dk + c];
            return Ws;
        };
        auto Q = matmul(X, headW(Wq), L, D, dk);
        auto K = matmul(X, headW(Wk), L, D, dk);
        auto V = matmul(X, headW(Wv), L, D, dk);
        float scale = 1.0f / sqrtf((float)dk);
        vector<float> S(L * L, 0.f);
        for (int i = 0; i < L; ++i)
            for (int j = 0; j < L; ++j) {
                float s = 0;
                for (int t = 0; t < dk; ++t) s += Q[i * dk + t] * K[j * dk + t];
                S[i * L + j] = s * scale;
            }
        if (!mask.empty())
            for (int t = 0; t < L * L; ++t) if (mask[t]) S[t] = -1e30f;
        for (int i = 0; i < L; ++i) softmaxInPlace(S, i * L, L);
        auto O = matmul(S, V, L, L, dk);
        for (int i = 0; i < L; ++i)                        // 拼回第 hi 头占的那几列
            for (int c = 0; c < dk; ++c) Ocat[i * D + hi * dk + c] = O[i * dk + c];
        copy(S.begin(), S.end(), attnAll.begin() + (size_t)hi * L * L);
    }
    if (attnOut) *attnOut = attnAll;
    return matmul(Ocat, Wo, L, D, D);                      // 最后过输出投影
}
```

逐行拆解：

- `int dk = D / h`：每个头的维度。**面试时先问一句"h 必须整除 D 吧"**，这是在展示你有工程感。
- `headW`：从 D×D 权重里切出 D×dk 的子块。**关键点：切的是列**。解释："Wq 的每一列对应输出的一维。第 hi 个头管输出的 [hi·dk, (hi+1)·dk) 这几维，所以取这段列。"
- `matmul(X, ..., L, D, dk)`：X 是 L×D、子权重 D×dk → 输出 L×dk。**尺寸参数必须同步改成 dk**，这是最容易错的地方。
- `Ocat[i * D + hi * dk + c]`：往 L×D 的大矩阵里"按列拼"。第 i 个 token 输出的第 (hi·dk + c) 维，就是第 hi 头输出的第 c 维。
- `attnAll`：把所有头的注意力矩阵存一起，shape h×L×L，方便可视化不同头看什么。
- 最后 `matmul(Ocat, Wo, L, D, D)`：投影回 D 维。**Wo 不能忘**，这是"多头之间做信息混合"的唯一机会。

我在本地验证过的三个性质（可以作为"我怎么确认写对了"的答案讲出来）： 1. `h=1 且 Wo=单位阵` 时，多头实现和单头实现**逐元素完全一致**（数值误差 1e-5 内）——这是最强的自检手段。 2. 每个头、每一行的 attention 权重和恒为 1。 3. 输出 shape 恒为 L×D，与输入一致（这是能堆叠 N 层的前提）。

追问 Q&A： - **Q：参数量？** A：Wq/Wk/Wv/Wo 各 D×D，共 **$4D^2$**（加 bias 再 +4D）。**和头数 h 无关**——这是常被考的常识。 - **Q：为什么多头比单头好？** A：不同头在不同子空间学不同的依赖模式（位置邻近、语法、指代、跨模态对应），类似 CNN 的多通道分工；单头只有一套注意力分布。 - **Q：MQA / GQA 是什么？省在哪？** A：MQA 让所有头共享同一份 K/V（h 个头只有 1 组 K/V）；GQA 折中，h 个头分 g 组、每组共享一组。省的是**推理时的 KV cache**：MHA 每 token 存 2D 个数，MQA 存 2D/h，GQA 存 2D·g/h。训练 FLOPs 基本不变。 - **Q：为什么 Q、K、V 用不同权重？** A：三者角色不同——Q 是"我要找什么"，K 是"我能被什么匹配到"，V 是"被匹配上之后提供什么内容"。同权重会限制表达。 - **Q：C++ 里"拆头"和 PyTorch 里 `view + transpose` 是什么关系？** A：同一件事。PyTorch 是 `[B,L,D] → view(B,L,h,dk) → transpose(1,2) → [B,h,L,dk]`，C++ 这里是直接在权重上按列切、输出按列拼，等价于 transpose 后连续化。**如果面试官追问 PyTorch 版，我按 4.2 节那样写给他**（这句是兜底话术，背下来）。 - **Q：注意力头之间真的分工不同吗？有证据吗？** A：可视化证据很多（有的头盯相邻 token、有的盯句法依存、有的盯罕见词/分隔符），也有研究报告存在冗余头（可剪枝）——答"部分是冗余的，所以有 head pruning 这类工作"很稳。

现场最易错 3 点：`dk` 忘了替换进 `matmul` 的尺寸参数；拼回时用 `hi*dk*c` 之类的错误偏移；忘了乘 Wo。

2.8 CART 分类树（传统 ML 基础题，代码最长的）

面试官原话："手写一个决策树" / "CART 的划分准则是什么" / 然后追一串：剪枝、连续特征、和 ID3/C4.5 的区别、GBDT 怎么从它推出来。

30 秒口述脚本：

> "CART 是二叉树：每次在所有特征的所有候选阈值里，选让加权基尼不纯度下降最多的那个 (特征, 阈值) 做二分切分，然后递归到左右子集。停止条件是深度到上限、样本太少、或节点已纯。叶子取多数类。"

公式：

```
基尼不纯度    Gini(S) = 1 - Σ_c (|S_c| / |S|)²
切分增益      Gain = Gini(S) - (|S_L|·Gini(S_L) + |S_R|·Gini(S_R)) / |S|
候选阈值      相邻不同取值的中点 (v_p + v_{p+1}) / 2
```

代码（完整可跑；`pool` 是节点池，用 `int` 下标代替指针，避免手写内存管理）：

```cpp
struct TreeNode { int feat = -1; float thr = 0; int left = -1, right = -1; int label = 0; };

double gini(const vector<int>& cnt, int n) {
    if (n <= 0) return 0.0;
    double s = 0;
    for (int c : cnt) { double p = (double)c / n; s += p * p; }
    return 1.0 - s;
}

int buildTree(vector<TreeNode>& pool, const vector<float>& X, const vector<int>& y,
              const vector<int>& idx, int d, int K, int depth, int maxDepth, int minLeaf) {
    int id = (int)pool.size();
    pool.push_back(TreeNode());

    vector<int> cnt(K, 0);
    for (int i : idx) ++cnt[y[i]];
    pool[id].label = (int)(max_element(cnt.begin(), cnt.end()) - cnt.begin());   // 叶子取值 = 多数类

    if (depth >= maxDepth || (int)idx.size() < 2 * minLeaf) return id;           // 停止条件 1/2
    double parent = gini(cnt, (int)idx.size());
    if (parent < 1e-12) return id;                                               // 停止条件 3: 已纯

    double bestGain = 1e-12;
    int bf = -1; float bt = 0;
    int n = (int)idx.size();
    for (int f = 0; f < d; ++f) {
        vector<pair<float, int>> v;
        v.reserve(n);
        for (int i : idx) v.push_back({X[i * d + f], i});
        sort(v.begin(), v.end());                                    // 排序后扫一遍就能拿到所有候选切分的不纯度
        vector<int> lc(K, 0), rc = cnt;
        for (int p = 0; p + 1 < n; ++p) {
            int yi = y[v[p].second];
            ++lc[yi]; --rc[yi];
            if (v[p].first == v[p + 1].first) continue;               // 同值不能切
            int nl = p + 1, nr = n - nl;
            if (nl < minLeaf || nr < minLeaf) continue;
            double g = parent - (nl * gini(lc, nl) + nr * gini(rc, nr)) / n;
            if (g > bestGain) { bestGain = g; bf = f; bt = (v[p].first + v[p + 1].first) * 0.5f; }
        }
    }
    if (bf < 0) return id;

    vector<int> il, ir;
    for (int i : idx) (X[i * d + bf] <= bt ? il : ir).push_back(i);
    pool[id].feat = bf; pool[id].thr = bt;
    pool[id].left = buildTree(pool, X, y, il, d, K, depth + 1, maxDepth, minLeaf);
    pool[id].right = buildTree(pool, X, y, ir, d, K, depth + 1, maxDepth, minLeaf);
    return id;
}

int predictTree(const vector<TreeNode>& pool, const vector<float>& x, int d) {
    int u = 0;
    while (pool[u].feat >= 0) u = (x[pool[u].feat] <= pool[u].thr) ? pool[u].left : pool[u].right;
    return pool[u].label;
}
```

逐行拆解（这是最容易写崩的一题，重点看下面五条）：

1. **节点池 `pool` + `int` 下标**：`int id = pool.size(); pool.push_back(...)` 先占位，再递归填充 `pool[id].left/right`。用下标而不是 `TreeNode*` 的原因：`push_back` 可能触发扩容，**指针会失效**，下标不会。面试时讲这句"我用下标避免 realloc 后指针失效"，是很硬的工程细节。
2. **三个停止条件**必须都写：`depth >= maxDepth`、`样本数 < 2*minLeaf`（左/右都至少 minLeaf 个才能切）、`parent < 1e-12`（已纯，再切是浪费）。漏第三个会导致无限递归/极深的树。
3. **前缀扫描技巧**：对每个特征，把样本按该特征值排序，然后从左往右扫：每挪动一个样本，把它从右子集搬到左子集（`++lc[yi]; --rc[yi];`），此时左右两侧的类别计数是 O(1) 更新，不用每次重新统计。**总复杂度从 $O(n^2 \cdot d)$ 降到 $O(n \cdot d \cdot \log n)$（排序主导）**。这一段是本题最大的加分点，主动讲。
4. **同值不能切**：`if (v[p].first == v[p+1].first) continue;` —— 两个样本特征值相同，任何阈值都分不开它们，切了是无意义分裂。
5. **阈值取中点**：`(v[p].first + v[p+1].first) / 2`，并且预测时用 `<=` 走左，保证阈值语义一致（写 `<=` 就不要在预测里写 `<`）。
6. **重新划分再递归**：确定最优 (bf, bt) 后，用一次线性扫描把 `idx` 分成 `il/ir`。没有在扫描阶段缓存划分结果，是因为那样要为每个特征存一份临时数组，代码更长；面试写"慢一点但清楚"的版本完全 OK，主动说一句"工程上可以缓存"。

追问 Q&A：- **Q：CART / ID3 / C4.5 的区别?**（几乎必问） A：ID3 用**信息增益**、多叉分裂、只能处理离散特征、偏好多取值特征；C4.5 用**信息增益率**（除以特征自身的分裂信息，惩罚多值特征）、支持连续特征与缺失值、仍倾向多叉；CART 用**基尼指数**、**严格二叉**、分类用基尼 / 回归用 MSE、支持剪枝，是 sklearn 和 GBDT 家族的基础。 - **Q：为什么 CART 用基尼不用熵?** A：基尼是 `1 - Σp²`，只有乘法和加法，没有 log，计算快（大样本下差别明显）；两者曲线形状高度相似，划分结果基本一致。**还有一个细节**：基尼可理解为"随机抽两个样本类别不同的概率"。 - **Q：剪枝怎么做?** A：预剪枝（本代码的三种停止条件）：快，但贪心视野短，可能过早停；后剪枝（代价复杂度剪枝 CCP）：先长满，再自底向上用 `R(T) + α|T|` 判断是否合并叶子，α 由交叉验证选——更准但更贵。sklearn 的 `ccp_alpha` 就是它。 - **Q：为什么树容易过拟合?** A：分裂太细，叶子只剩 1 个样本 → 方差爆炸；阈值微动就换一棵树（不稳定）。缓解：限制深度/叶最小样本/后剪枝/集成（RF 降方差、GBDT 降偏差）。 - **Q：连续特征怎么处理?** A：排序 + 取相邻中点（本代码）；大样本用分位分桶（LightGBM 的直方图，把候选阈值从 n 个降到 255 个桶）。 - **Q：特征重要性怎么算?** A：把每次分裂带来的"加权不纯度下降 × 该节点样本数"按特征累加，再归一化（sklearn 的 `feature_importances_`）。**缺点**：偏好多取值特征，对相关特征会分摊；更可靠的是置换重要性（打乱该列看指标降多少）。 - **Q：从 CART 怎么推到 GBDT / XGBoost?**（面到这一步说明前面都过了） A：GBDT = 串行加法模型，每棵新树拟合当前模型的**负梯度（残差）**，实现"用树做梯度下降"；XGBoost 在此基础上：① 用损失函数的**二阶泰勒展开**（一阶梯度 g、二阶 h）；② 目标函数显式加正则 `γT + λ/2·Σw²`；③ 最优叶权重有闭式解 `w* = -Σg / (Σh + λ)`，分裂增益有解析式；④ 工程上直方图 + 特征预排序 + 并行。LightGBM 换成 leaf-wise 生长 + GOSS（梯度采样）+ EFB（互斥特征捆绑）。 - **Q：回归树和分类树的区别?** A：划分准则从基尼换成**MSE 下降**，叶子值取均值而不是众数；其余（二分、递归、贪心阈值搜索）完全一样。 - **Q：随机森林和 GBDT 的区别?** A：RF = bagging（并行、样本+特征双采样、投票），主要降**方差**；GBDT = boosting（串行、拟合残差），主要降**偏差**。

现场最易错 3 点：漏停止条件导致栈溢出；索引池用指针被 realloc 打崩；候选阈值没跳过同值导致无效分裂。

---

3. 力扣部分（C++ 版）

3.1 华为机试形态（决定你练什么）

- 校招机试：**3 题 / 400 分 / 120 分钟**，约 简单100 + 中等200 + 困难300，**ACM 模式**（自己读标准输入、自己打印），可用本地 IDE 调试。**做出 200 分就进面**，第一题必须稳。
- 技术面手撕：面试官给题、你共享屏幕写，通常 LeetCode 中等。要求"能跑 + 说复杂度"。
- 华为偏好：**字符串模拟、DFS/BFS、DP、二叉树、双指针**。图论进阶（网络流、线段树）几乎不考。
- 语言：机试支持 C/C++，你直接用 C++，不用换。

3.2 IO 模板（很多人挂在这，不是挂算法）

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```cpp
int main() {
    ios::sync_with_stdio(false);   // 关掉 C 与 C++ 流同步，提速
    cin.tie(nullptr);              // 解绑 cin/cout，提速

    // 场景 1：第一行 n，之后 n 个整数
    int n;
    if (!(cin >> n)) return 0;
    vector<long long> a(n);
    long long sum = 0;
    for (int i = 0; i < n; ++i) { cin >> a[i]; sum += a[i]; }
    cout << sum << "\n";

    // 场景 2：多组数据直到 EOF（华为常见）
    // int a, b; while (cin >> a >> b) cout << a + b << "\n";

    // 场景 3：一行里用逗号分隔 -> 先读成字符串再切
    // string line; getline(cin, line);
    // stringstream ss(line); string tok;
    // while (getline(ss, tok, ',')) v.push_back(stoi(tok));

    // 场景 4：先读表头行再读数据（cin >> 与 getline 混用必须先吃掉换行）
    // cin >> n; cin.ignore(); getline(cin, line);
    return 0;
}
```

三条铁律：1. `cin >>` 和 `getline` **不要混着用**，混了就先 `cin.ignore()` 吃掉残留换行——这是 ACM 模式第一大坑。2. 多组数据一律写 `while (cin >> x)`，别假设只有一组。3. 输出用 `"\n"` 不用 `endl`（`endl` 强制 flush，数据量大时拖慢）。

3.3 七个模板（现场默写）

```cpp
// T1 快排（随机化 pivot，避免有序数组退化）
void quickSort(vector<int>& a, int l, int r) {
    if (l >= r) return;
    int p = l + rand() % (r - l + 1);
    swap(a[l], a[p]);
    int pivot = a[l], i = l;
    for (int j = l + 1; j <= r; ++j) if (a[j] < pivot) swap(a[++i], a[j]);
    swap(a[l], a[i]);
    quickSort(a, l, i - 1);
    quickSort(a, i + 1, r);
}

// T2 二分（左边界 / lower_bound 手写版）
int lowerBound(const vector<int>& a, int x) {
    int lo = 0, hi = (int)a.size();          // 注意 hi 是 size 而不是 size-1（左闭右开）
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;        // 防溢出写法
        if (a[mid] < x) lo = mid + 1;
        else hi = mid;
    }
    return lo;
}

// T3 滑动窗口（无重复字符最长子串）
int lengthOfLongestSubstring(const string& s) {
    vector<int> last(128, -1);
    int left = 0, ans = 0;
    for (int i = 0; i < (int)s.size(); ++i) {
        unsigned char c = s[i];
        if (last[c] >= left) left = last[c] + 1;
        last[c] = i;
        ans = max(ans, i - left + 1);
    }
    return ans;
}

// T4 优先队列 TopK（n log k）
vector<int> topK(const vector<int>& nums, int k) {
    priority_queue<int, vector<int>, greater<int>> pq;   // 小顶堆
    for (int x : nums) {
        pq.push(x);
        if ((int)pq.size() > k) pq.pop();
    }
    vector<int> res;
```

```
    while (!pq.empty()) { res.push_back(pq.top()); pq.pop(); }
    return res;
}

// T5 并查集（路径压缩 + 按秩合并）
struct DSU {
    vector<int> p, r;
    DSU(int n) : p(n), r(n, 0) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
    bool unite(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (r[ra] < r[rb]) swap(ra, rb);
        p[rb] = ra;
        if (r[ra] == r[rb]) ++r[ra];
        return true;
    }
};

// T6 二叉树层序（BFS 模板，所有"层"类题都从它改）
struct TNode { int val; TNode *left = nullptr, *right = nullptr; };
vector<vector<int>> levelOrder(TNode* root) {
    vector<vector<int>> res;
    if (!root) return res;
    queue<TNode*> q; q.push(root);
    while (!q.empty()) {
        int sz = q.size();                    // 关键：先固定这一层的节点数
        vector<int> layer;
        while (sz--) {
            TNode* u = q.front(); q.pop();
            layer.push_back(u->val);
            if (u->left) q.push(u->left);
            if (u->right) q.push(u->right);
        }
        res.push_back(layer);
    }
    return res;
}

// T7 01 背包（一维滚动数组，内层必须倒序）
int knapsack01(const vector<int>& w, const vector<int>& v, int W) {
    vector<int> dp(W + 1, 0);
    for (int i = 0; i < (int)w.size(); ++i)
        for (int c = W; c >= w[i]; --c)       // 倒序：保证每个物品只用一次
            dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
    return dp[W];
}
```

为什么 01 背包要倒序：正序时 `dp[c-w[i]]` 可能是本轮刚被更新的值（= 同一物品用了多次，退化成完全背包）；倒序保证用的是上一轮的值。

3.4 STL 速查（只列现场真正用到的 20 条）

需求	写法
数组	`vector<int> a(n, 0);`
二维	`vector<vector<int>> g(n, vector<int>(m, 0));`
排序降序	`sort(a.begin(), a.end(), greater<int>());`
自定义排序	`sort(v.begin(), v.end(), [](auto& x, auto& y){ return x.first < y.first; });` **必须严格弱序**
去重	`sort` 后 `a.erase(unique(a.begin(), a.end()), a.end());`
查找	`binary_search` / `lower_bound` / `upper_bound` （**前提：已排序**）
最大/最小	`*max_element(a.begin(), a.end())` （返回迭代器！）
求和	`accumulate(a.begin(), a.end(), 0LL)` （**初值用 0LL 防溢出**）
计数	`count(a.begin(), a.end(), x)`
反转	`reverse(s.begin(), s.end());`

需求	写法
填充	`fill(dp.begin(), dp.end(), 0);`
字符串转数	`stoi / stoll / stod`（大数用 `stoll`），`to_string(x)`
字符判断	`isdigit(c)`、`isalpha(c)`、`toupper(c)`
哈希表	`unordered_map<int,int> mp; mp[k]++;`（**注意 `mp[k]` 会插入默认值**，查存在用 `count` / `find`）
计数数组（小范围）	`int cnt[26] = {0};`（比 map 快一个量级）
集合有序	`set<int> s; s.insert(x); s.erase(x);`（O(log n)，中位数/前驱后继用它）
小顶堆	`priority_queue<int, vector<int>, greater<int>>`
双向队列	`deque<int> dq; dq.push_front(x);`（单调队列/滑动窗口最大值）
栈	`stack<char> st;`
位运算	`__builtin_popcount(x)`、`x & -x`（lowbit）

3.5 20 题清单（C++ 关键点）

#	题	C++ 关键点
1	两数之和 / 三数之和	`unordered_map`；三数之和先排序 + 双指针 + 跳过重复
2	无重复字符最长子串	`vector<int> last(128,-1)`（别用 unordered_map）
3	最长回文子串	中心扩展 2n-1 个中心
4	有效括号	`stack<char>`，空栈时 `top()` 会崩
5	字符串反转 / 翻转单词 / 大数相加	`string` 拼接；大数用进位 `int c`
6	合并两个有序链表 / 反转链表	哑结点 `ListNode dummy, *t = &dummy;`
7	环形链表 / 相交链表	快慢指针；相交走两遍换头
8	LRU 缓存	`list<pair<int,int>>` + `unordered_map<int, list<...>::iterator>`，**注意 list 迭代器在 splice 后仍有效**
9	二叉树层序 / 最大深度 / 之字形	BFS 先存 `sz = q.size()`；之字形用 `reverse` 隔层
10	验证 BST	传上下界 `long long lo, hi`（**节点值可能是 INT_MIN/MAX**）
11	岛屿数量 / 最大面积	DFS 淹没，或 DSU；大输入注意递归深度
12	腐烂的橘子	多源 BFS：所有源先入队，再按层计数
13	全排列 / 子集 / 组合总和	`sort` + 回溯 + 同层跳过 `i>start && a[i]==a[i-1]`
14	爬楼梯 / 打家劫舍	`long long` 防溢出（70 题会超 int？不会，但要养成习惯）
15	编辑距离 / LCS	二维 dp，`vector<vector<int>>`，注意初始化第 0 行/列
16	最大子数组和	Kadane，`int cur = 0, best = a[0]`
17	第 K 大 / 前 K 高频	小顶堆；高频用 `unordered_map` 计数再堆
18	搜索旋转排序数组	二分时先判断哪半边有序
19	零钱兑换	完全背包：外层物品内层容量**正序**
20	手写快排 / 归并 / 堆排	见 3.3；归并要求辅助数组

3.6 C++ 现场易错 8 条（每条都真的会挂）

1. **溢出**：`int` 相乘/累加超 2e9 直接 UB。看到"和/乘积/路径长度"就先想 `long long`。

2. **`size()` 是无符号**：`for (int i = 0; i < a.size() - 1; ++i)` 在 `a` 为空时 `size()-1` 变成巨大正数 → 死循环。写法：`int n = (int)a.size();` 先转 int。
3. **`sort` 的比较函数必须严格弱序**：写 `<=` 或 `>=` 在某些 STL 实现下直接越界崩溃。
4. **`unordered_map` 用 `[]` 会插入默认值**：判断存在用 `count` 或 `find`，别用 `mp[k] != 0`。
5. **迭代器失效**：`vector` 在 `push_back` 后旧迭代器/指针可能失效；`unordered_map` 在 rehash 后所有迭代器失效（`list` 的插入不失效）。
6. **`cin >>` 后接 `getline`**：必须先 `cin.ignore()`。
7. **递归深度**：DFS 在 1e5 级别链式数据上会爆栈，改成显式 `stack` 或迭代。
8. **多组数据忘了重置全局数组**：`fill` 一遍，别靠默认初始化。

---

4. 万一面试官坚持要 Python / PyTorch

4.1 先说三句兜底话术（背下来）

1. "核心逻辑我刚才用 C++ 展开写完了，PyTorch 版是把循环换成张量算子，我写一下映射关系。"
2. "权重投影就是 `nn.Linear`，拆头用 `view + transpose`，打分是 `Q @ K.transpose(-2,-1)`，缩放除 `sqrt(d_k)`。"
3. "我平时主力是 C++/工程侧，PyTorch 我读得懂、改得动，写这种标准实现没问题。"（**不说"我不会 Python"，说"我能写、只是首选 C++"**）

4.2 三张对照表（C++ ↔ numpy ↔ torch）

要做的事	C++	numpy	torch
建矩阵 n×m 全零	`vector<float> A(n*m, 0.f)`	`np.zeros((n, m))`	`torch.zeros(n, m)`
取元素 (i,j)	`A[i*m + j]`	`A[i, j]`	`A[i, j]`
矩阵乘	`matmul(A,B,n,k,m)`	`A @ B`	`A @ B`
逐元素乘	`for` 循环	`A * B`	`A * B`
按行求和	`for` 累加	`A.sum(-1, keepdims=True)`	`A.sum(dim=-1, keepdim=True)`
按行最大值	`for` 找 max	`A.max(-1, keepdims=True)`	`A.max(dim=-1, keepdim=True).values`
softmax（按行）	手写三段循环	手写 `e = np.exp(A - A.max(-1, keepdims=True)); e / e.sum(-1, keepdims=True)`	`torch.softmax(A, dim=-1)`
形状变换	无（本来就是扁平的）	`A.reshape(n, h, dk)`	`A.view(n, h, dk)` / `.reshape`
转置后两维交换	索引里换位置	`A.transpose(1, 2)`	`A.transpose(1, 2)`
生成全 1 / 单位阵	`vector<float>(n,1.f)`	`np.ones(n)` / `np.eye(n)`	`torch.ones(n)` / `torch.eye(n)`
取一列	`A[i*m + j]`	`A[:, j]`	`A[:, j]`
均值 / 方差	手动累加再除	`A.mean(-1)` / `A.var(-1)`	`A.mean(-1)` / `A.var(-1, unbiased=False)`
屏蔽赋值	`if (mask[i]) S[i] = -1e30f;`	`np.where(mask, -1e9, S)`	`S.masked_fill(mask, float('-inf'))`
拼回多头	`Ocat[i*D + hi*dk + c] = ...`	`np.concatenate` / 先分配再赋值	`.transpose(1,2).contiguous().view(B,L,D)`
随机初始化	`mt19937`	`np.random.default_rng(0)`	`torch.manual_seed(0)`

4.3 三段可直接背的 PyTorch 版

（1）MHA（面试要 Python 时写这段，40 行内）

```python
import math, torch, torch.nn as nn

class MHA(nn.Module):
    def __init__(self, d_model, n_head):
        super().__init__()
        assert d_model % n_head == 0
        self.h, self.dk = n_head, d_model // n_head
        self.wq, self.wk, self.wv, self.wo = [nn.Linear(d_model, d_model) for _ in range(4)]

    def forward(self, q, k, v, mask=None):
        B, Lq, D = q.shape
        Lk = k.size(1)
        Q = self.wq(q).view(B, Lq, self.h, self.dk).transpose(1, 2)   # [B,h,Lq,dk]
        K = self.wk(k).view(B, Lk, self.h, self.dk).transpose(1, 2)
        V = self.wv(v).view(B, Lk, self.h, self.dk).transpose(1, 2)
        s = Q @ K.transpose(-2, -1) / math.sqrt(self.dk)              # [B,h,Lq,Lk]
        if mask is not None:
            s = s.masked_fill(mask.bool(), float('-inf'))             # True = 屏蔽
        a = torch.softmax(s, dim=-1)
        o = (a @ V).transpose(1, 2).contiguous().view(B, Lq, D)       # 拼头
        return self.wo(o), a
```

对照讲：`view(B,Lq,h,dk).transpose(1,2)` 就是 C++ 里"按列切头"；`contiguous().view(B,Lq,D)` 就是 C++ 里的 `0cat` 拼列。**`contiguous()` 不能省**：transpose 后内存不连续，`view` 会报错，这是面试官最爱抓的一点。

（2）softmax + 交叉熵（要"手写不用内置"时）

```python
def softmax(x, dim=-1):
    x = x - x.max(dim=dim, keepdim=True).values
    e = x.exp()
    return e / e.sum(dim=dim, keepdim=True)

def cross_entropy(logits, target):               # logits: [N,C]  target: [N]
    m = logits.max(dim=1, keepdim=True).values
    lse = m.squeeze(1) + torch.log(torch.exp(logits - m).sum(dim=1))
    return (lse - logits.gather(1, target[:, None]).squeeze(1)).mean()
```

（3）kmeans（numpy 向量化，10 行）

```python
import numpy as np

def kmeans(X, k, iters=100, seed=0):
    rng = np.random.default_rng(seed)
    C = X[rng.choice(len(X), k, replace=False)].copy()
    for _ in range(iters):
        d = ((X[:, None, :] - C[None, :, :]) ** 2).sum(-1)     # n×k 距离矩阵
        lab = d.argmin(1)
        newC = C.copy()
        for j in range(k):
            m = lab == j
            if m.any():                                        # 空簇保留旧中心
                newC[j] = X[m].mean(0)
        if np.allclose(newC, C): break
        C = newC
    return lab, C
```

> 说明：C++ 版全部实测通过；上面 Python 代码的对等逻辑（同构的 numpy 实现）也已验证，但这台机器没有 torch 环境，**torch 三段建议你在自己环境跑一遍**再上场。

5. 三天练习法（C++ 版，针对"能看不能写"）

5.1 每一天只做"不看书写出来"

天	任务	验收标准（必须逐条对上）
D1 上午	softmax + 交叉熵 + LayerNorm	编译过；softmax 对 1e3 输入不出 nan；LN 输出 mean≈0/var≈1；能答"为什么减 max""为什么除 √d""LN 和 BN 区别"
D1 下午	IoU + NMS	能算手写例子 81/119；能说 Soft-NMS 三个改进
D2 上午	kmeans	三簇能分开；能答"k 怎么选""为什么局部最优（举我给的 125/150 vs 150/150 实测）""空簇怎么办"
D2 下午	self-attention → 改多头	**15 分钟内**单头写完；改多头只改 4 处（dk、权重切片、尺寸参数、拼回）；能答 √dk 的方差论证、FlashAttention 省什么
D3 上午	CART	二分类训练集能到 100%；能答 CART/ID3/C4.5 区别、剪枝两种、怎么推到 GBDT
D3 下午	力扣 4 题（C++，ACM 模式）	用 3.2 模板读入；每题先说复杂度再写；写完自己造 3 个边界用例

5.2 自检清单（面试前一晚过一遍，全部打勾才算准备好）

- [ ] 能默写 `matmul` 的三层循环和三个索引表达式
- [ ] 能一口气说出"softmax 三步 + 为什么减 max"
- [ ] 能说清 Q/K/V 各自的角色，以及为什么除以 √d_k（方差论证）
- [ ] 能说清 self-attention 的时间/显存复杂度，以及 FlashAttention 省的是什么（**省 IO 不省 FLOPs**）
- [ ] 能说清多头"按列切、按列拼"，h=1 时退化为单头（我实测一致）
- [ ] 能列出 mask 的三种类型，并说出"全屏蔽行会 NaN"
- [ ] 能说 CART 用基尼、ID3 用信息增益、C4.5 用增益率，以及为什么 CART 是二叉树
- [ ] 能说出 kmeans 三个局限 + kmeans++ + 空簇处理
- [ ] 能说 NMS 的三个改进方向
- [ ] 力扣：能默写快排/二分/滑窗/优先队列/DSU 五个模板，且知道 01 背包为什么倒序
- [ ] 能说出 C++ 现场 8 条易错里的 5 条（尤其 `size()` 无符号、`long long` 溢出）

5.3 一个加速技巧：把代码"压缩成骨架"

真正要背的不是完整代码，而是骨架。以 MHA 为例，骨架只有 6 行：

```
Q,K,V = XWq, XWk, XWv                          (L×D)
S = Q·K^T / √dk      -> L×L
S = mask(S)                                     (屏蔽位置 -1e30)
A = rowSoftmax(S)                               (逐行归一)
O = A·V                                          (L×D)
Out = Concat(各头 O) · Wo
```

这 6 行能背下来，代码现场就能一行一行"翻译"出来。其余五题同理，我在每题的"口述脚本"里已经给了这个骨架，练习时按骨架复述、不要背整段代码。

附录：配套可运行文件与验证结果

文件：`ai_interview_cpp.cpp`（与本指南同目录，纯 STL，无第三方依赖）

```
g++ -O2 -std=c++17 ai_interview_cpp.cpp -o ai && ./ai
```

本次实测输出（26 项全部 OK）：

模块	通过的断言
matmul	2×3 · 3×2 = [[4,5],[10,11]]
softmax	和为 1；单调性；1e3 输入不溢出；平移不变；与 double 参考值一致（5 项）
交叉熵	与 -log softmax 参考值一致；1e3 级 logits 不溢出（2 项）
LayerNorm	mean≈0、var≈1
IoU / NMS	IoU=81/119；无交集为 0；NMS 保留 {0,2}（3 项）
kmeans	单次 seed=7 一致性 125/150（局部最优实例）；20 次初始化后 150/150
self-attention	行和=1；因果 mask 上三角=0；mask 后行和仍=1；O == A · V（4 项）
multi-head	h=1+Wo=I 与单头逐元素一致；attn 形状 h · L · L；每头行和=1；输出 L×D（4 项）
CART	二分类 ACC=1.0；根阈值 1.670 落在簇间；三分类 ACC=1.0；树确实分裂（4 项）

把 `ai_interview_cpp.cpp` 当练习本用：先在纸上写，再打开文件对答案，最后删掉自己重写一遍。

手撕 AI 题详解 —— Python 版（零基础也能写出来）

> **说明**：本文档为对外分享版，前雇主项目的量化指标已做量级化处理，方法论与话术保持原样。
> 定位：**算法题（力扣）继续用 C++**（见另一份《手撕AI题详解_C++版》第 3 章），**AI 题（softmax / MHA / kmeans / 决策树）用 Python**——这份文档专门解决"Python 写不出来"。
> 全部代码已实测：**39 项断言全部通过**，其中 torch 部分装了 CPU 版 torch 2.14 真跑过，包括与官方 `F.scaled_dot_product_attention`、`F.cross_entropy` 的逐元素对比。配套可运行文件：`ai_interview_py.py`（同目录），`python3 ai_interview_py.py`。
> 阅读顺序：第 1、2 章（语法地基，1 小时）→ 第 3 章逐题（主体）→ 第 4 章 torch → 第 5 章验证法。

0. 先给你一个心理预期

你说"Python 基础不好"，但**手撕 AI 题需要的 Python 只有 30 个写法**，而且全是"数学公式直译"。真正的门槛不是语法，是三件事：

真正的门槛	为什么 C++ 选手会卡	本文怎么解决
numpy 的"轴"概念（axis / keepdims）	C++ 里你写循环很自由，numpy 你要先想"沿哪一维聚合"	第 2.2 节用一张"循环 ↔ axis"对照表讲透
广播规则（shape 怎么对齐）	C++ 没有这概念，写错就报 shape 不匹配	第 2.3 节给三条规则 + `None` 插轴的口诀
"一行顶你二十行"的 API（argmin / where / bincount / gather）	你不知道有这个函数，只能硬写循环	第 2.5 节列 12 个必会 API，每个配 C++ 等价写法

一句话心法：C++ 里你写的是"过程"，Python/numpy 里你先想"我要的结果是什么形状"，再选一个能产出这个形状的函数。**每次写代码前，先在纸上写 shape**。

1. Python 语法速成（只讲手撕用得到的）

> 目标：让你能读懂并写出第 3 章的所有代码。全部用 C++ 对照，跳过装饰器、生成器、异步这些用不到的。

1.1 十条语法对照表（必背）

要做的事	C++	Python	备注
定义函数	`int f(int x) { return x+1; }`	`def f(x): return x + 1`	**没有类型声明**，返回值靠 `return`
定义类	`class A { public: int x; };`	`class A:` + `__init__`	`def __init__(self, x): self.x = x`
成员变量	`this->x`	`self.x`	方法第一个参数永远是 `self`
循环	`for (int i = 0; i < n; ++i)`	`for i in range(n):`	`range(n)` = 0..n-1
遍历元素	`for (auto& v : a)`	`for v in a:`	要下标用 `enumerate(a)` → `for i, v in enumerate(a):`
数组	`vector<int> a(n, 0);`	`a = [0] * n`	或用 numpy（见第 2 章）
长度	`a.size()`	`len(a)`	Python 的 `len` 对 str/list/ndarray 都适用
条件	`x > 0 ? a : b`	`a if x > 0 else b`	或 `np.where`（数组版）
整数除法	`7 / 2 == 3`	`7 // 2 == 3`	**`/` 在 Python 里是浮点除法**，`7/2=3.5`
幂	`pow(x, 2)` / `x*x`	`x ** 2`	

1.2 五个"语法不同但一定用到"的点

```
# 1) 缩进就是作用域（没有大括号！）。一个 tab 或 4 个空格，全文件保持一致。
def f(x):
    if x > 0:
        return 1        # 这行属于 if
    return 0            # 这行属于函数

# 2) 列表推导式：一行写完"循环 + 筛选 + 收集"
ious = [iou(boxes[i], boxes[j]) for j in rest]          # 等价 C++: for(...) v.push_back(...)
pos = [x for x in a if x > 0]                           # 带筛选

# 3) 布尔数组直接当下标用（numpy 的核心技巧，C++ 里没有）
X[mask]        # 取出所有 mask 为 True 的行
y[~mask]       # ~ 是布尔取反

# 4) 多重赋值 / 交换
a, b = b, a
lo, hi = 0, len(a)

# 5) 返回值可以是多个
def kmeans(...):
    return labels, centers      # 返回元组，调用方 `lab, cen = kmeans(...)`
```

1.3 你会看到但可能没写过的符号（第 3 章会反复出现）

符号	含义	例子
`@`	矩阵乘（Python 3.5+）	`Q @ K.T`
`.T`	转置	`K.T`
`:`	切片	`x[:, sl]` / `x[:-1]`
`None`（在索引里）	插入一个新轴	`X[:, None, :]` 见 2.3
`**2`	平方	`(p ** 2).sum()`
`-1`（在 axis 里）	"最后一维"	`x.sum(-1)`
`//`	整除	`D // h`
`~`	布尔取反（numpy）	`y[~mask]`
`np.inf` / `-1e9`	无穷 / 极小值（做 mask）	`np.where(mask, -1e9, scores)`

---

2. numpy 地基：读懂 Python 手撕代码的钥匙

> 这一章是本文档的核心。**看懂这五个小节，第 3 章的代码你会觉得比 C++ 还简单。**

2.1 ndarray 是什么

```
import numpy as np
x = np.array([[1., 2., 3.], [4., 5., 6.]])   # 2 行 3 列
x.shape      # (2, 3)   —— 元组，写法像 C++ 的 pair，用 x.shape[0] 取行数
x.dtype      # float64
```

关键区别：C++ 你管"一维 vector + 自己算下标"；numpy 里 shape 是数据结构的一部分，所有运算都按 shape 自动对齐。代价是：**shape 写错就报错**——这是好事，等于免费的类型检查。

2.2 axis 与 keepdims（第一个大坎）

一句话：`axis=k` 表示"**沿第 k 维做聚合，做完这一维就消失**"。

```
x = np.array([[1., 2., 3.],
              [4., 5., 6.]])        # shape (2,3)
x.sum(axis=0)   # [5, 7, 9]       沿"行"方向压 → 每列求和，shape 从 (2,3) 变 (3,)
```

```
x.sum(axis=1)   # [6, 15]         沿"列"方向压 → 每行求和，shape 变 (2,)
x.sum(axis=-1)  # [6, 15]         -1 = 最后一维，等价 axis=1（推荐用 -1，代码通用）
x.max(axis=1)   # [3, 6]          每行最大值
```

C++ 对照：`axis=1` 就是"外层循环遍历行、内层累加列"，但 numpy 把内层循环省掉了：

```
// C++ 求每行和
for (int i = 0; i < n; ++i) { float s = 0; for (int j = 0; j < m; ++j) s += x[i][j]; rowsum[i] = s; }
```

```
rowsum = x.sum(axis=1)          # 一行搞定
```

keepdims=True 是干什么的（softmax 里必用）：

```
x.sum(axis=1)                # shape (2,)     ← 少了一维，没法直接除回 (2,3)
x.sum(axis=1, keepdims=True)  # shape (2,1)    ← 保住维度，可广播到 (2,3)
x / x.sum(axis=1, keepdims=True)     # 每行归一化，正确
x / x.sum(axis=1)                    # 报错：(2,3) 除 (2,) 语义不明确
```

口诀：只要这个聚合结果后面还要参与逐元素运算（相减/相除），就必须 `keepdims=True`。

2.3 广播（第二个大坎）

规则三条，按顺序判断，满足就停： 1. 从**最后一维**往前比，维度相等 → 可以。 2. 某一维是 **1** → 自动扩展成对方长度（复制）。 3. 维度数不同 → 缺的那边在前面补 1。

```
a = np.zeros((3, 4))
b = np.zeros((   4))     # 补成 (1,4)，再扩成 (3,4)      ✅
c = np.zeros((3, 1))     # 扩成 (3,4)                    ✅
d = np.zeros((2, 4))     # 3 != 2 且都不是 1             ❌ 报错
```

`None` 插轴的用法（kmeans 里那句长代码就是它）：

```
X = np.zeros((5, 2))          # 5 个样本，每样本 2 维
X[:, None, :].shape           # (5, 1, 2)   ← 在中间插一个长度 1 的轴
C = np.zeros((3, 2))          # 3 个中心
C[None, :, :].shape           # (1, 3, 2)
X[:, None, :] - C[None, :, :] # (5,1,2) - (1,3,2) → (5,3,2)   广播成立!
```

再 `.sum(-1)` → `(5,3)` = 5 个点到 3 个中心的距离平方。**用 C++ 写这段是三重循环，numpy 三行。**

2.4 索引：切片、布尔索引、高级索引

```
x = np.arange(12).reshape(3, 4)
x[0]          # 第 0 行
x[:, 1]       # 第 1 列          ← 逗号左边是行、右边是列
x[1:3]        # 第 1、2 行       ← 切片左闭右开
x[:, 1:3]     # 第 1、2 列
x[:, -1]      # 最后一列

# 布尔索引（最常用）
mask = x > 5
x[mask]       # 取出所有 >5 的元素，变成长度 k 的一维数组
y[labels == 2]         # 取出标签为 2 的那些样本（C++ 里要写循环判断）
y[~mask]               # 取反

# 高级索引（交叉取"每行的第 target 个元素"，交叉熵里用）
logits[np.arange(3), [2, 0, 1]]   # 取出 logits[0,2], logits[1,0], logits[2,1]
```

2.5 12 个必会 API（每个都给 C++ 等价思路）

API	作用	C++ 里你会怎么写
`x.max(axis, keepdims)`	按轴最大值	循环比较
`x.sum(axis)` / `x.mean(axis)` / `x.var(axis)`	求和/均值/方差	累加再除
`np.exp(x)` / `np.sqrt(x)` / `np.log(x)`	逐元素数学	循环 + `expf`
`np.where(cond, a, b)`	逐元素三目	`for` + `if`

API	作用	C++ 里你会怎么写
`x.argmin(axis)` / `x.argmax(axis)`	**最小/最大值的位置**（不是值）	循环记下标
`np.argsort(x)` / `np.argsort(-x)`	排序后的**下标**（降序用负号）	`sort` + 下标数组
`np.bincount(y)`	统计 0..k 每个整数出现几次	`int cnt[k] = {0}`
`np.unique(x)`	去重并排序	`sort` + `erase(unique)`
`np.concatenate(list, axis)`	沿某维拼接	两次 `insert`
`np.stack(list, axis)`	堆成新维度	新建矩阵逐元素拷
`np.allclose(a, b, atol)`	浮点近似相等（自检用）	手写 `fabs(a-b)<eps`
`x.copy()`	**深拷贝**（Python 里赋值是引用！）	C++ 的 `=` 就是值拷贝

> **最容易踩的坑**：`a = b` 在 Python 里是**引用**，不是拷贝。需要独立数组就写 `b = a.copy()`。kmeans 里 `new_centers = centers.copy()` 就是为这个——不 copy 会把上一步的中心改掉。

---

3. 逐题详解（numpy 版）

> 每题六段：**题目 → 30 秒口述 → 公式 → 代码 → 逐行讲解（重点 Python/numpy 语法）→ 追问 Q&A → 最易错 3 点**。

3.1 softmax（3 行，但每行都要会讲）

面试官原话："用 Python 写个 softmax。"

30 秒口述：

> "三步：先沿最后一维减去每行最大值防溢出，再取 exp，最后除以每行的和。numpy 里就是 `x - x.max(keepdims=True)`、`np.exp`、`e / e.sum(keepdims=True)`。keepdims 不能省，否则形状对不上。"

代码：

```python
def softmax(x, axis=-1):
    x = x - x.max(axis=axis, keepdims=True)     # ① 平移：每行减去该行最大值
    e = np.exp(x)                               # ② 逐元素取指数
    return e / e.sum(axis=axis, keepdims=True)  # ③ 每行归一化
```

逐行讲解（Python/numpy 语法是这个题的考点）：

写法	语法点	为什么必须这么写
`x.max(axis=-1, keepdims=True)`	`axis=-1` = 最后一维；`keepdims=True` 保住这一维为长度 1	结果 shape 是 (2,1) 而不是 (2,)，才能和 (2,3) 广播相减。**写成 `x.max(axis=1)` 会报 "operands could not be broadcast"**
`x - (...)`	numpy 的 `-` 是**逐元素**，且自动广播	不需要任何循环
`np.exp(x)`	逐元素 exp（不是 `math.exp`，那个只能处理标量）	`math.exp` 传入数组会 TypeError
`e / e.sum(axis=-1, keepdims=True)`	逐元素相除，同样靠广播	换成 `e / e.sum(axis=-1)[:, None]` 也等价（两种写法都要能认）

手写时的一个自检：`softmax(x).sum(-1)` 必须全是 1.0。我在 `ai_interview_py.py` 里就是这么断言的。

追问 Q&A：- **Q：为什么减 max？** A：`ex` 上溢成 inf，再除 inf 得 nan；分子分母同乘 `e^(-max)` 值不变。 - **Q：`axis=0` 和 `axis=1` 分别是什么？** A：`axis=1`（=最后一维）是"每个样本自己归一化"——softmax 永远用这个；`axis=0` 是"同一位置上跨样本归一化"，那是错的。 - **Q：形状是 1 维时怎么办？** A：`axis=-1` 自动适配，所以函数写成 `axis=-1` 最通用。 - **Q：温度 / mask？** A：把 `x` 换成 `x/T`；mask 位置置 `-1e9`（不要用 `-np.inf`，因为 `-inf - (-inf)` 会出 nan；全屏蔽行仍是 nan，工业实

现要单独处理）。 - **Q：softmax 和 log_softmax 区别？** A：log_softmax = `x - max - log(sum(exp(x-max)))`，在对数域直接算，交叉熵用它更稳。

最易错 3 点：忘了 `keepdims`；用 `math.exp`；`axis` 写成 0。

3.2 交叉熵（考你对"高级索引"和 logsumexp 的理解）

面试官原话："手写交叉熵，不用 nn.CrossEntropyLoss。"

30 秒口述：

> "loss 等于 logsumexp(每行 logits) 减去正确类的 logit，再对 batch 取均值。我不先算 softmax 再取 log，直接在对数域算，避免小概率下溢。取正确类 logit 用高级索引 `logits[np.arange(N), target]`。"

代码：

```python
def cross_entropy(logits, target, weight=None):
    m = logits.max(axis=1, keepdims=True)                  # [N,1]
    lse = m[:, 0] + np.log(np.exp(logits - m).sum(axis=1))   # [N] logsumexp
    nll = lse - logits[np.arange(len(target)), target]       # [N] 取正确类的 logit
    if weight is not None:
        w = weight[target]                                 # [N]
        return (nll * w).sum() / w.sum()
    return nll.mean()
```

逐行讲解：

- `m = logits.max(axis=1, keepdims=True)` → shape (N,1)。**为什么这里要 keepdims**：下一行要 `logits - m`，需要 (N,C) - (N,1) 才能广播。
- `m[:, 0]` → 把 (N,1) 取成 (N,)。**语法点**：逗号左边 `:` 表示所有行，右边 `0` 表示第 0 列。
- `np.exp(logits - m).sum(axis=1)` → 每行 exp 之和，(N,)。
- `np.log(...)` → 逐元素 log。
- **`logits[np.arange(len(target)), target]`** ← 本题唯一的"技巧点"：两个下标数组交叉索引，等价于 C++ 的 `cpp for (int i = 0; i < N; ++i) picked[i] = logits[i][target[i]];` 这是 numpy 高级索引的标准用法，**面试官会专门看你会不会这个**（不会的话只能写循环，也正确但显得生疏）。
- `weight[target]` → 用 target 去查权重，得到每个样本的权重 (N,)。
- `(nll * w).sum() / w.sum()` → 加权平均。C++ 里你要累加两个变量再除。

追问 Q&A： - **Q：batch 版和单样本版的关系？** A：单样本就是 N=1；`mean` 是 batch 平均，框架的 `reduction='mean'` 就是它。 - **Q：`ignore_index` 怎么做？** A：把 target 等于 ignore_index 的样本 mask 掉：`valid = target != ig; return (nll[valid]).mean()`。 - **Q：为什么梯度是 p - y？** A：对 logit 求导，softmax + CE 的雅可比与 CE 的导数抵消，得到极简形式——这也是为什么框架把 softmax 和 CE 合成一个算子（数值更稳、更快）。 - **Q：多标签怎么办？** A：用 sigmoid + BCE：`F.binary_cross_entropy_with_logits(logits, targets)`，逐元素算 `log(1+e^-z) - y·z`。

最易错 3 点：`m` 忘了 keepdims；用 `softmax` 再 `log`（小概率下溢成 -inf）；交叉索引写成 `logits[target]`（那是取第 target 行，形状全错）。

3.3 LayerNorm（考 axis 和广播的理解）

代码：

```python
def layer_norm(x, gamma, beta, eps=1e-5):
    mu = x.mean(axis=-1, keepdims=True)          # [..., 1]
    var = x.var(axis=-1, keepdims=True)          # [..., 1]   (除 D，与框架一致)
    return gamma * (x - mu) / np.sqrt(var + eps) + beta
```

逐行讲解：

写法	语法点
`x.mean(axis=-1, keepdims=True)`	对最后一维求均值且保持维度，才能与 x 广播
`x.var(axis=-1)`	**numpy 默认是总体方差（除 D）**，正好与框架一致；如果要无偏方差写 `ddof=1`
`gamma * (...) + beta`	gamma/beta 形状 (D,)，自动广播到 (..., D)，**不用手动 reshape**
`np.sqrt(var + eps)`	eps 在 sqrt **里面**

追问：LN vs BN（统计量维度）、RMSNorm（去掉均值）、Pre-LN vs Post-LN、BN 推理用滑动平均。细节见 C++ 版 2.3 节。

最易错：`axis=0`（那是 BN 的方向）；eps 放外面；用 `ddof=1`。

3.4 IoU + NMS（考 while 循环 + argsort + 布尔索引）

代码：

```
def iou(box_a, box_b):
    ix1, iy1 = max(box_a[0], box_b[0]), max(box_a[1], box_b[1])
    ix2, iy2 = min(box_a[2], box_b[2]), min(box_a[3], box_b[3])
    iw, ih = max(0.0, ix2 - ix1), max(0.0, iy2 - iy1)       # clamp 到 0
    inter = iw * ih
    area_a = max(0.0, box_a[2] - box_a[0]) * max(0.0, box_a[3] - box_a[1])
    area_b = max(0.0, box_b[2] - box_b[0]) * max(0.0, box_b[3] - box_b[1])
    union = area_a + area_b - inter
    return inter / union if union > 0 else 0.0

def nms(boxes, scores, thr=0.5):
    order = np.argsort(-np.asarray(scores))        # 降序排列后的下标
    keep = []
    while order.size > 0:
        i = int(order[0])                          # 当前最高分
        keep.append(i)
        if order.size == 1:
            break
        rest = order[1:]
        ious = np.array([iou(boxes[i], boxes[j]) for j in rest])   # 列表推导式
        order = rest[ious <= thr]                  # 布尔索引筛选
    return keep
```

逐行讲解：

- `np.argsort(-scores)`：`argsort` 返回**下标**而不是值；加负号实现降序。**这句不会就写不出 NMS**。
- `while order.size > 0`：numpy 数组用 `.size`（元素个数）或 `len()`；`order[0]` 是第一个元素。
- `keep = []` + `keep.append(i)`：Python 的 list，等价 C++ 的 `vector<int>`。
- `[iou(boxes[i], boxes[j]) for j in rest]`：**列表推导式**，等价 C++ 的 `cpp vector<float> ious; for (int j : rest) ious.push_back(iou(boxes[i], boxes[j]));`
- `ious <= thr` 得到一个**布尔数组**，直接拿来索引 `rest` → 筛出保留的下标。这是 numpy 相对 C++ 最爽的地方。
- `int(order[0])`：numpy 的整数是 `np.int64`，转成 Python `int` 更安全（有些库不接受 np 类型）。

追问：复杂度、Soft-NMS、IoU-aware、DETR 去 NMS、GIoU/DIoU/CIoU（详见 C++ 版 2.4 节）。

最易错：忘 clamp（负面积）；`argsort` 忘了负号（变成从小到大）；忘 `union` 减 `inter`。

3.5 K-Means（numpy 广播的最佳练习题）

面试官原话："手写 kmeans。"

30 秒口述：

> "E 步算每个点到每个中心的距离平方取 argmin，M 步用簇内均值更新中心，交替到中心移动量小于阈值。距离矩阵用广播一次算出来：`(X[:,None,:] - C[None,:,:])**2` 再沿最后一维求和，得到 N×k 的矩阵。要处理空簇，工程上还要多次初始化取 inertia 最小的。"

代码：

```
def kmeans(X, k, iters=100, tol=1e-4, seed=0, n_init=1):
    X = np.asarray(X, dtype=np.float64)
    rng = np.random.default_rng(seed)
    best = (None, None, np.inf)
    for _ in range(n_init):
        centers = X[rng.choice(len(X), k, replace=False)].copy()   # 随机选 k 个样本当中心
        labels = np.zeros(len(X), dtype=int)
        for _ in range(iters):
            d = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(axis=-1)   # [N,k]
            labels = d.argmin(axis=1)                                       # [N]
            new_centers = centers.copy()
            for j in range(k):
                mask = labels == j                  # 布尔数组 [N]
                if mask.any():                      # 空簇保留旧中心
                    new_centers[j] = X[mask].mean(axis=0)
            shift = np.linalg.norm(new_centers - centers)
            centers = new_centers
            if shift < tol:
                break
        inertia = ((X - centers[labels]) ** 2).sum()
        if inertia < best[2]:
            best = (labels, centers, inertia)
    return best[0], best[1]
```

逐行讲解（这题是 numpy 的"考试题"）：

① 距离矩阵那一行，拆开看：

```
X[:, None, :]                           # (N,d) -> (N,1,d)      给每个样本插一个"中心"维
centers[None, :, :]                     # (k,d) -> (1,k,d)      给每个中心插一个"样本"维
X[:, None, :] - centers[None, :, :]     # 广播 -> (N,k,d)       每个样本减每个中心
(...) ** 2                              # 逐元素平方
.sum(axis=-1)                           # (N,k,d) 沿 d 求和 -> (N,k)   ← 距离平方矩阵
```

- 语法点 1：**索引里的 `None` 是"插一个新轴"**（等价 `np.newaxis`）。C++ 里你没法这么干，这是 numpy 独有的表达方式。
- 语法点 2：**广播让 (N,1,d) 和 (1,k,d) 变成 (N,k,d)**——规则见 2.3 节。
- 语法点 3：`.sum(axis=-1)` 沿最后一维压掉，得到 N×k。**写成 `axis=1` 也对（因为 d 是第 2 维，-1 就是 1）**，但 `-1` 更通用。
- 性能提示：N 和 k 都很大时这个 (N,k,d) 中间张量吃内存，工程上用 `scipy.spatial.distance.cdist` 或分块。面试提一句"大规模会分块/用 cdist"即可。

② **`d.argmin(axis=1)`**：沿每一行找最小值的**下标**（不是值）。C++ 对照：

```
for (int i = 0; i < N; ++i) { float best = 1e30f; for (int j = 0; j < k; ++j) if (d[i][j] < best) { best = d[i][j]; lab[i] = j; } }
```

③ **`centers.copy()`**：Python 的 `=` 是引用！不 copy 就会在上一步的数组上原地改，导致 `shift` 永远算成 0、提前收敛。**这是 Python 新手最典型的静默 bug**，也值得在面试里主动说一句"这里必须 deep copy"。

④ **`mask = labels == j` → 布尔数组；`X[mask]` → 取出这些样本；`.mean(axis=0)` → 沿样本维求均值**，得到新的中心 (d,)。`mask.any()` 判空簇。

⑤ **`np.linalg.norm(v)`**：向量二范数。等价 C++ 里 `sqrt(sum(v*v))`。

⑥ **`inertia = ((X - centers[labels]) ** 2).sum()`**：`centers[labels]` 是**用标签数组反查中心**（高级索引！），得到每个点所属的中心 (N,d)，再算距离平方总和。这句很 numpy，值得单独记：**`centers[labels]` 是"按标签把中心分配给每个点"**。

追问 Q&A（kmeans 的追问比代码更重要）：

- **Q：k 怎么选？** A：肘部法（inertia 随 k 的拐点）、轮廓系数 silhouette、Gap statistic。
- **Q：初始化？** A：k-means++（按"到已有中心的最近距离平方"成比例采样），或跑 `n_init` 次取 inertia 最小。
- **Q：一定收敛到全局最优吗？——不是。这条我有实测数据：** 我在三个高斯簇（150 个点）上跑 20 个随机种子，**单次初始化的结果：min=125、中位=150、max=150，也就是 20 次里有 7 次没聚对**；改成 `n_init=20` 取最优后稳定 150/150。**面试官问到"局部最优"时，直接讲这个数字。**
- **Q：复杂度？** A：O(N · k · d · iters)。加速：MiniBatch、Elkan 三角不等式、先 PCA 降维。
- **Q：假设与局限？** A：凸簇、各向同性、方差/大小相近；对异常值敏感；硬分配（软分配要 GMM）。

- **Q：numpy 里怎么避免 (N,k,d) 的内存爆炸？** A：用 `scipy.spatial.distance.cdist(X, C, 'sqeuclidean')`，或分块计算，或利用 `||x||² + ||c||² - 2x·c` 展开成两次矩阵乘。

最易错 3 点：忘 `copy()` 导致原地修改；空簇不处理（除零得 nan）；用 `argmax` 而不是 `argmin`。

3.6 Self-Attention（单头，numpy 版只要 8 行）

面试官原话："用 Python 手写 self-attention。"

30 秒口述：

> "X 乘三个权重得到 Q、K、V；Q 乘 K 转置得到 L×L 分数，除 √d 缩放；mask 位置置 -1e9；沿最后一维 softmax；最后乘 V。numpy 里矩阵乘用 `@`，转置用 `.T`。"

代码：

```python
def self_attention(X, Wq, Wk, Wv, mask=None):
    Q = X @ Wq                              # [L,D]
    K = X @ Wk
    V = X @ Wv
    dk = Q.shape[-1]                        # = D
    scores = Q @ K.T / math.sqrt(dk)        # [L,D] @ [D,L] -> [L,L]
    if mask is not None:
        scores = np.where(mask, -1e9, scores)    # 逐元素三目
    attn = softmax(scores, axis=-1)         # 沿最后一维（每个 query 对所有 key）
    return attn @ V, attn                   # [L,L] @ [L,D] -> [L,D]
```

逐行讲解：

写法	语法点	对应 C++
`X @ Wq`	`@` 是矩阵乘运算符（Python 3.5+）；等价 `np.dot(X, Wq)`	`matmul(X, Wq, L, D, D)`
`K.T`	转置（2 维就是行列互换）	索引里换位置 `K[j*D+t]`
`Q.shape[-1]`	取最后一维长度，**比硬写 D 更安全**	传参 D
`Q @ K.T / math.sqrt(dk)`	矩阵乘完再逐元素除（`/` 对标量是逐元素）	循环里 `* scale`
`np.where(mask, -1e9, scores)`	**逐元素三目**：mask 为 True 取 -1e9，否则取原值	`if (mask[i]) S[i] = -1e30f;`
`softmax(scores, axis=-1)`	复用 3.1 的函数，按行归一化	`softmaxInPlace(S, i*L, L)`
`attn @ V`	[L,L] @ [L,D]	`matmul(attn, V, L, L, D)`

注意 mask 的两种写法（面试可能问你偏好哪种）：- `np.where(mask, -1e9, scores)`：生成新数组，不改原值。- `scores[mask] = -1e9`：**就地修改**（更省内存），但会改到调用方的数组，需要先 copy。

追问 Q&A：- **Q：为什么除 √dk？** A：q·k 的方差随 dk 线性增长，不缩放会让 softmax 饱和、梯度消失。- **Q：复杂度？** A：时间 $O(L^2D)$、显存 $O(L^2)$。**FlashAttention 省的是显存和 HBM 读写，不减少 FLOPs。**- **Q：`K.T` 在多 batch 下怎么写？** A：3 维要用 `K.transpose(-2, -1)`；**`.T` 只对 2 维有效**（这是踩过的坑，值得主动说）。- **Q：mask 一行全 True 会怎样？** A：`exp(-1e9 - max)` 全是 0，分母 0 → nan。要么保证每行至少一个有效位，要么对全屏蔽行单独置 0。- **Q：self 和 cross 的区别？** A：Q 的来源。**多模态融合就是 cross-attention：图像 token 当 K/V、文本 token 当 Q（或反之）**——这条一定要主动讲，能接上你简历里的视觉-激光融合。

最易错 3 点：`K.T` 用错（写成 `K`）；softmax 不指定 `axis=-1`；mask 用 0 表示屏蔽。

3.7 Multi-Head Attention（重点题，numpy 版 20 行）

面试官原话："改成多头。"

30 秒口述：

> "多头就是把 D 按**列**切成 h 份，每份 dk=D/h。第 i 个头取 Wq/Wk/Wv 的第 i 个列块，各自算一遍 attention，得到 h 个 L×dk，沿最后一维 concat 回 L×D，最后乘 Wo。切片用 `Wq[:, hi*dk:(hi+1)*dk]`。"

代码：

```
def multi_head_attention(X, Wq, Wk, Wv, Wo, h, mask=None):
    L, D = X.shape
    dk = D // h
    outs, attns = [], []
    for hi in range(h):
        sl = slice(hi * dk, (hi + 1) * dk)        # 第 hi 个头占的列区间
        Q = X @ Wq[:, sl]                         # [L,D] @ [D,dk] -> [L,dk]
        K = X @ Wk[:, sl]
        V = X @ Wv[:, sl]
        scores = Q @ K.T / math.sqrt(dk)
        if mask is not None:
            scores = np.where(mask, -1e9, scores)
        a = softmax(scores, axis=-1)
        outs.append(a @ V)                        # [L,dk]
        attns.append(a)
    O = np.concatenate(outs, axis=-1)             # 沿最后一维拼 [L,D]
    return O @ Wo, np.stack(attns, axis=0)        # [h,L,L]
```

逐行讲解：

- `sl = slice(hi*dk, (hi+1)*dk)`：等价于 `hi*dk : (hi+1)*dk` 这个切片，写成 `slice` 对象是为了复用。
- `Wq[:, sl]`：**逗号左边是行（全取），右边是列（只取这一段）** → 得到 (D, dk)。这就是"按列切头"，**和 C++ 版里 `Ws[r*dk+c] = W[r*D + hi*dk + c]` 是同一件事**。
- `X @ Wq[:, sl]`：(L,D) @ (D,dk) → (L,dk)，每个头在自己的 dk 维子空间里算。
- `np.concatenate(outs, axis=-1)`：h 个 (L,dk) 沿最后一维拼 → (L,D)。**C++ 里就是 `Ocat[i*D + hi*dk + c]` 那个双重循环。**
- `np.stack(attns, axis=0)`：h 个 (L,L) 堆成 (h,L,L)。**`stack` 是"新建一维"，`concatenate` 是"在已有维上接"**——这两个函数别搞混。
- `O @ Wo`：输出投影，不能忘。

我在本地实测过的三个性质（可当"你怎么确认写对了"的答案）：1. **`h=1` 且 `Wo=单位阵` 时，多头版和单头版逐元素完全一致**（我断言里用 `np.allclose` 验证过）。2. 每个头每一行的 attention 权重和 = 1。3. 输出 shape 恒为 (L,D)。

追问 Q&A：- **Q：参数量？** A：$4D^2$（Q/K/V/O 各 D×D），**与 h 无关**。- **Q：为什么多头有用？** A：不同头在不同子空间学不同依赖（位置、句法、指代、跨模态对应）。- **Q：MQA/GQA 省什么？** A：省推理时的 **KV cache**：MHA 每 token 存 2D，MQA 存 2D/h，GQA 存 2D·g/h；训练 FLOPs 基本不变。- **Q：numpy 用列表 append 再 concat，效率会不会差？** A：h 很小（8~32），完全无所谓；真要快就一次分配 `O = np.zeros((L, D))` 再按列赋值。- **Q：PyTorch 版怎么写？** A：见第 4 章，核心差别是 `view + transpose` 代替切片循环。

最易错 3 点：切行而不是切列（应该切 `W` 的**列**）；`dk` 忘了替换进缩放；忘了乘 `Wo`。

3.8 CART 分类树（Python 里最长的一题，但递归比 C++ 好写）

面试官原话："手写决策树 / CART 怎么分裂。"

30 秒口述：

> "CART 是二叉树，用基尼不纯度。对每个特征的每对相邻取值中点试切分，选加权基尼下降最多的做二分，递归到达到停止条件。叶子取多数类。停止条件：深度上限、样本数小于 2 倍最小叶子样本、节点已纯。"

代码：

```
class Node:
    __slots__ = ("feat", "thr", "left", "right", "value")
    def __init__(self):
        self.feat, self.thr, self.left, self.right, self.value = -1, 0.0, None, None, 0

def _gini(y, n_cls):
    if len(y) == 0:
```

```
        return 0.0
    p = np.bincount(y, minlength=n_cls) / len(y)     # 每类占比
    return 1.0 - (p ** 2).sum()

def _build(X, y, depth, max_depth, min_leaf, min_gain, n_cls):
    node = Node()
    node.value = int(np.bincount(y, minlength=n_cls).argmax())    # 叶子值 = 多数类
    if depth >= max_depth or len(np.unique(y)) == 1 or len(y) < 2 * min_leaf:
        return node
    parent = _gini(y, n_cls)
    if parent < 1e-12:
        return node
    n = len(y)
    best_gain, best = min_gain, None
    for f in range(X.shape[1]):
        vals = np.unique(X[:, f])
        thrs = (vals[:-1] + vals[1:]) / 2.0          # 相邻中点
        for t in thrs:
            m = X[:, f] <= t                          # 布尔数组：走左边
            nl = int(m.sum())
            nr = n - nl
            if nl < min_leaf or nr < min_leaf:
                continue
            g = parent - (nl * _gini(y[m], n_cls) + nr * _gini(y[~m], n_cls)) / n
            if g > best_gain:
                best_gain, best = g, (f, t, m)
    if best is None:
        return node
    f, t, m = best
    node.feat, node.thr = f, t
    node.left = _build(X[m], y[m], depth + 1, max_depth, min_leaf, min_gain, n_cls)
    node.right = _build(X[~m], y[~m], depth + 1, max_depth, min_leaf, min_gain, n_cls)
    return node
```

逐行讲解（Python 语法点为主）：

写法	语法点
`class Node:` + `def __init__(self)`	构造函数；`self.x` 是成员变量，等价 C++ 的 `this->x`
`__slots__ = (...)`	声明只有这几个属性，省内存（面试可以不提，写不写都对）
`self.feat, self.thr, ... = -1, 0.0, None, None, 0`	Python 支持**多重赋值**，一行初始化五个成员
`None`	空指针，等价 C++ 的 `nullptr`
`np.bincount(y, minlength=n_cls)`	统计每个整数类别出现几次 → 长度 n_cls 的计数数组
`.argmax()`	返回最大值的位置 = 多数类的类别号
`p = bincount / len(y)`	计数数组除样本数 → 概率（逐元素除）
`1.0 - (p ** 2).sum()`	`**` 是幂，`(p**2).sum()` 就是 Σp^2
`X[:, f]`	第 f 列（所有行）
`np.unique(...)`	去重并升序排序 → 候选切点来源
`vals[:-1]`、`vals[1:]`	切片：去掉最后一个 / 去掉第一个，两者相加除 2 得到所有中点（**`[:-1]` 这种写法在 C++ 里要额外循环，numpy 一行**）
`m = X[:, f] <= t`	布尔数组，**注意 Python 里布尔运算用 `~` 取反，不是 `!`**
`y[m]`、`y[~m]`	布尔索引取子集 / 取反后的子集
`int(m.sum())`	布尔数组求和 = True 的个数（`np.bool_` 转 int）
`if best is None:`	判断 None 用 `is`，**不要用 `==`**
`f, t, m = best`	元组解包，一行拆出三个值

为什么 Python 版比 C++ 版短：C++ 要维护 `idx` 下标数组、节点池、前缀扫描；Python 直接用布尔索引切 `X[m]`、`y[m]`，列表和对象都是引用传递，递归天然简洁。**代价是慢**（每次切都复制数组），面试时补一句"生产会用分位分桶 + 前缀扫描"即可。

追问 Q&A：- **Q：CART / ID3 / C4.5 区别？** 信息增益 / 增益率 / 基尼；ID3 多叉、CART 二叉。- **Q：为什么用基尼？** 无 log，算得快；与熵的划分结果高度一致。- **Q：剪枝？** 预剪枝（本代码的三个停止条件）vs 后剪枝（代价复杂度 α，自底向上合并）。- **Q：连续特征？** 排序取中点（本代码）；大样本用直方图分桶（LightGBM 255 桶）。- **Q：特征重要性？** 加权不纯度下降累积；更可靠用置换重要性。- **Q：怎么推到 GBDT/XGBoost？** 残差/负梯度 → 二阶泰勒展开 + 正则 $\gamma T + \lambda/2 \cdot \Sigma w^2$ → 最优叶权重 $w^* = -\Sigma g/(\Sigma h+\lambda)$ → 直方图并行。LightGBM 是 leaf-wise + GOSS + EFB。- **Q：`np.unique` 每个特征都调一次，会不会慢？** 会（O(n log n)·d）；工程上排序一次、或用直方图；面试主动提"可以排序+前缀扫描避免重复统计"。

最易错 3 点：漏停止条件（递归爆栈）；阈值用取值本身而不是中点；`y[~m]` 写成 `y[!m]`（Python 语法错）。

---

4. PyTorch 版（面试官说"用框架写"时）

> 为什么还要准备这一版：多模态岗的团队大概率天天写 torch，面试官很可能直接说"用 PyTorch 写一下吧"。 好消息：**你已经会 numpy 版了，torch 版只差 8 个语法差异**。

4.1 numpy → torch 的 8 个差异（背完就能改）

#	numpy	torch	说明
1	`x.max(axis=-1, keepdims=True)`	`x.max(dim=-1, keepdim=True).values`	torch 的 `max` 返回 (values, indices)，**必须取 `.values`**；参数名是 `dim`、`keepdim`（没有 s）
2	`x.sum(-1)`	`x.sum(-1)` 或 `x.sum(dim=-1)`	通用
3	`np.where(mask, -1e9, s)`	`s.masked_fill(mask, float('-inf'))`	torch 用 `masked_fill` 就地填；注意**语义：True = 屏蔽**
4	`logits[np.arange(N), t]`	`logits.gather(1, t[:, None]).squeeze(1)`	torch 没有高级索引，用 `gather` 按**维度**取
5	`.reshape(...)`	`.view(...)` / `.reshape(...)`	**`view` 要求内存连续**，transpose 之后必须先 `.contiguous()`
6	`x.T`	`x.transpose(-2, -1)`	torch 的 `.T` 只对 2 维且已弃用，统一用 `transpose`
7	标量就是 float	要 `.item()`	打印或比较时用 `x.item()` 取值
8	无梯度	`loss.backward()` + `p.grad`	手撕一般不用，但"能接进训练"是加分点

最大的一个坑（面试官爱看）：`view` 和 `contiguous` 的关系。

```
x = torch.randn(2, 4, 8)
y = x.transpose(1, 2)        # [2,8,4]，但内存不连续
y.view(2, 32)                # ❌ RuntimeError: view size is not compatible
y.contiguous().view(2, 32)   # ✅ 先复制成连续内存，再 view
y.reshape(2, 32)             # ✅ reshape 内部会自动处理（但可能悄悄复制）
```

讲法："拆头和拼头都要经过 transpose，所以拼头时必须 `contiguous()`，否则 `view` 报错。"

4.2 完整 MHA（已实测通过，与官方实现逐元素一致）

```
import math, torch, torch.nn as nn

def mha_core(Q, K, V, h, mask=None):
    """Q/K/V: [B,L,D], mask: 可广播到 [B,h,Lq,Lk] 的 bool, True = 屏蔽。"""
    B, Lq, D = Q.shape
    Lk = K.size(1)
    dk = D // h
    Qh = Q.view(B, Lq, h, dk).transpose(1, 2)      # [B,h,Lq,dk]
    Kh = K.view(B, Lk, h, dk).transpose(1, 2)
    Vh = V.view(B, Lk, h, dk).transpose(1, 2)
    s = Qh @ Kh.transpose(-2, -1) / math.sqrt(dk)  # [B,h,Lq,Lk]
    if mask is not None:
        s = s.masked_fill(mask, float('-inf'))
    a = torch.softmax(s, dim=-1)
    o = (a @ Vh).transpose(1, 2).contiguous().view(B, Lq, D)   # 拼头
    return o, a
```

```python
class MHA(nn.Module):
    def __init__(self, d_model, n_head):
        super().__init__()
        assert d_model % n_head == 0
        self.h = n_head
        self.wq = nn.Linear(d_model, d_model)
        self.wk = nn.Linear(d_model, d_model)
        self.wv = nn.Linear(d_model, d_model)
        self.wo = nn.Linear(d_model, d_model)

    def forward(self, q, k, v, mask=None):
        out, attn = mha_core(self.wq(q), self.wk(k), self.wv(v), self.h, mask)
        return self.wo(out), attn
```

与 numpy 版的对应关系（面试时可以这么讲）：

numpy 版	torch 版	含义
`Wq[:, sl]` 切片循环	`view(B,L,h,dk).transpose(1,2)`	都是"把 D 切成 h 个 dk"
`np.concatenate(outs, -1)`	`.transpose(1,2).contiguous().view(B,L,D)`	都是"拼回 D"
`np.where(mask, -1e9, s)`	`s.masked_fill(mask, -inf)`	都是"屏蔽"
手动 `softmax(s, axis=-1)`	`torch.softmax(s, dim=-1)`	同

4.3 torch 版 softmax + 交叉熵（与 F 一致，已实测）

```python
def torch_softmax(x, dim=-1):
    x = x - x.max(dim=dim, keepdim=True).values
    e = x.exp()
    return e / e.sum(dim=dim, keepdim=True)

def torch_cross_entropy(logits, target, weight=None):
    m = logits.max(dim=1, keepdim=True).values
    lse = m.squeeze(1) + torch.log(torch.exp(logits - m).sum(dim=1))
    nll = lse - logits.gather(1, target[:, None]).squeeze(1)
    if weight is not None:
        w = weight[target]
        return (nll * w).sum() / w.sum()
    return nll.mean()
```

实测结果：`torch_softmax` 与 `F.softmax` 一致；`torch_cross_entropy` 与 `F.cross_entropy` 一致（**含 `weight` 参数时也一致**）。

4.4 torch 版 kmeans（用 cdist 替代手写距离矩阵）

```python
def torch_kmeans(X, k, centers0, iters=100):
    X = torch.as_tensor(X, dtype=torch.float32)
    C = torch.as_tensor(centers0, dtype=torch.float32).clone()
    for _ in range(iters):
        d = torch.cdist(X, C)                # [N,k] 成对距离，省掉 (N,k,d) 中间张量
        lab = d.argmin(dim=1)
        newC = C.clone()
        for j in range(k):
            m = lab == j
            if m.any():
                newC[j] = X[m].mean(dim=0)
        if torch.allclose(newC, C, atol=1e-5):
            break
        C = newC
    return lab
```

实测：在同一初始中心下，torch 版与 numpy 版的分配结果**逐点完全一致**。

---

5. 怎么知道自己写对了（面试最值钱的技能）

面试官常问："你怎么确认这个实现是对的？" 大部分人说"我跑了一下没报错"。你可以给三档答案：

5.1 第一档：数学性质自检（不需要参照物）

题	必查性质
softmax	每行和为 1；对加常数不变（平移不变性）；大数（1e3）不出 nan
交叉熵	等于 `-log(softmax(x)[target])`；恒为正
LayerNorm	输出每个样本 mean≈0、var≈1
MHA	每个头每行 attention 和为 1；输出 shape 与输入一致；mask 位置权重为 0
kmeans	inertia 单调不增；多次初始化后 inertia 稳定
CART	训练集能拟合到 100%（过拟合是树的本性，正好用来验证实现没写错）

5.2 第二档：拿官方实现当参照物（最硬的答案）

```
# torch 版 attention 的正确性，直接和官方 SDPA 对拍
ref = F.scaled_dot_product_attention(Qh, Kh, Vh)                    # 无 mask
ref_causal = F.scaled_dot_product_attention(Qh, Kh, Vh, is_causal=True)
# 交叉熵对拍
F.cross_entropy(logits, target)
```

注意一个大坑：`F.scaled_dot_product_attention(attn_mask=...)` 的 **bool 语义是 `True = 允许参与`**，和我们手写 `masked_fill(mask, -inf)`（`True = 屏蔽`）**正好相反**。面试时主动提这一点，会显得你真的用过。

我的实测：无 mask 和因果 mask 两种情况，手写版与 SDPA **逐元素一致（1e-5 内）**。

5.3 第三档：退化测试（特殊输入下应退化成简单形式）

- 多头 attention 设 `h=1` 且 `Wo=单位阵` → 应与单头完全一致（**我实测一致**）。
- softmax 输入是 one-hot 的大值 → 输出应几乎还是 one-hot。
- kmeans 设 `k=1` → 所有点一个标签，中心 = 全局均值。

面试话术：

> "我一般做三层验证：先查数学性质（比如 attention 每行和为 1），再用官方算子对拍（SDPA / F.cross_entropy），最后做退化测试（h=1 应该退化成单头）。"

这段话能让面试官对你的工程严谨性直接上一个档。

---

6. 练习计划（针对 Python 弱的 3 天版）

时间	任务	验收
D1 上午	第 1、2 章（语法 + numpy 五个概念）	能不看文档写出：按行 softmax、按列求和、`X[:,None,:]` 的含义、布尔索引取子集
D1 下午	3.1 softmax + 3.2 交叉熵（numpy）	不看书写出；能说 `keepdims` 为什么必须、高级索引怎么取 target
D2 上午	3.5 kmeans（numpy）	能默写距离矩阵那一行并解释广播；能讲局部最优的实测数字（20 次里 7 次失败）
D2 下午	3.6 单头 → 3.7 多头（numpy）	单头 10 分钟写完；多头只改 4 处（切片、dk、concat、Wo）
D3 上午	3.8 CART（numpy）	二分类训练集 100%；能讲基尼/停止条件/GBDT 推导
D3 下午	第 4 章 torch 版 + 第 5 章验证法	能把 numpy 版 10 分钟内改成 torch 版（`view/transpose/contiguous/masked_fill/gather` 五个词不出错）

面试前一晚自检清单（全打勾再睡）

- [ ] `keepdims=True` / `keepdim=True` 什么时候必须加
- [ ] `axis=-1` 是"最后一维"，softmax/LN 都用它
- [ ] 广播三规则，以及 `X[:, None, :]` 是在中间插一个轴
- [ ] `logits[np.arange(N), target]` 是交叉索引取正确类
- [ ] `argsort(-scores)` 是降序下标（NMS 必用）
- [ ] `center[labels]` 是"按标签分配中心"（kmeans inertia 那句）
- [ ] `copy()` 必须有，Python 赋值是引用
- [ ] `@` 矩阵乘、`.T` 转置（3 维要用 `transpose(-2,-1)`）
- [ ] torch 的 `max(...).values`、`gather`、`masked_fill`、`contiguous().view`
- [ ] SDPA 的 bool mask 语义与 `masked_fill` **相反**
- [ ] 多层验证三件套：数学性质 → 官方对拍 → 退化测试

附录：配套文件与实测结果

文件：`ai_interview_py.py`（numpy + torch，与本文档代码一致）

```
python3 ai_interview_py.py       # 需要 numpy；装了 torch 会自动跑 torch 部分
```

本机实测（python3 + numpy + torch 2.14.1+cpu）：39 项断言全部通过，0 失败

模块	断言数	关键结论
softmax / log_softmax	5	行和为 1；1e3 级不溢出；平移不变；与手算一致；log_softmax 取 exp 可还原
cross_entropy	3	与按定义手算一致；带 weight 可跑；1e3 级 logits 不溢出
layer_norm	1	每个样本 mean≈0、var≈1
IoU / NMS	3	IoU=81/119；无交集为 0；NMS 保留
kmeans	2	**单次初始化 20 个 seed：min=125、中位=150、max=150，7/20 没聚对**；`n_init=20` 后稳定 150/150
self-attention	5	行和=1；输出=A·V；因果 mask 上三角=0；mask 后行和仍=1；下三角比例保持
multi-head	4	形状 [h,L,L]/[L,D]；每头行和=1；**h=1 且 Wo=I 时与单头逐元素一致**
CART	3	二分类/三分类 ACC=1.0；根阈值落在簇间（thr=1.523）
torch softmax/CE	3	与 `F.softmax`、`F.cross_entropy`（含 weight）一致
torch MHA	7	与 `F.scaled_dot_product_attention`（无 mask / 因果 mask）逐元素一致；mask/padding 位置权重为 0；行和=1
torch nn.Module	2	前向形状正确；反向传播有梯度
torch kmeans	1	与 numpy 版在同初始中心下分配完全一致

怎么用这份文件练：先关掉文件在纸上写 → 打开对照 → 删掉重写 → 跑 `python3 ai_interview_py.py` 看断言是否全绿。